

AI&ML Stage2 Programming



High performance. Delivered.

consulting | technology | outsourcing

Goals

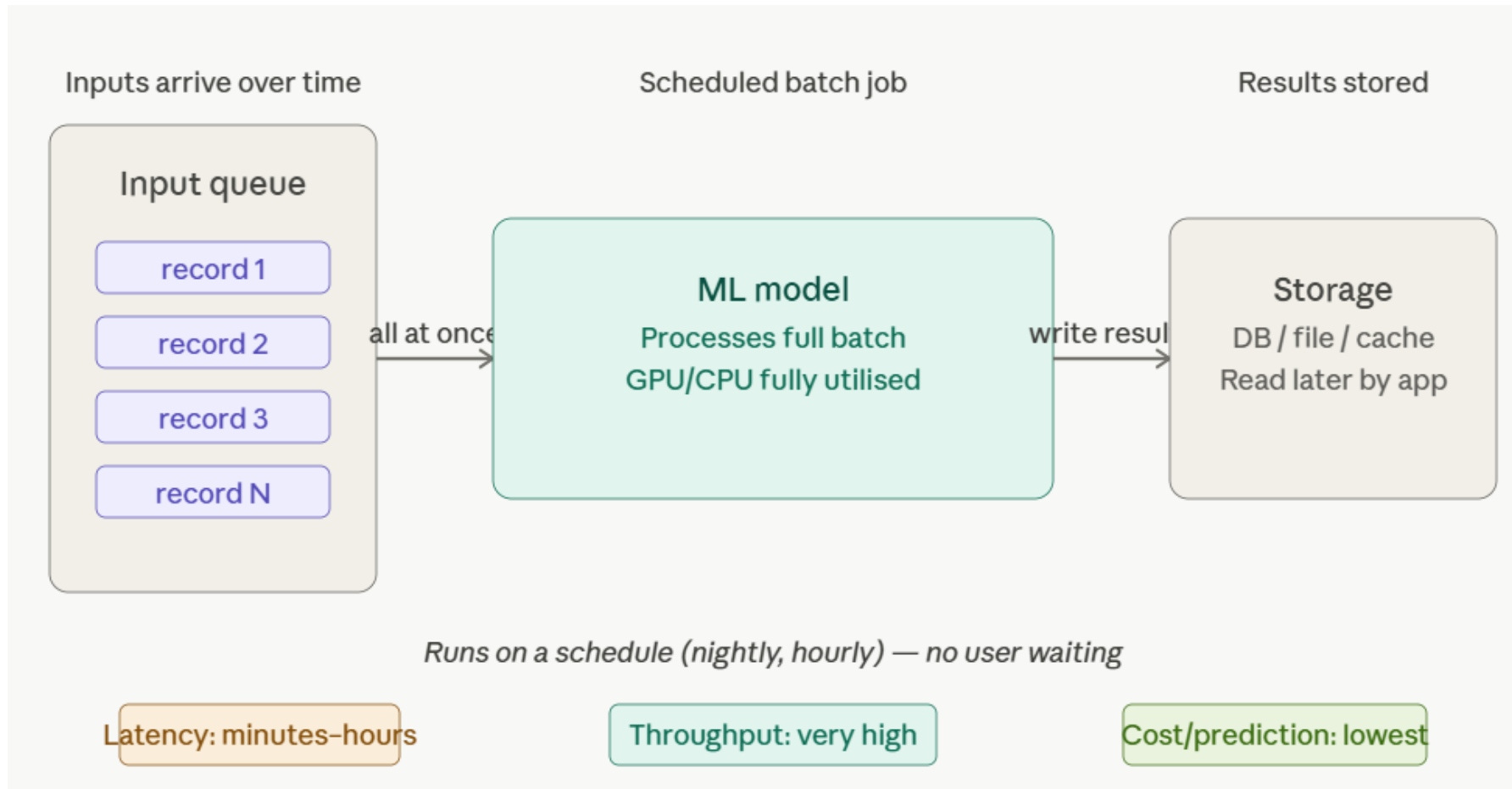
- Machine Learning Engineering and ML System Design
- Deep Learning and Scalable Model Serving
- NLP, LLMs and Generative AI Engineering
- Cloud-Native MLOps and Platform Automation

- ML Architecture Patterns
 - Batch vs real-time vs streaming inference
 - Synchronous/async microservices; routing
 - Multi-tenant serving; versioning and rollback
- Modeling & Evaluation
 - Supervised (linear/logistic, tree/GBM); unsupervised (K-Means, PCA)
 - CV, hyperparameter tuning; metric selection
 - Model packaging (pickle/ONNX) and contracts

Batch inference

- Inputs are collected and processed together as a group, on a schedule or when a threshold is reached.
- **Characteristics:** High throughput, high latency, lowest cost per prediction. No user waiting — results are written to storage for later use.
- **Best for:** Nightly report generation, retraining pipelines, processing large datasets (embeddings, recommendations, fraud scoring on historical transactions).
- **Example:** Every night, run all 1M user profiles through a recommendation model and write results to a database.

Batch inference



Batch Inference Real Time Scenarios

- **Scenario 1 — E-commerce recommendations** (nightly, 20 users): Pre-computes personalised product lists for every user using ThreadPoolExecutor. In production this runs overnight on millions of users and writes results to Redis or DynamoDB for instant homepage loads.
- **Scenario 2 — Fraud detection** (daily, 30 transactions): Scores prior-day transactions for suspicious patterns (high amount, foreign country, unusual hour). Flagged results go to a human review queue. 6 workers → cost index drops to 16 vs 100 for single-threaded.

Batch Inference Real Time Scenarios

- **Scenario 3 — Sentiment analysis** (on-demand, 20 reviews): Classifies customer reviews as positive/neutral/negative with a confidence score. In production this feeds into a product quality dashboard updated daily.
- **Scenario 4 — Credit risk scoring** (monthly, 20 applications): Scores loan applications into approved/review/rejected tiers. The top-5 highest-risk applications are surfaced for manual underwriter review.

Transformers Models

- **Transformer models** are a type of neural network architecture that revolutionized fields like natural language processing (NLP), computer vision, and more.
- They were introduced in the landmark paper Attention Is All You Need by researchers at Google Brain in 2017.

Transformers Models

-  **Core Idea: Attention Instead of Recurrence**
- Unlike older models (like RNNs or LSTMs), transformers don't process data sequentially.
- Instead, they rely on a mechanism called **self-attention**, which lets the model look at *all parts of the input at once* and decide what matters most.

Transformers Models

-  **Key Components**
- **1. Self-Attention Mechanism**
 - Assigns importance (weights) to different words/tokens in a sequence
 - Helps capture long-range dependencies efficiently
- **2. Multi-Head Attention**
 - Runs multiple attention processes in parallel
 - Each “head” learns different relationships (syntax, meaning, etc.)
- **3. Positional Encoding**
 - Since transformers don’t inherently understand order, positional encodings inject sequence information
- **4. Feedforward Layers**
 - Fully connected layers applied after attention to refine representations

Transformers Models

-  **Architecture Overview**
- A typical transformer has two main parts:
- **Encoder**: Reads and understands input
- **Decoder**: Generates output (e.g., translated text)
- Many modern models use only one part:
- Encoder-only → e.g., BERT
- Decoder-only → e.g., GPT

Transformers Models

- ⚡ **Why Transformers Are Powerful**
- Parallel processing → faster training than RNNs
- Better at long-range dependencies
- Scales extremely well with data and compute

Transformers Models

-  **Applications**
- Machine translation (Google Translate)
- Chatbots and assistants
- Text summarization
- Image processing (Vision Transformers)
- Code generation

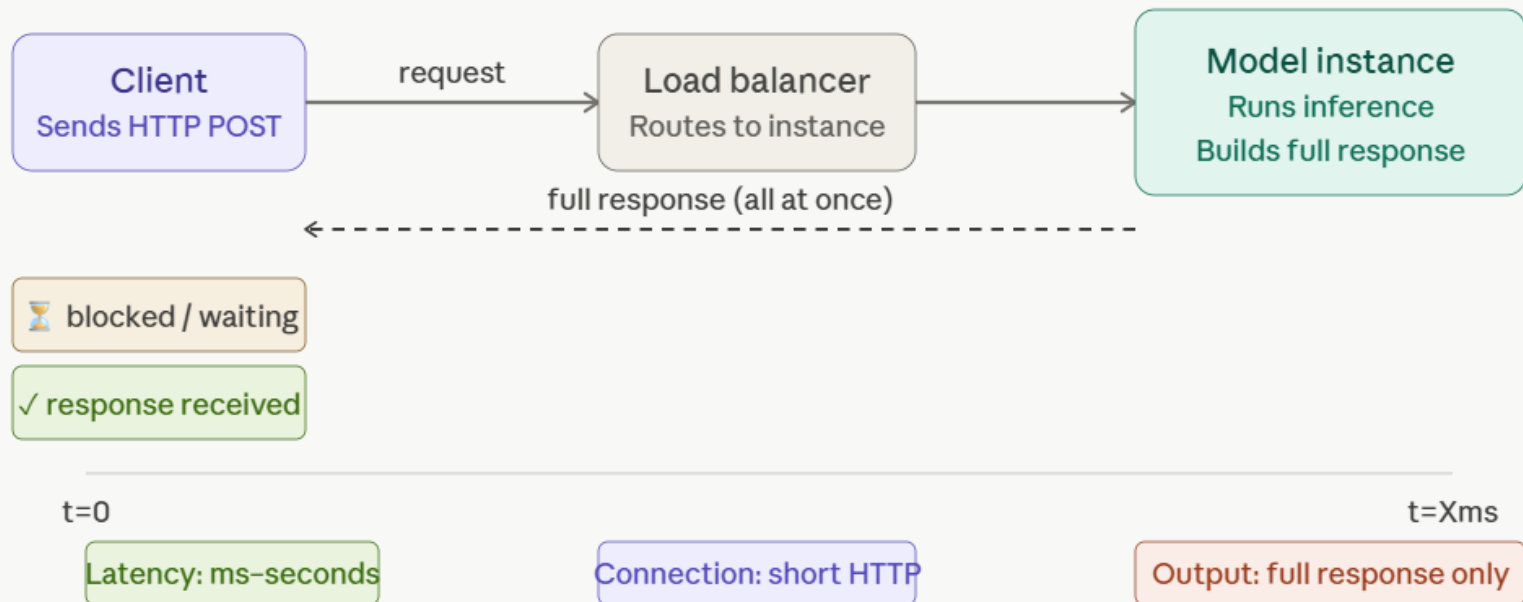
Real-Time (Request-Response) Inference

- A single input arrives, the model processes it immediately, and the result is returned synchronously — typically over HTTP.
- **Characteristics:** Low latency (milliseconds to seconds), low throughput per instance, higher cost per prediction. The caller blocks until the response arrives.
- **Best for:** Search ranking, content moderation, spam detection, image classification on upload, chatbot single-turn replies.
- **Example:** A user submits a photo → API calls a vision model → returns labels within 200ms.

Real-Time (Request-Response) Inference



Real-time inference is a synchronous request-response loop — the client blocks until the full prediction comes back.



Real-Time (Request-Response) Inference

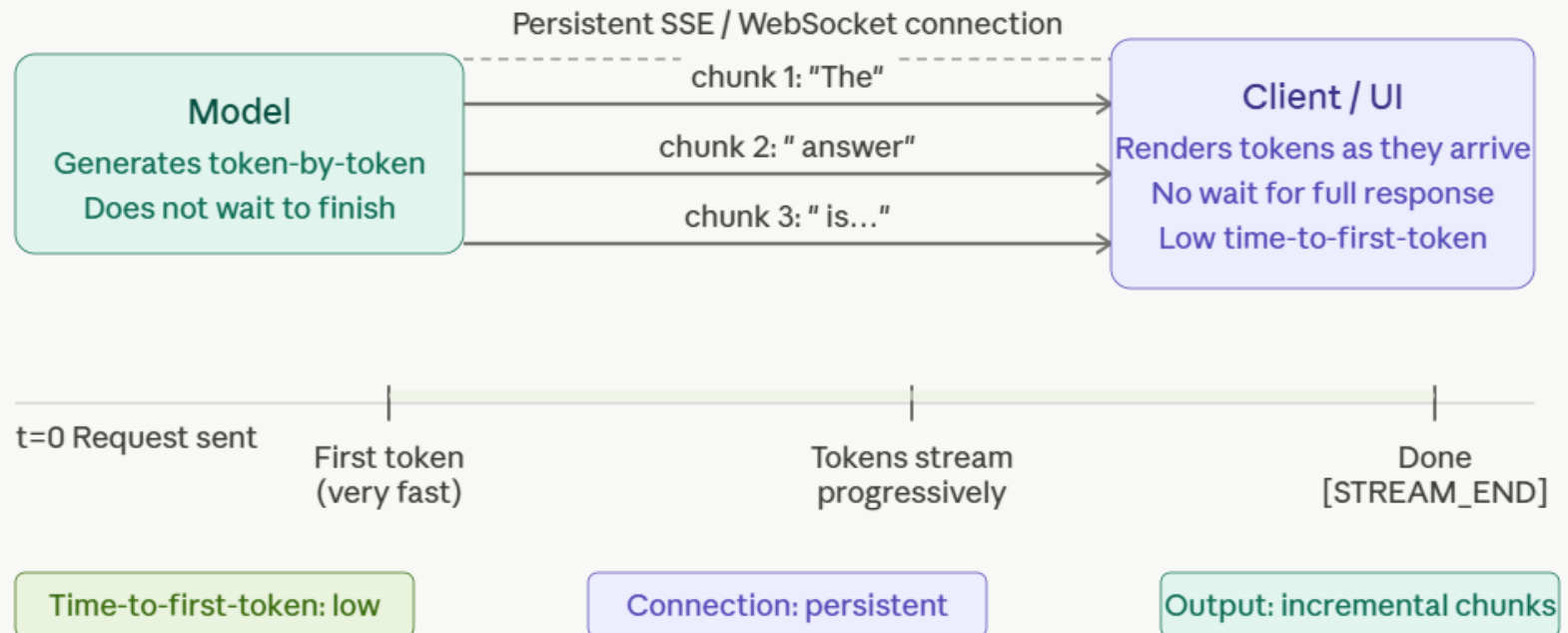
Scenario	Example
Chatbot	User message sentiment detected instantly
Product review form	Review classified while user submits
Customer support	Angry customer message flagged immediately
Social media monitoring	New post classified as positive/negative
Fraud detection	Transaction checked instantly
Recommendation system	User action triggers immediate suggestion
AI counselling app	Student message classified immediately

Streaming Inference

- The model generates output incrementally and sends tokens/chunks to the client as they're produced, rather than waiting for the full response.
- **Characteristics:** Time-to-first-token is low even if total generation is long. The client starts rendering immediately. Requires a persistent connection (SSE or WebSocket).
- **Best for:** LLM text generation, code completion, any output where partial results are useful and total latency would otherwise feel long.
- **Example:** You ask Claude a question — words appear progressively rather than all at once after a 5-second wait.

Streaming Inference

Streaming inference keeps a persistent connection open and pushes tokens to the client as they're generated — so the user sees output almost immediately even for long responses.



- **Scenario 1 — LLM Chat Assistant (Token Streaming)**
- **Business Use Case**
- Simulates conversational AI systems such as:
 - ChatGPT
 - Claude
 - Gemini
- The response is streamed **token-by-token**, allowing users to start reading immediately rather than waiting for the entire answer.

Streaming Inference – Realtime Scenario



Metric	Value
TTFT	280 ms
Total Response Time	4799 ms
Tokens Generated	102
Throughput	21.3 tokens/sec

- **Interpretation**

- User sees the **first token in 280 ms**
- Full response completes in **4.8 seconds**
- Streaming reduces perceived waiting time dramatically

- Without streaming:

- User waits 4.8 seconds staring at blank screen

- With streaming:

- 280 ms → first token visible
- 320 ms → response continues
- 4.8 s → response complete

- **Business Benefits**
 - Improved responsiveness
 - Better conversational experience
 - Reduced perceived latency
 - Higher engagement

- **Scenario 2 — Live Meeting Transcription (Speech Streaming)**
- **Business Use Case**
- Simulates real-time speech transcription systems:
 - Microsoft Teams
 - Google Meet
 - Zoom Live Captions
- Speech is converted into text continuously while participants speak.

Streaming Inference – Realtime Scenario



Parameter	Value
Speech Speed	~130 WPM
TTFT	400–500 ms
Delay	Less than one sentence

- **Interpretation**
- Captions appear nearly in real time and stay slightly behind the speaker.
- Instead of waiting:
 - (wait until sentence ends)
- Users see:
 - Good
Good morning
Good morning everyone

Streaming Inference – Realtime Scenario



- **Business Benefits**
- Accessibility
- Meeting transcription
- Live subtitles
- Improved collaboration

- **Scenario 3 — AI Clinical Note Generator**
- **Business Use Case**
- Simulates AI-powered clinical documentation inside an Electronic Health Record (EHR).
- As the doctor speaks, a structured SOAP note is generated live.
- SOAP Structure:
- Subjective
- Objective
- Assessment
- Plan

Streaming Inference – Realtime Scenario



Metric	Value
TTFT	342 ms
Total Time	4327 ms
Tokens Generated	132

- **Interpretation**
- Revenue insights appear immediately while supporting references arrive progressively.
- **Business Benefits**
 - Faster decision making
 - Evidence-backed responses
 - Real-time financial intelligence
 - Trustworthy AI outputs

Streaming Inference – Realtime Scenario



Metric	Meaning
TTFT	Time To First Token
Throughput	Tokens generated per second
Total Time	Full response completion time
Prefill	Prompt processing before generation
Streaming	Incremental output delivery
Jitter	Small timing variation between outputs
RAG	Retrieval-Augmented Generation

Streaming Inference – Realtime Scenario



Scenario	Streaming Unit	Example
Chat Assistant	Token	ChatGPT
Meeting Transcription	Word	Teams captions
Clinical Notes	Structured text	EHR documentation
Code Assistant	Character/Token	Copilot
Financial Q&A	Token + citations	Bloomberg GPT

Side-by-Side Comparison

	Batch	Real-Time	Streaming
Latency	High (minutes-hours)	Low (ms-s)	Low time-to-first-token
Throughput	Very high	Medium	Medium
Cost/prediction	Lowest	Higher	Similar to real-time
Connection	None (async)	Short HTTP request	Persistent (SSE/WS)
Output type	Stored results	Full response	Incremental chunks
User-facing?	Rarely	Yes	Yes

How They Combine in Practice

- A real system often uses all three:
- **Batch** pre-computes embeddings or recommendations overnight
- **Real-time** serves a classification decision during a user action
- **Streaming** delivers an LLM-generated response word-by-word in the UI

Technical Requirements

Backend



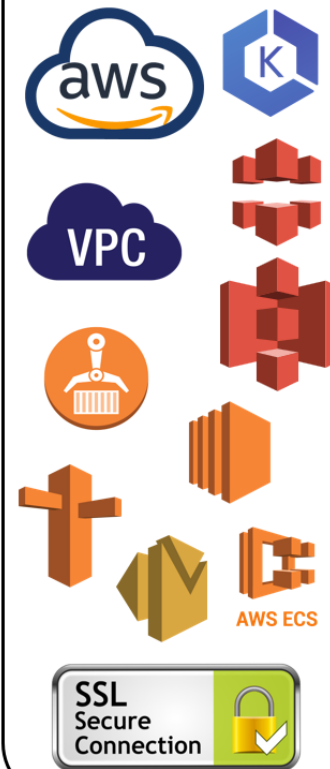
Frontend



DevOps



Cloud



Testing



NFR and SLA



➤ Design documentation	➤ Responsive web design - supported view port 767px (Mobile and Desktop)
➤ Microservice response time < 200 ms-Partially(Jmeter Reports)	➤ Data Metrics and Logging
➤ Secure access over HTTPS/TLS (Encryption over wire)	➤ CI&CD Pipelines to be deployed via Jenkins Docker.
➤ AWS security	➤ Release wise pipeline configuration for hot fixes and feature releases
➤ Application security	➤ Proper Error page should be displayed
➤ Secure data store for PII information (Encryption at rest)	➤ Test Data preparation, management and data cleanup should be there.
➤ Page view times < 2s	➤ Continuous failures shall be communicated at priority via alerts to the support teams to be acted upon.
➤ Latency < 15s	➤ Proper error handling should be there for microservice.
➤ Throughput - 100 req/s	➤ Test Data preparation, management and data cleanup should be there.
➤ Availability - 99.999	➤ Continuous failures shall be communicated at priority via alerts to the support teams to be acted upon.
➤ Quality of video – 480p/720p- Future scope	➤ Proper error handling should be there for microservice.
➤ Concurrent user session/ API – 5000	➤ Email notification to admin in case server is down for particular amount of time
➤ Concurrent user on video – 1000	➤ Accessibility

Microservices Agenda



01 What Are Microservices?

Architecture fundamentals

02 Synchronous Communication

REST & gRPC patterns

03 Asynchronous Communication

Queues, events & brokers

04 Side-by-Side Comparison

Latency, coupling & resilience

05 Hybrid Architectures

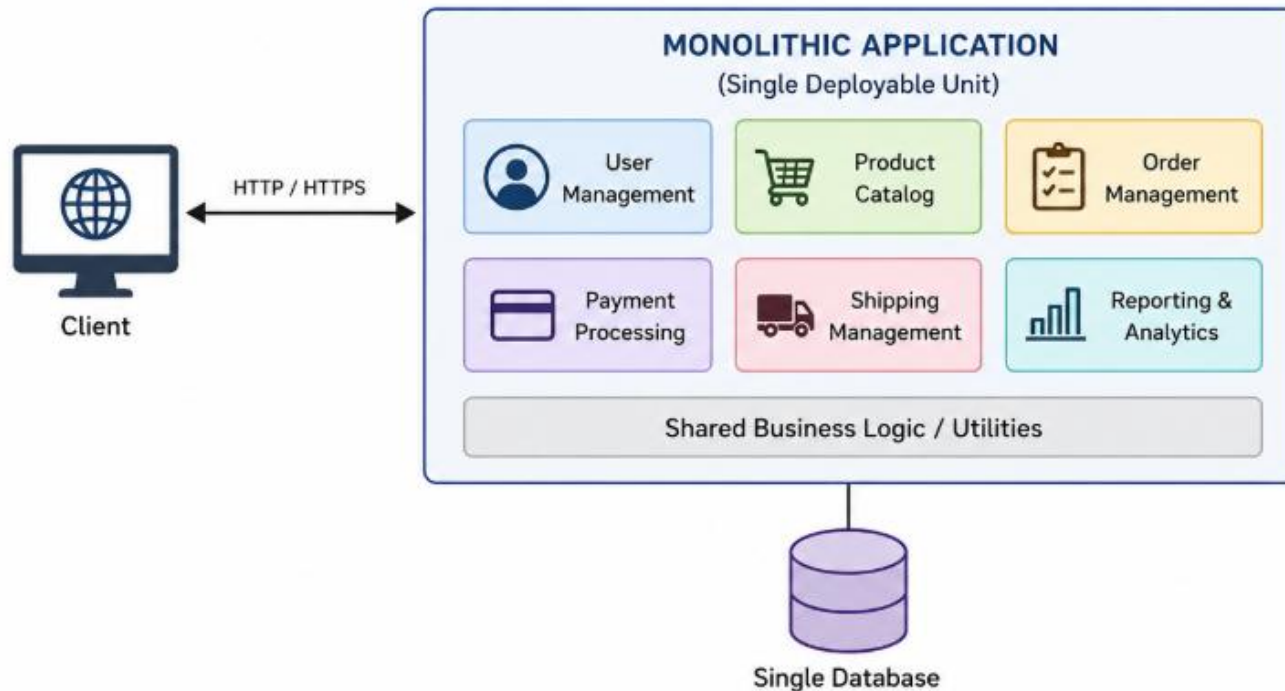
Choosing the right mix

06 Real-World Use Cases

E-commerce, fintech, IoT

Monolithic Architecture

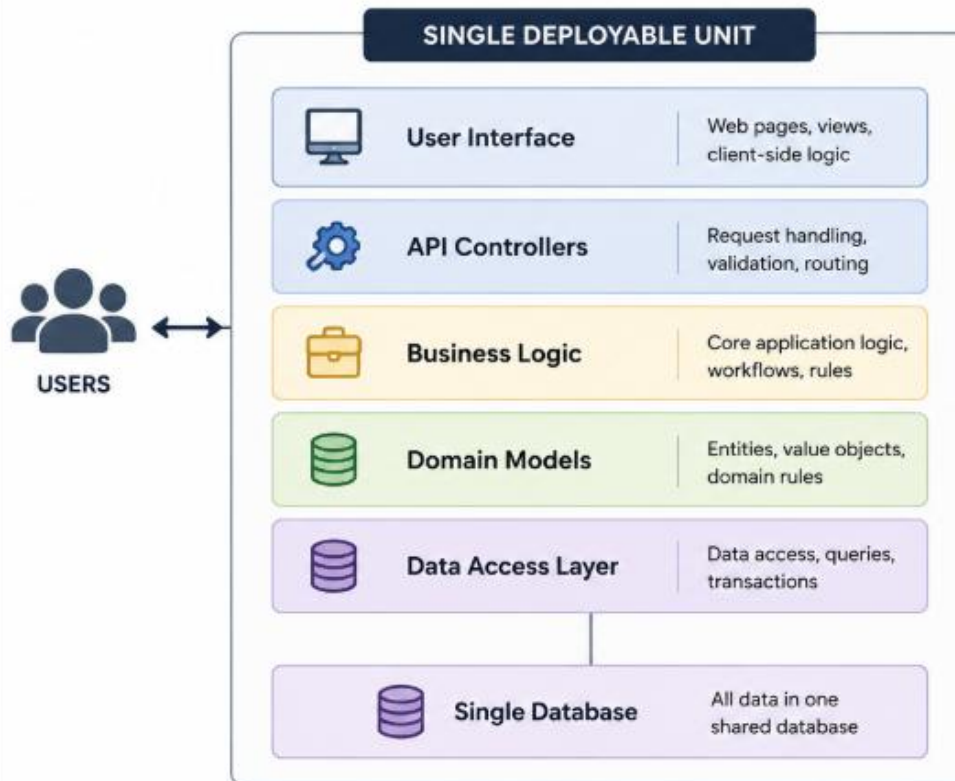
All functionality is built and deployed as a single unit



- All features are part of a single codebase.
- Deployed as a single unit (WAR / JAR / EXE, etc.).
- Modules share the same resources and database.
- Any change requires rebuilding and redeploying the entire application.
- Simpler to develop initially, but becomes complex to scale and maintain as the application grows.

Monolithic Architecture

A single deployable unit where all application components are tightly coupled and run as one process.



BEST SUITED FOR

Small applications, prototypes, or teams in the early stages of development with limited complexity and scale.



ADVANTAGES

- ✓ **Simple to develop initially**
Faster to build a working application in the early stages.
- ✓ **Easy to test end-to-end**
Everything runs as one process, making testing straightforward.
- ✓ **Simple deployment (one unit)**
Deploy the whole application as a single artifact.



DISADVANTAGES

- ⚠ **Scales as one — no granularity**
You must scale the entire application even if only one part needs more capacity.
- ⚠ **One language / tech stack**
The entire application must use the same technology.
- ✗ **Grows into a 'Big Ball of Mud'**
As features grow, the codebase becomes hard to understand and maintain.
- ✗ **Long build & deploy cycles**
Small changes require rebuilding and deploying the whole application.
- ✗ **One bug can crash everything**
A failure in one module can bring down the entire system.

Challenges in Monolithic Architecture

Challenges of Monolithic Applications



Inflexible

Tight coupling makes it hard to adapt to changes.



Unreliable

A small issue can impact the entire application.



Unscalable

Scaling the whole application is costly and inefficient.



Blocks Continuous Development

Teams depend on each other and release cycles slow down.



Slow Development

Large codebase and complex logic slow down new feature delivery.

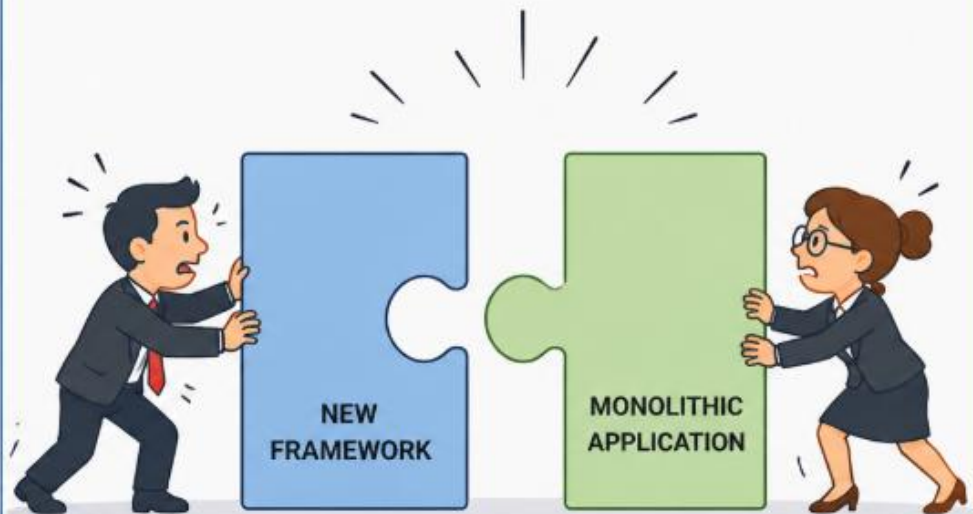


Large & Complex Applications

Harder to understand, test, maintain and onboard new developers.

WHY TEAMS MOVE BEYOND MONOLITHS

As applications grow, the monolith becomes harder to evolve.



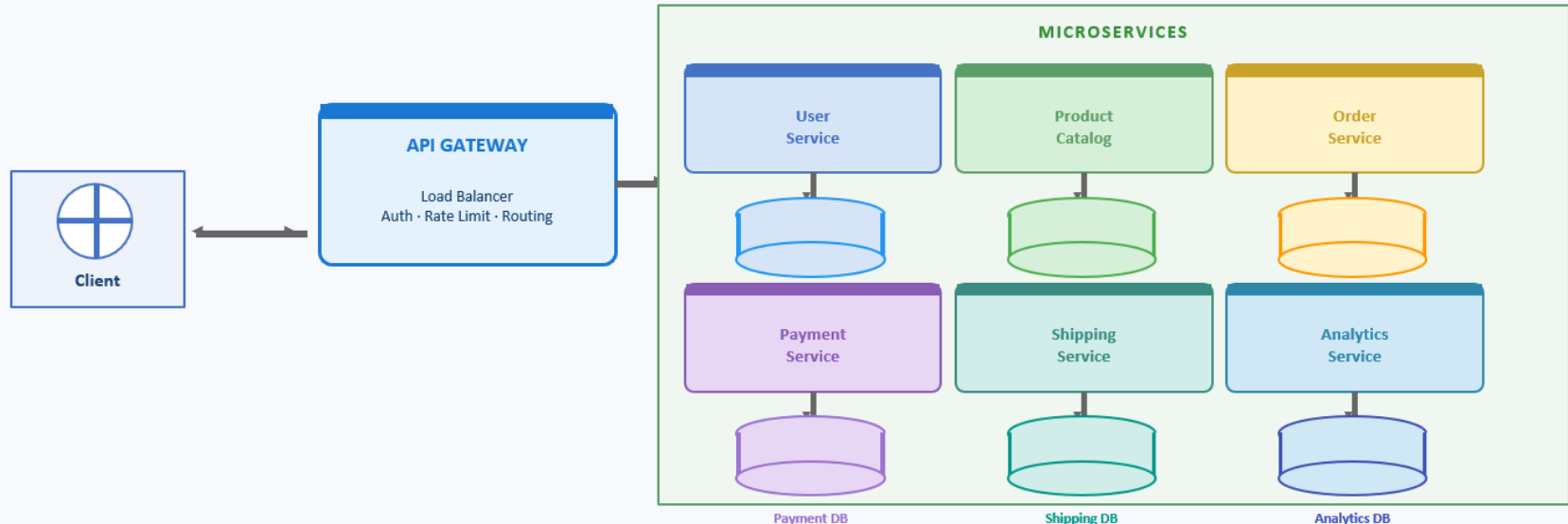
The Result:

It becomes really expensive and time-consuming.

More dependencies. More complexity. More cost.

Microservice Architecture

Each service is independently deployable and owns its own data



Message Broker / Event Bus (Kafka · RabbitMQ · SQS) — Services communicate asynchronously

- Each service is independently developed, deployed, and scaled.
- Services own their own database — no shared data store.
- Inter-service communication via REST / gRPC (sync) or message queues (async).
- Failure in one service does not cascade to others — fault isolation.
- Higher operational complexity; requires robust CI/CD, observability, and service discovery.

Microservice Architecture

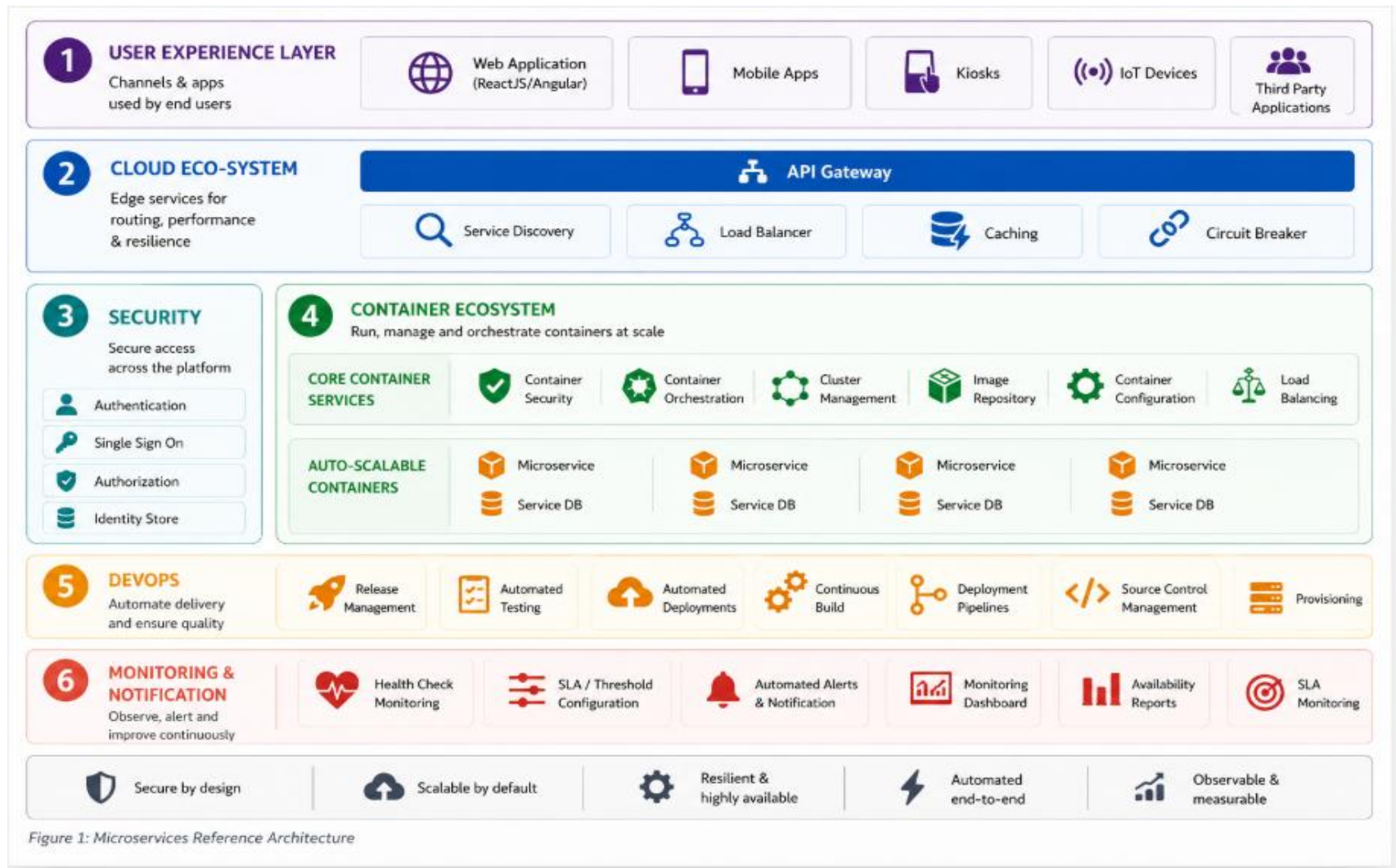
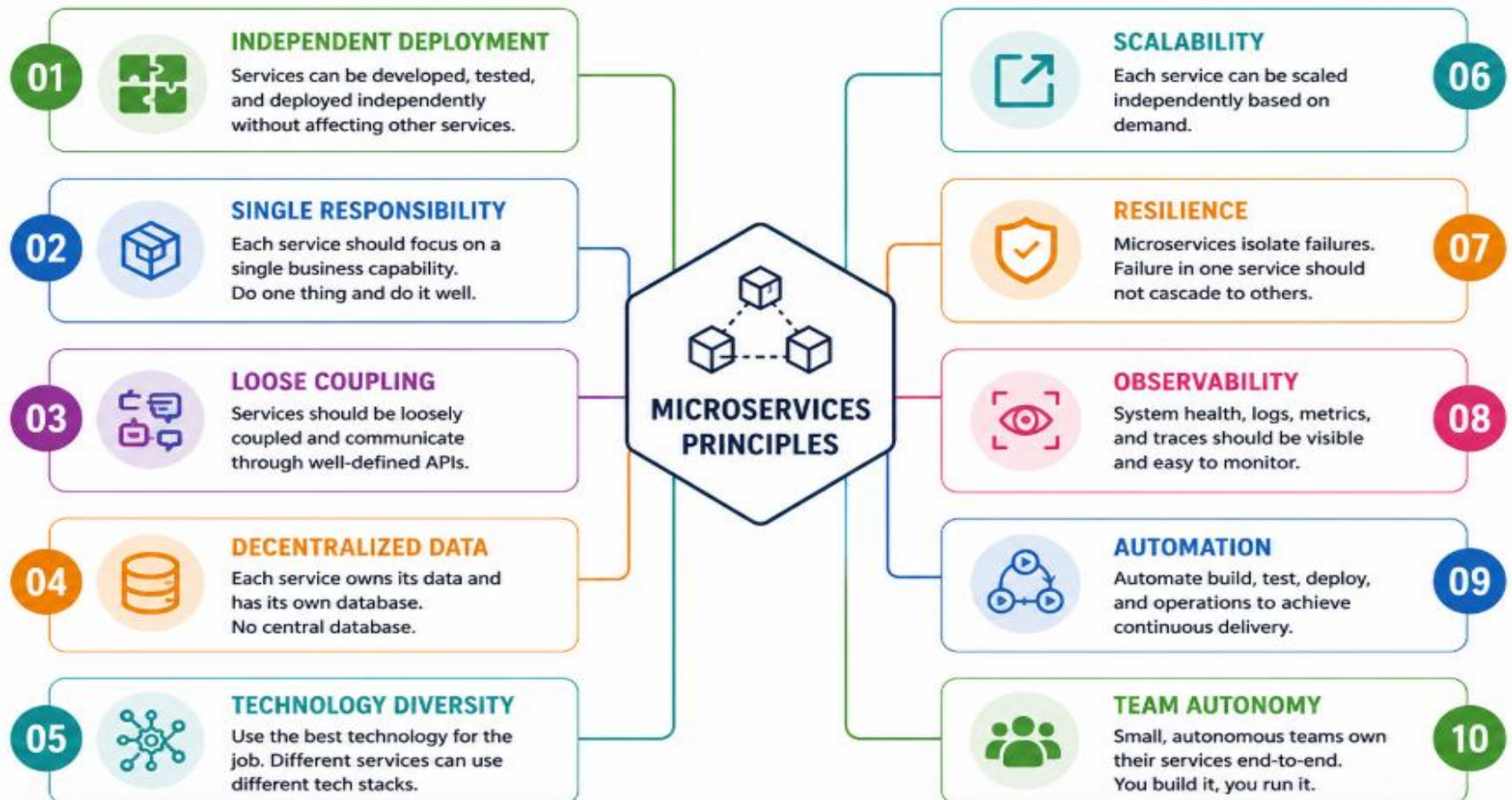


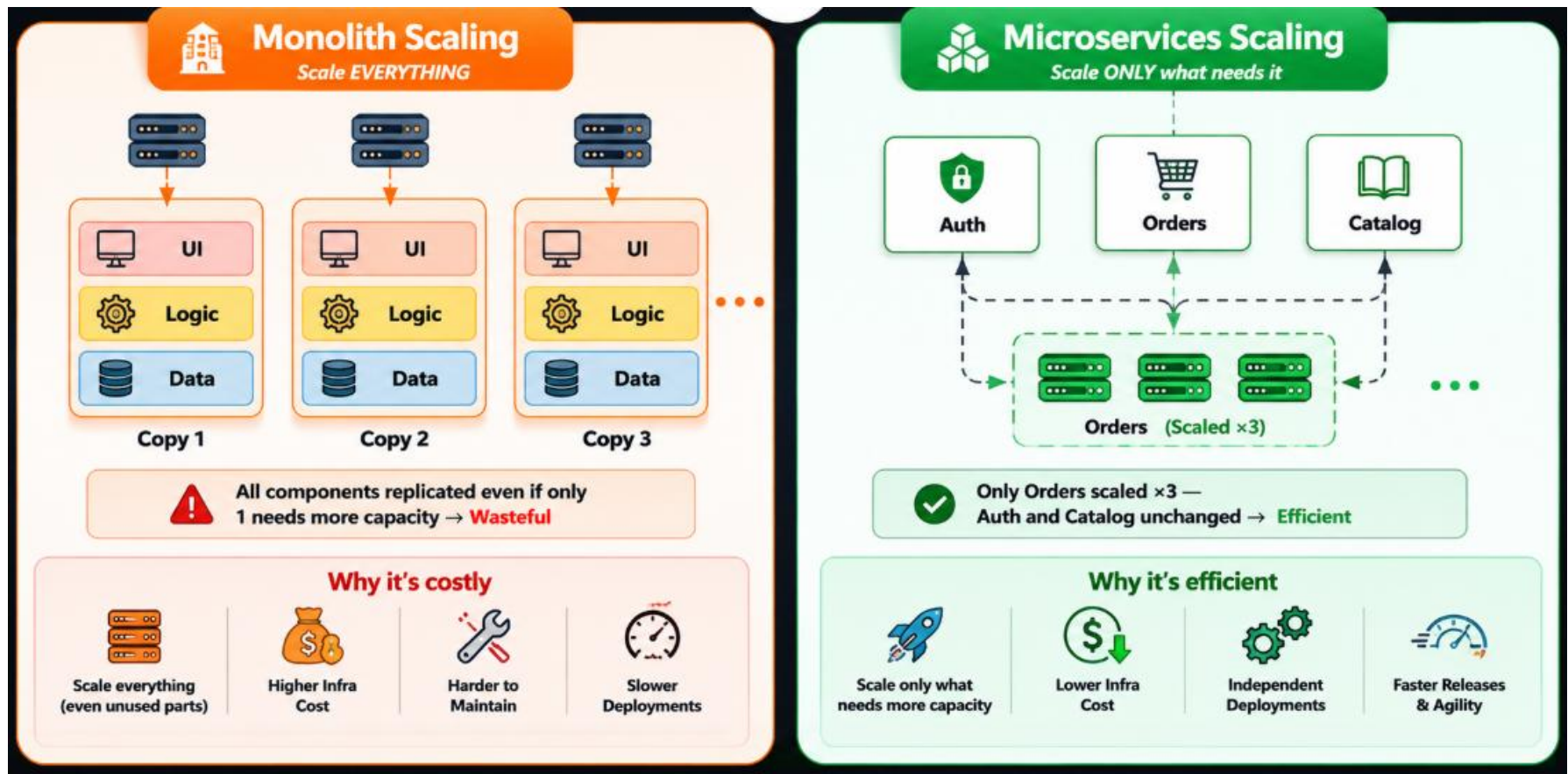
Figure 1: Microservices Reference Architecture

Principles of Micro service

MICROSERVICES ARCHITECTURE – PRINCIPLES



Horizontal Scaling



Head to Head Comparison

DIMENSION	 MONOLITHIC	 MICROSERVICES
 Codebase	 One repo, one artifact	 Many repos / mono-repo, many artifacts
 Deployment	 Deploy everything at once	 Deploy each service independently
 Scaling	 Scale the whole app	 Scale individual services
 Data	 Shared central database	 Database-per-service
 Team Model	 Functional teams (FE/BE/DB)	 Cross-functional product teams
 Resilience	 Single point of failure	 Failure isolated to one service
 Complexity	 Low initially, grows fast	 Higher ops complexity from day one
 Best Fit	 Small teams, early-stage product	 Large orgs, complex domains

Team Structure and Deployment



Conway's Law

"Organizations design systems that mirror their communication structure."



Monolith → Single large team with heavy coordination



Microservices → Small, autonomous product squads ("2 pizza rule")

– Jeff Bezos, Amazon



Deployment Pipeline



Code Change

Any change rebuilds ALL

Only that service rebuilt



Test Suite

Full regression suite

Service-scoped tests



Deploy

Full app downtime risk

Zero-downtime rolling



Typical Migration Path: Monolith → Microservices

1



Identify Bounded Contexts

Identify business domains and boundaries.

2



Strangler Fig Pattern

Carve out features and route traffic to new services.

3



Extract Services One-by-One

Extract and migrate services iteratively with tests.

4



API Gateway + Event Bus

Introduce an API gateway and messaging for communication.

5



Decommission Monolith

Reduce monolith surface area and retire it.

TWELVE-FACTOR APPS AND MICROSERVICES



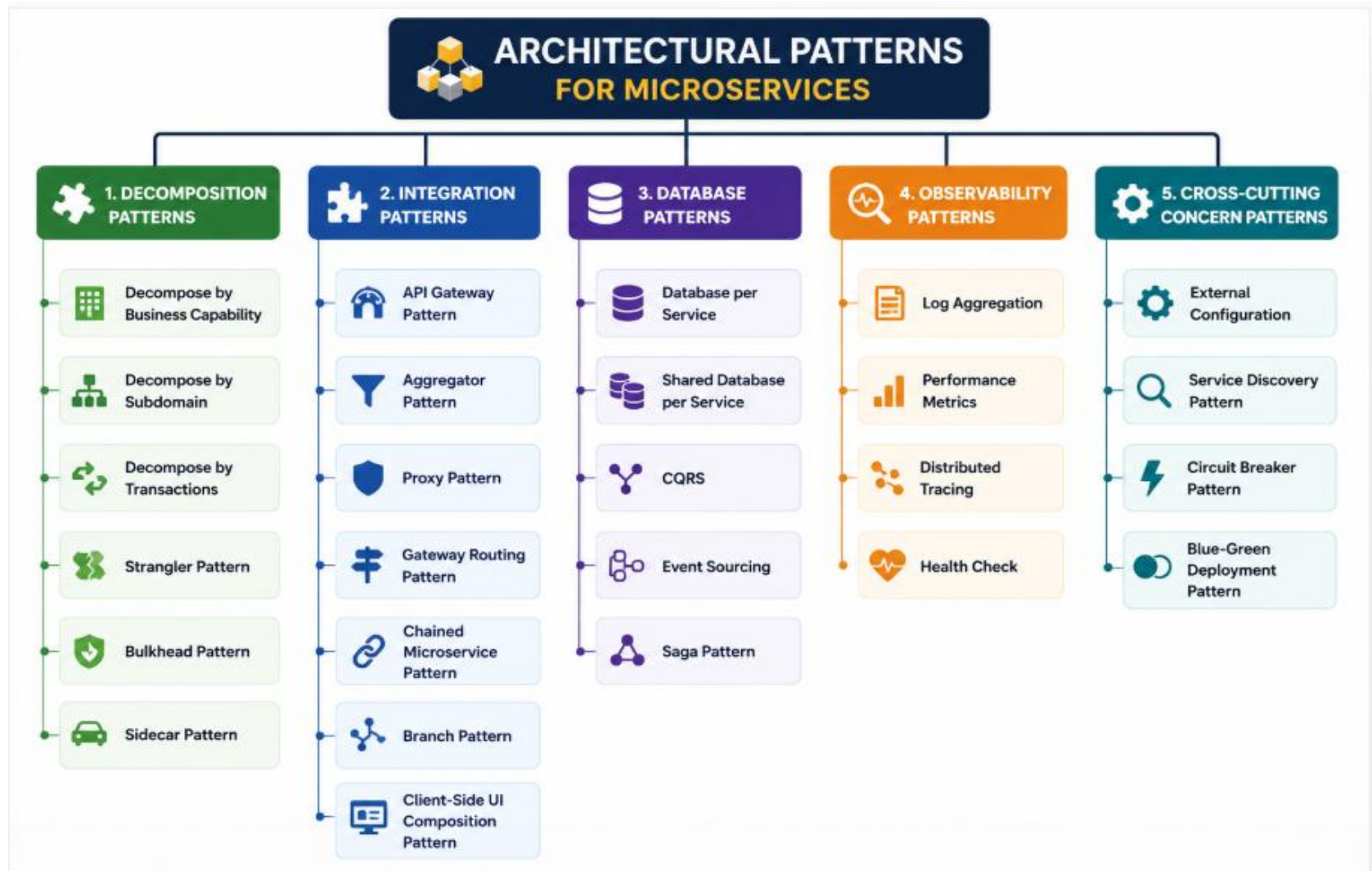
TABLE 1 SOLUTION COMPONENTS FOR 12 FACTOR APPS

FACTORS	BRIEF DETAILS (Ref: https://www.12factor.net/)	MICROSERVICE ECOSYSTEM COMPONENT
 Codebase	One codebase tracked in revision control, many deploys	Source control systems such as gitlab or bitbucket can be leveraged for this. Source control systems provide in-built support for code revisions and version controls. Deployment can be done through build pipeline and CI/CD pipeline.
 Dependencies	Explicitly declare and isolate dependencies	Build and packaging libraries such as Maven, Gradle and npm allow us to declare the dependent libraries along with the library version.
 Config	Store config in the environment	Environment specific configurations (such as URLs, connection strings etc.) can be injected to the configuration files (such as application.properties in Spring application) in the build pipeline.
 Backing services	Treat backing services as attached resources	Backing services such as storage, database or an external service should be accessible and managed through configuration files to ensure portability.
 Build, release, run	Strictly separate build and run stages	Source code branches and automated CI/CD pipelines can be leveraged to manage environment specific releases.
 Processes	Execute the app as one or more stateless processes	Stateless is the core tenets of microservices. Implementing token-based security helps us implement stateless authentication and authorization.

TABLE 1 SOLUTION COMPONENTS FOR 12 FACTOR APPS

FACTORS	BRIEF DETAILS (Ref: https://www.12factor.net/)	MICROSERVICE ECOSYSTEM COMPONENT
 Port binding	Export services via port binding	The services can be made visible through exposed ports. Container infrastructure provides configuration files (such as service.yaml in Docker) to bind the ports for services.
 Concurrency	Scale out via the process model	By leveraging independent deployment feature of microservices, we can individually scale the most needed microservice by using on-demand scaling feature of containers.
 Disposability	Maximize robustness with fast startup and graceful shutdown	Individual containers/pods can be started quickly. Container orchestrator can handle container shutdown gracefully.
 Dev/prod parity	Keep development, staging, and production as similar as possible	We can achieve parity in environment dependencies, server dependencies, configurations through a container model.
 Logs	Treat logs as event streams	Each microservice can log to standard output, which can be picked up by tools such as Kibana or Splunk to manage and visualize the logs centrally.
 Admin processes	Run admin/management tasks as one-off processes	Container orchestration is managed by tools such as Kubernetes, while log management is carried out by tools such as Kibana or Splunk. Other application-specific administration can be deployed as a separate microservice.

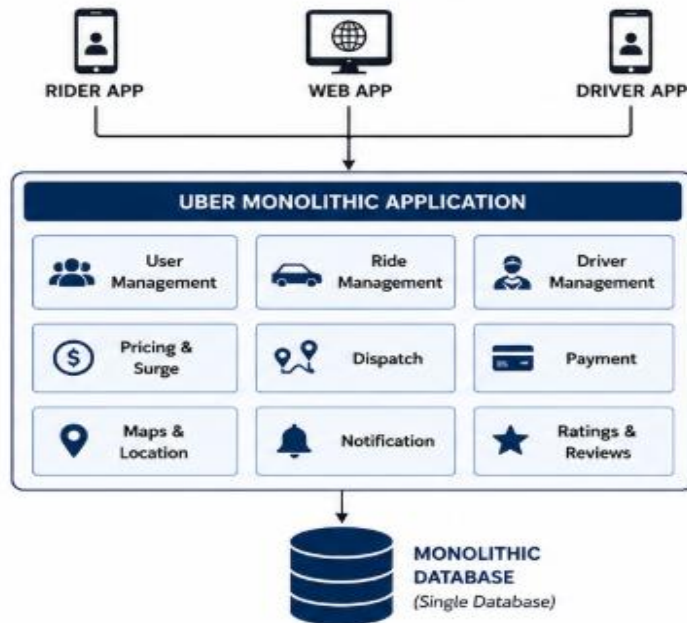
Architectural Patterns For Microservices



Uber Case Study

UBER CASE STUDY – MONOLITHIC ARCHITECTURE

In the early days, Uber used a single monolithic application to handle all functionality.

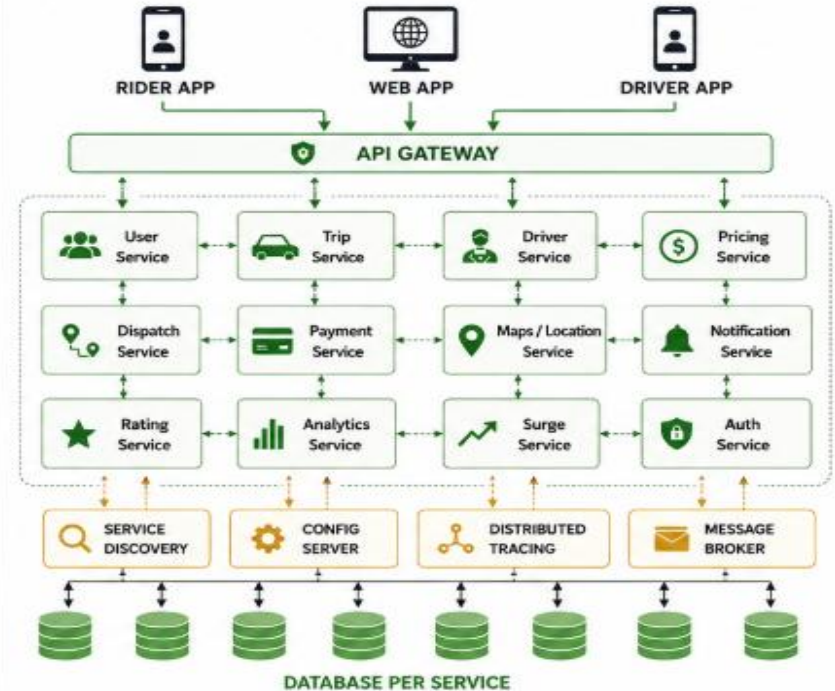


CHALLENGES

- Hard to scale (entire application scales even for small changes)
- Tight coupling of modules
- Longer release cycles
- Harder to maintain with growing team
- Single point of failure
- Technology stack rigidity

UBER CASE STUDY – MICROSERVICES ARCHITECTURE

Uber decomposed the monolith into independent microservices, each responsible for a specific business capability.



BENEFITS

- Independent scaling of services
- Faster deployment and release cycles
- Better fault isolation and reliability
- Technology flexibility
- Team autonomy and productivity
- High availability and resiliency

When to Use Which

 CHOOSE MONOLITH WHEN...	VS	 CHOOSE MICROSERVICES WHEN...
 Early-stage startup / MVP Get to market quickly with minimal architecture overhead.		 Multiple teams working in parallel Teams can own services independently and move faster.
 Small team (< 8 engineers) Easier collaboration and fewer synchronization costs.		 Different scaling needs per domain Scale only the services that need more resources.
 Domain is not yet well understood Requirements are evolving and likely to change.		 High availability requirements Failures in one service won't bring down the entire system.
 Speed to first launch matters most Ship fast and iterate based on real user feedback.		 Need for polyglot tech stacks Use the best technology for each service or business need.
 Simple, low-traffic application Lower complexity and infrastructure costs are enough.		 Complex domain with clear boundaries Well-defined domain boundaries map well to independent services.
 Limited DevOps / infra maturity Less operational overhead and simpler deployment.		 Strong DevOps & CI/CD culture Automated delivery, monitoring, and observability are in place.



Remember

There is no one-size-fits-all. Choose the architecture that fits your current context and can evolve as your needs grow.

Real World Examples

NETFLIX

Monolith (2008) → Microservices (2009+)



Why the change?

Needed to scale streaming globally; DVD-rental monolith couldn't handle 100M+ users.



Key takeaway

Built microservices to achieve massive scale, resilience, and rapid feature delivery.

SHOPIFY

Rails Monolith → Modular Monolith → Selective Microservices



Why the change?

Scaled to \$6B+ GMV still on a refined monolith; only extracted true hot-path services.



Key takeaway

Took an evolutionary approach — extracted only what provided real business impact.

AMAZON

Monolith (2001) → Service-oriented (2002+)



Why the change?

Jeff Bezos mandated all data be exposed only via APIs — birth of SOA → microservices.



Key takeaway

API-first principles enabled independent scaling, innovation, and organizational agility.

BASECAMP / HEY

Rails Monolith → Still a Monolith



Why stay monolithic?

DHH publicly defends the majestic monolith. Small team, full control, fast iteration.



Key takeaway

Monoliths can be the right choice — simplicity and speed over premature complexity.

Key Take Aways

1 Monoliths aren't bad



They're often the right choice for small teams and early products. Start simple.

2 Microservices aren't free



You trade application complexity for operational complexity. Plan your infra.

3 Understand your domain first



Good bounded contexts make microservices natural. Bad ones make them a nightmare.

4 Team size drives architecture



Conway's Law is real. Structure your teams around the system you want to build.

5 The Strangler Fig is your friend



Incrementally extract services from a monolith — never do a big-bang rewrite.

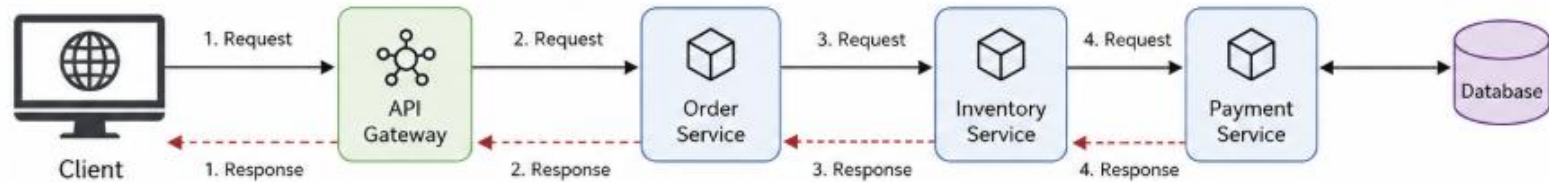
6 Observability is non-negotiable



Logs, metrics, distributed tracing must be in place before you add services.

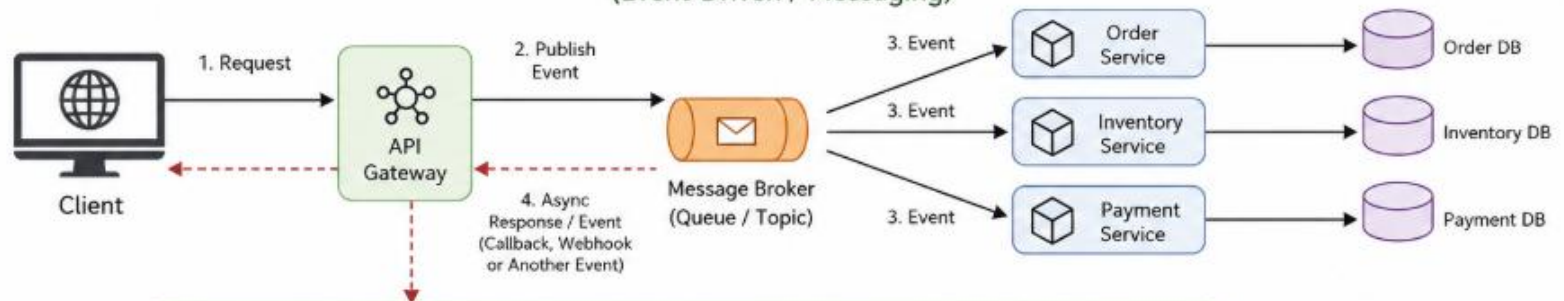
Synchronous & Asynchronous Microservice

SYNCHRONOUS MICROSERVICES ARCHITECTURE (Request/Response)



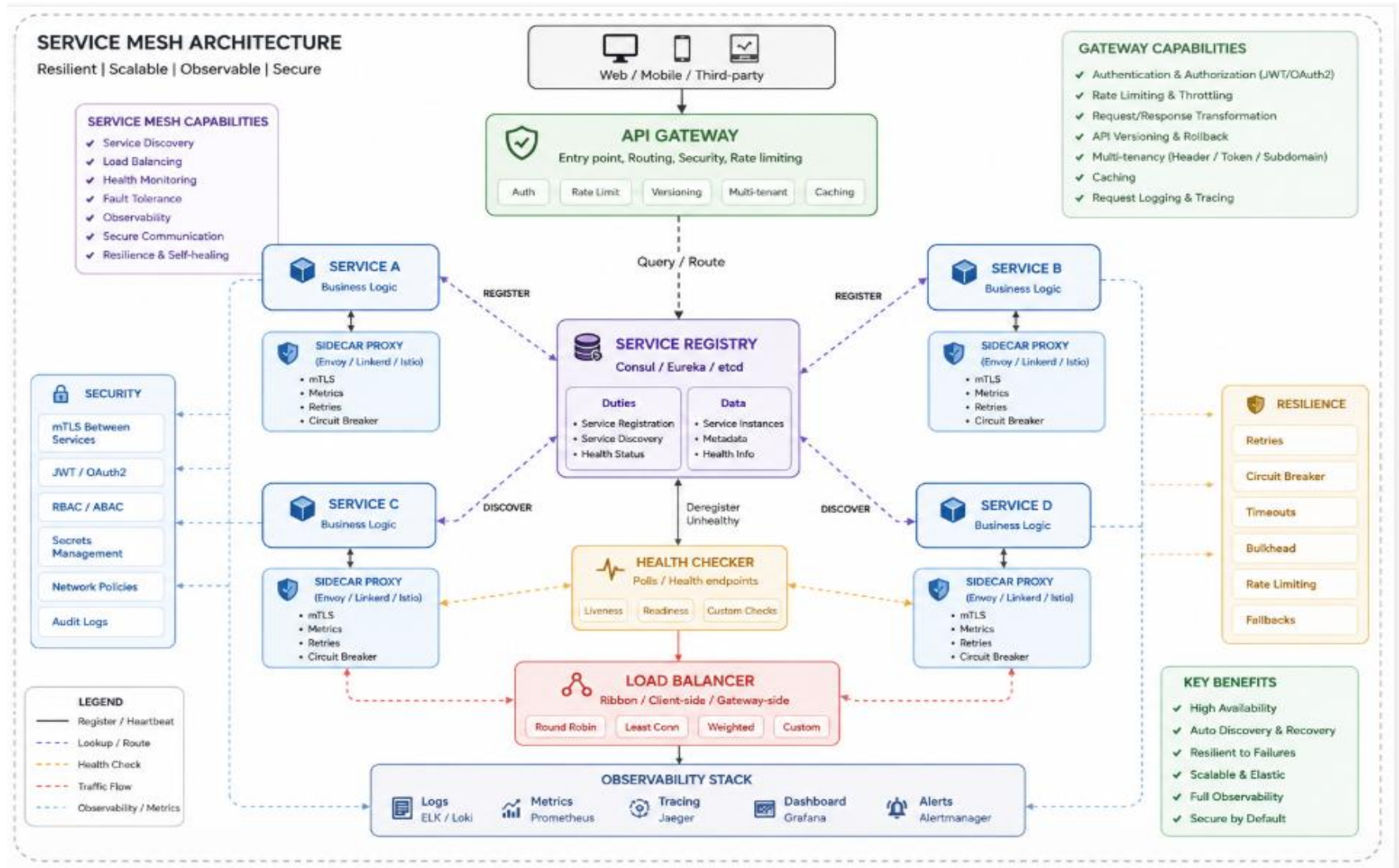
- Each service call waits for a response before proceeding to the next step.
- If one service is slow or unavailable, it can impact the entire flow.
- Easier to implement but has tighter coupling and lower resilience.

ASYNCHRONOUS MICROSERVICES ARCHITECTURE (Event-Driven / Messaging)

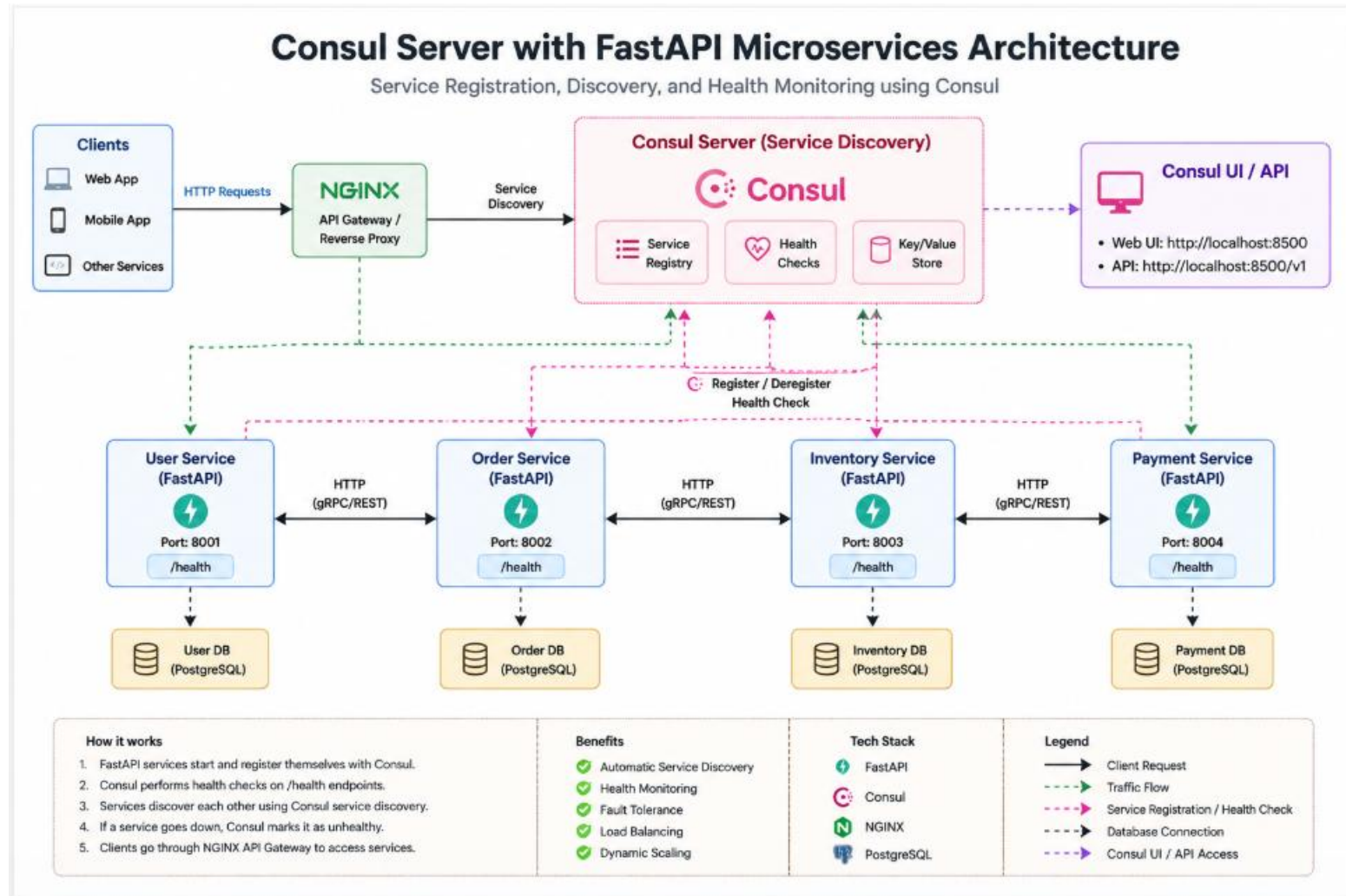


- Services communicate by publishing and consuming events.
- Services are decoupled and can work independently.
- More resilient, scalable, and better for high-throughput systems.

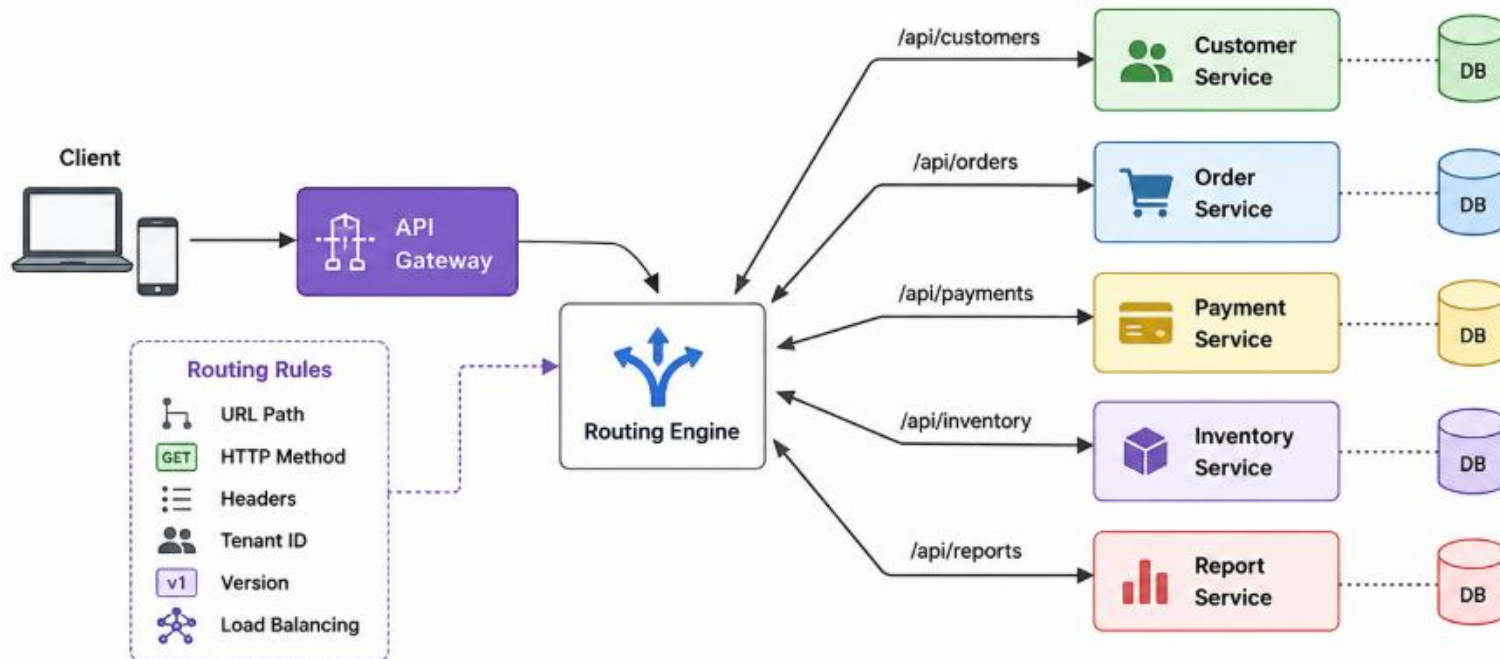
Service Discovery



Consul Server



Routing Microservices



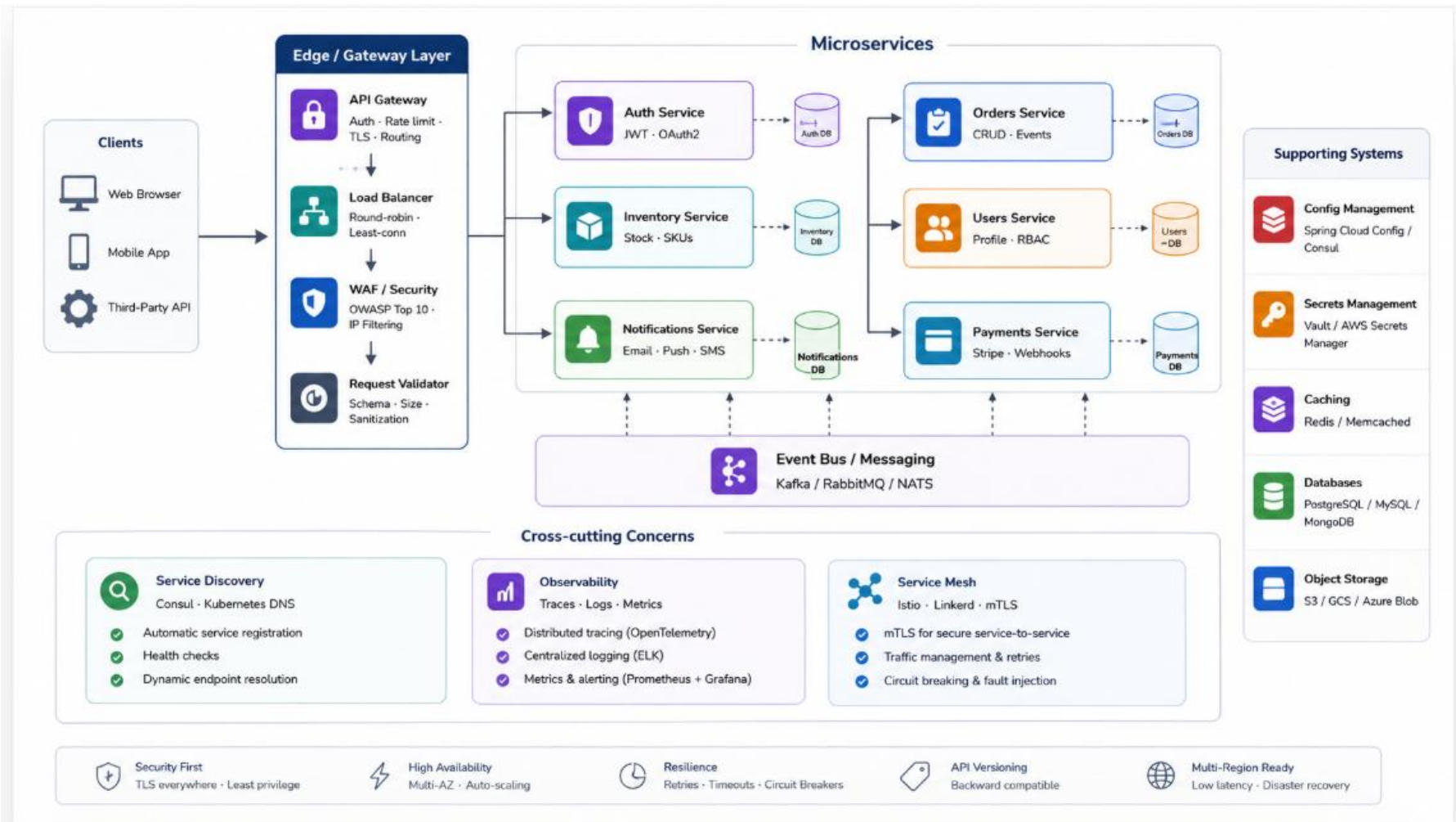
How it works

1. Client sends request to API Gateway
2. Gateway forwards request to Routing Engine
3. Routing Engine matches rules
4. Request is routed to appropriate microservice
5. Microservice processes request and sends response back

Example Requests and Routing

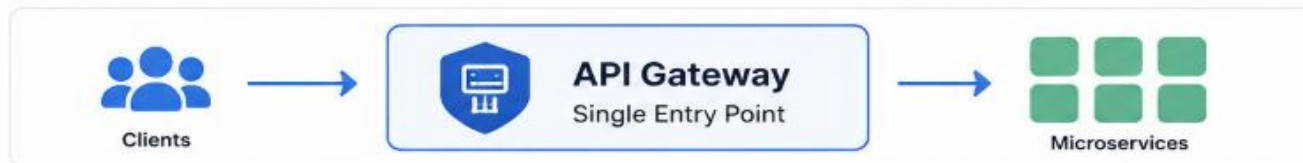
Request	Method	Route To
/api/customers	GET	Customer Service
/api/orders	POST	Order Service
/api/payments	POST	Payment Service
/api/inventory	GET	Inventory Service
/api/reports	GET	Report Service

Routing Microservices



API Gateway

The **single entry point** for all client traffic



Authentication

Validate JWT tokens, API keys, and OAuth2 flows before any request reaches a service.

- ✓ JWT / OAuth2 / API Key
- ✓ Token validation
- ✓ Unauthorized access blocked



Rate Limiting

Throttle requests per client, IP, or route to protect services from overload.

- ✓ Per IP / Per Client / Per Route
- ✓ Burst & sustained limits
- ✓ Protects from abuse & spikes



TLS Termination

Handles HTTPS at the edge — internal traffic can use plain HTTP or mTLS.

- ✓ HTTPS offload at gateway
- ✓ mTLS support
- ✓ Stronger security & performance



Routing Rules

Path based, header based, and weighted routing to direct traffic to the right service.

- ✓ Path / Header / Query / Method
- ✓ Weighted / Canary / Blue-Green
- ✓ Version & region based routing



Response Caching

Caches common responses at the edge to reduce load and latency.

- ✓ TTL based caching
- ✓ Cache invalidation
- ✓ Lower latency & cost



Protocol Translation

Converts REST ↔ gRPC or handles WebSocket upgrades transparently.

- ✓ REST to gRPC / gRPC to REST
- ✓ WebSocket support
- ✓ Backward compatibility



Secure

Centralized security enforcement



Performant

Better latency, caching and compression



Scalable

Handle more traffic with control



Observable

Centralized logs, metrics and tracing



Developer Friendly

Consistent APIs and policies

Load Balancing Strategies

Distribute incoming traffic across multiple service instances to improve performance, scalability, and reliability.

☆ Default



Round Robin

Requests cycle evenly across all instances in sequence.



Client → Instance 1 → Instance 2 → Instance 3

✓ Pros

- ✓ Simple & easy to implement
- ✓ No state or data required
- ✓ Works well for identical instances

✗ Cons

- ✗ Ignores actual server load
- ✗ Can send traffic to busy or unhealthy instances
- ✗ Poor for long-lived requests

 Best for

Small scale applications with similar instances and short-lived requests.

👍 Recommended



Least Connections

Routes traffic to the instance with the fewest active connections.



Client → Instance 1 (12) → Instance 2 (5) → Instance 3 (8)

✓ Pros

- ✓ Handles uneven workloads better
- ✓ Better performance for long requests
- ✓ Adaptive to traffic patterns

✗ Cons

- ✗ Requires connection tracking
- ✗ Slightly more complex
- ✗ More memory and CPU overhead

 Best for

Applications with varying request durations or uneven traffic distribution.

⚠ Use with care



Sticky Sessions

Ensures the same client is always routed to the same instance.



Client → Instance 1 → Instance 2 → Instance 3

✓ Pros

- ✓ Required for stateful applications
- ✓ Maintains session affinity
- ✓ Useful for WebSocket connections

✗ Cons

- ✗ Breaks horizontal scaling
- ✗ Uneven load distribution
- ✗ Session loss if instance fails

 Best for

Stateful applications, user sessions, and WebSocket based systems.



Key Takeaway

Choose the right load balancing strategy based on your application type, traffic pattern, and instance behavior.



Improves Availability



Enhances Scalability



Optimizes Performance



Reduces Downtime

The **three pillars** for understanding distributed systems



Traces

Request Flow

Follow a single request as it travels across all services. Essential for debugging latency and identifying failure points in complex call chains.

What it helps you do

- ✓ Visualize end-to-end request flow
- ✓ Find performance bottlenecks
- ✓ Identify failing services and dependencies
- ✓ Understand latency at each hop

Popular Tools



Jaeger



Zipkin



Tempo



Logs

Events

Structured log events from every service. Include a correlation ID matching the trace ID so logs and traces link together automatically.

What it helps you do

- ✓ Understand what happened
- ✓ Debug errors and exceptions
- ✓ Audit and compliance
- ✓ Search and analyze events

Popular Tools



Loki



Elasticsearch



CloudWatch



Metrics

Aggregated Data

Numeric time-series data: request rate, error rate, latency percentiles (p50/p95/p99), and saturation. These feed your dashboards and alerts.

What it helps you do

- ✓ Monitor system health
- ✓ Track SLIs / SLAs
- ✓ Detect anomalies early
- ✓ Capacity planning and alerting

Popular Tools



Prometheus



Grafana



Datadog

How They Work Together



1. Request Received
A user makes a request to your system.



2. Trace Created
A trace ID is created and propagated across services.



3. Logs Generated
Each service writes logs with the same trace ID.



4. Metrics Collected
Services emit metrics continuously.



5. Correlate & Analyze
Use trace ID to connect traces, logs, and metrics for full visibility.



Together, traces, logs, and metrics give you end-to-end visibility, faster troubleshooting, and better system reliability.

Key Takeaways

Build systems that scale, stay reliable, and are easy to operate.



01

Gateway first

Centralize authentication, rate limiting, and TLS at the API gateway before traffic touches any service.



02

Decouple with LB

Use least-connections for mixed workloads; reserve sticky sessions only for unavoidably stateful services.



03

Mesh at scale

Introduce a service mesh when manual mTLS, retry logic, and circuit breakers become a cross-team burden.



04

Instrument everything

Emit traces with correlation IDs, structured logs, and RED metrics (Rate, Errors, Duration) from day one.



Secure

Protect traffic and data



Scalable

Handle more traffic with confidence



Resilient

Detect issues early and recover fast



Observable

See what's happening, everywhere

Multi-Tenant Serving

Isolation models, resource sharing & tenant lifecycle management



Isolation Models



Core Capabilities



Tenant Lifecycle



Isolation Models



SILO

Dedicated everything

Each tenant gets their own stack: compute, database, networking. **Maximum isolation** but highest cost and operational overhead.

Architecture



Pros

- ✓ Strongest data isolation
- ✓ Independent scaling
- ✓ Full customization
- ✓ Simpler compliance

Cons

- ✗ Highest infrastructure cost
- ✗ Lower resource utilization
- ✗ Harder to operate at scale
- ✗ Longer provisioning time

★ Best for

Highly regulated workloads, large enterprises, specific compliance needs.

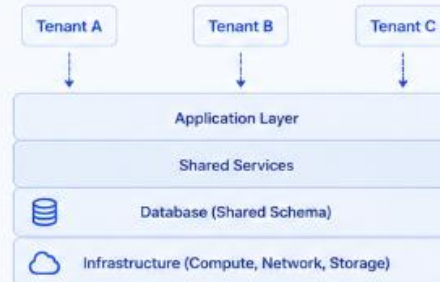


POOL

Fully shared stack

All tenants share the same infrastructure. **Lowest cost**, but requires careful partitioning to prevent data leakage and noisy neighbour issues.

Architecture



Pros

- ✓ Lowest cost
- ✓ High resource utilization
- ✓ Easiest to operate
- ✓ Fast provisioning

Cons

- ✗ Data isolation risk
- ✗ Noisy neighbour issues
- ✗ Complex quotas & limits
- ✗ Limited customization

★ Best for

Small to medium SaaS, startups, cost-sensitive workloads with similar usage patterns.



BRIDGE

Tiered hybrid

Free/standard tenants share pooled resources; premium/enterprise tenants get dedicated infrastructure. **Best of both worlds.**

Architecture



Pros

- ✓ Cost-efficient tiers
- ✓ Upgrade path for tenants
- ✓ SLA flexibility
- ✓ Balanced isolation & scale

Cons

- ✗ Two models to operate
- ✗ Routing complexity
- ✗ Higher architectural complexity
- ✗ Migration planning required

★ Best for

Growing SaaS platforms, different tenant tiers, customers with varying requirements.

Tenant Routing



How the system identifies and routes each incoming request to the right tenant context



Resolution Strategies (evaluated in order)

1 Subdomain
Identify tenant from the request subdomain.

Example
acme.app.io → acme-001

2 URL Path
Extract tenant from a specific path segment.

Example
/tenant/acme/... → acme-001

3 Custom Domain
Map the request host to a tenant.

Example
app.acme.com → acme-001

4 JWT Claim
Extract tenant identity from JWT claims.

Example
tid: acme-001 → acme-001

Resolved Tenant Context		
Attached to the request and used across all services and policies		
tenant_id	acme-001	
tier	enterprise	
region	us-east-1	
db_schema	acme	
feature_flags	{ ... }	
rate_limit	10 000 rpm	
plan_expires_at	2026-12-31T23:59:59Z	
policies	[authz, quota, data-isolation]	

Why it matters

Correct isolation
Requests are routed to the right tenant context.

Policy enforcement
Apply tenant-specific authZ, quotas and limits.

Operational efficiency
Faster lookups, better caching, lower overhead.

Observability
End-to-end visibility by tenant.

Resource Isolation

Isolation options across compute, data, and network to meet different security, compliance, and cost requirements



Compute

Isolate CPU & memory to protect workloads and ensure fairness.



Namespace / cgroup

Process isolation using Linux namespaces; CPU & memory limits enforced via cgroups or container resource specs.

Pros

- ✓ Lightweight & flexible
- ✓ High density
- ✓ Fast provisioning

Cons

- ✗ No strong runtime isolation
- ✗ Kernel-level risk
- ✗ Requires capacity governance



Pod / task quotas

Kubernetes ResourceQuota or ECS task limits per tenant namespace to prevent runaway consumption.

Pros

- ✓ Enforced by platform
- ✓ Granular controls
- ✓ Prevents noisy neighbours

Cons

- ✗ Not hard isolation
- ✗ Quota misconfiguration risk
- ✗ Limited by cluster capacity



Data

Isolate data at the right level for security, simplicity, and cost.



Schema-per-tenant (Pool)

Single DB cluster; each tenant owns a schema. Simple to operate but needs row-level guard rails.

Pros

- ✓ Lower cost
- ✓ Easy backups & maintenance
- ✓ Good resource utilization

Cons

- ✗ Weaker isolation
- ✗ Row-level security required
- ✗ Noisy neighbour risk



DB-per-tenant (Silo)

Dedicated database instance per tenant. Full isolation with independent scaling and backups.

Pros

- ✓ Strong isolation
- ✓ Independent performance
- ✓ Simpler compliance

Cons

- ✗ Higher cost
- ✗ More operational overhead
- ✗ Lower resource utilization



Network

Isolate network traffic to enforce boundaries and control access.



VLAN / VPC per tenant

Hard network boundaries—tenant traffic never traverses shared segments. Ideal for high-compliance.

Pros

- ✓ Strong isolation
- ✓ Meets strict compliance
- ✓ Clear blast radius

Cons

- ✗ Higher cost
- ✗ More complex to manage
- ✗ Slower provisioning



Shared VPC + security groups

Single VPC with per-tenant security groups and NACLs. Cost-efficient for standard tiers.

Pros

- ✓ Cost-effective
- ✓ Simpler networking
- ✓ Scales easily

Cons

- ✗ Weaker isolation
- ✗ Policy complexity risk
- ✗ Shared failure domain



Choose the right isolation level based on your tenant tier, compliance needs, and cost targets. You can mix models across tiers.

Machine Learning

- Machine Learning is a branch of AI where computers learn patterns from data instead of being explicitly programmed for every rule.
- **Core idea**
- Instead of writing:
- “If email contains X, mark as spam”
- you train a model on thousands of examples of spam and non-spam emails, so it learns the pattern itself.

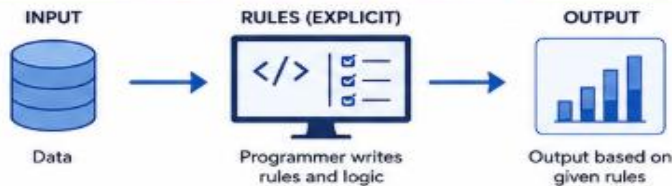
Machine Learning

TRADITIONAL PROGRAMMING vs. MACHINE LEARNING



Machine learning enables systems to learn from data and improve automatically — without being explicitly programmed for every scenario.

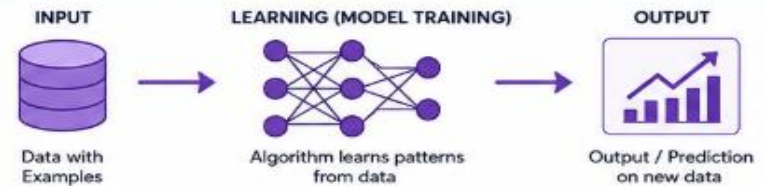
TRADITIONAL PROGRAMMING



How it works

Programmer writes explicit instructions.
System follows rules step-by-step to produce output.

MACHINE LEARNING



How it works

System learns patterns from data.
It generalizes and makes predictions on unseen data.

ASPECT	TRADITIONAL PROGRAMMING	MACHINE LEARNING	KEY DIFFERENCE
 Approach	Rules + Data → Output	Data + Output → Rules (Model)	Who creates the rules
 Process	Programmer writes rules manually	Model learns rules automatically	Manual vs. automatic rule creation
 Key Characteristic	Deterministic, explicit logic	Learns patterns from data	Fixed logic vs. learned patterns
 Flexibility	Less flexible to changes	Adapts to new data and patterns	Rigid vs. adaptive
 When to Prefer	Known, stable rules	Dynamic, complex domains	Simplicity vs. complexity
 Examples	Payroll calculation, Invoice generation, Tax computation	Image recognition, Fraud detection, Recommendation systems	Rule-based tasks vs. data-driven tasks

WHEN TO PREFER



CHOOSE TRADITIONAL PROGRAMMING WHEN:

- ✓ Rules are clear, fixed, and well-defined
- ✓ Problem is simple and stable
- ✓ High accuracy and consistency are critical
- ✓ Less data is available



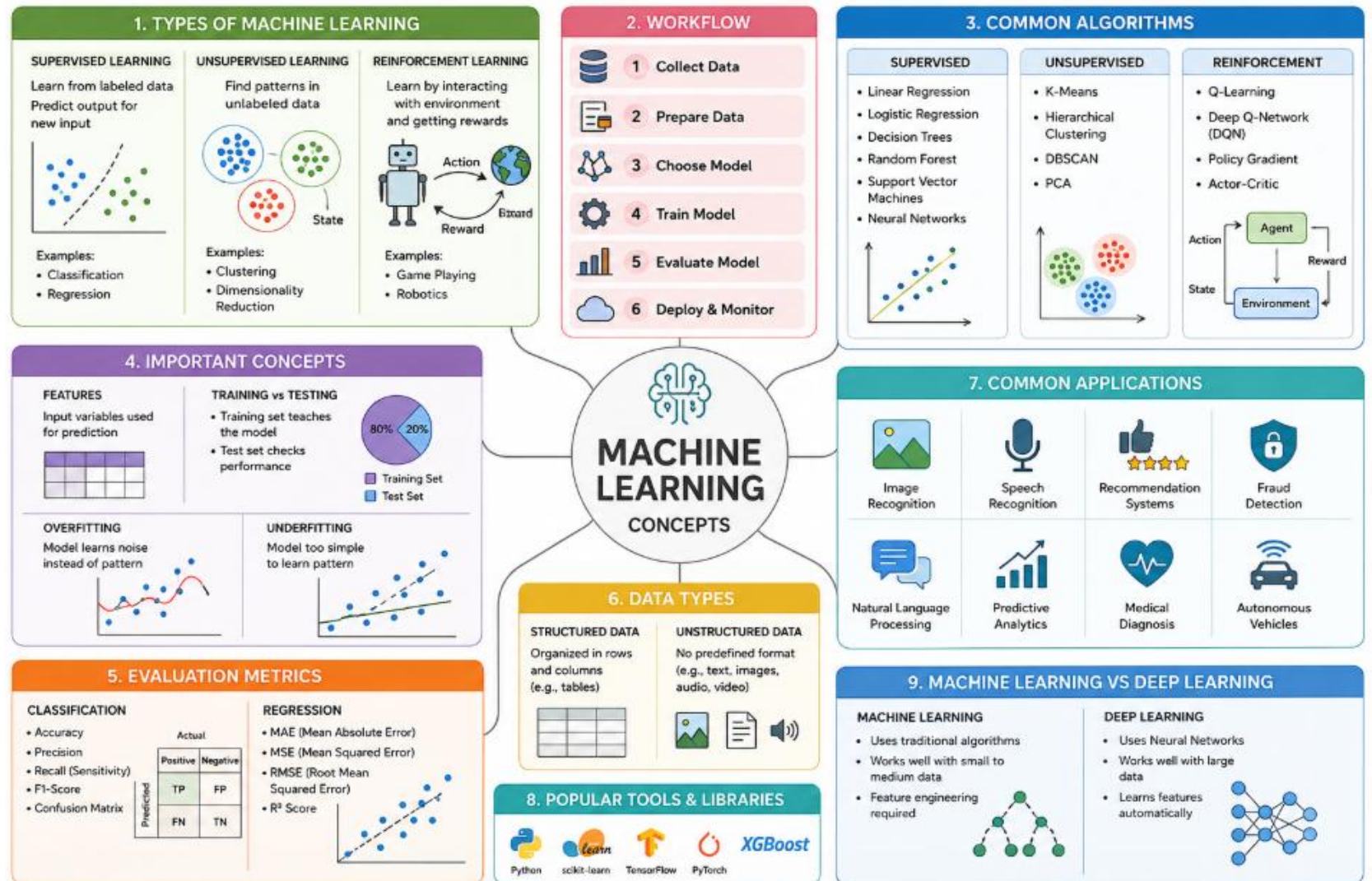
CHOOSE MACHINE LEARNING WHEN:

- ✓ Rules are unknown or change frequently
- ✓ Problem is complex and data is abundant
- ✓ Need system that improves over time
- ✓ Looking for predictions or insights



In many real-world systems, Traditional Programming and Machine Learning work together to build intelligent and reliable solutions.

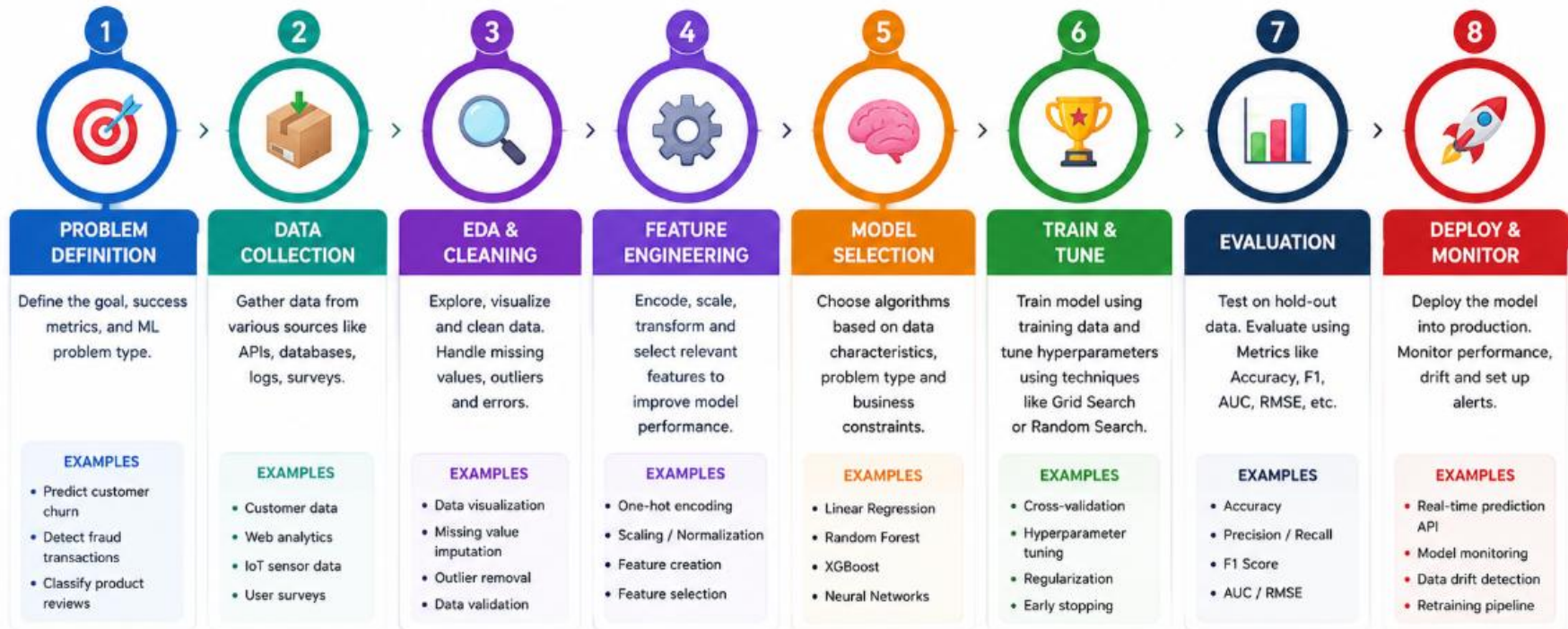
Machine Learning



Machine Learning Life Cycle



A structured approach to build, evaluate and deploy ML models effectively.



BEST PRACTICES



Start with a clear problem and success metrics.



High quality data leads to better models.



Iterate and experiment continuously.



Monitor in production and retrain when needed.

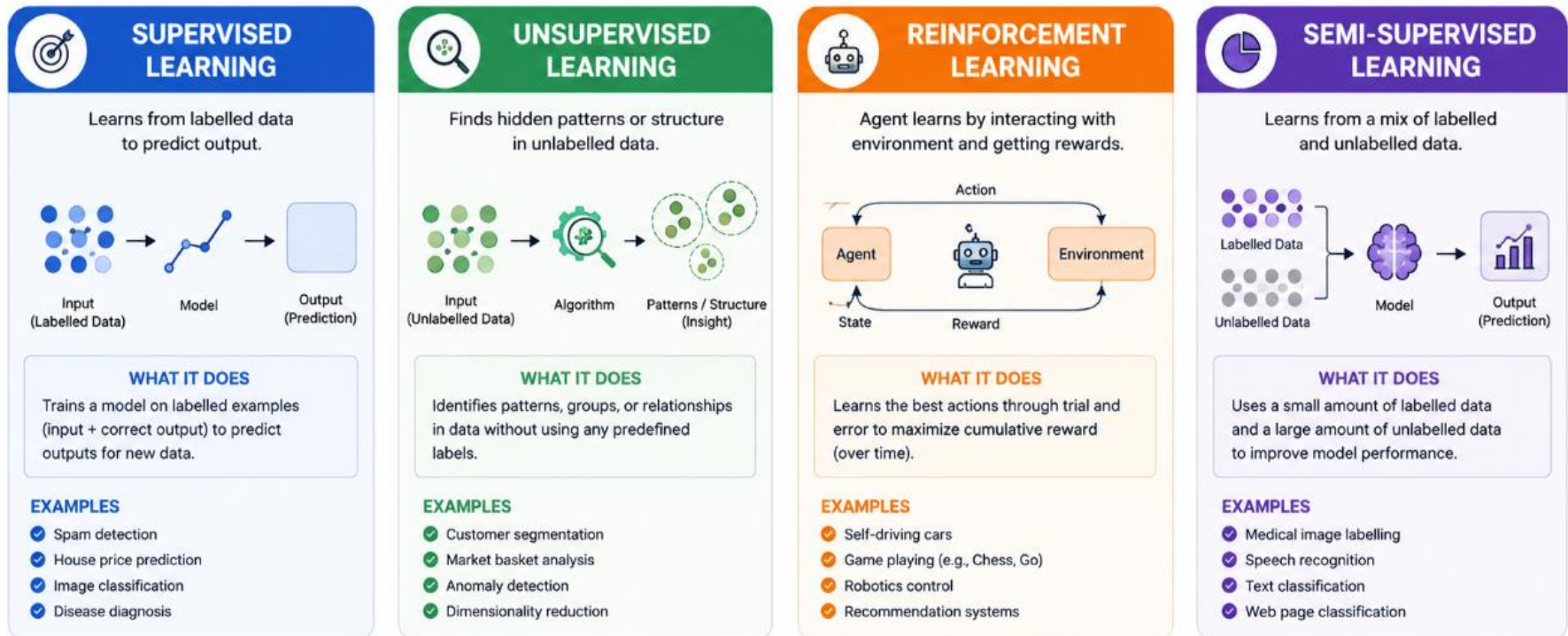


Collaboration between data scientists and business teams is key.

Machine Learning Types



Machine Learning algorithms learn from data to make predictions or decisions.



★ **KEY TAKEAWAY:** Choose the right type of machine learning based on the type of data you have and the problem you want to solve.

Machine Learning Types



Key concepts every machine learning practitioner should know



1. FEATURE (X)

Input variable used for predictions.
Examples: age, salary, location, transaction amount.



2. LABEL / TARGET (y)

Output we want to predict.
Examples: churn (0/1), house price (\$), disease (Yes/No).



3. TRAINING SET

Data used to teach the model.
Typically ~70–80% of total available data.



4. TEST SET

Held-out data to evaluate final model performance.
Never seen during training.



5. MODEL

The mathematical function learned from data.
 $\hat{y} = f(X; \theta)$ where θ = learned parameters.



6. HYPERPARAMETER

Settings chosen BEFORE training.
Examples: learning rate, K in KNN, max depth in trees.



7. OVERFITTING

Model memorizes training data but fails on new data.
High train accuracy, low test accuracy.



8. UNDERFITTING

Model too simple — fails to capture patterns even in training data. High bias.



9. CROSS-VALIDATION

Split data K times to robustly estimate generalisation.
Prevents data leakage.



10. GRADIENT DESCENT

Optimisation algorithm that updates model parameters in the direction of steepest loss reduction.



TRAINER TIP: Quick quiz after this slide:



What is the difference between a feature and a label?



Feature = Input (predictor)



Label = Output (what we predict)

Bias – Variance Trade off

The balance between model simplicity and flexibility determines how well your model generalizes.

THE FUNDAMENTAL PRINCIPLE

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

↓
Error from overly simple assumptions

↓
Error from sensitivity to small fluctuations in data

↓
Error from randomness in the data itself (unavoidable)

1. HIGH BIAS — UNDERFITTING



- ✗ Model too simple to capture the underlying pattern.
- ✗ Ignores important trends.

⚠ Symptoms: High training error AND high test error



How to fix:

Use a more complex model, add relevant features, reduce regularisation.

2. SWEET SPOT — GOOD FIT



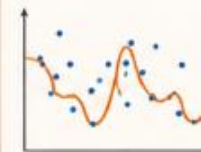
- ✓ Balances bias and variance.
- ✓ Captures true patterns without fitting the noise.

🎯 Goal: Low training error AND low test error



This is the goal of every ML project.

3. HIGH VARIANCE — OVERFITTING



- ✗ Model too complex and memorizes the training data.
- ✗ Fits noise instead of the underlying pattern.

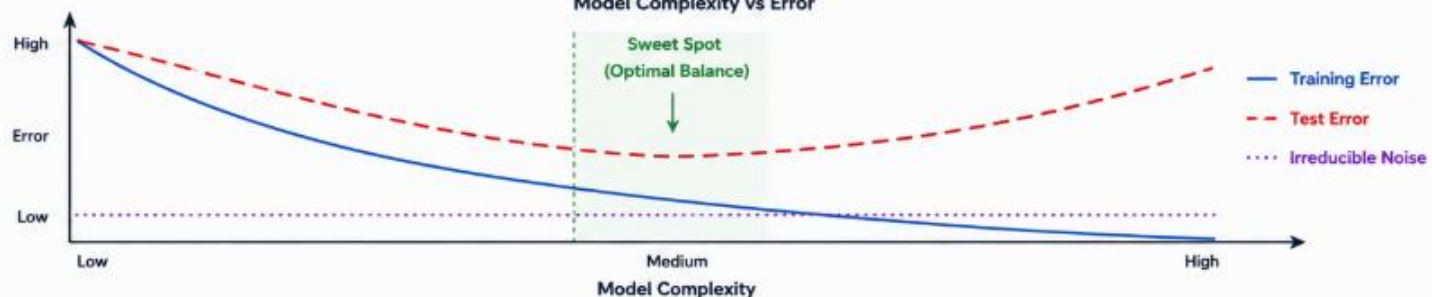
⚠ Symptoms: Low training error BUT high test error



How to fix:

Use regularisation, more data, dropout, pruning, or a simpler model.

Model Complexity vs Error



KEY TAKEAWAYS



Underfitting (High Bias):
Model is too simple → misses the pattern.



Overfitting (High Variance):
Model is too complex → fits noise.



Sweet Spot: Best generalisation performance on unseen data.



Use the right complexity + regularisation + enough data.



INTERVIEW TIP: Overfitting shows high train accuracy but low test accuracy. Check the train-test gap and validation curves!

Machine Learning Life Cycle

Clean, transform, and prepare your data to improve model performance.



1. MISSING VALUES

Handle incomplete data to avoid bias and information loss.

- Drop if >40% missing
- Impute: mean/median for numeric, mode for categorical
- Advanced: KNN imputation, MICE iterative imputation



Example: Fill missing age with median age.



2. ENCODING CATEGORICALS

Convert categorical data into numbers so models can understand.

- Label Encoding: ordinal features (Low=0, Mid=1, High=2)
- One-Hot Encoding: nominal features (no order implied)
- Target Encoding: replace with mean of target variable



Example: Color → Red, Blue, Green becomes [1,0,0], [0,1,0], [0,0,1]



3. FEATURE SCALING

Scale features to bring them to a similar range.

- StandardScaler: $z = (x - \mu) / \sigma \rightarrow \text{mean}=0, \text{std}=1$
- MinMaxScaler: $x' = (x - \min) / (\max - \min) \rightarrow [0,1]$
- Needed for: SVM, KNN, Neural Nets, PCA, Logistic Regression



Example: Scale income from [20K, 200K] to [0, 1] range.



4. IMBALANCED CLASSES

Handle skewed class distribution to avoid biased predictions.

- SMOTE: synthetic minority oversampling
- Undersampling: remove majority samples
- class_weight='balanced' in sklearn models
- Metric: use Precision-Recall AUC, not accuracy



Example: Fraud detection (1% fraud, 99% non-fraud).



5. FEATURE SELECTION

Select the most relevant features to improve performance and reduce noise.

- Filter: Correlation, Chi-Square, ANOVA F-test
- Wrapper: Recursive Feature Elimination (RFE)
- Embedded: LASSO L1 regularisation, Tree-based importance



Example: Keep top 20 features with highest importance.



6. TRAIN-TEST SPLIT

Split data to evaluate model fairly on unseen data.

- Standard: 70% train / 15% val / 15% test
- Stratified split preserves class distribution
- Time-series: always chronological — never random



Example: For time series, train on past, test on future.



KEY TAKEAWAY

Good preprocessing = better models.
Dirty data → misleading results.



Clean Data

Handle missing values and outliers.



Transform Smartly

Encode, scale, and select the right features.



Balance & Validate

Fix class imbalance and split data correctly.



Better Models

Leads to better generalization and real-world performance.

Model Evaluation

Model Evaluation

- It is the process of assessing a machine learning model's performance using specific metrics. It helps determine how well the model achieves the desired outcome.

Why to Evaluate the model?

- The reason for evaluating a machine learning model is to ensure that it performs effectively and meets the desired objectives, allowing for improvements in accuracy, efficiency, and generalization.

Model Evaluation Metrics



Evaluation Metrics	What is it?	When to Use?	Why to Use?
Confusion Matrix	A table used to evaluate the performance of a classification model.	Use it when you need to assess the accuracy and errors in classification models.	It provides insights into the types of errors a model makes, helping to improve performance.
Accuracy	Measures the percentage of correctly classified instances.	Used when class distribution is balanced.	Provides a general performance measure but is misleading for imbalanced datasets.
Precision	The ratio of correctly predicted positive observations to total predicted positives.	Used when False Positives (FP) need to be minimized (e.g., spam detection).	Ensures that the model is making positive predictions correctly.
Recall (Sensitivity)	The ratio of correctly predicted positive observations to actual positives.	Used when False Negatives (FN) need to be minimized (e.g., disease detection).	Ensures that positive cases are not missed.
F1-Score	Harmonic mean of Precision and Recall.	Used when there is an imbalance between Precision and Recall.	Provides a balanced measure between Precision and Recall.

Model Evaluation Metrics

Evaluation Technique	What is it?	When to Use?	Why to Use?
Mean Squared Error (MSE)	Measures the average squared difference between actual and predicted values.	Used in regression problems.	Penalizes larger errors more heavily.
Mean Absolute Error (MAE)	Measures the average absolute difference between actual and predicted values.	Used in regression when all errors should contribute equally.	Less sensitive to outliers than MSE.
ROC-AUC (Receiver Operating Characteristic - Area Under Curve)	Measures the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR).	Used when dealing with binary classification problems.	Helps evaluate the performance of probabilistic models.
Log Loss (Logarithmic Loss)	Measures how uncertain the model's probability predictions are.	Used for probabilistic classification tasks.	Penalizes confident incorrect predictions more heavily.

Validation Techniques

Validation Technique	Description	Pros	Cons
K-Fold Cross Validation	K-Fold Cross Validation splits the dataset into K parts, using each part as a test set once.	Reduces overfitting and improves model reliability.	Computationally expensive and time-consuming.
Leave-One-Out Cross-Validation (LOOCV)	Trains the model on all data except one point; repeats for each point.	Uses all data for training, low bias.	Computationally expensive, high variance.
Stratified Cross-Validation	A version of k-fold cross-validation that maintains class distribution in each fold.	Better for imbalanced datasets, ensures fair model evaluation.	More computationally intensive than simple k-fold.

Confusion Matrix

- A **Confusion Matrix** is a table used to evaluate the performance of a **classification model** in machine learning.
- It compares the **actual values** with the **predicted values**.
- It helps answer:
- “How many predictions were correct and where did the model make mistakes?”

Binary Classification of Confusion Matrix



Actual \ Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Confusion Matrix

A confusion matrix shows the performance of a classification model by comparing actual vs predicted values.

		PREDICTED (MODEL OUTPUT)	
		Positive (1)	Negative (0)
ACTUAL (GROUND TRUTH)	Positive (1)	 TP True Positive Predicted Positive and Actual Positive n = 85	 FN False Negative Predicted Negative but Actual Positive n = 15
	Negative (0)	 FP False Positive Predicted Positive but Actual Negative n = 10	 TN True Negative Predicted Negative and Actual Negative n = 90

TP (True Positive) : Correctly predicted positive
TN (True Negative) : Correctly predicted negative
FP (False Positive) : Incorrectly predicted positive (Type I error)
FN (False Negative) : Incorrectly predicted negative (Type II error)

Totals
 P (Actual Positives) = TP + FN = 100
 N (Actual Negatives) = TN + FP = 100
 Total Samples = 200

EVALUATION METRICS



1. ACCURACY

$$(TP + TN) / (TP + TN + FP + FN) = 87.5\%$$

Overall correctness of the model.
Can be misleading on imbalanced data.



2. PRECISION (Positive Predictive Value)

$$TP / (TP + FP) = 89.5\%$$

Of all predicted positives, how many are actually positive?



3. RECALL (SENSITIVITY, TRUE POSITIVE RATE)

$$TP / (TP + FN) = 85.0\%$$

Of all actual positives, how many did we correctly catch?



4. F1-SCORE

$$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 87.2\%$$

Harmonic mean of Precision and Recall.
Balances both.



5. SPECIFICITY (TRUE NEGATIVE RATE)

$$TN / (TN + FP) = 90.0\%$$

Of all actual negatives, how many did we correctly reject?

WHEN TO USE WHAT?

-  High Recall: Disease detection, fraud detection (Don't miss positives)
-  High Precision: Spam filtering, ad targeting (Avoid false alarms)
-  F1-Score: When classes are imbalanced and both precision & recall matter
-  Specificity: Medical tests, quality control (Correctly identify negatives)

EXAMPLE SUMMARY (FROM ABOVE)

TP (True Positive)	85
FN (False Negative)	15
FP (False Positive)	10
TN (True Negative)	90
Total Samples	200
Actual Positives (P)	100
Actual Negatives (N)	100

IMPORTANT NOTES

- Accuracy is NOT reliable for imbalanced datasets.
- False Negative (FN) can be very costly in critical applications (e.g., cancer, fraud).
- False Positive (FP) can be costly in spam, ad targeting, or security.
- Always choose the metric based on the problem and business goal.

QUICK FORMULA RECAP

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
Precision = $TP / (TP + FP)$
Recall = $TP / (TP + FN)$
F1-Score = $2 \times (P \times R) / (P + R)$
Specificity = $TN / (TN + FP)$
 Where: P = Precision, R = Recall



TRAINER TIP: When the cost of missing a positive is high → prioritize Recall. When the cost of false alarms is high → prioritize Precision. F1 when classes are imbalanced.



Confusion Matrix

- **1. True Positive (TP)**
 - Model correctly predicts a positive class.
 - Example:
 - Actual: Fraud transaction
 - Predicted: Fraud 
- **2. True Negative (TN)**
 - Model correctly predicts a negative class.
 - Example:
 - Actual: Not fraud
 - Predicted: Not fraud 

Confusion Matrix

- **3. False Positive (FP)**

- Model predicts positive, but actual is negative.

- Example:

- Actual: Legitimate transaction

- Predicted: Fraud ✖

- Also called **Type I Error**

- **4. False Negative (FN)**

- Model predicts negative, but actual is positive.

- Example:

- Actual: Fraud transaction

- Predicted: Legitimate ✖

- Also called **Type II Error**

Real Time Confusion Matrix

Actual / Predicted	Diabetes	No Diabetes
Diabetes	90	10
No Diabetes	15	85

Meaning:

- TP = 90 → correctly detected patients
- FN = 10 → diabetic patients missed
- FP = 15 → healthy people wrongly predicted as diabetic
- TN = 85 → healthy correctly predicted

Metrics Derived from Confusion Matrix

Accuracy

How many predictions are correct overall?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Real Time Confusion Matrix

Actual / Predicted	Diabetes	No Diabetes
Diabetes	90	10
No Diabetes	15	85

Meaning:

- TP = 90 → correctly detected patients
- FN = 10 → diabetic patients missed
- FP = 15 → healthy people wrongly predicted as diabetic
- TN = 85 → healthy correctly predicted

Metrics Derived from Confusion Matrix

Accuracy

How many predictions are correct overall?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Real Time Confusion Matrix

Accuracy

How many predictions are correct overall?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

Out of predicted positives, how many are correct?

$$Precision = \frac{TP}{TP + FP}$$

Example:

Spam email prediction → avoid false alarms.

Recall (Sensitivity)

Out of actual positives, how many did we detect?

$$Recall = \frac{TP}{TP + FN}$$

Real Time Confusion Matrix

Example:

Disease detection → missing disease is dangerous.

F1 Score

Balance between Precision and Recall

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Training-Friendly Intuition

Think of a **student exam result prediction system**:

- TP → failed students correctly identified
- FP → good students wrongly marked as weak
- FN → weak students missed
- TN → good students correctly identified

When to Use

- Classification algorithms:
 - Decision Tree
 - Random Forest
 - Logistic Regression
 - SVM
 - Naive Bayes
 - XGBoost



What ROC(Receiver Operating Characteristic) Curve Actually Means



- The ROC curve shows:
- **How good a model is at separating two classes** (Yes/No, Fraud/Not Fraud, Disease/Healthy).
- It compares:
 - **True Positive Rate (TPR / Recall / Sensitivity)** → Correct positive predictions
 - **False Positive Rate (FPR)** → Wrong positive predictions

What ROC(Receiver Operating Characteristic) Curve Actually Means



- The ROC curve shows:
- **Formula**
 - Recall (TPR):
$$TPR = \frac{TP}{TP+FN}$$
- False Positive Rate:
 - $FPR = \frac{FP}{FP+TN}$
- Where:
 - TP = True Positive
 - FP = False Positive
 - TN = True Negative
 - FN = False Negative

Why ROC Curve Is Needed

- Suppose a bank predicts fraud.
- Model gives probability:

Transaction	Fraud Probability
T1	0.95
T2	0.85
T3	0.70
T4	0.40
T5	0.20

Why ROC Curve Is Needed

- We need a **threshold**.
- Example:
- **Threshold = 0.5**
- If probability $> 0.5 \rightarrow$ Fraud
- Prediction:
- T1 
- T2 
- T3 
- T4 
- T5 

Why ROC Curve Is Needed

- **Threshold = 0.2**
- Now model becomes more aggressive:
- T1 
- T2 
- T3 
- T4 
- T5 
- You catch more frauds (**high recall**) but may wrongly flag genuine transactions (**more false positives**).
- ROC checks model performance at **ALL thresholds**, not just one threshold.
- That is why ROC is powerful.

ROC Curve & AOC Score

Evaluate classification models across all thresholds and handle imbalanced data effectively.

1. WHAT IT MEASURES



ROC (Receiver Operating Characteristic) curve shows the trade-off between True Positive Rate (Recall) and False Positive Rate at every decision threshold (0 → 1). Threshold controls the balance between sensitivity and specificity.

2. AUC INTERPRETATION



1.0	= Perfect	(Model is perfect)
0.9 – 1.0	= Excellent	(Outstanding performance)
0.8 – 0.9	= Good	(Strong performance)
0.7 – 0.8	= Fair	(Moderate performance)
0.5 – 0.7	= Poor	(Weak performance)
0.5	= Random (Coin flip)	(No discriminative power)
< 0.5	= Worse than random	(Model is doing the opposite)

3. WHEN TO USE AUC



- ✓ When dealing with imbalanced datasets
- ✓ When accuracy is misleading
- ✓ When comparing multiple classifiers objectively
- ✓ When decision threshold needs to vary based on business needs

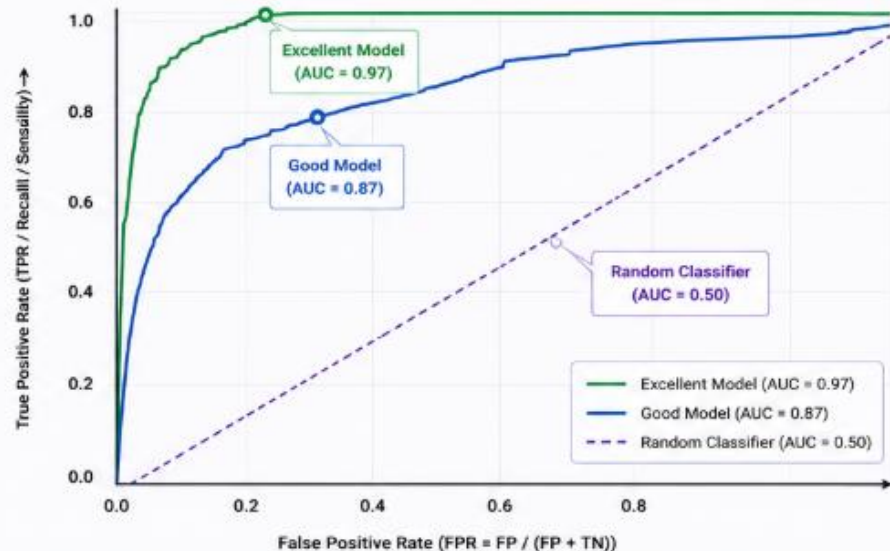
4. INTERVIEW INSIGHT



Q: Why AUC for imbalanced data?

A: Accuracy depends on class distribution and a single threshold. AUC evaluates the model's ability to rank positive instances higher than negative ones across ALL possible thresholds.

ROC CURVE COMPARISON



HOW THRESHOLD AFFECTS CLASSIFICATION

	Threshold = 0.1 (Conservative)	Threshold = 0.3	Threshold = 0.5 (Balanced)	Threshold = 0.7	Threshold = 0.9 (Aggressive)
Effect	Predicts more positives	More true positives, more false positives	Balanced trade-off	Fewer false positives, more false negatives	Predicts fewer positives
Result	High Recall Low Precision	Moderate Recall Moderate Precision	Balanced Recall and Precision	Low Recall High Precision	Low Recall High Precision



REAL-WORLD TIP

A fraud detection model with 99% accuracy may still have AUC = 0.60 if fraud is only 1% of transactions. Always report AUC along with accuracy for imbalanced problems.



Remember:

Higher AUC → Better model at ranking positives above negatives across thresholds.

ROC 3. Understanding the ROC Graph Curve



- In your image:
- **X-axis**
- False Positive Rate
- Meaning:
- How many wrong alarms you create
- Example:
- Genuine customers wrongly flagged as fraud.
- **Y-axis**
- True Positive Rate (Recall)
- Meaning:
- How many real positives you correctly identify
- Example:
- Actual fraud transactions caught.

Why Top-Left Corner Is Best

- Imagine this point:
- Recall = 1.0 (100% fraud detected)
- FPR = 0.0 (no false alarm)
- That means:
 - ✓ Catch all positives
 - ✓ No mistakes
- Perfect model.
- So best ROC curves go toward the **top-left corner**.

What AUC Means

- AUC = **Area Under ROC Curve**
- It summarizes model performance into **one number**.
- Range:
 - $0 \rightarrow 1$
- Think like this:
 - AUC tells how well the model ranks positives higher than negatives.
- Interpretation:

AUC	Meaning
1.0	Perfect
0.9–1	Excellent
0.8–0.9	Good
0.7–0.8	Fair
0.5	Random guessing
<0.5	Worse than random

Real Meaning of AUC (Interview Explanation)



- Suppose:
 - Positive case = Fraud
 - Negative case = Normal transaction
- $AUC = 0.90$ means:
- If I randomly choose one fraud and one genuine transaction, the model has **90% chance** of giving a higher score to fraud.
- This explanation is interview gold.

Why Random Model Is 0.5

- Look at the diagonal line in image.
- That line means:
 - Model is guessing randomly.
 - Like coin toss.
 - 50% probability.
- Because:
 - True positives increase at same speed as false positives.
 - No intelligence.

ROC vs Accuracy (Most Important Concept)



- Suppose:
- 1000 transactions
 - Fraud = 10
 - Genuine = 990
- Model predicts:
- “All genuine”
- Accuracy:
 - $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- Accuracy = $990/1000 = \mathbf{99\%}$

ROC vs Accuracy (Most Important Concept)



- Looks amazing?
 - No.
- Because fraud detection failed completely.
 - ROC/AUC reveals truth.
 - AUC may be poor.
- That is why AUC is preferred for **imbalanced datasets**.
- Examples:
 - Fraud detection
 - Disease prediction
 - Spam filtering
 - Defect detection
 - Rare event prediction

ROC vs Precision-Recall Curve

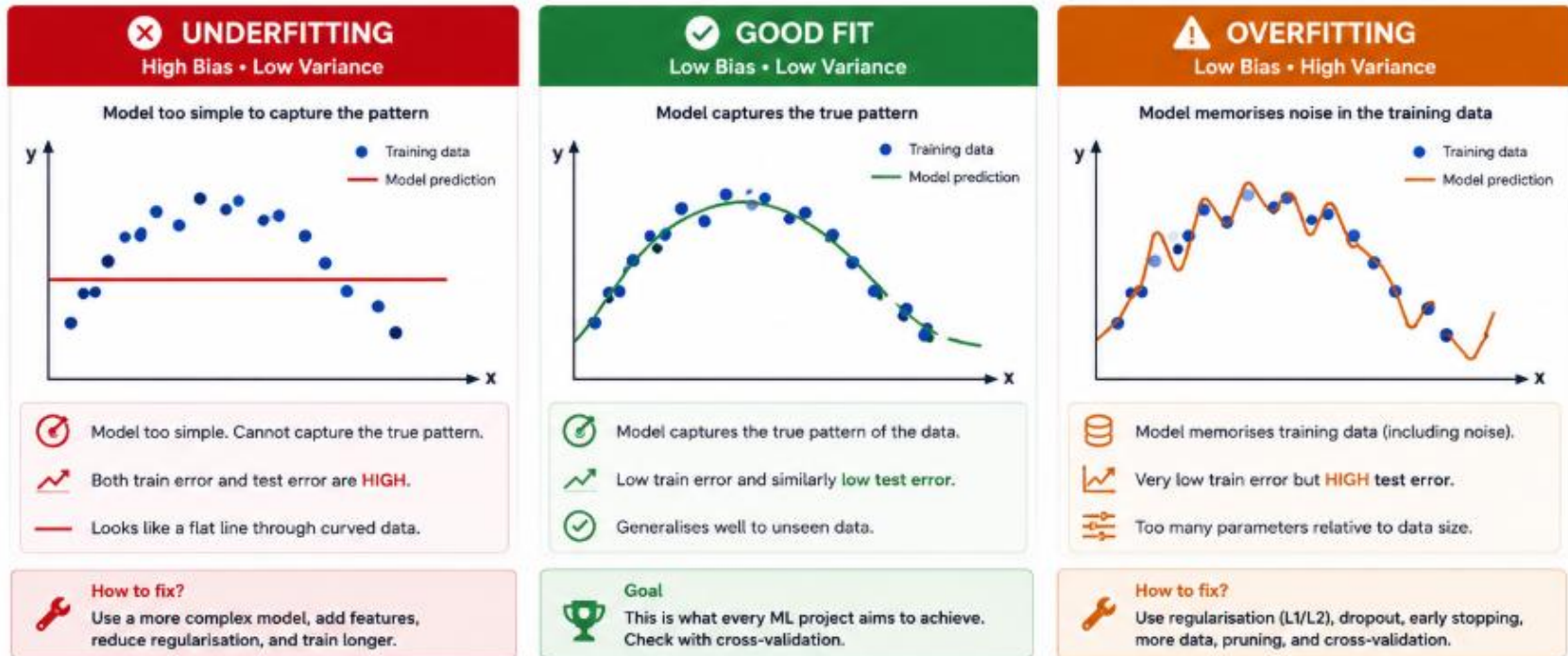
- Use ROC when:
 - ✓ Balanced or moderately imbalanced data
 - ✓ Need overall classifier comparison
- Use Precision-Recall when:
 - ✓ Extremely imbalanced dataset
 - ✓ Positive class is more important
- Example:
 - Cancer detection.
 - Missing cancer is dangerous

Easy Interview One-Line Answer

- “ROC curve measures model performance across all classification thresholds using TPR and FPR, while AUC summarizes the model’s ability to separate positive and negative classes into a single score.”
- Remember:
 - **ROC = Trade-off graph**
 - **AUC = Final score**
- Higher AUC → Better model separation.

Over Fitting vs Under Fitting

The goal is to find the right balance between bias and variance to achieve the best generalization.



Aspect	Underfitting	Good Fit	Overfitting
Bias	High	Low	Low
Variance	Low	Low	High
Train Error	High	Low	Very Low
Test Error	High	Low	High
Generalization	Poor	Good	Poor

TRAINER TIP: Show 3 curves and ask: Which is overfitting? →

Example Answer: The third one is overfitting. → **Answer:** Reduce complexity, add regularisation, more data, or early stopping.

95% train accuracy, 62% test accuracy — what do you do?

K-Fold Cross Validation

- Instead of training/testing once, we train and test **multiple times** to get a more reliable score.

1. Why Single Train-Test Split Is Problematic

- Suppose you have 1000 student records.
- You split:
 - 80% Train
 - 20% Test
- Example:
 - Train = 800
 - Test = 200

1. Why Single Train-Test Split Is Problematic

- Problem:
- What if by chance:
 - Easy samples go into test set?
 - Hard samples go into training set?
- Then accuracy changes.
- Example:
- Run 1:
- Train/Test split gives:
 - **Accuracy = 95%**
- Run 2 (different random split):
 - **Accuracy = 82%**
- Why?
 - Because results depend on which samples entered train/test.
 - That makes evaluation unreliable.

2. What K-Fold Cross Validation Solves

- Instead of one split:
- We divide data into **K equal parts (folds)**.
- In your image:
 - $K = 5$
- Dataset split into:
 - Fold1, Fold2, Fold3, Fold4, Fold5
- Each fold gets chance to become test data.

Cross Validation and Model Selection

Why? A single train-test split is unreliable — results depend on which samples end up in each set.

K-Fold Cross-Validation (K = 5)

Fold 1	TEST	Train	Train	Train	Train	→ Score 1
Fold 2	Train	TEST	Train	Train	Train	→ Score 2
Fold 3	Train	Train	TEST	Train	Train	→ Score 3
Fold 4	Train	Train	Train	TEST	Train	→ Score 4
Fold 5	Train	Train	Train	Train	TEST	→ Score 5

Final Score = Mean(Score 1...5) ± Standard Deviation

Stratified K-Fold

Maintains class proportions in each fold.

Leave-One-Out (LOO)

K = n (one sample per fold). Very expensive.

Time-Series Split

Train on past, test on future. Never shuffle.

✳️ TRAINER: Interview Q: 'Why use K-Fold instead of a single split?' More reliable estimate — every sample is used for both training and testing across K iterations.

3. Understanding the Diagram

- You see:
- Row 1:
 - TEST | Train | Train | Train | Train
- Meaning:
 - Fold1 = Test
 - Remaining = Training
- Model score:
 - **Score 1**

4. Final Score

Instead of trusting one score:

We average them.

Suppose:

Fold	Accuracy
1	90
2	92
3	88
4	91
5	89

Mean:

$$Mean = \frac{90 + 92 + 88 + 91 + 89}{5} = 90$$

Final Accuracy = **90%**

Much more trustworthy.

Why K-Fold Is Better Than Single Split

Single Split	K-Fold
One test only	Multiple tests
Unstable	Reliable
Randomness impacts more	Less randomness
Biased	Better estimate

Hands-on: Steps for Evaluating and Choosing the Best Model



1. Define Your Problem and Goal

Type of Task:

- Regression (e.g., predicting prices).
- Classification (e.g., spam detection).
- Regression: R^2 , Mean Squared Error (MSE).
- Classification: F1-Score, Accuracy.

2. Prepare Your Data Preprocessing:

- Handle missing values, outliers, and scale features.
- Splitting: Training (60–70%), Validation (20–30%), Test (10–20%).

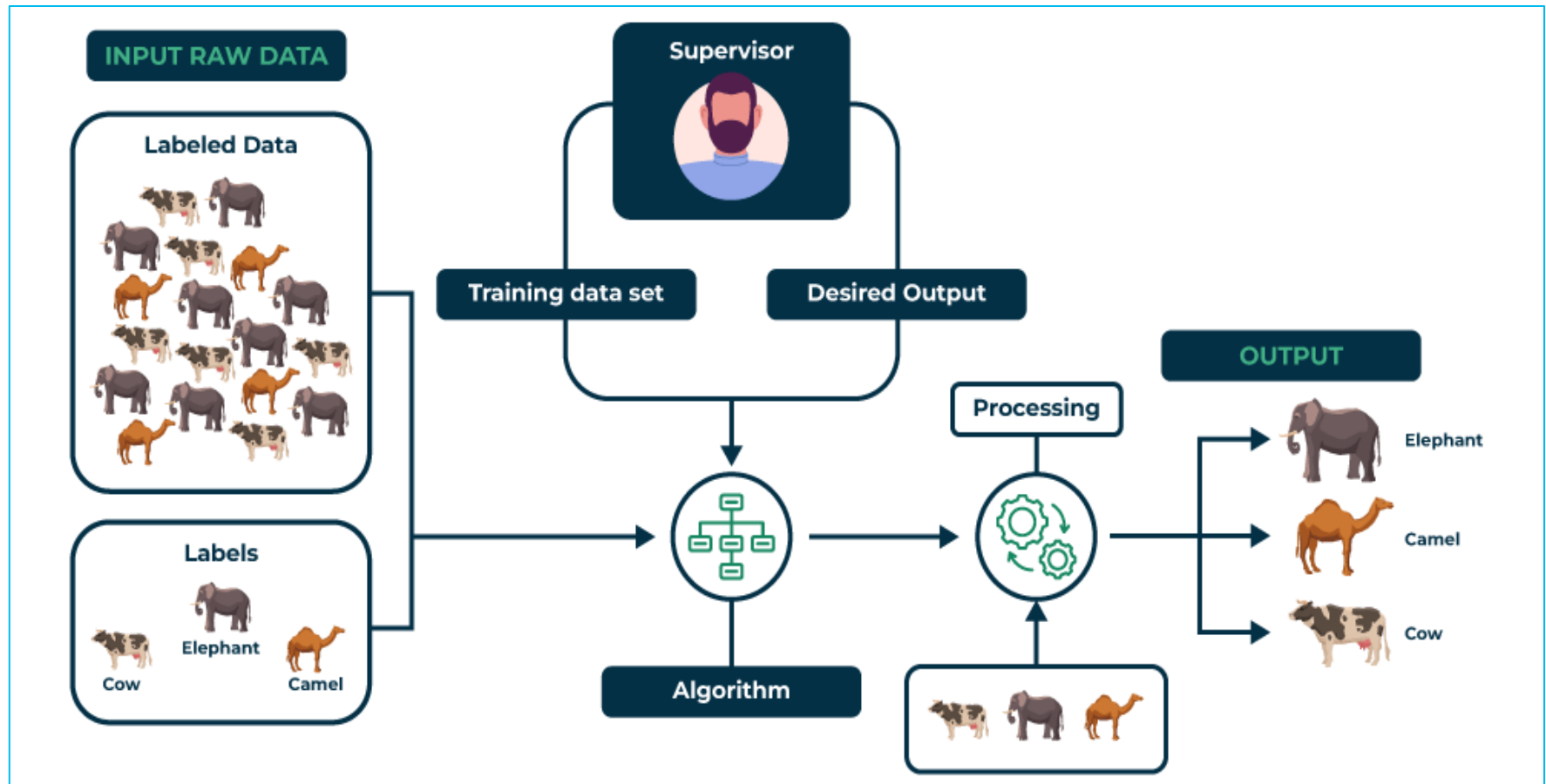
3. Choose Candidate Models

- Diversify: Use models from different families (e.g., Linear, Tree-Based).
- Examples: Regression: Linear Regression, Random Forest Regressor.
- Classification: Logistic Regression, SVM, Neural Networks.

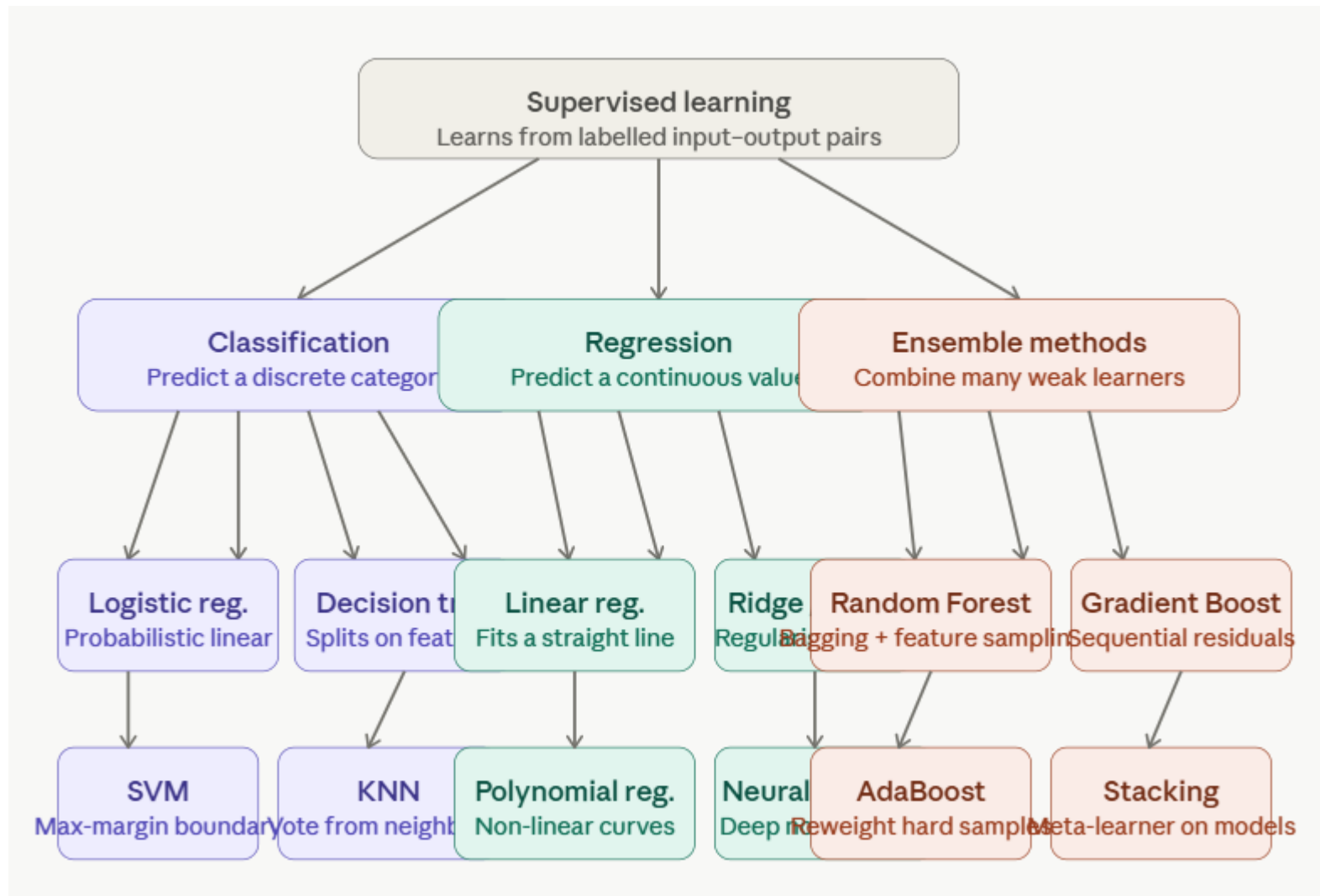
4. Train and Evaluate

- Train models on the training set.
- Use the validation set for performance comparison.
- Metric comparison ensures the best candidate.

Supervised Learning

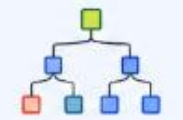
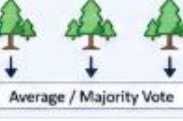
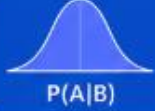


Supervised Learning



Supervised Learning

Learning from labelled data to predict outputs

 1. Linear Regression	What it does: Finds the best straight line that predicts a continuous value.	Use for:  Regression (Continuous)	Best for: • House price prediction • Sales forecasting • Salary prediction	
 2. Logistic Regression	What it does: Models the probability of a class using a logistic (S-shaped) function.	Use for:  Classification (Binary / Multi-class)	Best for: • Spam detection • Customer churn • Disease prediction	
 3. Decision Tree	What it does: Splits data using rules to make decisions like a flowchart.	Use for:  Classification & Regression	Best for: • Fraud detection • Risk assessment • Customer segmentation	
 4. Random Forest	What it does: Combines many decision trees to improve accuracy and stability.	Use for:  Classification & Regression	Best for: • Credit scoring • Product recommendation • Medical diagnosis	
 5. K-Nearest Neighbours (KNN)	What it does: Predicts based on the majority class (or average value) of nearest points.	Use for:  Classification & Regression	Best for: • Image recognition • Recommender systems • Anomaly detection	
 6. Naïve Bayes	What it does: Uses Bayes' theorem with the 'naïve' assumption of feature independence.	Use for:  Classification (Probabilistic)	Best for: • Text classification • Spam filtering • Sentiment analysis	



Key Takeaway: Supervised learning uses labelled data (inputs + known outputs) to learn patterns and make predictions on new, unseen data.

Linear Regression – Definition & Foundations



Linear Regression finds the **best-fit line** (or hyperplane) that **minimises** prediction errors between input features **X** and a continuous target variable **y**.



WHY WE NEED IT



Many real outcomes are continuous: house prices, temperatures, sales revenue. Linear Regression provides a fast, interpretable baseline model with direct feature impact coefficients.



WHEN TO USE

- ✓ Relationship between X and y is linear
- ✓ Interpretability is required
- ✓ Need a fast baseline before complex models
- ✓ Continuous, numeric output variable



WHEN NOT TO USE

- ✗ Target is categorical (use Logistic Regression)
- ✗ Strong non-linear relationships in data
- ✗ Many correlated features (multicollinearity)
- ✗ Heavy outliers without robust preprocessing



KEY EQUATIONS

Simple:

$$\hat{y} = \beta_0 + \beta_1 x$$

Multiple:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

Cost (MSE):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Optimise
(Gradient Descent):

$$\beta_j \leftarrow \beta_j - \alpha \frac{\partial MSE}{\partial \beta_j}$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} \quad | \quad 1.0 = \text{perfect fit}$$



ADVANTAGES



Fast to train and predict



Highly interpretable coefficients



TRAINER TIP: Interview: "Explain Linear Regression to a 5-year-old." Then: "What is multicollinearity and how do you detect it?" (Variance Inflation Factor — **VIF > 10** is problematic).

Linear Regression – RealTime



What is a Residual?

Residual = Actual - Predicted
Measures the vertical distance from a point to the regression line.

How the Regression Line is Found



Finds the "best-fit" line that minimises the sum of squared residuals (MSE)



Estimates coefficients (β_0, β_1) using Optimisation (Gradient Descent / Normal Equation)



Used to predict new values of y for any given x

Variants: Ridge (L2) • LASSO (L1) • ElasticNet • Polynomial Regression

INDUSTRY EXAMPLES



House Price Prediction



Features: sq.ft, bedrooms, location score, age



Target: Sale price (₹ or \$)



Used by: Magicbricks, Zillow, 99acres



Sales Forecasting



Features: ad spend, seasonality, promotions



Target: Quarterly revenue



Used by: Retail, FMCG, e-commerce chains



Drug Dosage Modelling



Features: patient weight, age, kidney function



Target: Optimal dose (mg)



Used by: Clinical trials, ICU management



Weather Prediction



Features: pressure, humidity, wind speed



Target: Temperature (°C)



Used by: IMD, AccuWeather, aviation systems

Goodness of Fit

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

- $R^2 = 1.0$ → Perfect fit
- $R^2 = 0$ → Model no better than mean
- $R^2 < 0$ → Poor model



TRAINER TIP: Lab: sklearn California housing dataset. Fit LinearRegression, print coefficients and R^2 . Plot predicted vs actual. Visualise residual distribution (histogram) and residuals vs fitted values.



Easy to implement & interpret



Works well with linear relationships



Watch out for: non-linearity, outliers, multicollinearity

Linear Regression – RealTime

- Your **Simple Linear Regression** result:
- R2 Score: 0.4186
RMSE: 4.4233
Intercept: 8.3177
Coefficient: 0.0447
- Let's interpret it properly.
- **1. Model Equation**
- Your model becomes:
- $Sales = 8.32 + 0.0447(TV)$
- Meaning:
- Sales depends **only on TV advertisement spend.**

Linear Regression – RealTime

- **2. R^2 Score Interpretation**
- You got:
- $R^2 = 0.4186$
- Meaning:
- The model explains only **41.86% of sales variation.**

R^2	Meaning
>0.90	Excellent
0.80–0.90	Very Good
0.70–0.80	Good
0.50–0.70	Moderate
<0.50	Weak

Linear Regression – RealTime

- Business meaning:
 - TV ads alone explain only ~42% of sales.
- Remaining **58%** of sales variation is due to:
 - Radio advertisements
 - Newspaper ads
 - Discounts
 - Festival season
 - Brand popularity
 - Competition
 - Random factors
- This actually makes sense because your earlier multiple regression used:
 - TV + Radio + Newspaper
- and got:
 - **$R^2 \approx 0.70$**
- So adding variables improved prediction.

RMSE Interpretation

- You got:
 - $RMSE = 4.42$
- Meaning:
 - On average, prediction error is about **4.42 sales units**
- Example:
 - Actual sales = 20
- Predicted may be:
 - 15.6 or 24.4
- Compared to earlier multiple regression:
 - Multiple Regression $RMSE \approx 3.15$
 - Simple Regression $RMSE \approx 4.42$
- Smaller RMSE is better.
- So:
 -  Simple regression is worse than multiple regression

Intercept Interpretation

- You got:
- Intercept = 8.3177
- Meaning:
- If:
- TV Spend = 0
- Expected sales:
- **≈ 8.32**
- Interpretation:
- Even without TV ads, business still sells ~8.3 units.
- Why?
- Because:
- Existing customers
- Organic demand
- Word of mouth
- Brand recognition

Coefficient Interpretation

- You got:
- Coefficient = 0.0447
- Meaning:
- For every **1 unit increase in TV spend**:
- Sales increase by:
- **≈ 0.0447 units**
- Example:
- Increase TV spend by 100:
- Expected sales increase:
- **≈ 4.47**

Coefficient Interpretation

- Calculation:
- $100 \times 0.0447 = 4.47$
- Business interpretation:
- TV advertisement positively impacts sales.
- Positive coefficient = good sign.
- Negative coefficient would mean opposite effect.

Simple vs Multi Linear Regression

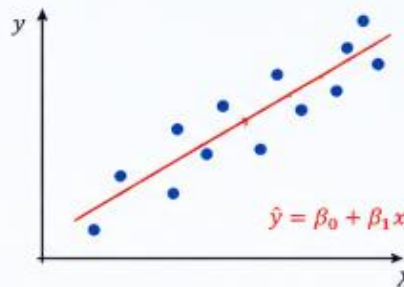
- The simple linear regression model is weaker because it explains only about 42% of sales variability ($R^2=0.42$) and has a higher prediction error (RMSE=4.42).
- This suggests TV advertisement alone is insufficient to explain sales, and additional factors such as radio and newspaper advertisements improve prediction accuracy.

Types of Regression



Regression techniques help us predict a continuous numeric value (y) from one or more input features (X).

1 SIMPLE LINEAR REGRESSION (One input feature)



Models a linear relationship between X and y .

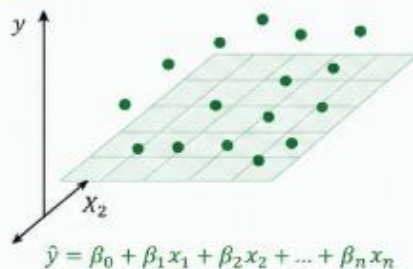
Use When:

- Relationship between X and y is approximately linear.

Example:

House price vs. size

2 MULTIPLE LINEAR REGRESSION (Multiple input features)



Models a linear relationship between y and two or more features.

Use When:

- Relationship is linear but involves multiple predictors.

Example:

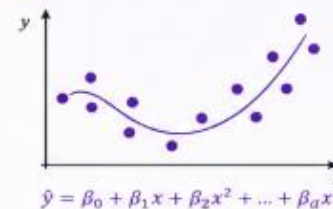
Salary prediction (experience, education, skills)



KEY TAKEAWAY

Use Simple Linear for one predictor and Multiple Linear when multiple factors linearly influence the target.

3 POLYNOMIAL REGRESSION (One input feature)



Models non-linear relationships by fitting a polynomial curve.

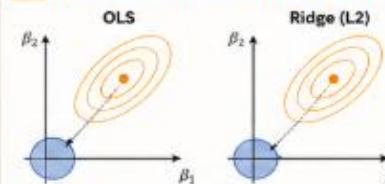
Use When:

- Relationship is non-linear but can be captured by a polynomial curve.

Example:

Salary vs. Years of Experience (curves upward)

4 RIDGE REGRESSION (L2 Regularization)



Linear regression with L2 regularization to reduce overfitting by shrinking coefficients.

Minimise:

$$MSE + \lambda \sum_{j=1}^n \beta_j^2$$

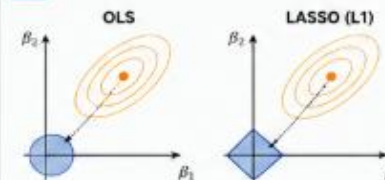
Use When:

- Features are many and correlated.
- Want to shrink coefficients but keep all features.

Example:

Predicting customer lifetime value

5 LASSO REGRESSION (L1 Regularization)



Linear regression with L1 regularization that performs feature selection.

Minimise:

$$MSE + \lambda \sum_{j=1}^n |\beta_j|$$

Use When:

- Need automatic feature selection.
- Expect only a few important features.

Example:

Marketing spend: selecting key channels that drive sales

QUICK COMPARISON

Type	Captures	Overfitting Control	Feature Selection	Interpretability	Best For
Simple Linear	Linear (1 feature)	Low	No	High	Simple insights, quick analysis
Multiple Linear	Linear (many features)	Low	No	Medium	Multiple factors analysis
Polynomial	Non-linear (curves)	Medium	No	Medium	Curved relationships
Ridge (L2)	Linear	High	No	High	Many correlated features
LASSO (L1)	Linear	High	Yes	Medium	High-dimensional data



LAB IDEA: Use sklearn to fit each type on the California Housing dataset. Compare R^2 , RMSE and number of selected features (for LASSO). Plot actual vs. predicted.

Simple vs Multi Linear Regression

Metric	Simple	Multiple
R^2	0.42	0.70
RMSE	4.42	3.15
Inputs	TV only	TV + Radio + Newspaper
Accuracy	Lower	Better

Conclusion:

Multiple regression is clearly better for this dataset.

Reason:

Sales depend on multiple advertisement channels.

Model packaging (pickle/ONNX) and contracts

- "Think of yourself as a chef.
- You've cooked a great dish — the model.
- Now we need to pack it in a container so the delivery team (production engineers) can serve it to customers.
- That's packaging.
- Contracts? That's the menu card — it tells customers exactly what they'll get."

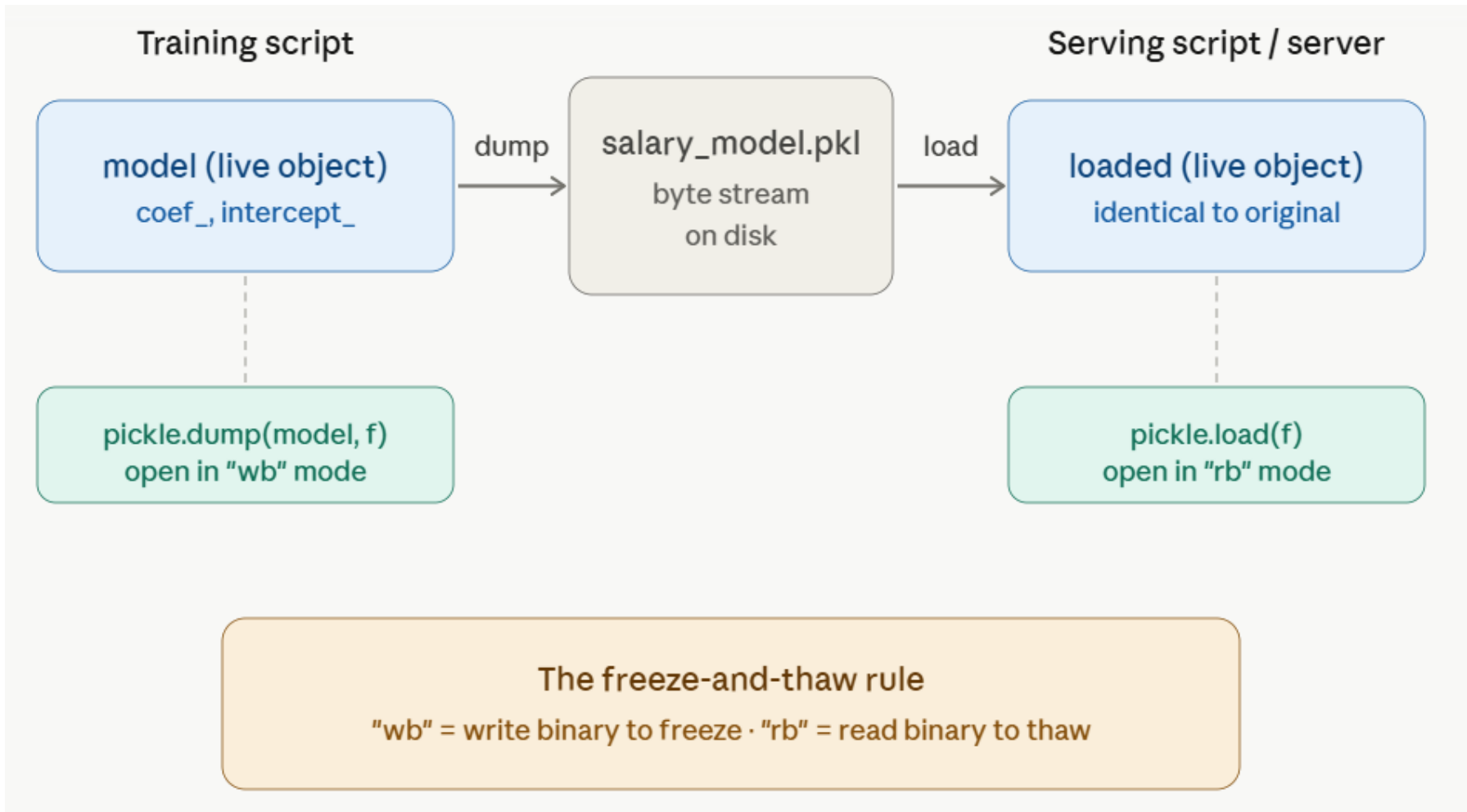
Model packaging (pickle/ONNX) and contracts

- **What Pickle actually is**
- Imagine you've cooked a perfect dish in your kitchen (trained your model in a notebook).
- The kitchen closes for the night.
- Tomorrow, a different chef needs to serve that exact dish to a customer — but they can't re-cook it from scratch.
- So tonight you **vacuum-seal and freeze it** (pickle it to a file).
- Tomorrow they **thaw it** (unpickle), and it's exactly as you made it.

Model packaging (pickle/ONNX) and contracts

- That freeze-and-thaw is what Pickle does for Python objects.
- The technical terms: **serialization** (turning a live Python object into a stream of bytes you can write to disk).
- **Deserialization** (rebuilding the live object from those bytes).

Model packaging (pickle/ONNX) and contracts



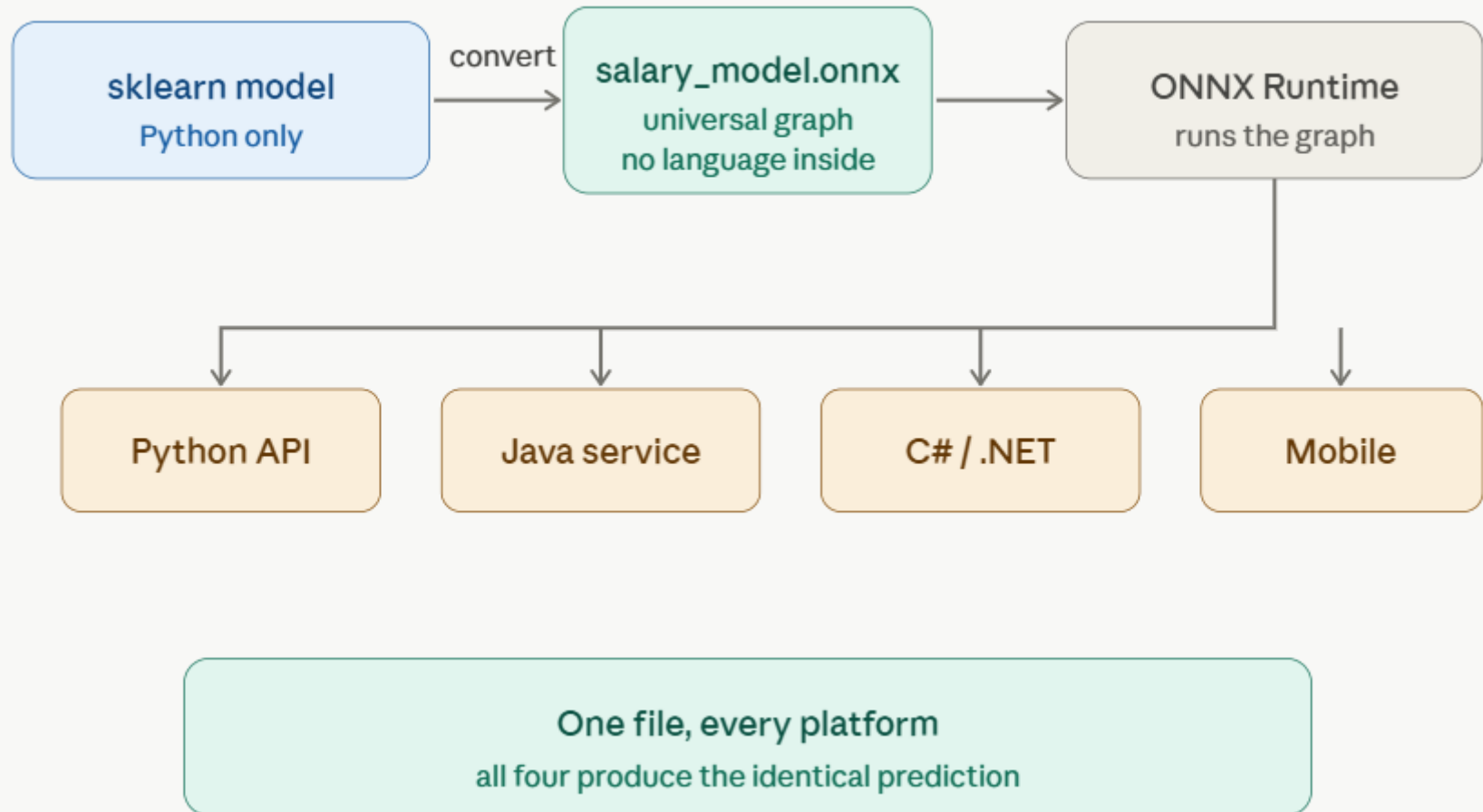
Model packaging (pickle/ONNX) and contracts

- **What is Model Packaging?**
- After training a machine learning model, we need to **save it** and use it later for prediction.
- This process is called **model packaging**.
- Example:
- Train Model → Save Model → Load Model → Predict

Model packaging (pickle/ONNX) and contracts

- ONNX — **Open Neural Network Exchange** — solves exactly that.
- Instead of freezing a Python object, ONNX rewrites your model as a **universal recipe card** that any kitchen in the world can read.
- It doesn't save "a scikit-learn LinearRegression object"; it saves the *math* — "multiply the input by this number, add that number" — as a standardized computation graph.
- A Java service, a C# app, a phone, or a browser can all read that recipe and produce identical results, without Python anywhere in sight.

Model packaging (pickle/ONNX) and contracts



Model packaging (pickle/ONNX) and contracts

- First, **it stores math, not a Python object** — that's why it travels across languages while Pickle can't.
- Second, you need **two different libraries**: `skl2onnx` to *convert* a scikit-learn model into the format, and `onnxruntime` to *run* it.
- Third, **the input must be float32** — this is the single most common beginner mistake; passing `int64` or `float64` causes shape errors or silently wrong results.
- Fourth, **you give your input a name** at conversion time (like `"float_input"`) and must use that same name when running predictions.

Model packaging (pickle/ONNX) and contracts

- When do you reach for ONNX over Pickle?
- Pickle for internal experiments and Python-only batch jobs;
- ONNX the moment a model needs to be served in a production API, deployed to mobile, run in real time, or called from a non-Python service.

Pickle vs ONNX

Pickle

Freeze a live Python object, thaw it later

- ✓ Built into Python — no install
- ✓ Two lines to save, two to load
- ✓ Saves any Python object as-is
- ✗ Python-only — can't run in Java, C#, mobile
- ✗ Breaks across library versions
- ⚠ Unsafe to load untrusted files
- = Inference at plain sklearn speed

Best for internal experiments & batch jobs

ONNX

Rewrite the model as a universal graph

- ✓ Runs in Python, Java, C#, JS, mobile
- ✓ Stable across library versions
- ✓ Optimized runtime — faster inference
- ✓ Safer to share externally
- ✗ Needs two libraries: skl2onnx + onnxruntime
- ⚠ Input must be float32 numpy
- ⚠ More setup to convert

Best for production APIs, mobile & cross-language

Pickle vs ONNX

Factor	Pickle	ONNX
What it stores	The live Python object	The math, as a portable graph
Install needed	None (built in)	<code>skl2onnx</code> + <code>onnxruntime</code>
Save	<code>pickle.dump(model, f)</code>	<code>convert_sklearn(...)</code> then write bytes
Load	<code>pickle.load(f)</code>	<code>ort.InferenceSession(path)</code>
Input type	Any Python / NumPy	<code>float32</code> NumPy only
Languages	Python only	Python, Java, C#, JS, C++, mobile
Version safety	Tied to exact library version	Stable across versions
Inference speed	Plain library speed	Optimized runtime, usually faster
Security	Never load untrusted files	Safer to share externally
Best for	Internal experiments, batch jobs	Production APIs, mobile, non-Python services

Polynomial Regression

- Polynomial Regression is used when the relationship is **not a straight line**.
- For salary prediction:
 - Salary usually does **not increase linearly** with experience.
- Example:
 - 1 year $\rightarrow$ ₹3 LPA
 - 3 years $\rightarrow$ ₹5 LPA
 - 5 years $\rightarrow$ ₹9 LPA
 - 10 years $\rightarrow$ ₹25 LPA
 - 15 years $\rightarrow$ ₹60 LPA
- Notice:
- Salary increases faster as experience grows.
- A straight line (Linear Regression) may fail.
- So we use **Polynomial Regression**.

Polynomial Regression

- Polynomial Regression is used when the relationship is **not a straight line**.
- For salary prediction:
 - Salary usually does **not increase linearly** with experience.
- Example:
 - 1 year $\rightarrow$ ₹3 LPA
 - 3 years $\rightarrow$ ₹5 LPA
 - 5 years $\rightarrow$ ₹9 LPA
 - 10 years $\rightarrow$ ₹25 LPA
 - 15 years $\rightarrow$ ₹60 LPA
- Notice:
- Salary increases faster as experience grows.
- A straight line (Linear Regression) may fail.
- So we use **Polynomial Regression**.

Polynomial Regression

- Predicted Salary: 37.05

R2 Score: 0.9927

Accuracy Percentage: 99.27

MSE: 4.77

RMSE: 2.18

MAE: 1.69

Intercept: 4.258

Coefficients:

[0.0, 0.0919, 0.2358, -0.0013]

Polynomial Regression

- **1. Model Equation**
- Since degree = 3, your polynomial model becomes:
- $Salary = 4.258 + 0.0919x + 0.2358x^2 - 0.0013x^3$
- Where:
- **x = Experience Years**
- Salary = predicted salary (LPA)
- Meaning:
- Salary depends on:
 - experience
 - experience²
 - experience³
- This creates a **curve relationship**, not a straight line.

Polynomial Regression

- **2. Predicted Salary**
- You predicted:
- 12 years experience $\rightarrow$ 37.05 LPA
- Interpretation:
- According to historical salary trend, someone with **12 years experience** is expected to earn approximately **37 LPA**
- Business meaning:
- This prediction comes from:
 - growth trend
 - non-linear salary increase
 - curve fitting

Polynomial Regression

- **3. R^2 Score Interpretation (MOST IMPORTANT)**

- You got:
- $R^2 = 0.9927$

- Meaning:

- Model explains **99.27% of salary variation**

- This is extremely high.

- Interpretation scale:

- Your model:

-  **Excellent fit**

- Meaning:

- Experience strongly explains salary in your dataset.

R^2	Meaning
>0.95	Excellent
$0.90-0.95$	Very Good
$0.80-0.90$	Good
$0.70-0.80$	Moderate
<0.70	Weak

Polynomial Regression

- **4. “Accuracy Percentage” Interpretation**
- You calculated:
 - 99.27%
- This comes from:
 - $R^2 \times 100$
- Technically:
 - Regression doesn't have classification accuracy.
- But for teaching/demo:
- You may say:
 - “Model explains about 99% of salary behavior.”
 - Good for understanding.

Polynomial Regression

- **5. RMSE Interpretation**
- You got:
 - $RMSE = 2.18$
- Meaning:
 - On average, salary prediction error is around **2.18 LPA**
- Example:
 - Actual salary = 35
- Prediction may be:
 - 32.8 or 37.2
- Small error.
- So:
 -  Good model.

Polynomial Regression

- **6. MAE Interpretation**

- You got:
 - $MAE = 1.69$
- Meaning:
 - Average prediction difference $\approx$ **1.69 LPA**
- This is easier to explain than RMSE.
- Simple English:
 - “On average, prediction differs from actual salary by about 1.7 LPA.”
 - Smaller = better.

Polynomial Regression

- **7. MSE Interpretation**
- You got:
 - $MSE = 4.77$
- Meaning:
- Squared prediction error.
- Less intuitive.
 - Mostly for optimization.
 - Lower is better.
- Since RMSE is only 2.18:
 -  acceptable.

Polynomial Regression

- **8. Coefficient Interpretation**
- Your coefficients:
- $[0.0, 0.0919, 0.2358, -0.0013]$
- Meaning:
- **x term**
- $0.0919 \times x$
- Small positive contribution.
- Salary increases with experience.

Polynomial Regression

- **x^2 term**
- $0.2358 \times x^2$
- Strong positive growth.
- Meaning:
- Salary grows faster as experience increases.
- This reflects reality.
- Senior people often get larger jumps.

Polynomial Regression

- **x^3 term**
- $-0.0013 \times x^3$
- Tiny negative curvature.
- Meaning:
- Salary growth slightly slows at very high experience.
- Also realistic.
- Nobody's salary grows infinitely.

Polynomial Regression

- **9. Is Model Too Good? (Important)**
- Because:
 - $R^2 = 99.27\%$
- We should ask:
 - Is this overfitting?
- In your case:
- Probably **No** because:
 - Dataset itself was generated using polynomial logic
 - Degree = 3 is reasonable
 - Metrics are consistent
- But in real HR datasets:
 - 99% is suspicious.
- Real salary prediction often:
 - $R^2 = 0.70-0.90$

Polynomial Regression

- The polynomial regression model performs extremely well with an R^2 score of 0.99, meaning it explains around 99% of salary variation.
- The RMSE of 2.18 indicates a small prediction error, and the model predicts around 37 LPA for a person with 12 years of experience.
- The quadratic term shows salary grows faster with experience, while the cubic term slightly controls excessive growth.

Ridge Regression

- Ridge regression is a **regularized linear regression** technique that addresses multicollinearity and overfitting by adding a penalty term to the loss function.

Why Ridge Regression

- Suppose your Linear Regression model is:
- $Y = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$
- If features are highly correlated:
 - Area and Number of Rooms
 - Salary and Years of Experience
 - Temperature and Humidity
- the coefficients can become very large and unstable.
- Ridge Regression solves this by adding a penalty term.

Ridge Regression

- Linear Regression minimizes:
- $RSS = \sum (y_i - \hat{y}_i)^2$
- **One goal only** — minimize prediction error.
- It finds coefficients that make predictions as close to actual values as possible. There are **no restrictions** on how large the coefficients can grow.
- **The Problem**
- When features are correlated or there are many predictors, OLS can produce **very large coefficients** to fit the training data perfectly — this is **overfitting**.

Ridge Regression

- Linear Regression minimizes:
- $RSS = \sum (y_i - \hat{y}_i)^2$
- **One goal only** — minimize prediction error.
- It finds coefficients that make predictions as close to actual values as possible. There are **no restrictions** on how large the coefficients can grow.
- **The Problem**
- When features are correlated or there are many predictors, OLS can produce **very large coefficients** to fit the training data perfectly — this is **overfitting**.

OLS- Ordinary Least Squares (OLS) — Deep Dive ML

- **The Core Idea**
- OLS answers one question:
- **"What coefficients minimize the total squared prediction error?"**
- $$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

What is a Residual?

- For each data point, the **residual** is the gap between actual and predicted:

$$e_i = y_i - \hat{y}_i$$

- OLS minimizes the **sum of all squared residuals** — squaring because:
- Negative and positive errors don't cancel out
- Large errors are penalized more heavily than small ones

Ridge Regression

- Ridge minimizes:
- $RSS + \lambda \sum b_j^2$
- or
- $\sum (y_i - \hat{y}_i)^2 + \lambda \sum b_j^2$
- Where:
- RSS = Residual Sum of Squares
- λ (lambda/alpha) = Regularization strength
- b_j = model coefficients

Ridge Regression

- **Two goals simultaneously:**
- Minimize prediction error (RSS term)
- Keep coefficients small (penalty term)

Ridge Regression

• Breaking Down the Ridge Penalty

$$\underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{fit the data well}} + \lambda \underbrace{\sum_{j=1}^p b_j^2}_{\text{keep coefficients small}}$$

Part	Symbol	Role
RSS	$\sum (y_i - \hat{y}_i)^2$	Measures how well model fits training data
Penalty	$\lambda \sum b_j^2$	Punishes large coefficient values
Lambda	λ	Controls the tradeoff between the two

Ridge Regression

The Role of λ (Lambda / Alpha)

λ is the **tuning knob** that balances fit vs. simplicity:

$\lambda = 0$	$\rightarrow$	penalty vanishes	$\rightarrow$	Ridge becomes OLS (no regularization)
λ small	$\rightarrow$	mild shrinkage	$\rightarrow$	coefficients slightly pulled toward 0
λ large	$\rightarrow$	heavy shrinkage	$\rightarrow$	coefficients strongly pulled toward 0
$\lambda \rightarrow \infty$	$\rightarrow$	all $b_j \rightarrow 0$	$\rightarrow$	model predicts only the mean

λ is never negative. A negative λ would *reward* large coefficients — the opposite of regularization.

OLS – Ordinary Least Square Method

Ridge Regression

- **Why Square the Coefficients? (b_j^2)**
- Squaring makes all terms **positive** (so negative coefficients are also penalized)
- Squaring penalizes **large coefficients disproportionately** — a coefficient of 10 contributes 100 to the penalty, while 1 contributes only 1
- This creates a smooth, differentiable penalty — easy to optimize mathematically

Ridge Regression

- **1. Simple Meaning**
- Linear Regression tries to reduce prediction error.
- Ridge Regression does the same, but also says:
 - “Keep coefficients small.”
- So Ridge Regression controls model complexity.

Ridge Regression

- **2. Formula**

- Linear Regression cost:

- $Loss = MSE$

- Ridge Regression cost:

- $Loss = MSE + \alpha \sum w_i^2$

- Where:

- MSE = prediction error

- α / alpha = regularization strength

- w_i = coefficients

Ridge Regression

- **3. Why Ridge Is Needed**
- Suppose features are highly related:
 - TV Ads
 - Online Ads
 - Marketing Budget
- These may move together.
 - Linear Regression may create unstable coefficients.
 - Ridge reduces this problem by shrinking coefficients.

Ridge Regression

- 4. Alpha Meaning

Alpha	Meaning
0	Same as Linear Regression
Small alpha	Light penalty
Large alpha	Strong penalty

Ridge Regression

- R2 Score: 0.9412297156334417
- Accuracy Percentage: 94.12297156334417
- MSE: 31.179867364156546
- RMSE: 5.583893566693096
- MAE: 4.6550446190875725
- Intercept: -9.925163374301789
- Coefficient: [4.40506893]

Ridge Regression

- **1. Model Equation**
- Your Ridge Regression equation becomes:
- $Salary = -9.92 + 4.405(Experience)$
- Meaning:
- Salary depends linearly on experience.

Ridge Regression

- **2. R^2 Score Interpretation**
- You got:
- $R^2 = 0.9412$
- Meaning:
- Model explains **94.12% of salary variation**
- This is very good.

R^2	Meaning
>0.95	Excellent
$0.90-0.95$	Very Good
$0.80-0.90$	Good
<0.80	Weak

Ridge Regression

- **3. Compare with Polynomial Regression**
- Earlier polynomial regression:
 - $R^2 = 99.27\%$
RMSE = 2.18
- Now Ridge:
 - $R^2 = 94.12\%$
RMSE = 5.58
- Comparison:

Metric	Polynomial	Ridge
R^2	99.27%	94.12%
RMSE	2.18	5.58
MAE	1.69	4.65

Ridge Regression

- Meaning:
- Polynomial fits your salary dataset better.
- Why?
 - Because salary growth is curved.
- Ridge here is still linear:
 - Experience $\rightarrow$ Salary
- But salary usually behaves:
 - Experience $\rightarrow$ curved salary growth
- Hence polynomial wins.

Ridge Regression

- **4. RMSE Interpretation**
- You got:
 - $\text{RMSE} = 5.58$
- Meaning:
 - Average salary prediction error $\approx$ **5.58 LPA**
- Example:
 - Actual salary = 40
- Prediction may be:
 - 34.4 or 45.6
 - Not bad.
- But polynomial (2.18) is much better

Ridge Regression

- **5. MAE Interpretation**
- You got:
 - $\text{MAE} = 4.65$
- Meaning:
 - On average prediction differs by around **4.65 LPA**
- Lower is better.
- Polynomial:
 - $\text{MAE} = 1.69$
 - Again better.

Ridge Regression

- **6. Intercept Interpretation**
- You got:
 - Intercept = -9.92
- Meaning:
 - When experience = 0
- Predicted salary:
 - **≈ -9.92 LPA**
- This is unrealistic.

Ridge Regression

- Why?
- Because regression line mathematically extrapolates.
- In real life:
- Salary cannot be negative.
- This tells us:
- ⚠ linear model is not naturally fitting salary growth.
- Polynomial handled this better.

Ridge Regression

- **7. Coefficient Interpretation**
- You got:
 - Coefficient = 4.405
- Meaning:
 - For every extra year of experience:
- Salary increases by:
 - **≈ 4.4 LPA**
- Example:
 - 5 years more experience:
 - $5 \times 4.405 = 22.02$
- Expected salary increase **≈ 22 LPA**

Ridge Regression

- **8. Predicted Salary for 12 Years**
- You got:
 - 42.93 LPA
- Interpretation:
 - According to Ridge Regression, a person with **12 years experience** may earn around **43 LPA**
- Earlier Polynomial predicted:
 - 37.05 LPA
- Difference occurs because:
 - Polynomial captures curve
 - Ridge assumes straight line

Realtime Examples

- A better **real-time example** for Ridge Regression is where:
 - Many input variables are highly correlated (multicollinearity).
 - Salary vs experience is not the best real-world Ridge example.

Realtime Examples

- 1. House Price Prediction (Best Real-Time Example)

Feature	Meaning
House size	sq.ft
Number of rooms	bedrooms
Construction area	built-up area
Land area	plot size
Locality score	neighborhood quality

Realtime Examples

- Problem:
 - Many features are related.
- Example:
 - House Size ↑
Rooms ↑
Construction Area ↑
Land Area ↑
 - These move together.
- This causes:
 - **Multicollinearity**
- Linear Regression becomes unstable.
- Example:
 - Model may produce weird coefficients:
 - House Size = +1000
 - Rooms = -800
 - Construction Area = +900
 - Makes no business sense.
- Ridge fixes this.
 - It shrinks coefficients and stabilizes prediction.

Realtime Examples

- **2. Marketing / Advertisement Spend (Excellent Example)**
- Predict sales using:
 - TV ads
 - Radio ads
 - Digital marketing
 - Facebook ads
 - Instagram ads
 - Google ads
- Problem:
 - These are correlated.
- If company increases marketing budget:
 - Everything increases.
- Linear regression struggles.
- Ridge helps.

Realtime Examples

- **3. Employee Salary Prediction (Corporate HR)**
- Predict salary using:
 - Experience
 - Skill score
 - Certification count
 - Project count
 - Performance score
 - Education
- Problem:
 - These are correlated.
- Example:
 - High experience → more projects
 - More projects → better performance
 - Better performance → higher salary
- Linear Regression may overfit.
- Ridge stabilizes.

Realtime Examples

- **4. Medical Insurance Cost Prediction (Classic)**
- Predict insurance premium using:
 - Age
 - BMI
 - Smoking
 - Medical history
 - Blood pressure
 - Cholesterol
- Some health variables are correlated.
- Ridge reduces instability.

Realtime Examples

- **5. Banking Loan Risk (Very Common)**
- Predict loan default using:
 - Income
 - Credit score
 - Existing loans
 - Spending behavior
 - EMI ratio
 - Savings
- Financial variables are correlated.
- Ridge works well.

Realtime Examples

- **Why Ridge Is Used in Real Companies**
- When:
 - ✓ Many input features exist
 - ✓ Features are related to each other
 - ✓ Overfitting happens
 - ✓ Coefficients become unstable
- Use:
- Ridge Regression

Realtime Examples

- **Best Interview Example**
- **House price prediction using correlated features such as house size, rooms, built-up area, and locality is a strong real-world example of Ridge Regression because these variables are highly correlated and Ridge helps reduce coefficient instability and overfitting.**

Lasso Regression

- **Lasso (Least Absolute Shrinkage and Selection Operator)** is an advanced version of Linear Regression.
- It is used when:
- We have many features and want the model to automatically identify the most important ones.
- **Predicting continuous values** (like Linear Regression)
- **Automatic feature selection** (its biggest advantage)

Lasso Regression

- **1. Why Lasso Regression?**
- Suppose we predict house prices using:
- Area
 - Bedrooms
 - Bathrooms
 - Parking
 - Age
 - Distance to City
 - School Rating
 - Crime Rate
 - Bus Stop Distance
 - Hospital Distance
- Not all features are useful.
- Lasso automatically removes unimportant features.

Lasso Regression

- **2. Formula**

- Linear Regression minimizes:

- $Loss = MSE$

- Lasso Regression minimizes:

- $Loss = MSE + \alpha \sum |w_i|$

- Where:

- MSE = Mean Squared Error

- α (alpha) = penalty strength

- $|w_i|$ = absolute value of coefficients

- This is called **L1 Regularization**.

Lasso Regression

- **Main Idea**
- Lasso pushes less important coefficients toward zero.
- Example:
- Before Lasso:

Feature	Coefficient
Area	2500
Bedrooms	120000
Bathrooms	60000
Bus Stop Distance	50
Railway Distance	20

Lasso Regression

- **Main Idea**
- **After Lasso:**

Feature	Coefficient
Area	2400
Bedrooms	115000
Bathrooms	58000
Bus Stop Distance	0
Railway Distance	0

Lasso removed:
Bus Stop Distance
Railway Distance

Lasso Regression

- **Model Performance**
- You got:
- R^2 Score = 0.9891
- Meaning:
- The model explains **98.91% of the variation in house prices.**
- Interpretation scale:

R^2 Score	Interpretation
> 0.95	Excellent
0.90 - 0.95	Very Good
0.80 - 0.90	Good
< 0.80	Needs Improvement

Lasso Regression

- **Model Performance**
- Your model:
-  **98.91% = Excellent**
- This means the selected property features explain almost all house price variations.

Lasso Regression

- **House Price Equation**
- Your Lasso model learned:
- #additional cost required for area say 100 sqft 4434, 1 additional bedroom 340704 and so on.,
- Price =
293097
+ 4434 × Area
+ 340704 × Bedrooms
+ 231550 × Bathrooms
+ 164672 × Parking
- 45234 × Property_Age
+ 5400 × School_Distance

Lasso Regression

- **Intercept Interpretation**

- Intercept = 293097
- Meaning:
- If all features are zero:
 - Area = 0
 - Bedrooms = 0
 - Bathrooms = 0
 - Parking = 0
 - Property Age = 0
 - School Distance = 0
- Base price:
 - ₹2,93,097
- In practice this has little business meaning because such a house cannot exist.
- Intercept is mainly a mathematical starting point.

Lasso Regression

- **Feature Importance Interpretation**
- **Area_SqFt**
 - 4434.26
- Meaning:
- For every additional 1 sq.ft:
 - House price increases by ₹4,434
- Example:
 - 100 sq.ft increase:
 - 100×4434
 - =
 - ₹4,43,400 increase
 - Area is highly important.

Lasso Regression

- **Bedrooms**
 - 340703.58
- Meaning:
- Adding one bedroom increases price by:
 - ₹3.41 Lakhs
- Keeping all other features constant.

Lasso Regression

- **Bedrooms**
 - 340703.58
- Meaning:
- Adding one bedroom increases price by:
 - ₹3.41 Lakhs
- Keeping all other features constant.

Lasso Regression

- **Did Lasso Remove Any Features?**
- Normally Lasso sets unimportant features to:
 - 0
- Your coefficients:
 - [4434
340703
231550
164672
-45234
5400]
- None are zero.
- Meaning:
 - Lasso found all six features useful.
 - No feature was removed.
 - This often happens when all features genuinely contribute to house price.

Lasso Regression

- **Which Feature Has Most Impact?**
- Ranking by coefficient magnitude:

Feature	Impact
Area_SqFt	Very High
Bedrooms	High
Bathrooms	High
Parking	Medium
Property_Age	Medium Negative
School_Distance	Very Low

Business conclusion:

Area, bedrooms, and bathrooms are the strongest drivers of house price.

- **Interview Answer**
- **The Lasso Regression model achieved an R^2 score of 98.91%, indicating an excellent fit.**
- **House area, bedrooms, bathrooms, and parking positively influence house prices, while property age negatively impacts price.**
- **Since none of the coefficients became zero, Lasso determined that all features contribute to house price prediction. 🚀**

Logistic Regression

- Despite the name “**Regression**”, Logistic Regression is actually used for:
- **Classification problems**
- It predicts categories like:
 - Yes / No
 - Pass / Fail
 - Spam / Not Spam
 - Disease / No Disease
 - Fraud / Not Fraud

Logistic Regression

- **1. Real-Time Example**
- Suppose we predict:
- Will a customer buy a product?
- Inputs:
 - Age
 - Salary
- Previous purchases
- Output:
 - 0 = No Purchase
 - 1 = Purchase
- This is Logistic Regression.

Logistic Regression

- **2. Why Not Linear Regression?**
- Suppose Linear Regression predicts:
 - 1.5
 - -0.4
 - 2.8
- Problem:
- Class labels must be:
 - 0 or 1
- Linear regression can produce impossible values.
- So Logistic Regression converts predictions into probabilities.

Logistic Regression

3. Logistic Regression Idea

- Instead of predicting directly:
- It predicts:
 - Probability between 0 and 1
- Example:
 - 0.92 $\rightarrow$ Fraud
 - 0.81 $\rightarrow$ Disease
 - 0.23 $\rightarrow$ Not Fraud
- Then threshold:
- Usually:
 - 0.5
- Rule:
- Probability $> 0.5 \rightarrow$ Class 1
- Probability $\leq 0.5 \rightarrow$ Class 0

Logistic Regression

- **4. Sigmoid Function (Heart of Logistic Regression)**
- Logistic regression uses a sigmoid curve.
- It converts any number to probability.
 - $P(y) = \frac{1}{1+e^{-x}}$
- Output always between:
 - 0 to 1

Score	Probability
-10	0.00
-2	0.11
0	0.50
2	0.88
10	1.00

Logistic Regression

- **5. Real-Time Examples**
- **Email Spam Detection**
- Input:
 - Email length
 - Suspicious words
 - Links count
- Output:
 - Spam = 1
 - Not Spam = 0

Logistic Regression

- **5. Real-Time Examples**
- **Medical Prediction**
- Input:
 - Age
 - Sugar level
 - BP
 - Cholesterol
- Output:
 - Disease = 1
 - Healthy = 0

Logistic Regression

- **5. Real-Time Examples**
- **Employee Attrition**
- Predict:
 - Employee leaves company?
- Output:
 - Leave = 1
 - Stay = 0

Logistic vs Linear Regression

Linear Regression	Logistic Regression
Predict numbers	Predict classes
Salary prediction	Pass/Fail
House price	Disease detection
Output any number	Output probability 0–1

Logistic vs Linear Regression

- **Interview Question**
- **Q: Why Logistic Regression for classification?**
- **Answer:**
- Logistic Regression is used for classification because it predicts probabilities between 0 and 1 using the sigmoid function, making it suitable for binary outcomes such as yes/no or pass/fail prediction.

Logistic Regression for Bank Loan

- Accuracy: 0.75

Loan Prediction: 1

Probability:

[[0.0858662 0.9141338]]

Logistic Regression for Bank Loan

- **1. Accuracy Interpretation**
- You got:
- Accuracy = 0.75
- Meaning:
- Model predicts correctly **75% of the time**

Accuracy	Meaning
>90%	Excellent
80–90%	Very good
70–80%	Good
60–70%	Moderate
<60%	Weak

Logistic Regression for Bank Loan

- So:
-  **75% = Good model**
- Business meaning:
- If bank processes:
 - 100 loan applications
- Model predicts correctly:
 - ≈ 75 applications
- Wrong predictions:
 - ≈ 25 applications
- Not bad for a small dataset (100 rows).

Logistic Regression for Bank Loan

- **2. Loan Prediction Interpretation**
- You got:
- Loan Prediction = 1
- Meaning:
- Loan Approved
- Because:
- 1 = Approved
0 = Rejected

Logistic Regression for Bank Loan

- **3. Probability Interpretation (MOST IMPORTANT)**

- Output:
 - $[[0.0858662 \ 0.9141338]]$
- This means:
 - Reject probability = $0.0858 = 8.58\%$
Approve probability = $0.9141 = 91.41\%$
- Since:
 - $91.41\% > 50\%$
- Prediction:
 - Approved (1)
- Simple English:
 - The bank model is **91% confident** this customer should receive loan approval.

Logistic Regression for Bank Loan

- **4. Why Accuracy Is Not Very High**
- Reasons:
- **Small dataset**
- Only:
 - 100 customers
- Machine learning usually performs better with:
 - 1000+
 - 10000+
- records.

Logistic Regression for Bank Loan

- **Interview Answer**
- **“The logistic regression model achieved 75% accuracy, meaning it predicts loan approval or rejection correctly for 75% of customers. For a customer with salary ₹75,000, credit score 740, and 5 years experience, the model predicts approval with around 91% confidence.”**

Lasso vs Logistic

Feature	Lasso Regression	Logistic Regression
Purpose	Predict continuous values	Predict categories
Output	Numeric value	Class (0/1)
Problem Type	Regression	Classification
Regularization	L1 Regularization	Classification algorithm
Example	Salary Prediction	Loan Approval
Output Example	₹12 LPA	Approved / Rejected
Formula Type	Linear Regression + L1	Sigmoid Function

Decision Tree Based Algorithms

-  **Big Picture: What is a Decision Tree?**
- **Decision Tree** is a supervised learning algorithm used for both classification and regression.
- Think of it as a **flowchart of “if–then” rules**:
 - Root node = starting question.
 - Branches = possible answers.
 - Leaf nodes = final decision/prediction.

The three things every decision tree needs

- Features — the information you give it
- These are the columns in your data table.
 - Income.
 - Age.
 - Credit score.
- Number of dependants.
 - The tree uses these to ask questions.
- In the morning example:
 - "Did alarm ring?",
 - "Is there traffic?" — those questions are based on features.

The three things every decision tree needs

- Labels — the answer you want to predict
- This is the one column in your data that says what actually happened.
 - Approved or Rejected.
 - Spam or Not Spam.
 - Will churn or Won't churn.
- The tree learns to predict this from the features.
- You must have labelled historical data to train a decision tree.

The three things every decision tree needs

- Training data — past examples to learn from
- This is a table of historical records where you already know both the features AND the label.
- The computer studies these examples to discover patterns — which features matter, and at what thresholds.
- The more examples, the better it learns.

Where decision trees are used every day



Bank loan approval

Income high enough? Credit score good?
Employment stable? Approve or reject in seconds.
Banks process millions of applications — humans cannot review each one manually.



Medical diagnosis support

Fever above 38°C? Cough present? Sore throat?
The tree narrows down likely conditions to help doctors triage patients faster, especially in busy clinics.



Email spam filter

Contains "win money"? Sender in unknown list? No subject line? Your email client makes this decision before you even open your inbox — thousands of times per day.



Customer churn prediction

Not logged in for 30 days? Raised a complaint? Usage dropped by 50%? The tree identifies customers at risk of cancelling — before they actually cancel.



Fraud detection

Transaction from a new country? Amount 10x the usual? Happening at 3am? Your bank's system raises a flag or blocks the transaction using a tree running in real time.



E-commerce recommendations

Bought a camera? Viewed tripods? Budget under ₹5,000? The tree predicts which product you are most likely to buy next and surfaces it on your screen.

What Problem Does a Decision Tree Solve?



- The business problem a bank faces
- A bank receives thousands of loan applications every day.
- Each application must be evaluated: should we lend this person money, or is the risk of them defaulting too high?
- Before machine learning, banks employed large teams of loan officers who would manually review each application — checking documents, making phone calls, and using their personal judgment.
- This process was slow, expensive, and inconsistent. Two different officers might reach different decisions on the same application.

What Problem Does a Decision Tree Solve?

The old problem: manually reviewing 10,000 applications per day requires ~500 loan officers. Each earns ₹8-12 lakh per year. That is ₹50+ crore annually just for the review process — before a single rupee of lending happens.

10,000+

Loan applications received daily by a mid-size bank

< 2 sec

Time a decision tree takes to evaluate one application

24 / 7

Availability — the model never sleeps, never has a bad day

What the bank wants to predict

Approve

The applicant is likely to repay the loan. Safe to lend.
Bank earns interest revenue.

Reject

The applicant is likely to default. Too risky. Bank avoids potential loss.

This is a classic binary classification problem — there are two possible output categories. The decision tree learns from thousands of past applications (where we know what happened) to predict outcomes for new ones.

Feature Selection

Features (inputs) the bank collects

- 1 Monthly income**
Higher income = more capacity to repay.
Measured in ₹ per month.
- 2 Credit score**
CIBIL score 300–900. History of past repayments. Higher = more trustworthy borrower.
- 3 Employment years**
Job stability. Longer employment = more stable income source.
- 4 Existing loan EMIs**
Current monthly debt obligations. High existing debt = less room for new repayment.
- 5 Loan amount requested**
How much the applicant wants to borrow.
Larger loans carry higher risk.
- 6 Age**
Proxy for financial maturity and career stage, though used carefully to avoid bias.

The label (what we want to predict)

Did this applicant fully repay the loan? **1 = Approved & repaid** / **0 = Defaulted or rejected**

Training Data

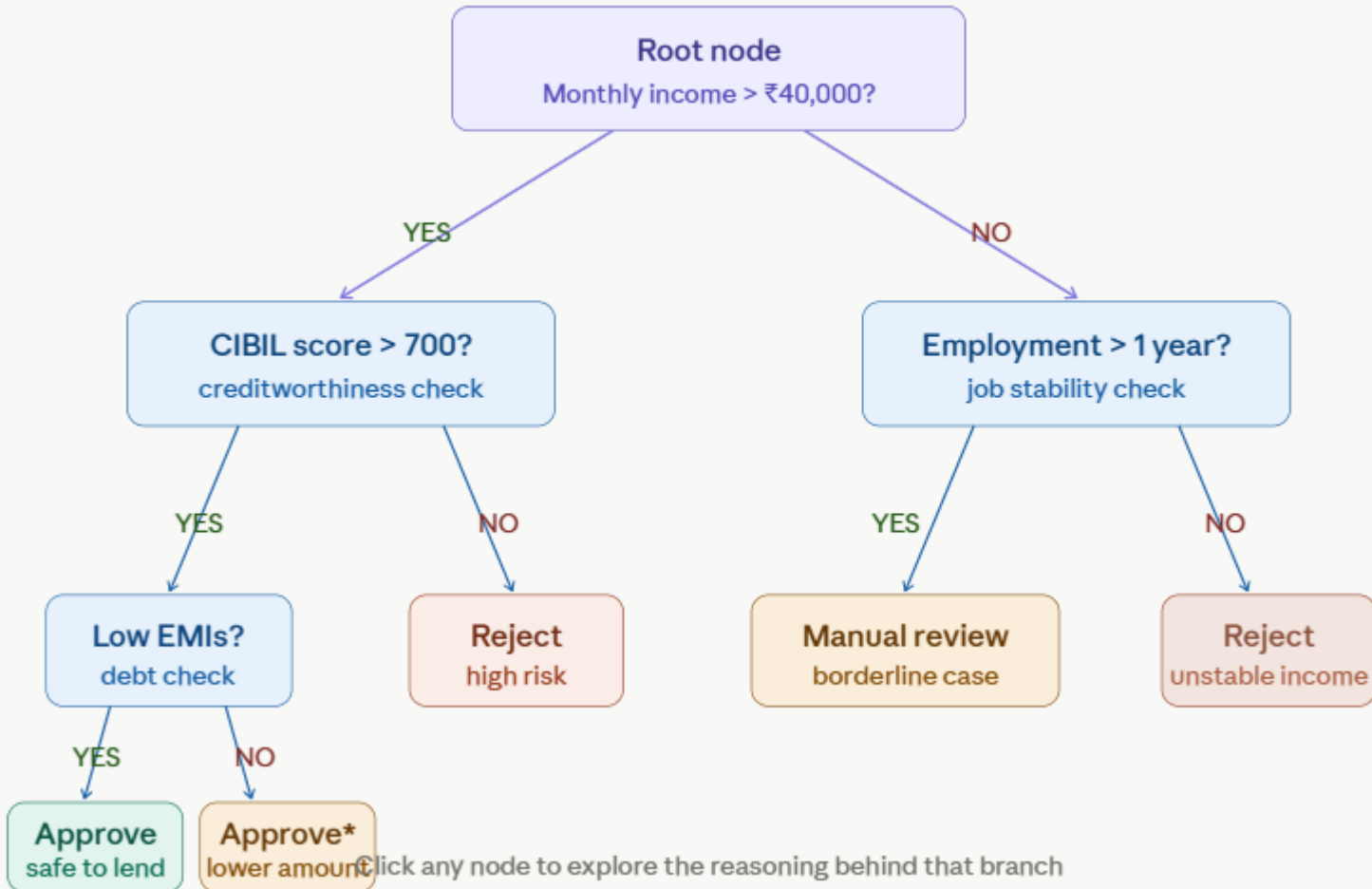
Sample rows from the training dataset

Applicant	Income (₹k)	CIBIL	Emp yrs	Loan ₹L	Label
Ravi Kumar	72	760	6	10	APPROVE
Meena Pillai	28	560	1	15	REJECT
Ajay Singh	95	810	10	25	APPROVE
Priya Nair	45	680	3	8	APPROVE*
Suresh Reddy	32	720	0	5	REJECT
Lakshmi Rao	58	740	8	12	APPROVE
Vikram Sharma	18	490	2	20	REJECT

* Priya's case is borderline — the tree will learn from these edge cases too

The bank feeds all 50,000+ historical records into the decision tree algorithm. The algorithm studies every row and figures out: "which questions, asked in which order, best separate the Approve rows from the Reject rows?"

Decision Tree Bank Learns



What the tree discovered from the data

- Income is the most important feature — it is asked first because it alone explains the most variation between approvals and rejections.
- The algorithm found this by testing all features and picking the one with the best separation.
- For high-income applicants, CIBIL score becomes the next most important question.
- Even a wealthy applicant with a history of missed payments is a risk.
- For low-income applicants, job stability matters most. Someone earning ₹30,000 with 5 years at the same employer is more reliable than someone earning ₹35,000 who just started a job 2 months ago.

Walk through 4 real applicant profiles

RK

Ravi Kumar

Age 34 · Income ₹75k · CIBIL 770 · Emp 7y

APPROVED



Income ₹75k > ₹40k?

YES — passes first check



CIBIL 770 > 700?

YES — excellent credit history



Existing EMI ₹6k is low?

YES — very low debt burden



Leaf reached

APPROVE — safe to lend

Why: Income well above threshold. Excellent CIBIL score. 7 years employment. Low existing debt. Strong candidate on all metrics.

Walk through 4 real applicant profiles

MP

Meena Pillai

Age 26 · Income ₹22k · CIBIL 540 · Emp 1y

REJECTED

- X Income ₹22k > ₹40k?**
NO — fails first check
- X Employment > 1 year?**
Exactly 1 year — borderline NO
- X Leaf reached**
REJECT — unstable income profile

Why: Income below threshold. CIBIL score far below minimum. Only 1 year employment. High loan amount relative to income. Multiple risk signals.

Walk through 4 real applicant profiles

AS

Ajay Singh

Age 42 · Income ₹95k · CIBIL 820 · Emp 14y

APPROVED



Income ₹95k > ₹40k?

YES — well above threshold



CIBIL 820 > 700?

YES — outstanding credit history



EMI ₹25k vs income ₹95k ratio OK?

Debt-to-income = 26% — acceptable



Leaf reached

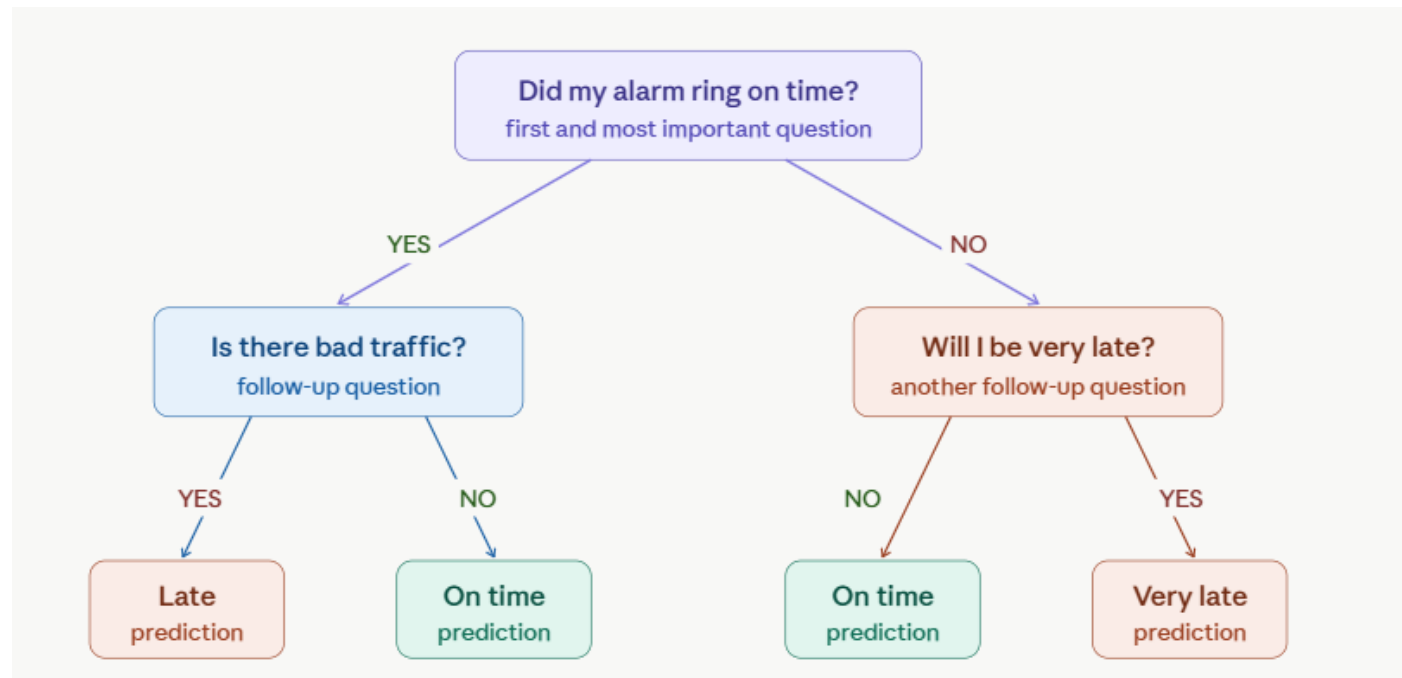
APPROVE — strong profile

Why: High income. Outstanding CIBIL score. 14 years stable employment. Despite higher EMIs, income comfortably covers all obligations. Premium customer.

What Problem Does a Decision Tree Solve?



- The morning routine — your first mental model
- Here is a tree that anyone on the team already uses every morning without realising it.
- Read through it top to bottom:



How Does a Tree Learn?

- The algorithm asks:
 - Which feature best separates approved and rejected loans?
- Possible features:
 - Income
 - Credit Score
 - Employment Status
 - Age
 - Existing Debt
- The tree evaluates each feature.
- The feature producing the purest groups becomes the split.

The problem: how does the tree know which question to ask first?



- In our bank loan tree, the algorithm asks "Is income > ₹40,000?" at the root — the very first question.
- But why income? Why not CIBIL score first? Why not employment years?
- The answer is Gini Impurity and Information Gain.
- These are the two numbers the algorithm calculates for every possible split, and picks the one with the highest gain.

Our Bank Loan DataSet

Our bank loan dataset — 10 applicants

We will use a small, simple dataset so every number is traceable by hand. The algorithm does the same thing — just on 50,000 rows instead of 10.

#	Applicant	Income (₹k)	CIBIL	Emp yrs	Decision
1	Ravi	75	770	7	APPROVE
2	Meena	22	540	1	REJECT
3	Ajay	95	820	14	APPROVE
4	Priya	47	690	3	REJECT
5	Suresh	32	720	0	REJECT
6	Lakshmi	58	740	8	APPROVE
7	Vikram	18	490	2	REJECT
8	Deepa	65	760	5	APPROVE
9	Kiran	38	610	2	REJECT
10	Anita	82	800	11	APPROVE

10

Total applicants

5

Approved (Ravi, Ajay, Lakshmi,
Deepa, Anita)

5

Rejected (Meena, Priya, Suresh,
Vikram, Kiran)

The dataset is perfectly balanced — 5 approved, 5 rejected. This is the worst starting point for a tree: maximum uncertainty. The algorithm will look for the split that reduces this uncertainty the most.

What does "impurity" mean? The jar analogy

- Imagine two jars of loan application files.
- Pick one file at random from each jar and try to guess if it is an approval or rejection.

Jar A — all approvals



5 green (approve), 0 red

Jar B — perfectly mixed



3 green, 3 red — 50/50

Pick any file — you know it is approved. Zero uncertainty.
This is **pure**. Impurity = 0.

Pick any file — you have no idea. Maximum uncertainty.
This is **maximally impure**. Impurity is at its highest.

The goal of every split in a decision tree is to turn one impure jar into two jars that are each more pure than the original. A perfect split creates one all-green jar and one all-red jar.

What does "impurity" mean? The jar analogy

Our starting node — the whole bank dataset

Before any split, we have all 10 applicants in one node. 5 approved (green) and 5 rejected (red).



10 applicants: 5 approve ($p = 0.5$), 5 reject ($p = 0.5$)

This is maximally impure — the algorithm has no idea which way to go yet

The algorithm will now try splitting this group in every possible way and measure: which split reduces impurity the most? That becomes the root node question.

Two splits the algorithm will test:

Split A

Income > ₹40k?

Left: Ravi, Ajay, Priya, Lakshmi, Deepa, Anita (6)

Right: Meena, Suresh, Vikram, Kiran (4)

Split B

CIBIL > 700?

Left: Ravi, Ajay, Suresh, Lakshmi, Deepa, Anita (6)

Right: Meena, Priya, Vikram, Kiran (4)

We will calculate Gini and Entropy for both splits and see which wins.



What is the Gini Index?

- The **Gini Index** (also called Gini Impurity) is a metric used by Decision Trees to decide **which feature to split on**.
- It measures how **pure** a node is:
 - A node is **pure** if all samples belong to one class (e.g., all “Yes” or all “No”).
 - A node is **impure** if samples are mixed across classes.

Gini Steps

Step 1 — Gini of the root node (before any split)

All 10 applicants together. 5 approved, 5 rejected.

$$p_{\text{approve}} = 5/10 = 0.5$$

$$p_{\text{reject}} = 5/10 = 0.5$$

$$\begin{aligned} \text{Gini}(\text{root}) &= 1 - (0.5^2 + 0.5^2) \\ &= 1 - (0.25 + 0.25) \\ &= 1 - 0.50 \\ &= 0.50 \quad \leftarrow \text{maximum impurity (worst possible)} \end{aligned}$$

Gini = 0.5 means we are completely in the dark. If we had to guess, we would be right only 50% of the time — no better than a coin flip.

Gini Steps

Step 1 — Gini of the root node (before any split)

All 10 applicants together. 5 approved, 5 rejected.

$$p_{\text{approve}} = 5/10 = 0.5$$

$$p_{\text{reject}} = 5/10 = 0.5$$

$$\begin{aligned}\text{Gini}(\text{root}) &= 1 - (0.5^2 + 0.5^2) \\ &= 1 - (0.25 + 0.25) \\ &= 1 - 0.50 \\ &= 0.50 \quad \leftarrow \text{maximum impurity (worst possible)}\end{aligned}$$

Gini = 0.5 means we are completely in the dark. If we had to guess, we would be right only 50% of the time — no better than a coin flip.

Gini Steps



Step 2 — Gini after Split A: Income > ₹40,000

Let's see what happens to our 10 applicants when we split on ₹40k income threshold.

Left node — Income > ₹40k (6 applicants)



5 APPROVE (Ravi, Ajay, Lakshmi, Deepa, Anita)
1 REJECT (Priya ₹47k)

Right node — Income ≤ ₹40k (4 applicants)



0 APPROVE
4 REJECT (Meena, Suresh, Vikram, Kiran)

Gini Steps

LEFT node (6 samples: 5 approve, 1 reject):

$$p_{\text{approve}} = 5/6 = 0.833$$

$$p_{\text{reject}} = 1/6 = 0.167$$

$$\begin{aligned} \text{Gini}(\text{left}) &= 1 - (0.833^2 + 0.167^2) \\ &= 1 - (0.694 + 0.028) \\ &= 1 - 0.722 \\ &= 0.278 \quad \leftarrow \text{much better than } 0.5! \end{aligned}$$

RIGHT node (4 samples: 0 approve, 4 reject):

$$p_{\text{approve}} = 0/4 = 0.0$$

$$p_{\text{reject}} = 4/4 = 1.0$$

$$\begin{aligned} \text{Gini}(\text{right}) &= 1 - (0.0^2 + 1.0^2) \\ &= 1 - (0 + 1) \\ &= 0.000 \quad \leftarrow \text{PERFECT! Completely pure.} \end{aligned}$$

$$\begin{aligned} \text{Weighted Gini (Split A)} &= (6/10) \times 0.278 + (4/10) \times 0.000 \\ &= 0.1668 + 0.000 \\ &= 0.1668 \end{aligned}$$

The right node is perfectly pure — all rejections! The algorithm loves this. The weighted Gini dropped from 0.5 all the way to 0.1668.

Gini Steps

Step 3 — Gini after Split B: CIBIL > 700

Left — CIBIL > 700 (6 applicants)



5 APPROVE, 1 REJECT (Suresh CIBIL 720)

Right — CIBIL ≤ 700 (4 applicants)



0 APPROVE, 4 REJECT (Meena, Priya, Vikram, Kiran)

LEFT node (6 samples: 5 approve, 1 reject):

$$\begin{aligned} \text{Gini}(\text{left}) &= 1 - ((5/6)^2 + (1/6)^2) \\ &= 1 - (0.694 + 0.028) = 0.278 \end{aligned}$$

RIGHT node (4 samples: 0 approve, 4 reject):

$$\text{Gini}(\text{right}) = 1 - (0^2 + 1^2) = 0.000$$

$$\begin{aligned} \text{Weighted Gini (Split B)} &= (6/10) \times 0.278 + (4/10) \times 0.000 \\ &= 0.1668 \end{aligned}$$

Split A (income) and Split B (CIBIL) produce the same weighted Gini of 0.1668! In a real bank dataset with thousands of rows, income would pull ahead. Here ties are broken by other rules (e.g. left-branch purity or random state).

- **Entropy in Decision Trees is a measure of uncertainty or disorder in a dataset, borrowed from information theory.**
- **It helps the tree decide the best feature to split on by quantifying how mixed the class distribution is.**
- In practice, lower entropy means purer nodes, while higher entropy signals the need for further splitting.

Definition of Entropy

- **Origin:** From information theory, used to measure randomness or unpredictability.
- **Formula:**

$$Entropy = - \sum_{i=1}^k p_i \log_2(p_i)$$

where p_i is the probability of class i .

- **Range:**
 - **0** → perfectly pure (all samples same class).
 - **High (>0.5)** → mixed classes, high uncertainty.

Step 1 — Entropy of the root node

Root: 5 approve, 5 reject ($p = 0.5$ each)

$$\begin{aligned}\text{Entropy}(\text{root}) &= -(0.5 \times \log_2(0.5)) - (0.5 \times \log_2(0.5)) \\ &= -(0.5 \times -1.0) - (0.5 \times -1.0) \\ &= 0.5 + 0.5 \\ &= 1.0 \quad \leftarrow \text{maximum possible entropy}\end{aligned}$$

Note: $\log_2(0.5) = \log_2(1/2) = -1$ (because $2^{-1} = 0.5$)

Entropy = 1.0 is the worst case — exactly as bad as a coin flip. Every bit of the dataset is maximally uncertain. The algorithm needs to find a split that reduces this toward 0.

Step 2 — Entropy after Split A: Income > ₹40k

LEFT node (6 samples: 5 approve, 1 reject):

$$p_{\text{approve}} = 5/6 = 0.8333$$

$$p_{\text{reject}} = 1/6 = 0.1667$$

$$\begin{aligned}\text{Entropy}(\text{left}) &= -(0.8333 \times \log_2(0.8333)) - (0.1667 \times \log_2(0.1667)) \\ &= -(0.8333 \times -0.2630) - (0.1667 \times -2.5850) \\ &= 0.2192 + 0.4308 \\ &= 0.6500\end{aligned}$$

[How to read: $\log_2(0.8333) \approx -0.263$ | $\log_2(0.1667) \approx -2.585$]

RIGHT node (4 samples: 0 approve, 4 reject):

$$p_{\text{approve}} = 0, \quad p_{\text{reject}} = 1.0$$

$$\text{Entropy}(\text{right}) = 0 \quad \leftarrow \text{rule: } 0 \times \log_2(0) = 0 \text{ (by convention)}$$

Weighted Entropy (Split A):

$$\begin{aligned}&= (6/10) \times 0.6500 + (4/10) \times 0.000 \\ &= 0.3900 + 0.000 \\ &= 0.3900\end{aligned}$$

Step 3 — Entropy after Split B: CIBIL > 700

LEFT node (6 samples: 5 approve, 1 reject):

Entropy(left) = 0.6500 (same proportions as Split A left)

RIGHT node (4 samples: 0 approve, 4 reject):

Entropy(right) = 0.0000

Weighted Entropy (Split B):

= $(6/10) \times 0.6500 + (4/10) \times 0.0000$

= 0.3900

Again both splits tie at 0.39 on this small dataset. With 50,000 real rows, income would edge ahead because the income threshold separates the groups more cleanly across the full distribution.

Information Gain

- Information Gain = Impurity(parent) – Weighted Impurity(children)
- Using Gini:
 - $IG = \text{Gini}(\text{parent}) - \text{Weighted Gini}(\text{children})$
- Using Entropy:
 - $IG = \text{Entropy}(\text{parent}) - \text{Weighted Entropy}(\text{children})$
- Higher Information Gain = better split = chosen by the algorithm

Information Gain

Information Gain for our two splits

Split A — Income > ₹40k

Using Gini:

$$IG = 0.5000 - 0.1668 = 0.3332$$

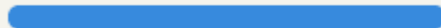
Using Entropy:

$$IG = 1.0000 - 0.3900 = 0.6100$$

Gini Gain: 0.3332



Entropy Gain: 0.6100



Split B — CIBIL > 700

Using Gini:

$$IG = 0.5000 - 0.1668 = 0.3332$$

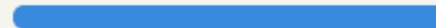
Using Entropy:

$$IG = 1.0000 - 0.3900 = 0.6100$$

Gini Gain: 0.3332



Entropy Gain: 0.6100



Both splits gain the same amount on our toy dataset. In production with 50,000 rows, income typically gains more because it separates high earners (who almost always repay) from low earners (who default at much higher rates) — a cleaner split than CIBIL alone.

Information Gain

What would a bad split look like?

Let's say the algorithm tried: Age > 30? (not a strong predictor of repayment)

Hypothetical bad split – Age > 30:

Left (Age > 30): 3 approve, 3 reject → Gini = 0.5, Entropy = 1.0

Right (Age ≤ 30): 2 approve, 2 reject → Gini = 0.5, Entropy = 1.0

Weighted Gini = $(6/10) \times 0.5 + (4/10) \times 0.5 = 0.5000$

Weighted Entropy = $(6/10) \times 1.0 + (4/10) \times 1.0 = 1.0000$

Gini IG = $0.5000 - 0.5000 = 0.0000$ ← zero gain!

Entropy IG = $1.0000 - 1.0000 = 0.0000$ ← zero gain!

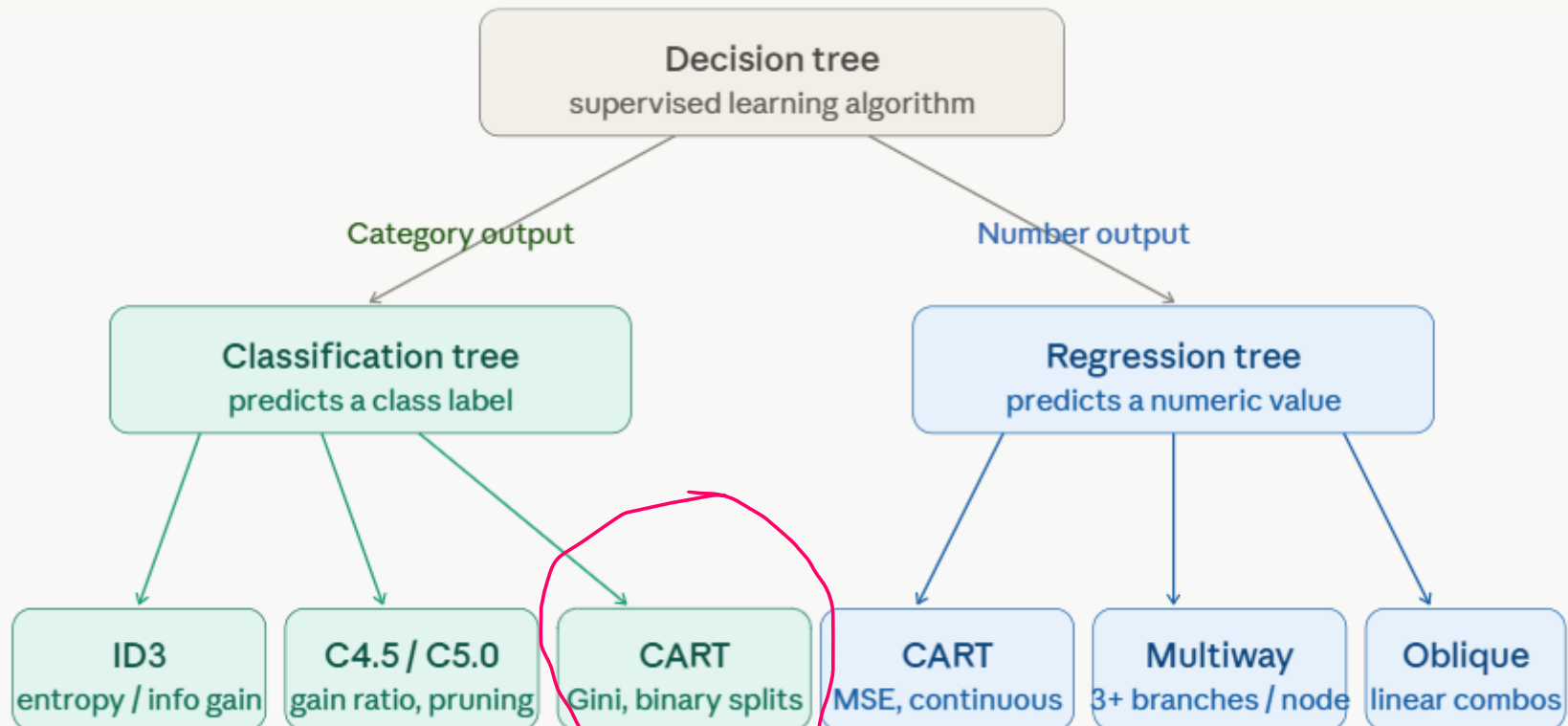
Age gains nothing. The split is useless. Each child group is just as mixed as the parent. The algorithm would never choose this split — income and CIBIL are far better.

Information Gain

The complete picture — how the root node is chosen

- 1 For every feature (income, CIBIL, employment, EMI, loan amount), the algorithm tries every unique threshold value as a potential split point.
- 2 For each candidate split, it calculates Gini or Entropy for both child nodes and computes the weighted average.
- 3 It subtracts the weighted child impurity from the parent impurity to get Information Gain.
- 4 The split with the highest Information Gain becomes the node's question. On a dataset of 50,000 rows and 6 features, this may evaluate thousands of candidate splits per node.

Types of Decision Trees



Deep Discussion

Types of Decision Trees

Algorithm	Key Feature	Use Case
ID3	Uses <i>Entropy & Information Gain</i> for splits	Works best with categorical data
C4.5	Extension of ID3, handles continuous data	General-purpose classification
CART	Uses <i>Gini Index</i> or variance reduction	Both classification & regression
CHAID	Uses chi-square tests for splits	Marketing, customer segmentation

Decision Tree Cart

- Your output:
 - ['No' 'Yes']
Accuracy: 1.0
- **What does ['No' 'Yes'] mean?**
- This is likely the unique values found in your target column:
- `print(df['LoanApproved'].unique())`
- Output:
 - ['No' 'Yes']
 - Meaning:
 - **No** = Loan Rejected
 - **Yes** = Loan Approved
 - This is your classification target.

Decision Tree Cart

- **What does Accuracy = 1.0 mean?**
- Accuracy: 1.0
- Accuracy is calculated as:
- $Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$
- An accuracy of **1.0** means:
- $1.0 = 100\%$
- The model correctly predicted every test record.

Should you celebrate?

- Not immediately. 🚨
- With Decision Trees, **100% accuracy can sometimes indicate overfitting.**
- **Scenario 1: Small Dataset**
- If your dataset has only:
 - 10 records
 - 20 records
 - 50 records
- then 100% accuracy is common.
- The tree may simply memorize the data.

Should you celebrate?

- **Scenario 2: Data Leakage**

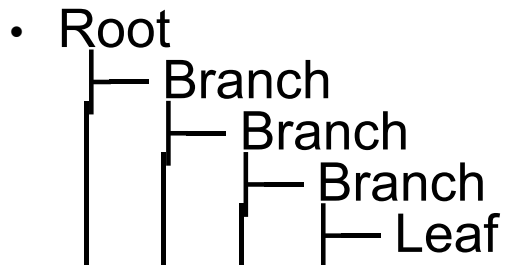
- Example:

- Income
CreditScore
LoanApproved

- If one feature directly reveals the answer, accuracy becomes artificially high.

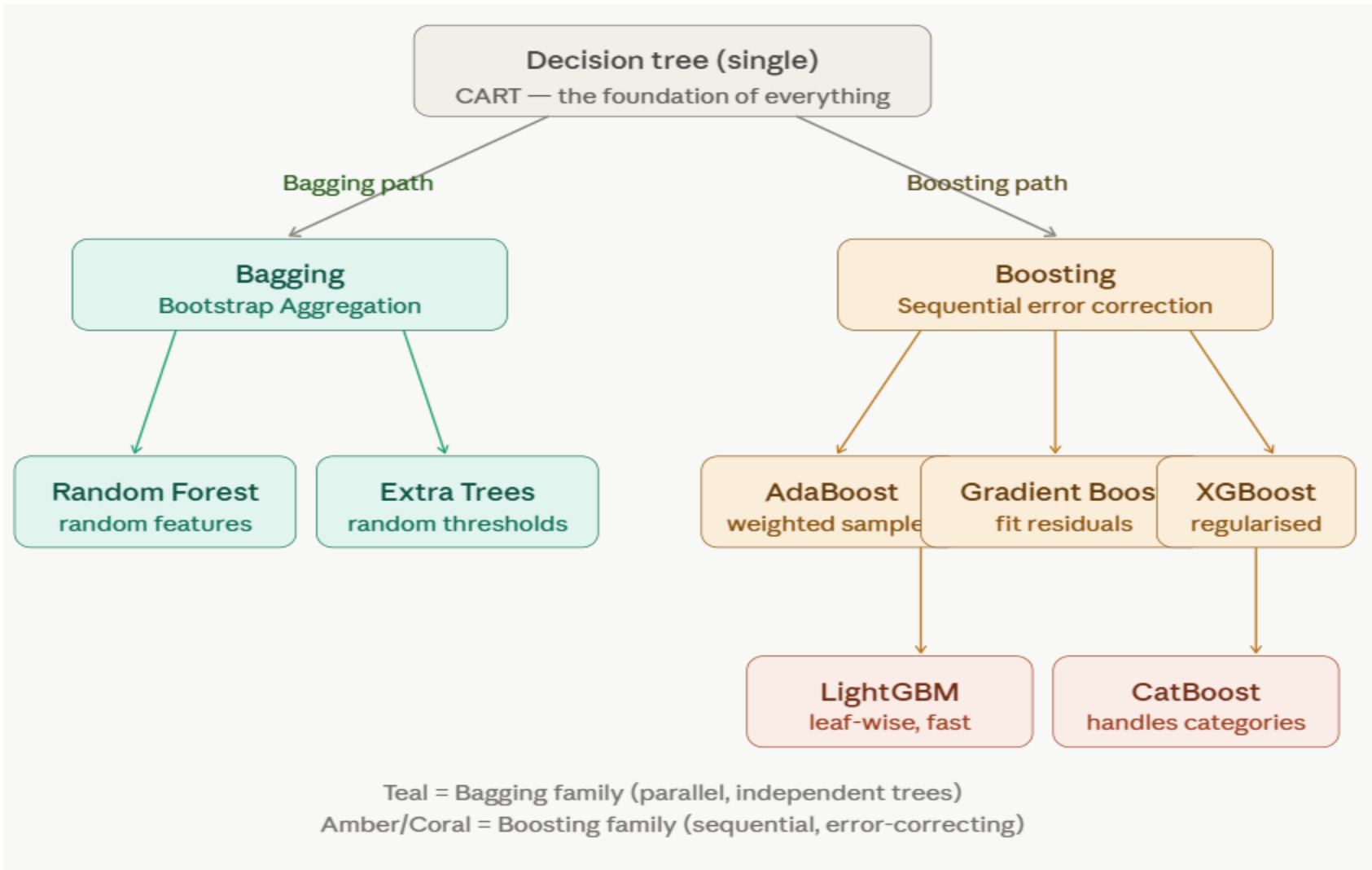
- **Scenario 3: Overfitting**

- Tree becomes too deep:



- The model memorizes training patterns instead of learning general rules.

Supervised Learning Tree Family



The two major strategies

- Bagging — train trees in parallel
- Each tree sees a different random sample of the data.
- Final prediction = vote or average.
- Reduces variance (overfitting).
- Stable and fast.

The two major strategies

- Boosting — train trees sequentially
- Each tree fixes the errors of the previous one.
- Final prediction = weighted sum.
- Reduces bias (underfitting).
- Higher accuracy, slower training.

Random Forest Tree

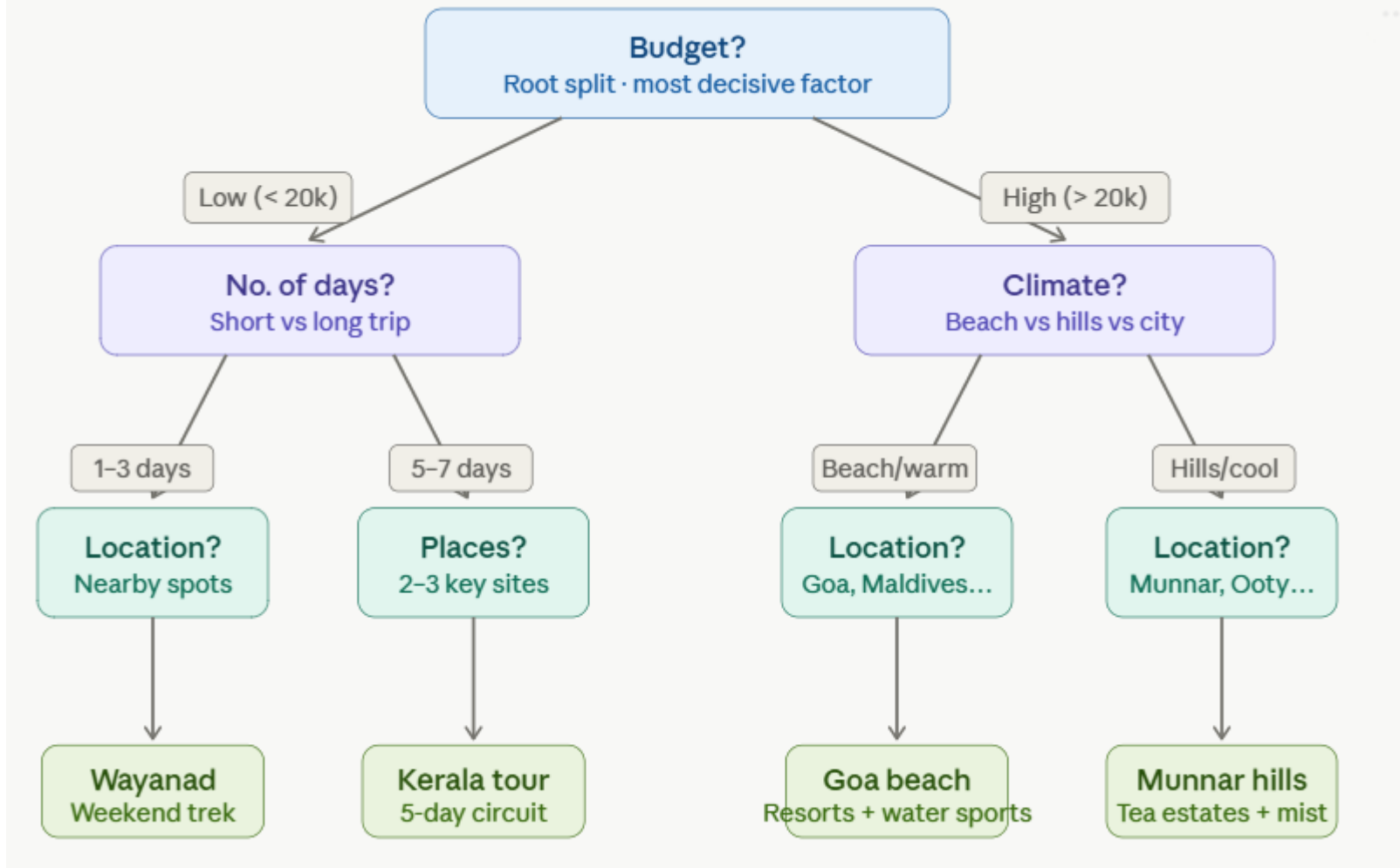
- Random Forest is an ensemble learning method that builds many decision trees and combines their predictions — hence the "forest."
- It was introduced by Leo Breiman in 2001.

Random Forest Tree

- **The Problem: The Single Decision Tree**
- A decision tree is a flowchart that splits data to reach a conclusion
- It is easy to interpret but often overfits by memorizing noise in the data
- High sensitivity: small changes in input can lead to vastly different outcomes

Random Forest Tree

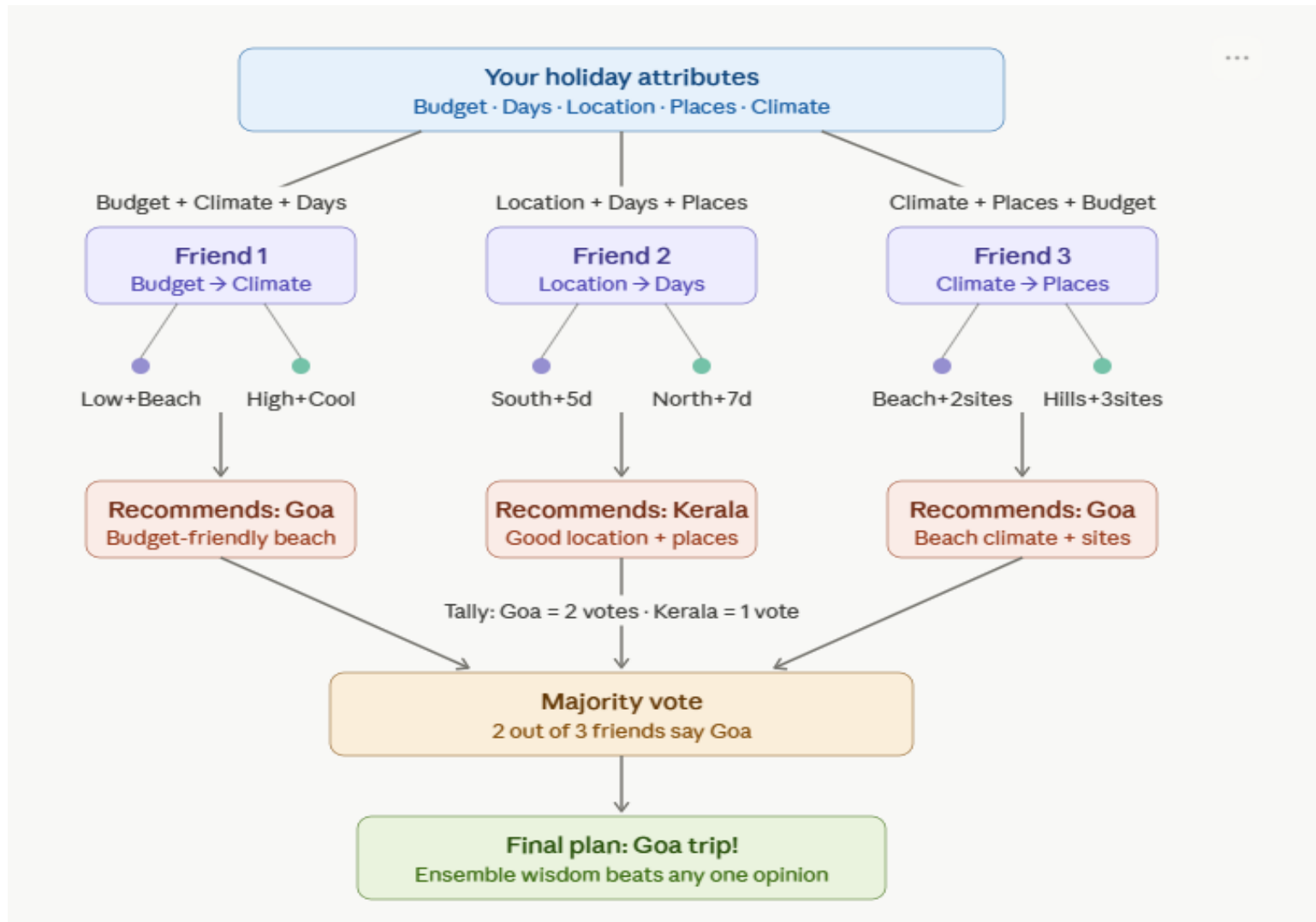
Decision Tree, where YOU are the single expert making all the decisions in order:



Random Forest Tree

- **The Solution: The Wisdom of the Crowd**
- Random Forest is an ensemble method: it builds many trees and combines them
- Concept of Diversification: each tree is trained on a slightly different subset of data
- Instead of one expert, we use a committee of diverse trees to vote on the result.
- In our Holiday plan example, if we consider Random Forest, each friend is a separate "tree" who looks at a different random combination of your attributes, then everyone votes on the best destination:

Random Forest Tree



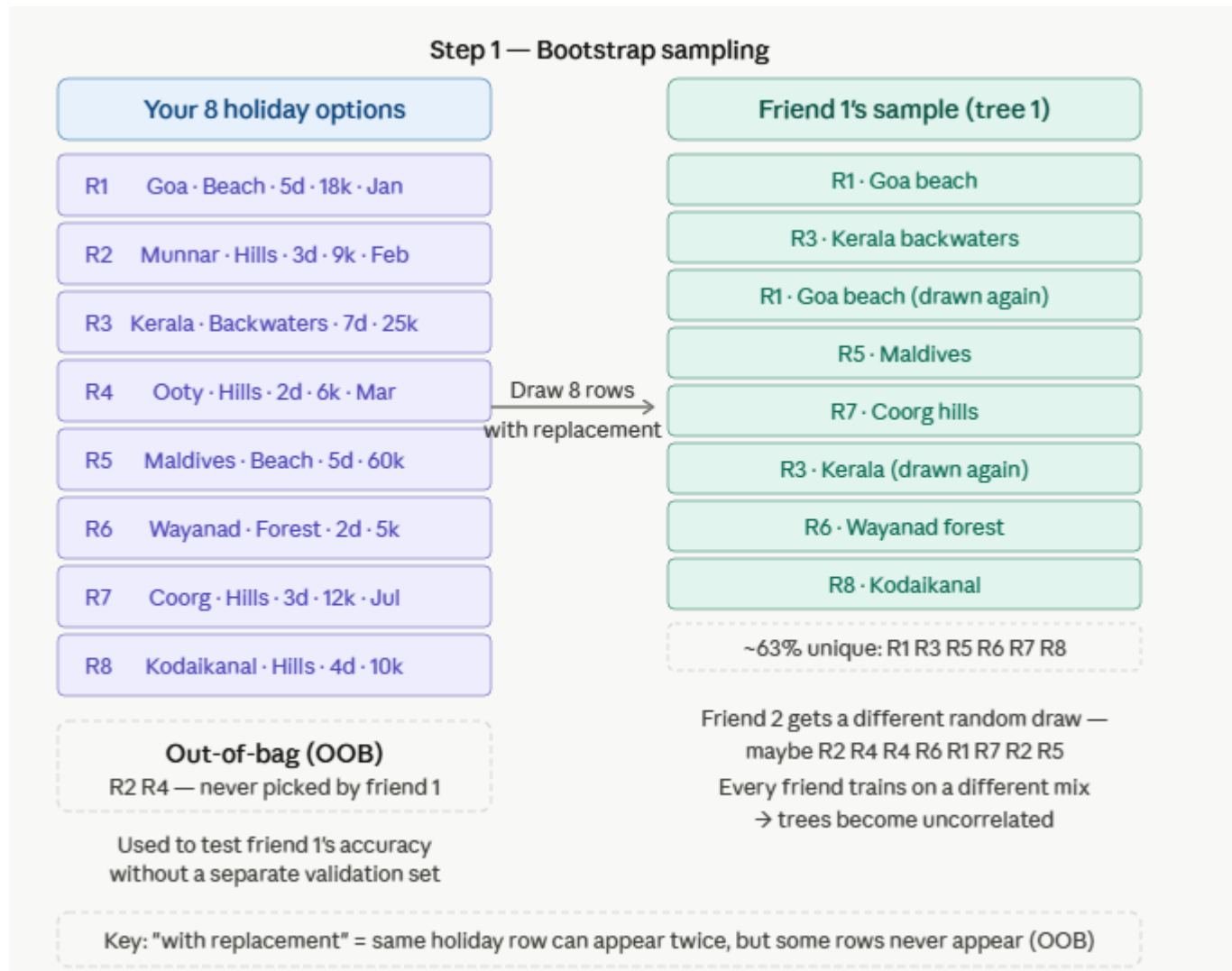
Decision Tree vs Random Forest Tree

Dimension	Decision Tree	Random Forest
No. of trees	1 tree	N trees (e.g. 100–500)
Training data	Full dataset	Bootstrap samples
Feature selection	All features at each split	Random subset per split
Overfitting	High risk	Low risk
Interpretability	Easily visualised	Black box
Speed	Very fast	Slower ($N \times$ tree cost)
Accuracy	Moderate	Higher (ensemble gain)
Best used when	Explainability needed, small dataset	Accuracy is priority, larger dataset

How it works (step by step):

- **Why it works so well:** A single deep decision tree has low bias but high variance (overfits).
- By averaging many such trees trained on different subsets, the variance cancels out while bias stays low — this is called the **bias-variance tradeoff** in action.

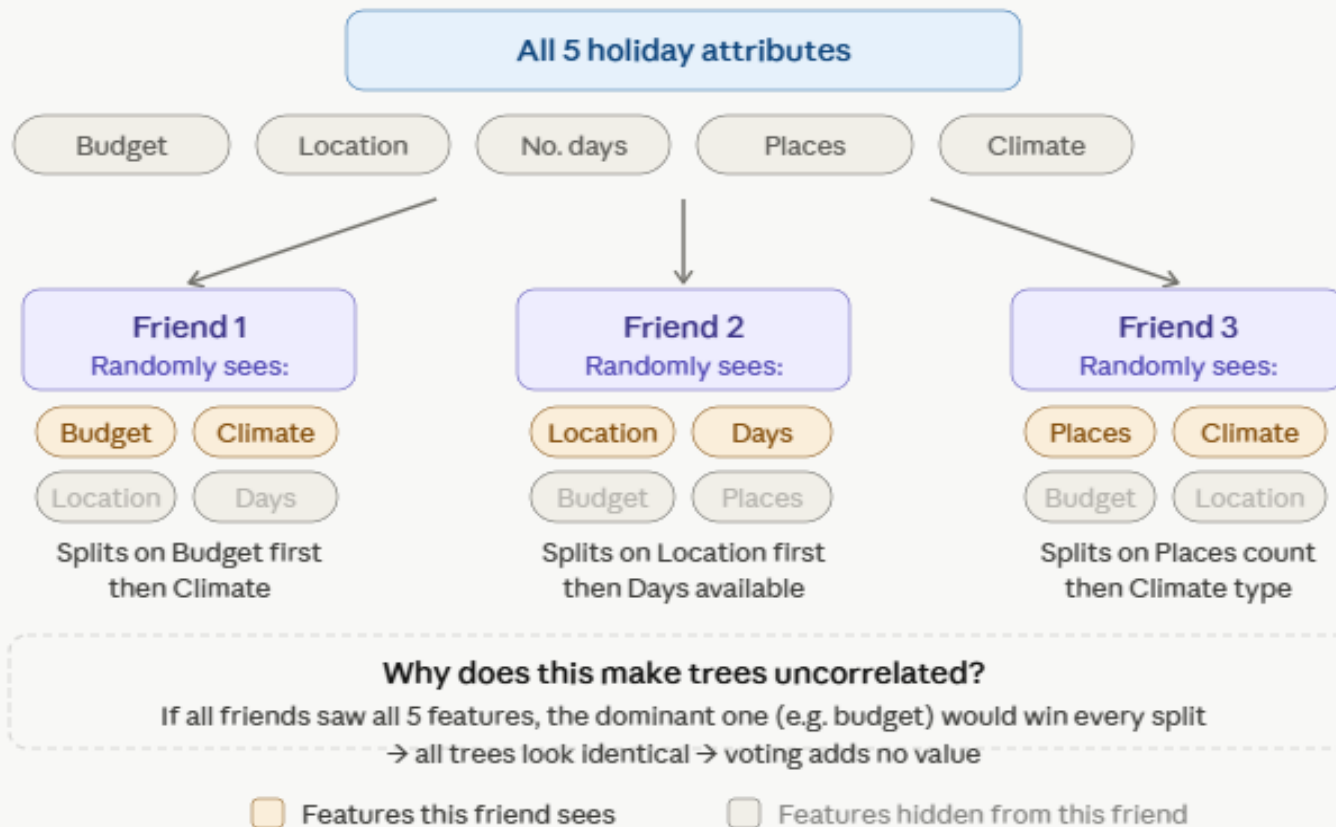
How it works (step by step):



How it works (step by step):

Step 2 — Feature randomness (VM features per split)

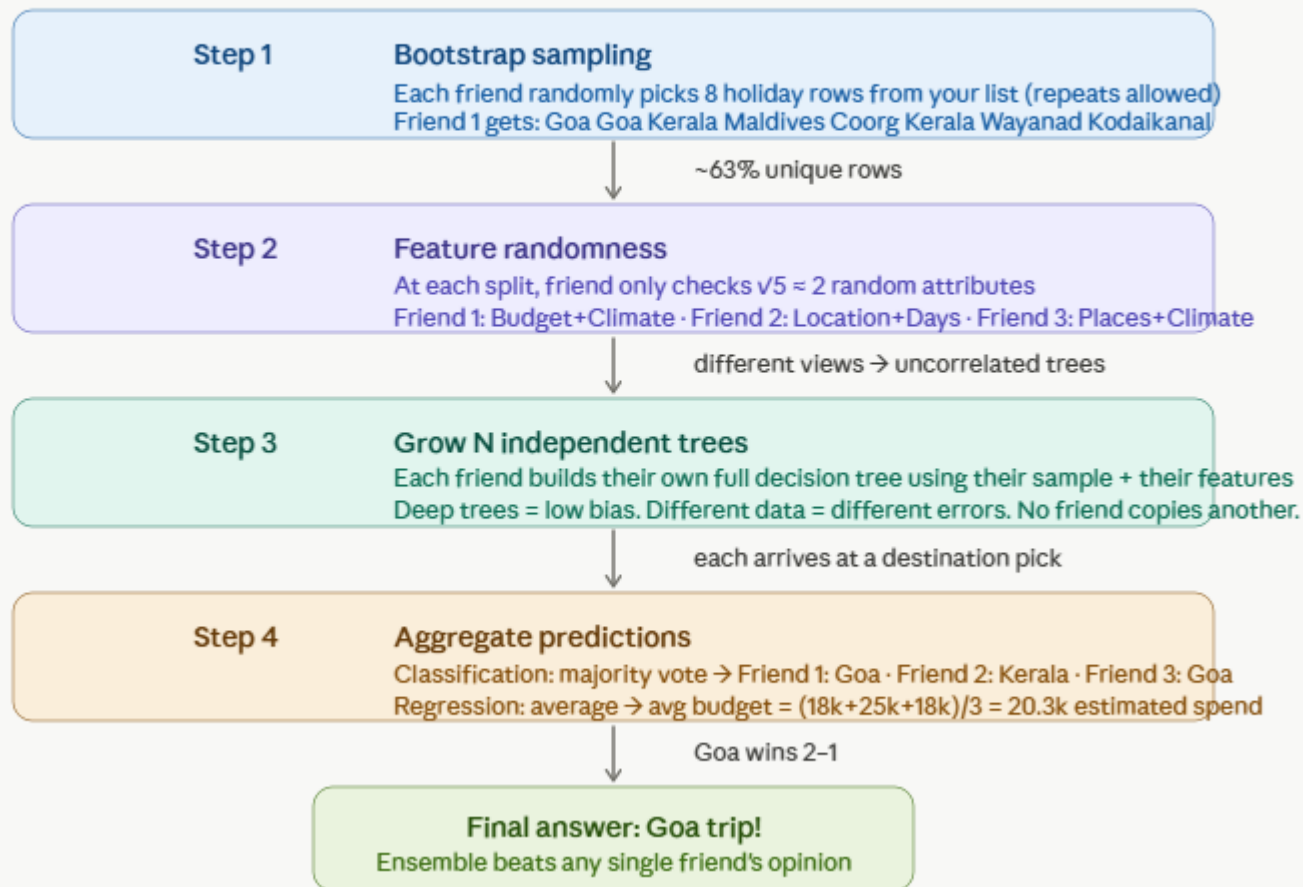
You have $M = 5$ attributes $\rightarrow$ each friend considers $\sqrt{5} \approx 2$ random attributes at each split



Finally — how all 4 steps connect end to end in your holiday scenario:

How it works (step by step):

Finally — how all 4 steps connect end to end in your holiday scenario:



How it works (step by step):

Random Forest — important parameters		
Parameter	Formula / default	Holiday analogy
n_estimators No. of trees	Default = 100 More = better, slower	Number of friends you ask for opinions
max_features Features per split	Classification: $\sqrt{M}$ Regression: $M/3$ M = total features	M=5 attrs → each friend sees only $\sqrt{5} \approx 2$ attributes per decision
max_depth Tree depth limit	Default = None Grow until pure leaves	How many questions each friend can ask
min_samples_split Min rows to split	Default = 2 Higher = smoother tree	Min trip options needed before friend splits further
min_samples_leaf Min rows in leaf	Default = 1 Higher = less overfit	Min trips in final recommendation bucket
bootstrap Use sampling?	Default = True False = each tree sees all data	True: each friend picks random trip subset
criterion Split quality measure	Gini (default) $Gini = 1 - \sum(p^2)$ Entropy = $-\sum(p \cdot \log_2 p)$	How "mixed" a bucket is — Goa+Goa = pure (0) Goa+Kerala = mixed (0.5)
oob_score Free validation score	Default = False Uses ~37% rows not drawn in bootstrap	Test friend 1's advice on trips they never studied (Ooty, Munnar etc.)

Realtime Scenarios



**Bank fraud detection**
Is this transaction fraudulent?

Transaction amount

Location

Time of day

Merchant category

Past spend pattern

+2 more

Verdict

Fraud flagged

Realtime Scenarios



Bank fraud detection — how RF decides

Tree votes (8/10 predict fraud)

Yes

Yes

Yes

Yes

Yes

Yes

Yes

Yes

No

No

Top features by importance



Real scenario: A ₹85,000 transaction at 2:47 AM from a new device in a different city — 8 of 10 trees flag fraud.
Card blocked instantly, SMS sent to customer.

Why RF here: Handles 100+ features (device, location, behaviour). Imbalanced classes (fraud is rare) — RF with `class_weight` handles it well. Processes millions of transactions per second via parallel trees.

Realtime Scenarios



Bank fraud detection — how RF decides

Tree votes (8/10 predict fraud)

Yes Yes Yes Yes Yes Yes Yes Yes No No

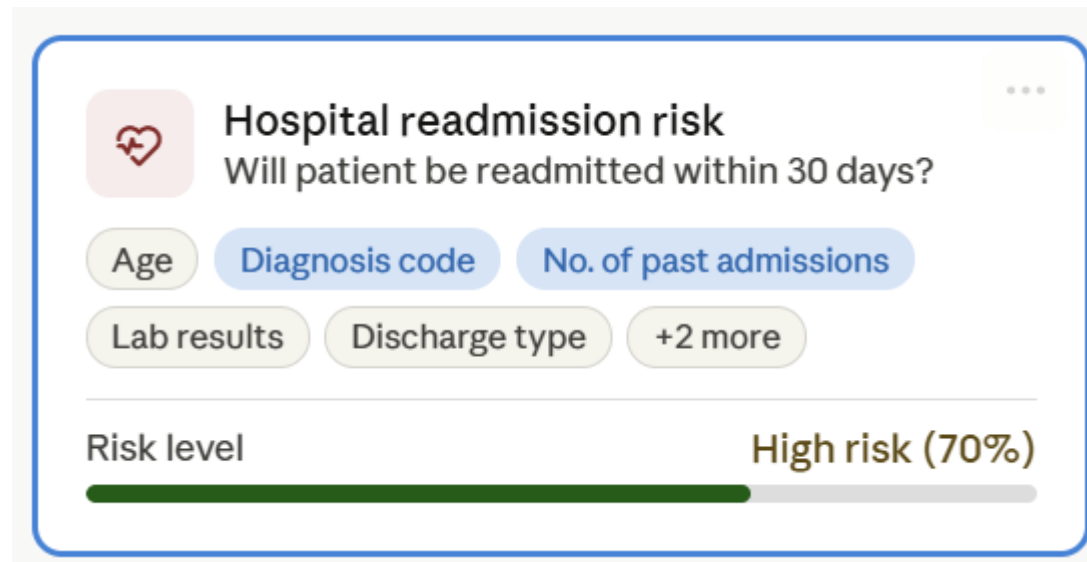
Top features by importance



Real scenario: A ₹85,000 transaction at 2:47 AM from a new device in a different city — 8 of 10 trees flag fraud.
Card blocked instantly, SMS sent to customer.

Why RF here: Handles 100+ features (device, location, behaviour). Imbalanced classes (fraud is rare) — RF with `class_weight` handles it well. Processes millions of transactions per second via parallel trees.

Realtime Scenarios



Realtime Scenarios



Hospital readmission risk — how RF decides

Tree votes (7/10 predict high)



Top features by importance



Real scenario: 68-year-old patient with diabetes, 3 prior admissions, 12 medications. 7 trees predict readmission. Hospital assigns a care coordinator before discharge.

Why RF here: Mixes numeric (age, lab values) and categorical (diagnosis codes) features without preprocessing. Feature importance reveals which factors drive readmission — clinically actionable.

Realtime Scenarios



House price prediction

What is this property worth?

Area (sq ft)

Location pin

Age of building

No. of rooms

Nearby schools

+2 more

Estimated price

₹81.4 lakhs

Realtime Scenarios

House price prediction — how RF decides

Tree predictions (averaged for regression)

₹82L ₹78L ₹85L ₹80L ₹83L ₹81L ₹79L ₹84L ₹80L ₹82L

Top features by importance



Real scenario: 1,450 sq ft flat in Kozhikode, 3 BHK, 8 years old, 2 good schools within 1km. Each of the 10 trees gives a price — average is ₹81.4 lakhs. Used by real-estate apps like Magicbricks.

Why RF here: Regression mode — averaging reduces noise in price estimates. Location is high-cardinality (1000s of pin codes) — RF handles this natively. No outlier sensitivity unlike linear regression.

Realtime Scenarios



Crop disease detection

Does this plant have a disease?

Leaf colour (RGB)

Spot pattern

Humidity

Temperature

Soil pH

+2 more

Diagnosis

Blight detected

Realtime Scenarios

Crop disease detection — how RF decides

Tree votes (8/10 predict blight)



Top features by importance



Real scenario: Sensor reading: yellow-brown patches, humidity 91%, temp 28°C. 8 trees diagnose late blight.

Alert sent to farmer: apply fungicide within 48 hours.

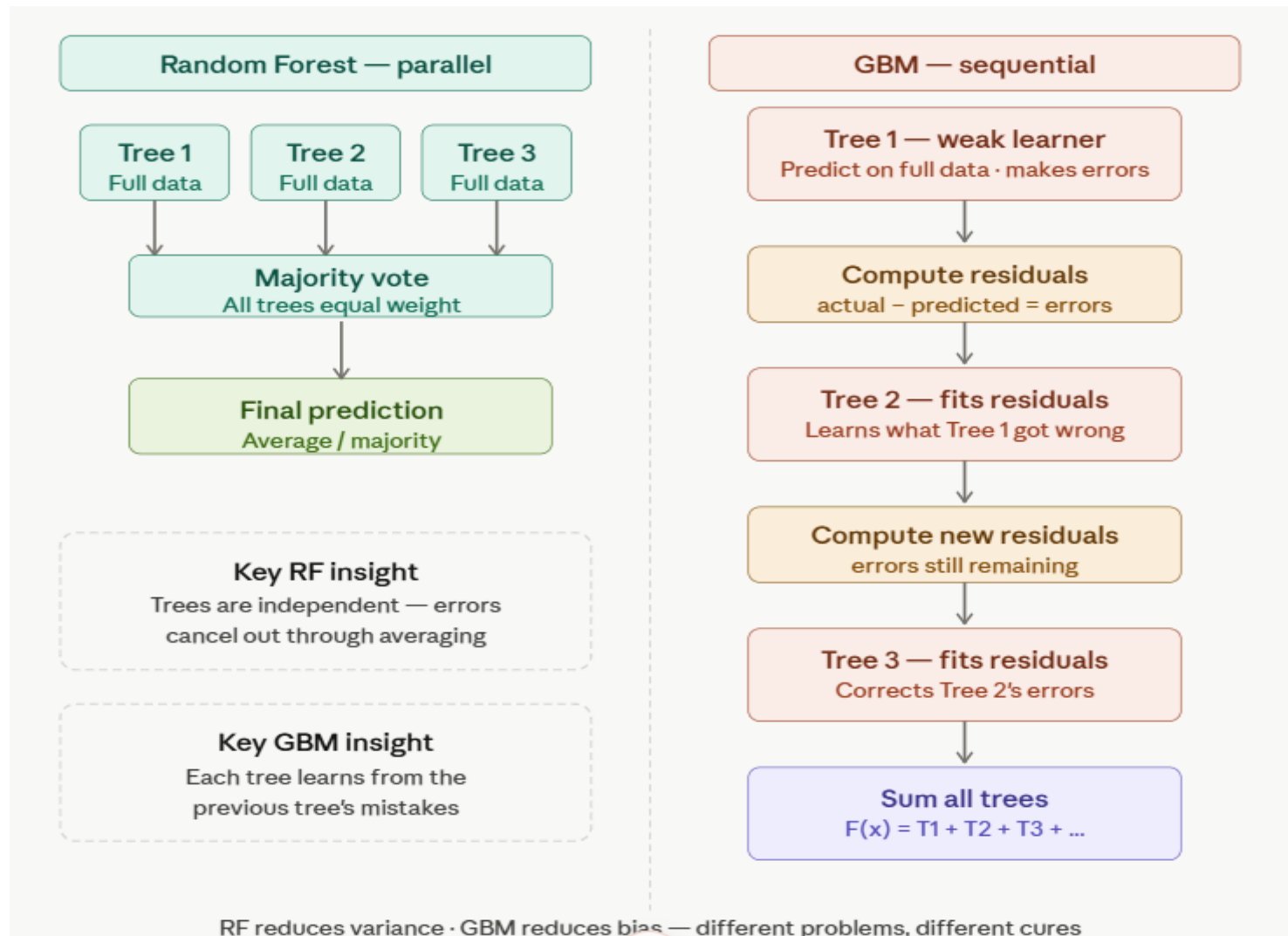
Why RF here: Handles image-derived features (colour histograms) plus sensor readings in the same model. Robust to missing sensor data — if humidity sensor fails, other features still vote.

Gradient Boosting Machine

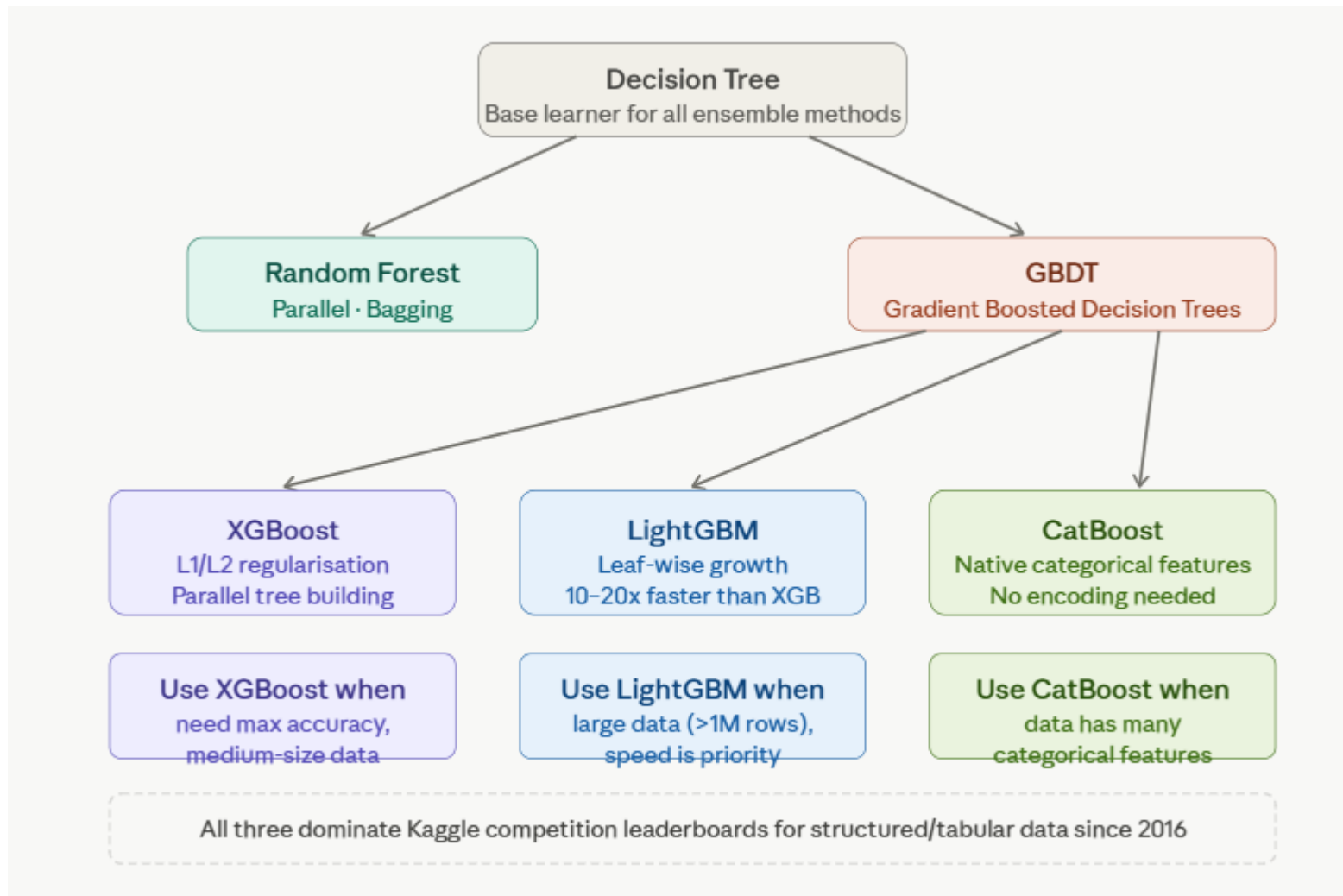
- The **Gradient Boosting Machine (GBM)** is one of the most powerful ensemble methods in machine learning.
- It builds models in a **stage-wise fashion**, where each new tree corrects the errors of the previous ones.
- Unlike Random Forest (which builds trees independently), GBM builds them **sequentially** to minimize loss.

- **Start with a weak learner** (usually a shallow decision tree).
- **Calculate residuals** → the difference between actual and predicted values.
- **Train the next tree** to predict these residuals.
- **Update the model** by adding the new tree's contribution.
- Repeat until the desired number of trees or until loss converges.
- 👉 Each tree focuses on the mistakes of the previous ones, making the ensemble stronger.

Gradient Boosting Machine



GBM Family



GBM Family



Dimension	Random Forest	XGBoost	LightGBM	CatBoost	...
Training style	Parallel (independent)	Sequential (boosting)	Sequential (boosting)	Sequential (boosting)	
Speed	Fast	Medium	Very fast	Medium	
Accuracy (tabular)	Good	Excellent	Excellent	Excellent	
Overfitting risk	Low	Medium	Medium	Low	
Categorical features	Needs encoding	Needs encoding	Built-in support	Best-in-class	
Missing values	Handles natively	Handles natively	Handles natively	Handles natively	
Hyperparameter tuning	Easy (few params)	Complex (many)	Moderate	Easy defaults	
Large data (>1M rows)	OK	OK	Best choice	OK	
Interpretability	Feature importance	SHAP values (best)	SHAP values	SHAP values	
Real use case	Hospital readmission, credit scoring	Fraud detection, loan default	Uber surge, ad CTR	Recommendation with categories	
Pick when	Quick strong baseline, robust, no tuning	Max accuracy, explainability needed	Speed + scale (>1M rows)	Many categorical columns	

GBM Family Realtime Scenarios

Finance

Fraud detection

Real-time transaction scoring

Telecom

Churn prediction

Customer retention scoring

Banking

Credit scoring

Loan default risk

Retail

Demand forecasting

Inventory & supply chain

Healthcare

Disease risk

Early diagnosis support

AdTech

Click-through rate

Ad ranking & bidding

GBM Family Realtime Scenarios

Fraud detection

[Deep dive ↗](#)

Models like XGBoost score each card transaction in <100ms, catching anomalies before authorization completes.

Top features used

Transaction amount Merchant category Time since last txn Location delta Device fingerprint

Feature importance (relative)

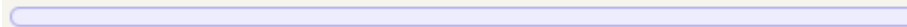
Location delta



Transaction amount



Time since last txn



⚡ Used by: Stripe, PayPal, Visa fraud systems — often XGBoost or LightGBM

GBM Family Realtime Scenarios

Churn prediction

[Deep dive ↗](#)

GB models flag at-risk customers 30–60 days before cancellation, enabling targeted retention campaigns.

Top features used

Usage frequency

Support tickets

Days since login

Contract type

Plan downgrade history

Feature importance (relative)

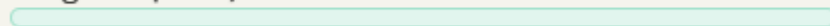
Days since login



Support tickets



Usage frequency



⚡ Used by: Telcos, SaaS platforms, subscription services — often paired with SHAP explainability

GBM Family Realtime Scenarios

Demand forecasting

[Deep dive ↗](#)

GB captures complex interactions between seasonality, promotions, and external events that ARIMA-style models miss.

Top features used

Day of week Rolling sales lag Promotional flag Holiday calendar Price elasticity

Feature importance (relative)

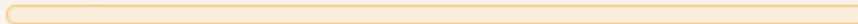
Rolling sales lag



Promotional flag



Holiday calendar



⚡ Used by: Amazon, Walmart, grocery chains — LightGBM popular for speed on large SKU datasets

GBM Family Realtime Scenarios

Click-through rate prediction

[Deep dive ↗](#)

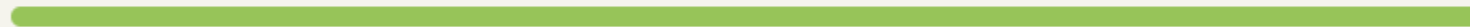
GB models predict the probability a user clicks an ad, driving real-time auction bidding decisions in milliseconds.

Top features used

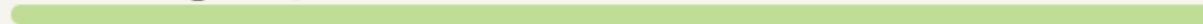
User browsing history Ad category Time of day Device type Historical CTR

Feature importance (relative)

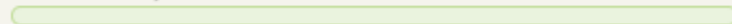
Historical CTR



User browsing history



Time of day



⚡ Used by: Google, Meta, LinkedIn — GBDT features often fed into deep learning for hybrid models



Why Gradient Boosting dominates these scenarios

- Gradient Boosting (XGBoost, LightGBM, CatBoost) works so well in practice because it handles messy tabular data naturally.
 - missing values
 - mixed types,
 - skewed distributions
 - non-linear interactions
- Without requiring the extensive preprocessing that neural networks demand.
- Each scenario shares a common pattern: the model sequentially builds hundreds of shallow decision trees, each one correcting the errors of the previous ensemble.
 - By the final tree, even subtle, high-order feature combinations (like "unusual amount + foreign merchant + 3am transaction") are captured.

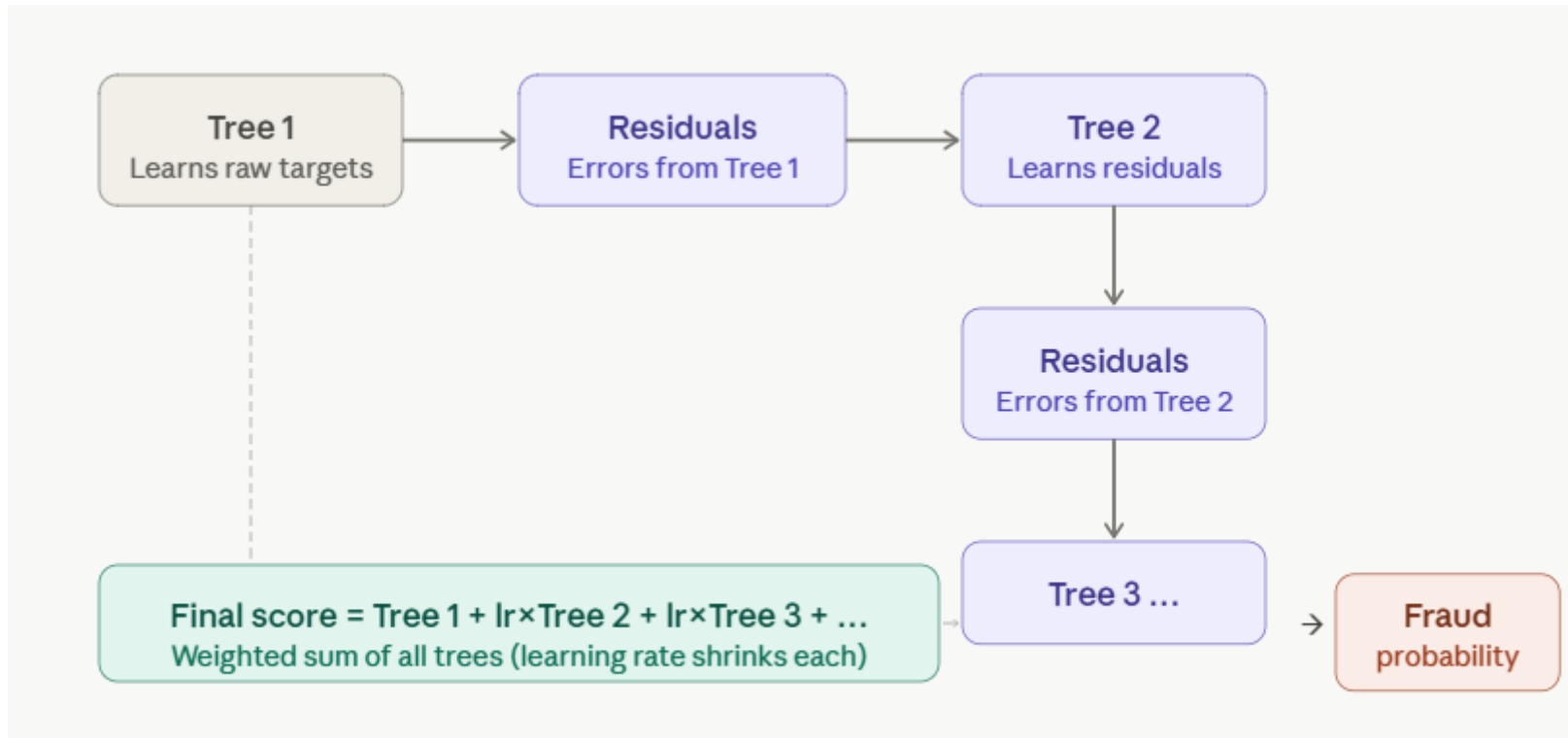


Why Gradient Boosting dominates these scenarios

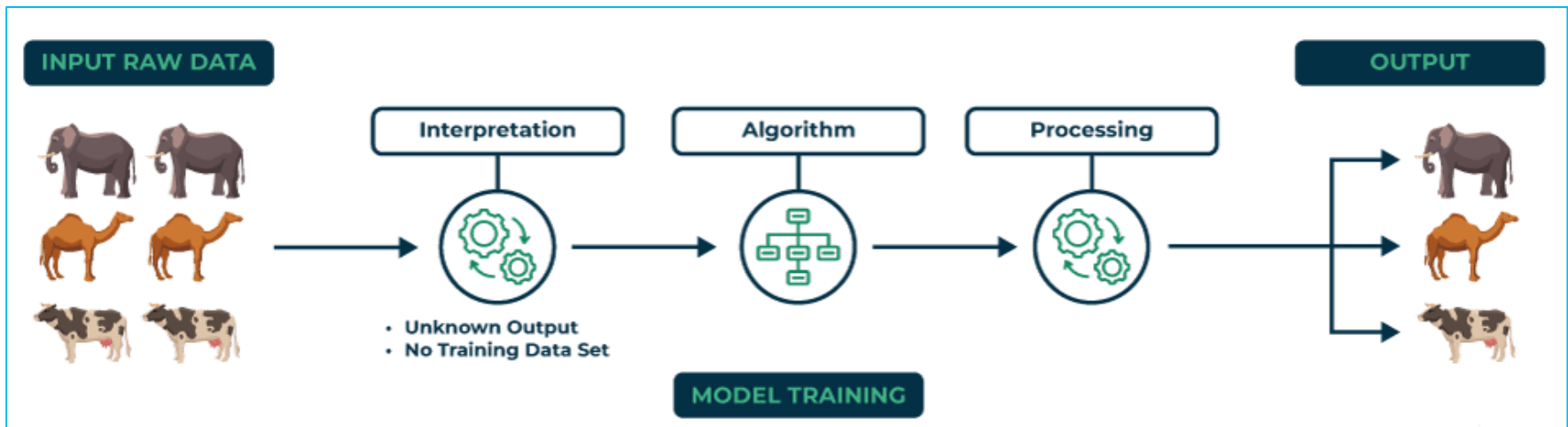
- **Key tradeoffs in production:**
- **Speed vs accuracy** — LightGBM is fastest (ad bidding), XGBoost is most stable (credit scoring)
- **Explainability** — regulated industries (healthcare, banking) pair GB with SHAP values to explain individual predictions
- **Retraining cadence** — fraud and CTR models retrain daily/hourly; churn and credit models weekly/monthly



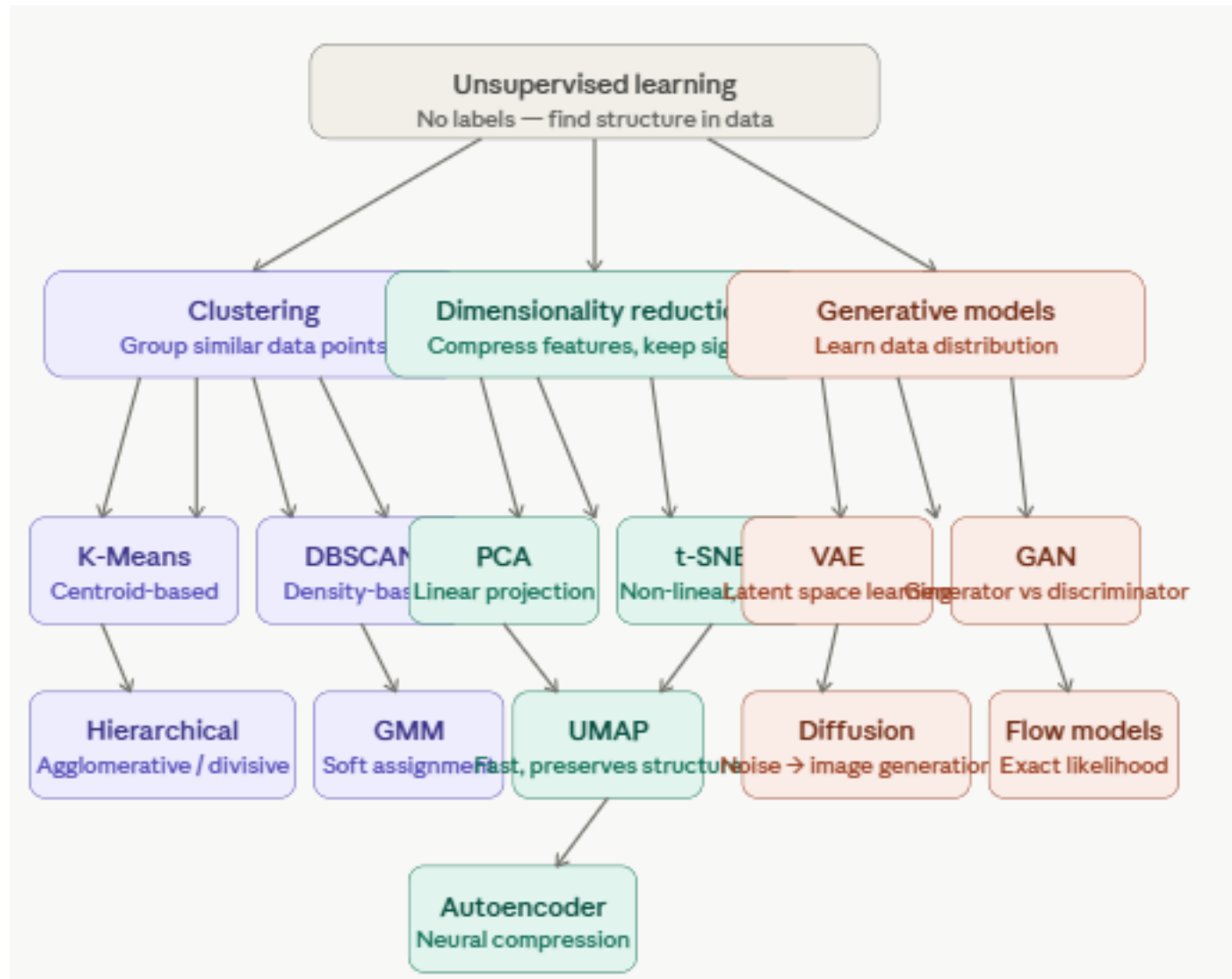
Why Gradient Boosting dominates these scenarios



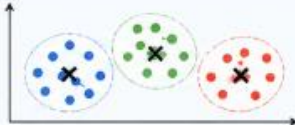
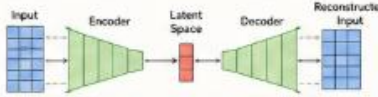
Unsupervised Learning



Unsupervised Learning



Unsupervised Learning

Unsupervised Learning Algorithms at a Glance																
Algorithm	What it does	Best for (Use Cases)	How it works (In Simple Terms)	Visual Representation												
1. K-Means Clustering (Partition-based) 	Groups data into K clusters by minimizing distance within each cluster.	- Customer segmentation - Market analysis - Image segmentation - Document clustering	- Choose K (number of clusters) - Randomly pick K centroids - Assign each point to nearest centroid - Update centroids (mean of points) - Repeat steps 3–4 until stable													
2. Hierarchical Clustering (Hierarchy-based) 	Builds a hierarchy of clusters (tree-like) without predefining the number of clusters.	- Gene expression analysis - Social network analysis - Image retrieval - Document clustering	- Treat each point as a cluster - Merge closest clusters step by step - Repeat until all points in one cluster - Cut the tree at desired level to get clusters													
3. DBSCAN (Density-based) 	Groups dense regions together and marks sparse points as outliers.	- Anomaly detection - Geospatial data - Customer segmentation - Noise removal	- Pick a point, find neighbors within ϵ (eps) - If neighbors $\geq$ MinPts $\rightarrow$ create cluster - Expand cluster by adding density-reachable points - Points not reachable $\rightarrow$ noise (outliers)													
4. Gaussian Mixture Models (GMM) (Probabilistic) 	Assumes data is generated from a mixture of several Gaussian (normal) distributions.	- Customer segmentation - Anomaly detection - Speech recognition - Image modeling	- Assume data comes from K Gaussians - Estimate parameters (mean, variance, weight) - Compute probability of each point for clusters - Assign to cluster with highest probability													
5. Association Rule Learning (Apriori) (Pattern Mining) 	Finds interesting relationships (rules) between items in transaction data.	- Market basket analysis - Cross-selling - Recommendation - Inventory planning	- Find frequent itemsets (commonly bought together) - Generate rules: $A \rightarrow B$ - Keep rules with high support & confidence	<table><thead><tr><th>Rule</th><th>Support</th><th>Confidence</th></tr></thead><tbody><tr><td>{Milk} $\rightarrow$ {Bread}</td><td>60%</td><td>75%</td></tr><tr><td>{Bread} $\rightarrow$ {Butter}</td><td>50%</td><td>80%</td></tr><tr><td>{Milk, Bread} $\rightarrow$ {Butter}</td><td>40%</td><td>90%</td></tr></tbody></table>	Rule	Support	Confidence	{Milk} $\rightarrow$ {Bread}	60%	75%	{Bread} $\rightarrow$ {Butter}	50%	80%	{Milk, Bread} $\rightarrow$ {Butter}	40%	90%
Rule	Support	Confidence														
{Milk} $\rightarrow$ {Bread}	60%	75%														
{Bread} $\rightarrow$ {Butter}	50%	80%														
{Milk, Bread} $\rightarrow$ {Butter}	40%	90%														
6. Principal Component Analysis (PCA) (Dimensionality Reduction) 	Reduces dimensionality while preserving most variance in the data.	- Data visualization - Feature reduction - Noise reduction - Preprocessing	- Find directions (components) of max variance - Project data onto top components - Keep components that explain most variance													
7. t-SNE (Visualization) 	Reduces high-dimensional data to 2D/3D for visualization (preserves local structure).	- Data visualization - Cluster inspection - Exploratory analysis	- Measure similarity between points in high-D space - Map to low-D space - Preserve local neighborhood structure													
8. Autoencoders (Representation Learning) 	Learns compressed representations of data by encoding and then decoding it.	- Anomaly detection - Image compression - Feature learning - Denosing	- Encoder compresses input to lower dimension - Decoder reconstructs the input - Model learns useful representations													
★ Key Takeaway: Unsupervised learning helps discover hidden patterns in data without labels. Choose the right algorithm based on your data type and goal.																

K-Means Algorithm

- K-Means is an unsupervised clustering algorithm that partitions n data points into K groups (clusters) where each point belongs to the cluster with the nearest centroid.
- The "K" is chosen by you — the algorithm doesn't know how many groups exist, it finds the best arrangement for whatever K you give it.

K-Means Algorithm

- **The real-world analogy for trainees:** Imagine you run a pizza chain and have 1,000 customer addresses.
- You want to open $K = 3$ delivery hubs. K-Means finds the 3 hub locations that minimise the total delivery distance for all customers.
- Each iteration moves the hubs closer to the ideal positions until no improvement is possible.

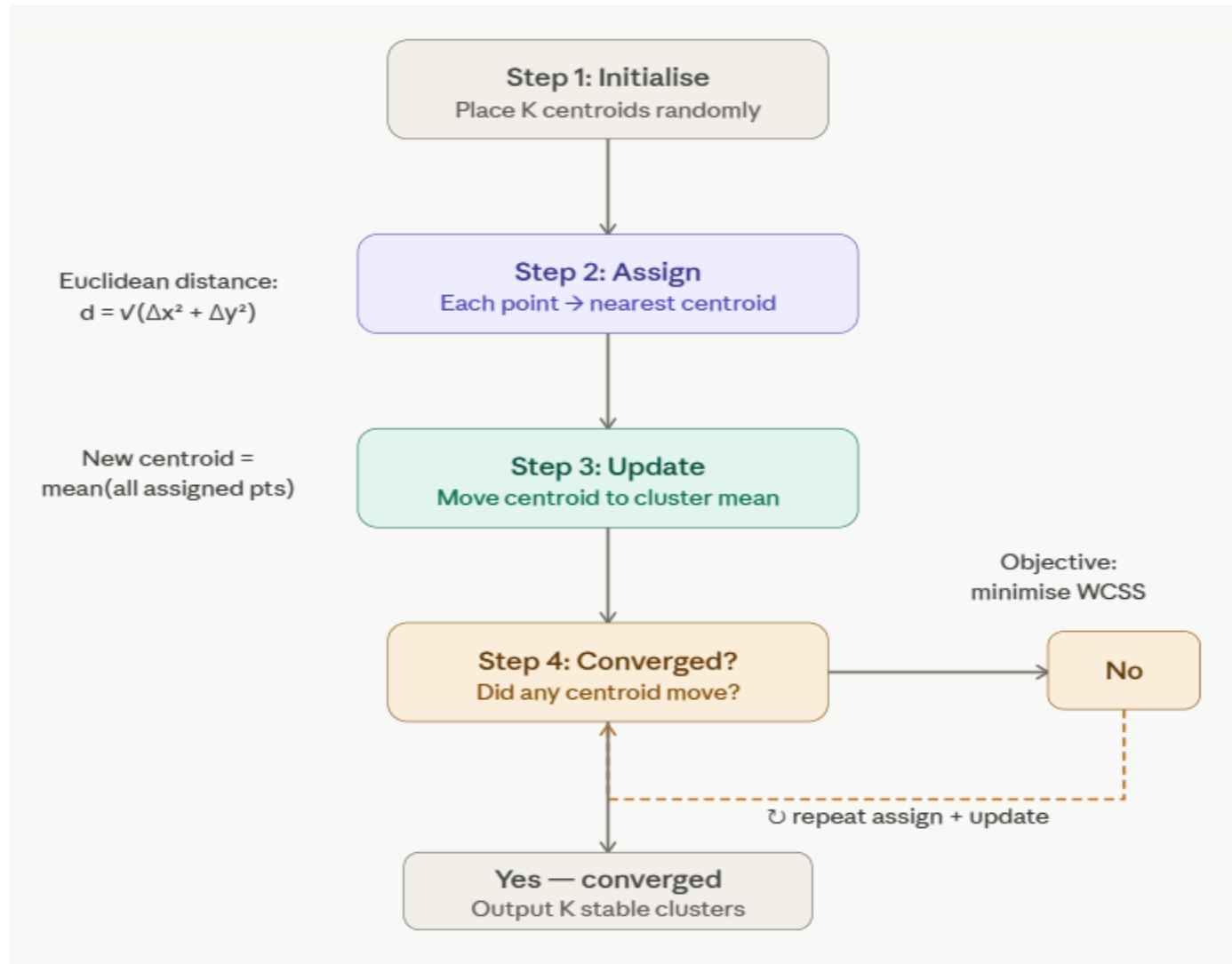
K-Means Algorithm Steps

- **The algorithm — step by step**
- The loop has exactly four steps that repeat until convergence:
- **Initialise** — randomly place K centroids in the data space
- **Assign** — assign every data point to its nearest centroid (Euclidean distance)
- **Update** — move each centroid to the mean (average) position of its assigned points
- **Check** — if centroids moved, go to step 2. If nothing moved, stop.

K-Means Algorithm Steps



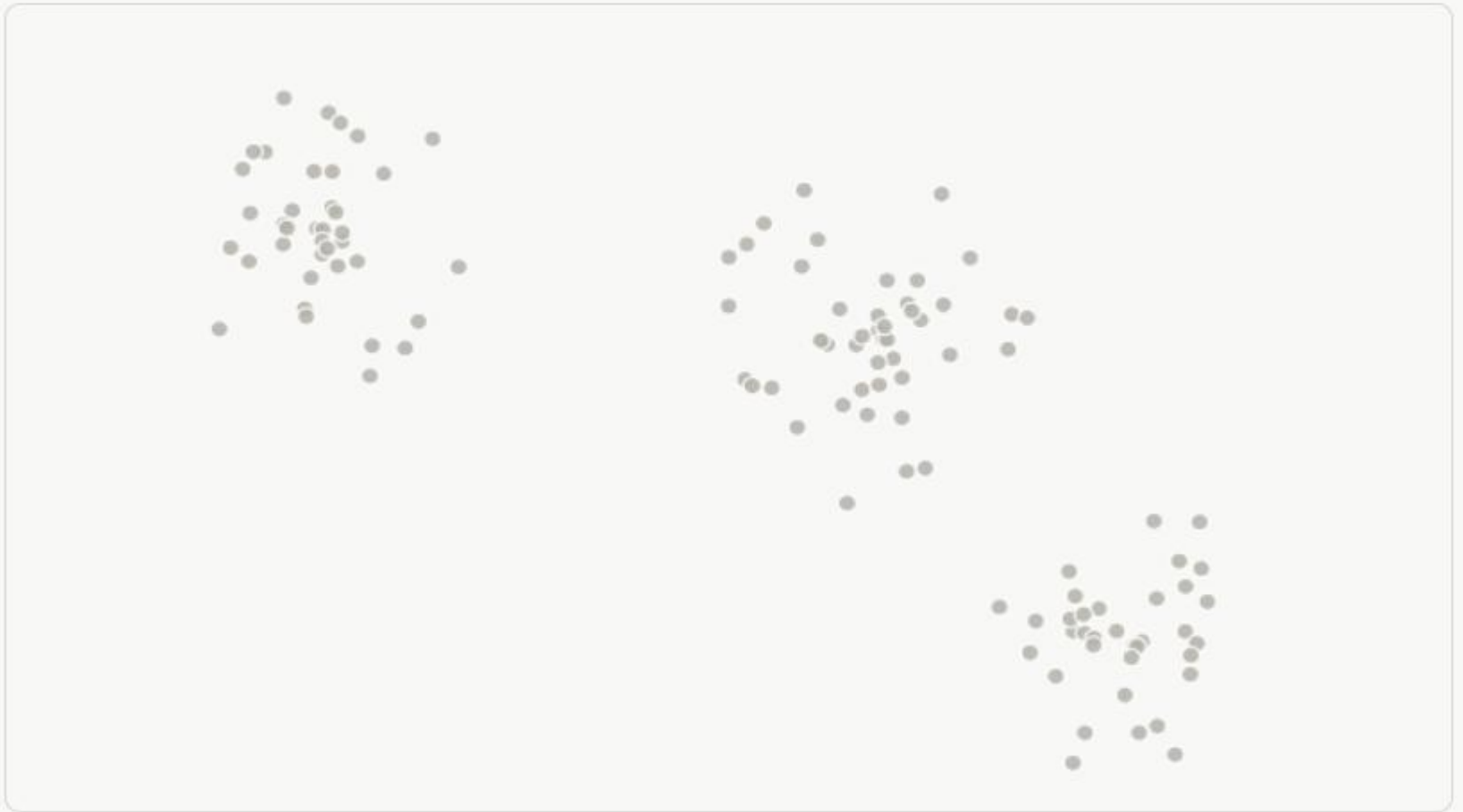
K-Means Algorithm Steps



K-Means Algorithm

Step 1 — The problem

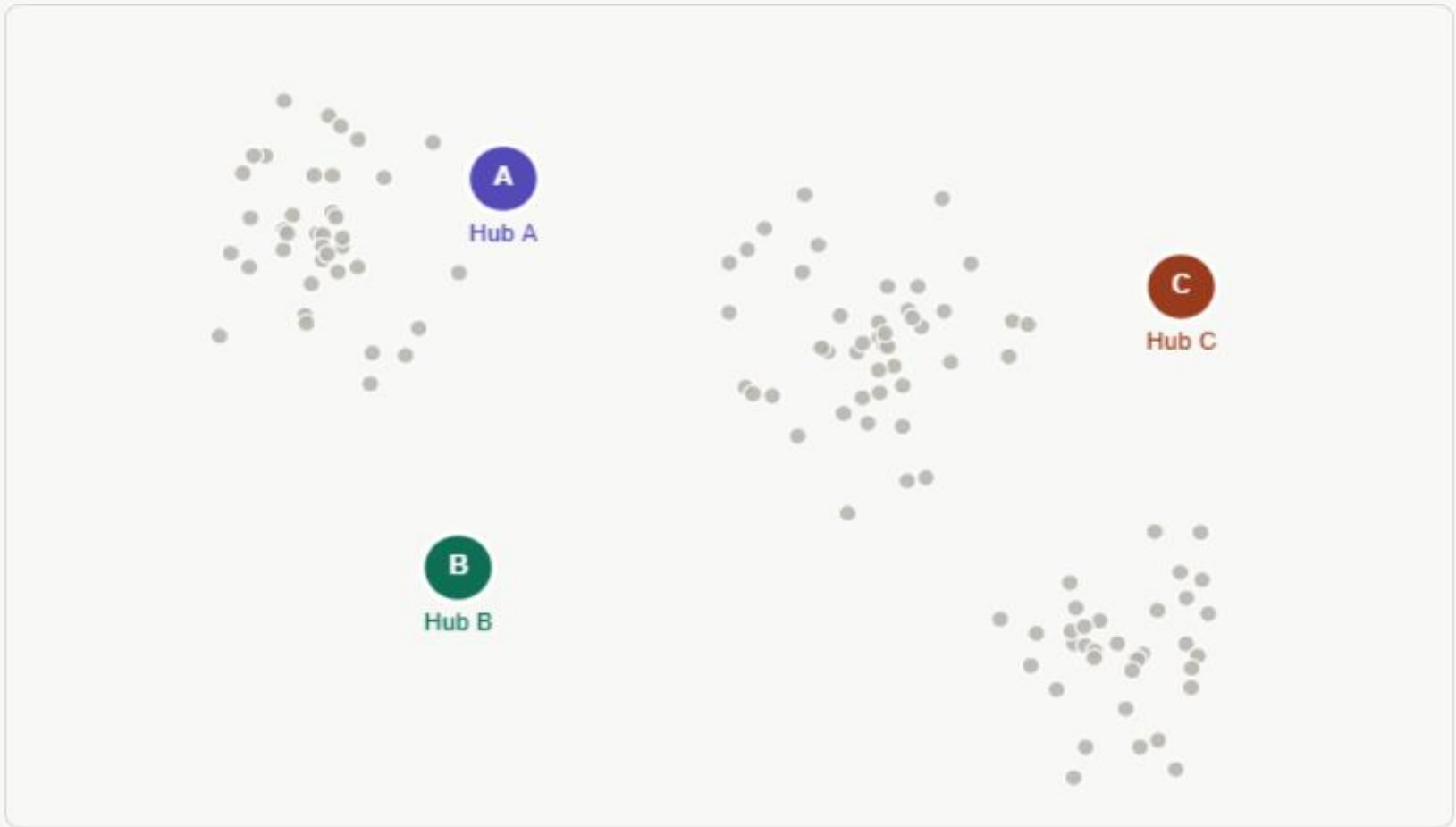
1,000 customer addresses spread across the city. You need to open $K = 3$ delivery hubs to serve them. Where should the hubs go?



K-Means Algorithm

Step 2 — Random initialisation

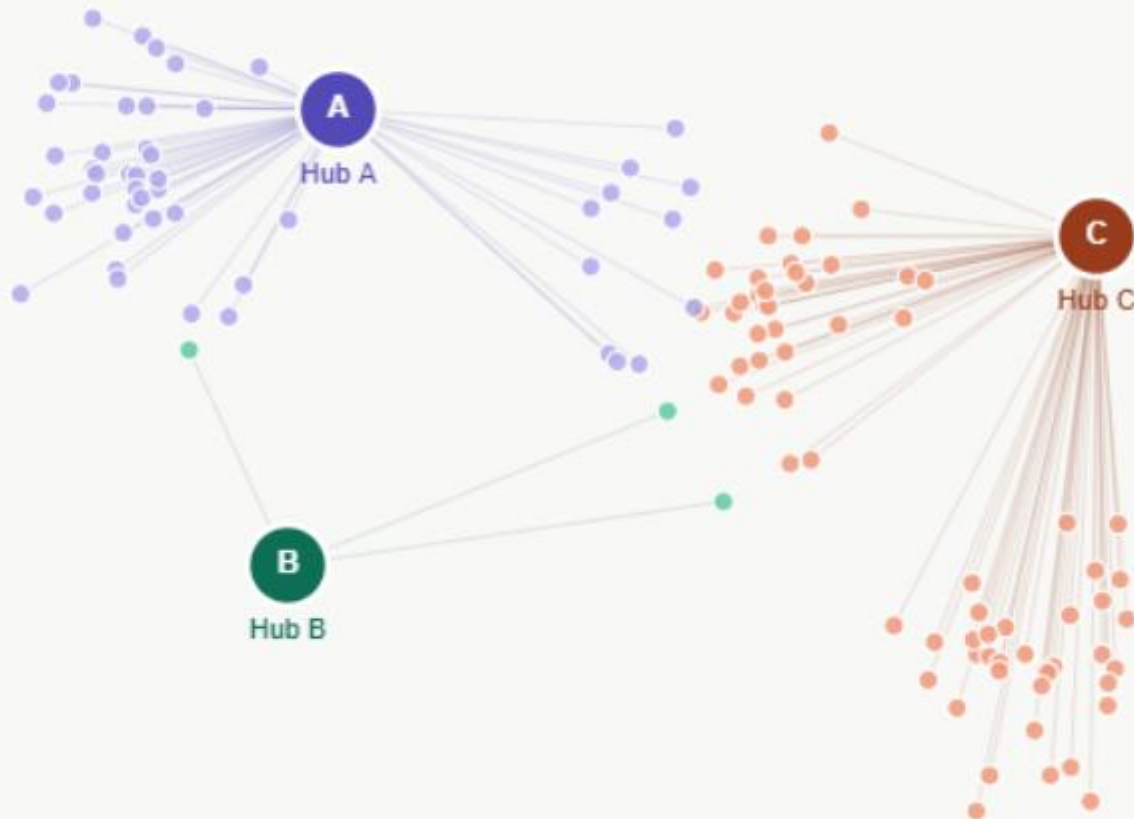
K-Means starts by placing 3 hubs at random locations. These are almost certainly not optimal — but we have to start somewhere.



K-Means Algorithm

Step 3 — Assign customers (iteration 1)

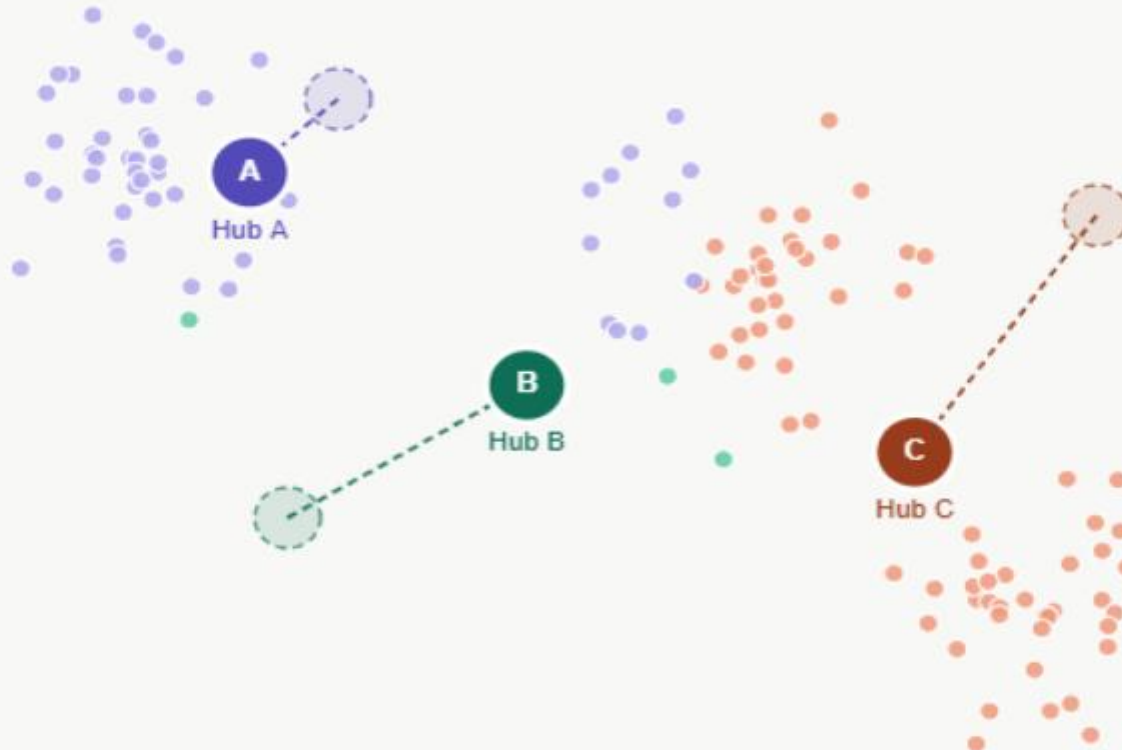
Each customer is assigned to the **nearest hub**. The city splits into 3 zones — coloured by which hub they'd order from.



K-Means Algorithm

Step 4 — Move hubs to cluster centres

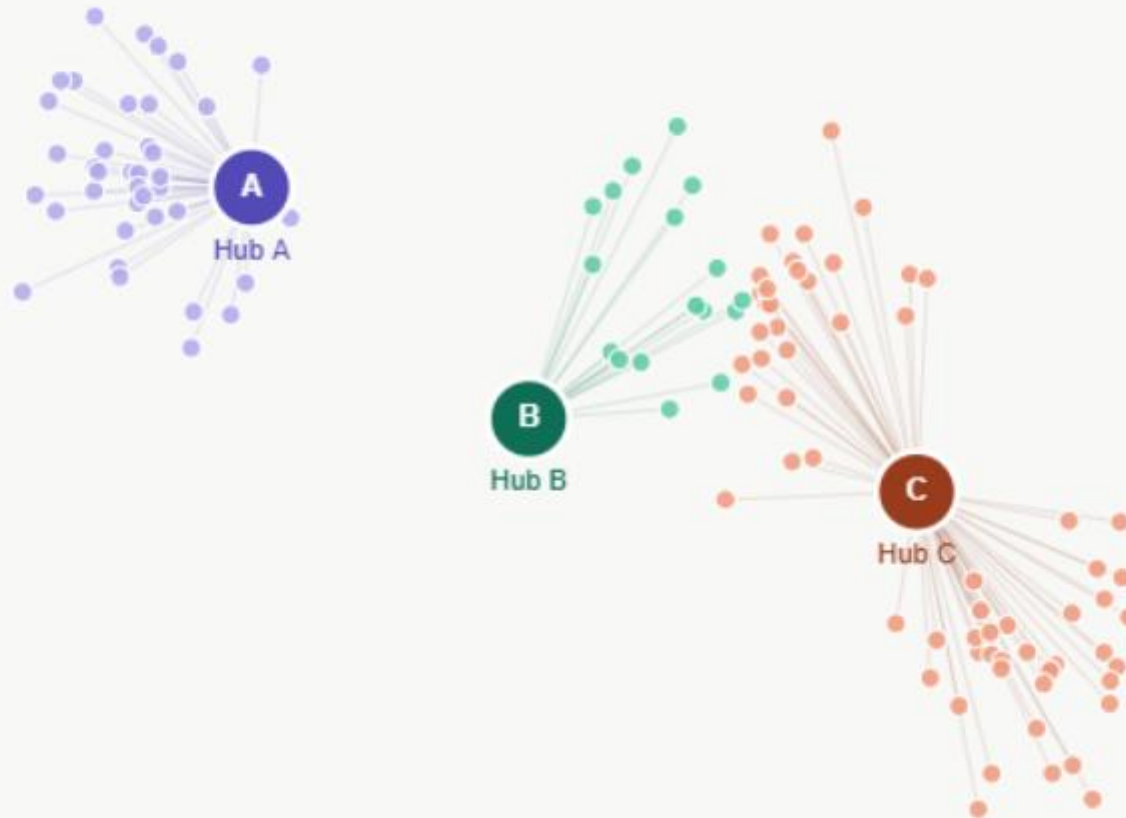
Each hub moves to the average position of its customers — the true geographic centre of its zone. Watch the hubs shift.



K-Means Algorithm

Step 5 — Reassign customers (iteration 2)

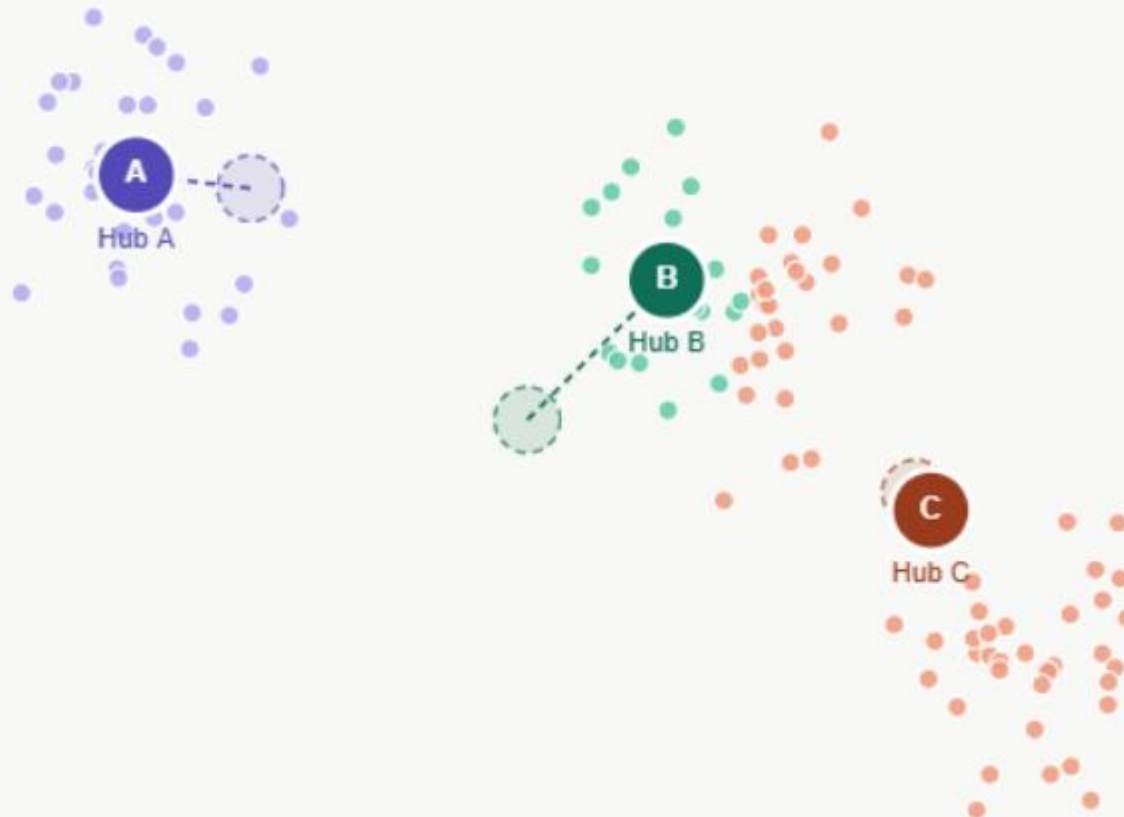
With hubs in new positions, **some customers switch zones**. The boundaries adjust to match the new hub locations.



K-Means Algorithm

Step 6 — Move hubs again

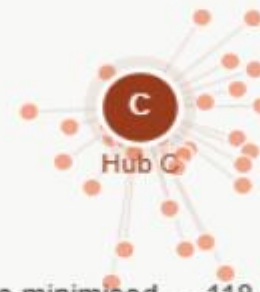
Hubs shift again toward their new cluster centres. **Each iteration reduces total delivery distance.**



K-Means Algorithm

Step 7 — Converged

Hubs stop moving. No customer would be closer to a different hub. The 3 optimal locations are found — minimum total delivery distance.



Total distance minimised — 118 customers, 3 hubs

K-Means Algorithm

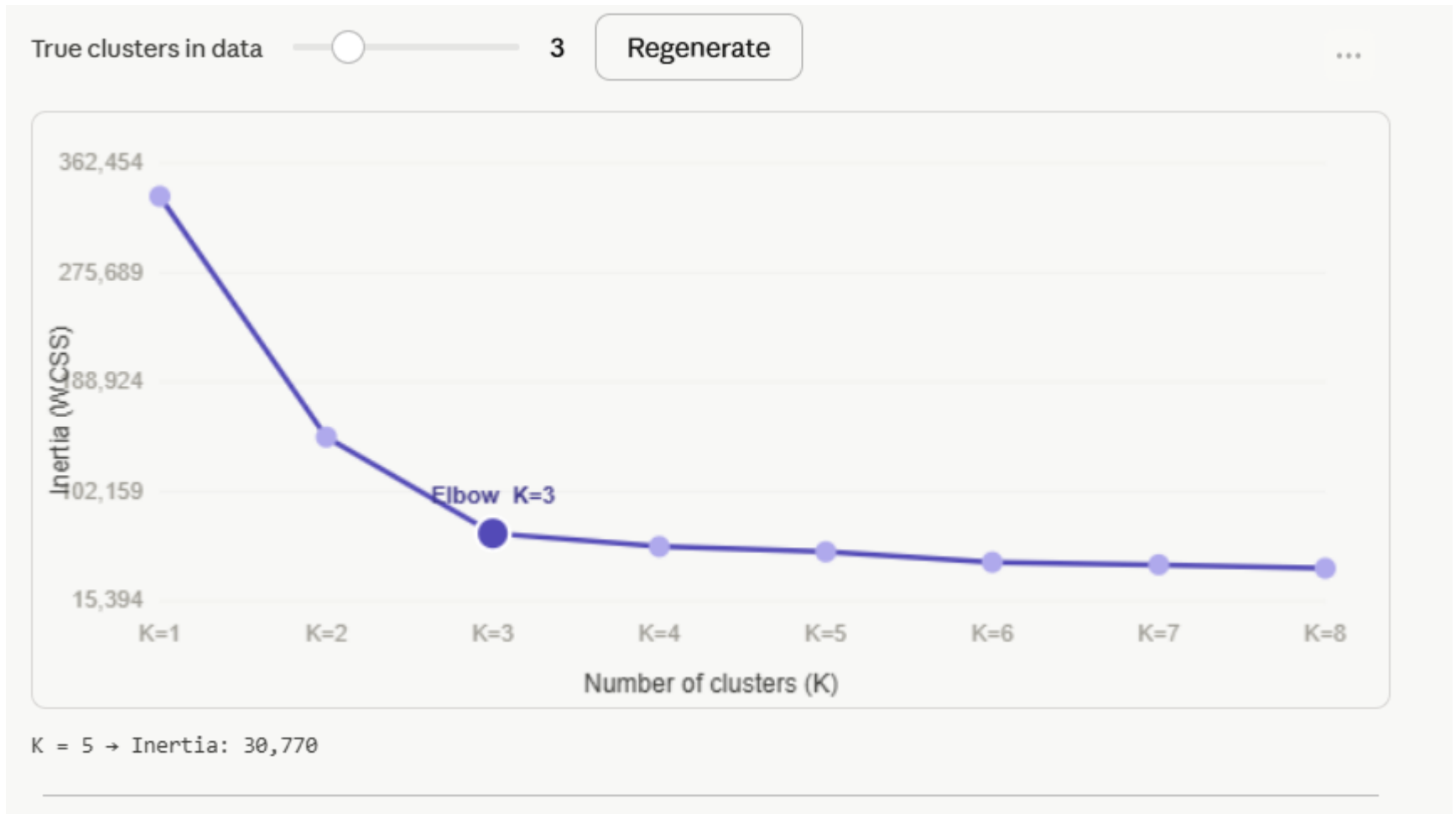
Walk through all 7 steps using the Next/Previous buttons. Each step maps directly to a K-Means concept:

Diagram step	K-Means concept
1,000 customer addresses	Data points ($n = 1,000$)
3 delivery hubs	Centroids ($K = 3$)
"Nearest hub" zone colouring	Cluster assignment step
Hub moves to geographic centre	Centroid update step
No customer switches zones	Convergence condition
Total delivery distance	WCSS / Inertia — what the algorithm minimises

How to choose K — the Elbow Method

- The single most common fresher question: "How do I know what K to use?"
- The answer is the Elbow Method
- Run K-Means for $K = 1, 2, 3, \dots$ and plot inertia.
- The elbow point where inertia stops dropping sharply is your best K.

How to choose K — the Elbow Method



Assumptions, strengths, and limitations

- **Assumptions K-Means makes** (train your freshers to always ask these):
- Clusters are roughly **spherical** — it uses Euclidean distance, so elongated or irregular shapes get split wrong
- Clusters are roughly **equal in size** — a very small cluster near a large one will get absorbed
- Features are on **similar scales** — always normalise/standardise your data before running K-Means

Assumptions, strengths, and limitations

- **Strengths:** Simple to understand and explain, fast on large datasets ($O(n \cdot K \cdot \text{iterations})$), scales well, easy to implement.
- **Limitations:** You must choose K in advance, sensitive to outliers (one outlier point drags a centroid), non-deterministic (different runs can give different results), struggles with non-convex cluster shapes.

Key terms for freshers to memorise

Term	Plain-English meaning
Centroid	The "centre of gravity" of a cluster — its mean position
WCSS / Inertia	Total squared distance from every point to its centroid — lower = tighter clusters
Convergence	When centroids stop moving — the algorithm is done
Elbow method	Plot inertia vs K and pick the "bend" in the curve
K-Means++	A smarter initialisation (scikit-learn default) that spreads initial centroids apart to avoid bad starts
Silhouette score	A quality metric (-1 to +1) measuring how well-separated clusters are

K-Means Realtime Scenarios

Customer segmentation



Group shoppers by spend, frequency & loyalty

Anomaly detection



Flag points far from any centroid

Image compression



→ K centroids



Replace millions of colors with K palette entries

Document clustering

Politics

Sports

Finance



Cluster TF-IDF vectors of text documents

Recommendation systems



Users in same cluster get similar suggestions

Urban & logistics planning



Optimal depot placement minimising travel cost

K-Means Realtime Scenarios

- **1. Customer segmentation** — Retail and e-commerce companies cluster shoppers by purchase frequency, basket size, and recency.
- Each cluster becomes a distinct marketing persona: high-value loyalists, bargain hunters, occasional browsers, etc.
- **2. Anomaly detection** — In cybersecurity and fraud detection, normal behaviour forms tight clusters.
- Any data point (transaction, login, packet) sitting far from all centroids is flagged as suspicious and routed for review.

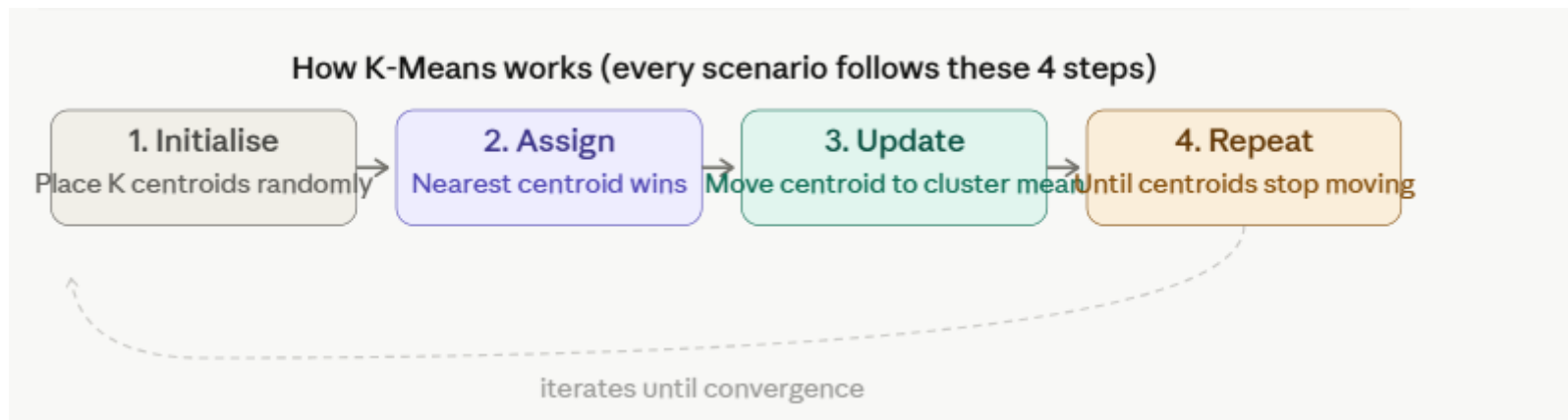
K-Means Realtime Scenarios

- **3. Image compression** — Pixels across the image are clustered by RGB value.
- The centroid of each cluster becomes the "representative color."
- Replacing millions of colors with K palette entries (e.g. K=64) dramatically shrinks file size with minimal visible loss — this is how older GIF compression worked.
- **4. Document / topic clustering** — News aggregators and search engines convert articles into TF-IDF or embedding vectors, then cluster them.
- Articles about the same topic land in the same cluster without any labels needed — a form of unsupervised topic modelling.

K-Means Realtime Scenarios

- **5. Recommendation systems** — Users are clustered by their ratings or viewing history.
- When you want recommendations for a user, you find their cluster and surface items popular within that cluster — a lightweight collaborative filtering approach used by streaming and e-commerce platforms.
- **6. Urban & logistics planning** — Given a set of demand points (stores, customers, deliveries), K-Means finds the K optimal depot or warehouse locations that minimise average travel distance — essentially solving a facility location problem.

K-Means Realtime Scenarios



K-Means Result Interpretation

- **Cluster 0 – Young Frequent Shoppers**
- Young customers
 - Low income
 - High spending score
 - Shop frequently
- Business Action:
- Offer student discounts
- Promote fashion and gadgets

K-Means Result Interpretation

- **Cluster 1 – Wealthy but Conservative**
- Older customers
 - High income
 - Low spending score
 - Rare purchases
- Business Action:
- Premium loyalty programs
- Personalized marketing

K-Means Result Interpretation

- **Cluster 2 – Valuable Regular Customers**
- Middle-aged
Good income
High spending
Regular buyers
- Business Action:
- Retain aggressively
- Cross-sell products

K-Means Result Interpretation

- **Cluster 3 – Low Engagement Customers**
- Average age
Low income
Low spending
Low purchase frequency
- Business Action:
- Coupons
- Re-engagement campaigns

K-Means Result Interpretation

- **Cluster 4 – VIP Customers**
- High income
Highest spending score
Highest order value
Most frequent purchases
- Business Action:
- VIP membership
- Dedicated support
- Exclusive offers

K-Means Result Interpretation

- **Cluster 4 – VIP Customers**

- High income
Highest spending score
Highest order value
Most frequent purchases
- Business Action:
- VIP membership
- Dedicated support
- Exclusive offers

K-Means Result Interpretation

- Cluster 0 → Budget Shoppers
- Cluster 1 → Conservative Rich
- Cluster 2 → Loyal Customers
- Cluster 3 → Occasional Buyers
- Cluster 4 → VIP Customers

- PCA (Principal Component Analysis) is a dimensionality reduction technique that converts many correlated features into a smaller number of new features called **Principal Components** while preserving most of the information in the data.
- **Simple Definition**
- PCA reduces the number of columns in a dataset while keeping most of the important information.

Why Do We Need PCA?

Customer	Age	Income	Salary	Spending	Credit Limit
C1	25	50K	55K	30K	60K
C2	30	60K	65K	40K	70K

- Income and Salary are related
- Income and Credit Limit are related
- Many columns carry similar information.
- Problems:
 - ✗ Slow training
 - ✗ More memory
 - ✗ Overfitting
 - ✗ Difficult visualization
- PCA solves this.

PCA Real time scenarios

● Face recognition

Each pixel is a feature. PCA compresses thousands of pixels into eigenfaces.

4,096D → 150D

Variance kept  92%

● Genomics / DNA analysis

SNP arrays have 500k+ genetic markers per sample. PCA reveals population structure.

500,000D → 10D

Variance kept  85%

● Finance / risk modelling

Correlated stock returns collapse into uncorrelated risk factors.

500D → 20D

Variance kept  78%

● NLP / text embeddings

TF-IDF or word-count matrices are reduced to dense semantic topic spaces.

50,000D → 300D

Variance kept  70%

● Manufacturing / quality control

Hundreds of sensor readings compress to a 2D fault detection chart.

200D → 2D

Variance kept  88%

● Medical imaging / EEG

Brain signal channels are decorrelated to isolate neural components.

256D → 30D

Variance kept  90%

PCA Real time scenarios

Scenario	Input dims	Output dims	Reduction
Face recognition	4,096 pixels	150 eigenfaces	27×
Genomics (SNPs)	500,000 markers	10 components	50,000×
Finance (stocks)	500 stocks	20 risk factors	25×
NLP (LSA)	50,000 vocab	300 topics	167×
Manufacturing sensors	200 signals	2 chart axes	100×
EEG brain signals	256 channels	30 components	8.5×

PCA to recognize Digits

- Each Number represented in 64 pixels

pixel_0	pixel_1	pixel_2	pixel_3	...	pixel_63	target
0	0	5	13	...	0	0

Value	Meaning
0	White / Empty
4	Light Gray
8	Medium Gray
12	Dark Gray
16	Black

PCA to recognize Digits

- **Problem Before PCA**
- For every digit:
 - 64 features
- Machine Learning must process:
 - 1797 rows $\times$ 64 columns
- Many pixels:
 - contain similar information
 - are empty
 - contribute very little

PCA to recognize Digits

- **After PCA**
- Suppose we use:
 - `PCA(n_components=20)`
- Now:
 - 64 Features
 - ↓
 - 20 Features

PCA to recognize Digits

Achievement	Before PCA	After PCA	Benefit
Number of Features	64 pixel columns	20 Principal Components	Reduced dimensionality
Data Complexity	High	Lower	Easier for ML algorithms
Training Speed	Slower	Faster	Less computation required
Memory Usage	More memory needed	Less memory needed	Efficient resource utilization
Redundant Information	Many correlated pixels	Correlation removed	Cleaner data
Noise Handling	Noise present in pixels	Much noise removed	Better generalization
Visualization	Impossible to visualize 64 dimensions	Easy to plot in 2D/3D	Better understanding
Storage Requirement	64 values per image	20 values per image	Reduced storage
Model Performance	May overfit	Less chance of overfitting	Better prediction stability
Interpretability of Patterns	Raw pixels difficult to analyze	Major patterns captured in PCs	Easier pattern discovery
Processing Large Datasets	Expensive	Efficient	Scalable solution
Information Retained	100%	Typically 90–95%	Minimal information loss

Cross Validation

- A technique to evaluate how well a model **generalizes** to unseen data by splitting the dataset into multiple train/test partitions.

Common Methods

Method	Description	Best For
K-Fold	Split data into K folds; train on K-1, test on 1, rotate	General use
Stratified K-Fold	K-Fold but preserves class distribution	Imbalanced classes
Leave-One-Out (LOO)	Each sample is its own test set	Small datasets
Time Series Split	Respects temporal order; no future leakage	Sequential data
Holdout	Simple train/val/test split	Large datasets

Cross Validation

- **Why it matters**

- Detects **overfitting** — high train score, low CV score
- Gives a **more reliable** performance estimate than a single split
- Reduces sensitivity to how the data was split

Hyperparameter Tuning

- Finding the optimal **configuration** of a model before training (not learned from data).

Strategies

Strategy	How it works	Pros / Cons
Grid Search	Exhaustive search over all combinations	Thorough / Slow
Random Search	Randomly samples combinations	Fast / Misses optimal
Bayesian Optimization	Builds a probabilistic model of the search space	Efficient / Complex
Halving / Successive	Prunes poor candidates early	Fast / Needs large data

Hyperparameter Tuning

- **Key hyperparameters by model type**
- **Trees / Forests** → max_depth, n_estimators, min_samples_split
- **Neural Nets** → learning_rate, batch_size, dropout, hidden layers
- **SVM** → C (regularization), kernel, gamma
- **Regularized Regression** → alpha (Ridge/Lasso), l1_ratio (ElasticNet)
- **Best practice**
- Always tune inside CV — wrap your tuner in an **outer CV loop** to avoid leaking validation signal into hyperparameter choices (nested cross-validation).

Metric Selection

Choosing the **right measure** for your problem — accuracy alone is almost never enough.

Classification

Metric	Formula	Use When
Accuracy	$(TP+TN)/Total$	Balanced classes
Precision	$TP/(TP+FP)$	Cost of false positives is high (spam)
Recall	$TP/(TP+FN)$	Cost of false negatives is high (cancer)
F1 Score	$2 \times (P \times R) / (P + R)$	Imbalanced, both errors matter
ROC-AUC	Area under ROC curve	Ranking/probability calibration
PR-AUC	Area under Precision-Recall	Heavily imbalanced data
MCC	Balanced metric for all 4 quadrants	Imbalanced, single-number summary

Metric Selection



Regression

Metric	Sensitive To	Use When
MAE	Outliers (less)	Robust, interpretable errors
MSE / RMSE	Outliers (more)	Large errors should be penalized more
R^2	Scale-free	Explaining variance, comparisons
MAPE	% error	Business contexts needing % interpretation
Huber Loss	Mix of MAE + MSE	Outlier-robust but smooth

Metric Selection



Regression

Metric	Sensitive To	Use When
MAE	Outliers (less)	Robust, interpretable errors
MSE / RMSE	Outliers (more)	Large errors should be penalized more
R^2	Scale-free	Explaining variance, comparisons
MAPE	% error	Business contexts needing % interpretation
Huber Loss	Mix of MAE + MSE	Outlier-robust but smooth

Metric Selection

Clustering (unsupervised)

Metric	Notes
Silhouette Score	Cohesion vs. separation, -1 to 1
Davies-Bouldin	Lower is better
Adjusted Rand Index	Needs ground truth labels

How They Interact

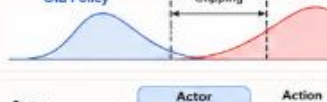
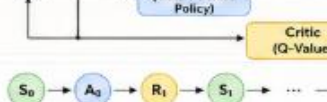
Data

- └─ Cross Validation (outer loop)
 - └─ Hyperparameter Tuning (inner loop / nested CV)
 - └─ Model trained & scored with chosen Metric

The **metric drives tuning** — tuning to accuracy on an imbalanced dataset gives you a useless model. The **CV structure ensures** your metric score is honest and not inflated by leakage. All three must be aligned with your **business objective**.

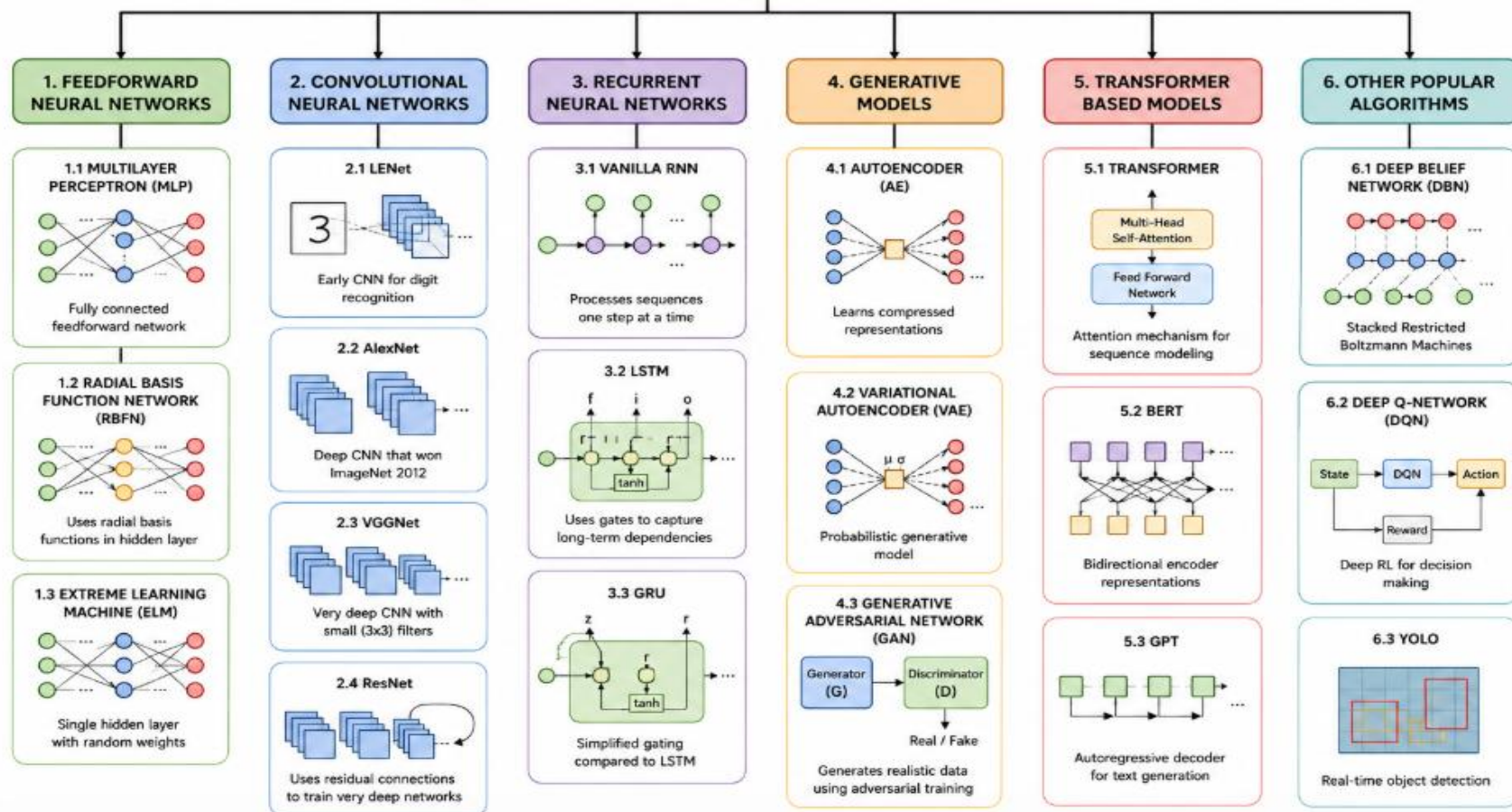
Reinforcement Learning

Reinforcement Learning Algorithms – At a Glance

Algorithm	Type	How it works (Idea)	Best for	Illustration / Diagram																									
1 Q-Learning	Model-Free (Value-Based)	Learns a Q-table that stores the expected future reward (Q-value) for each state-action pair. Updates using the Bellman equation.	- Discrete state/action spaces - Simple environments - Grid worlds, games	 Q-Table <table><thead><tr><th></th><th>A1</th><th>A2</th><th>...</th><th>A_n</th></tr></thead><tbody><tr><th>S1</th><td>1.2</td><td>0.7</td><td>...</td><td>0.1</td></tr><tr><th>S2</th><td>0.4</td><td>1.5</td><td>...</td><td>-0.3</td></tr><tr><th>...</th><td>...</td><td>...</td><td>...</td><td>...</td></tr><tr><th>S_n</th><td>0.5</td><td>0.2</td><td>...</td><td>0.9</td></tr></tbody></table>		A1	A2	...	A _n	S1	1.2	0.7	...	0.1	S2	0.4	1.5	...	-0.3	...	...	...	...	...	S _n	0.5	0.2	...	0.9
	A1	A2	...	A _n																									
S1	1.2	0.7	...	0.1																									
S2	0.4	1.5	...	-0.3																									
...	...	...	...	...																									
S _n	0.5	0.2	...	0.9																									
2 SARSA	Model-Free (Value-Based)	Similar to Q-Learning but updates Q-value using the action actually taken (on-policy).	- On-policy learning - Real-time decision making - Robotics, control tasks	 State Action Reward Next State Next Action S₁ A₁ R₂ S₂ A₂																									
3 Deep Q-Network (DQN)	Model-Free (Value-Based) (Deep Learning)	Uses a neural network to approximate Q-values instead of a table. Handles large state spaces like images.	- Atari games - High-dimensional states - Complex environments	 Q-values <table><tbody><tr><td>Action 1</td><td>0.3</td></tr><tr><td>Action 2</td><td>1.2</td></tr><tr><td>...</td><td>...</td></tr><tr><td>Action n</td><td>-0.1</td></tr></tbody></table>	Action 1	0.3	Action 2	1.2	...	...	Action n	-0.1																	
Action 1	0.3																												
Action 2	1.2																												
...	...																												
Action n	-0.1																												
4 Policy Gradient	Model-Free (Policy-Based)	Directly learns a policy (mapping from states to actions) and updates it to maximize expected reward.	- Continuous action spaces - Robotics, locomotion - Control problems	 Update Policy (e.g., REINFORCE)																									
5 Actor-Critic	Model-Free (Policy + Value)	Combines policy (actor) and value function (critic). Critic evaluates, actor improves.	- Continuous control - Robotics - Real-world applications																										
6 Proximal Policy Optimization (PPO)	Model-Free (Policy-Based) (Advanced)	Improved Policy Gradient. Uses a clipped objective to update policy in a stable way.	- Large, continuous spaces - Robotics, games - Real-world RL	 Restricts new policy to stay close to old policy for stable updates.																									
7 Deep Deterministic Policy Gradient (DDPG)	Model-Free (Actor-Critic) (Continuous)	Actor-Critic for continuous actions with deterministic policy. Uses experience replay & target networks.	- Continuous action control - Robotics - Autonomous systems																										
8 Monte Carlo Methods	Model-Free (Value-Based) (Sampling)	Learn value from complete episodes using actual returns. No bootstrapping.	- Episodic tasks - Finance, simulations - Small environments	 Episode $G_0 = \sum_{t=0}^T \gamma^t R_{t+1}$																									

★ **Key Takeaway:** Reinforcement Learning algorithms learn by trial and error. Choose the right algorithm based on state/action space, environment, and whether you want to learn values (what is good) or policies (what to do).

DEEP LEARNING ALGORITHMS

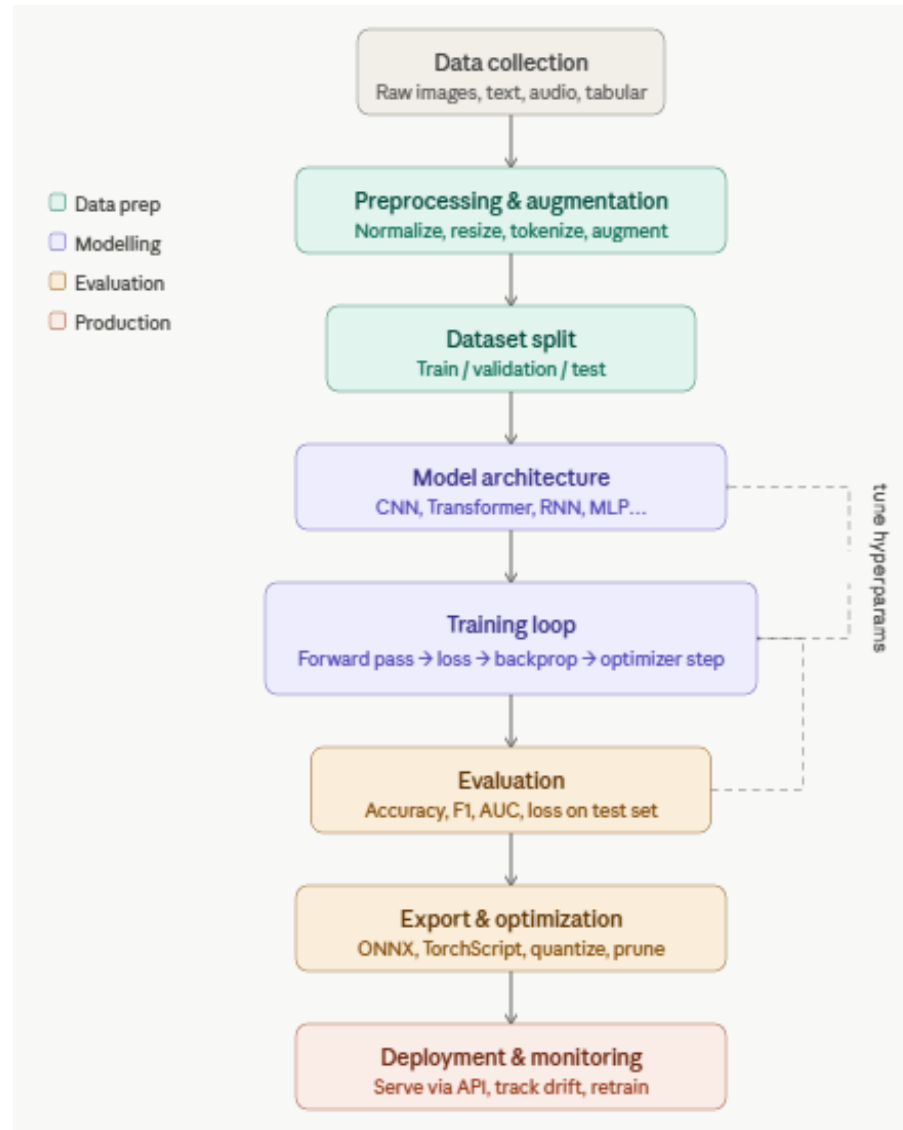


- DL Pipelines
 - TensorFlow/Keras training flows; callbacks; experiment tracking
 - GPU/CPU utilization; mixed precision (awareness)
 - Exporting SavedModel; ONNX conversion
- High-Performance Inference
 - TensorRT/Triton Serving (awareness)
 - Batch vs online inference; autoscaling
 - A/B and shadow deployments

Deep Learning Pipeline

- **A deep learning pipeline is a structured workflow that takes raw data through preprocessing, model design, training, evaluation, deployment, and monitoring — ensuring reproducibility and scalability in real-world AI projects.**
- It's the backbone of modern AI systems, used by companies like Tesla, Netflix, and Amazon to continuously improve their models.

Deep Learning Pipeline



Deep Learning Pipeline

1.Data Collection

- Gather images, text, audio, tabular data, etc.
- Sources: APIs, databases, sensors, web scraping.

2.Data Preprocessing

- Clean missing or noisy data.
- Normalize/scaling.
- Tokenization (NLP).
- Image resizing/augmentation.

3.Dataset Splitting

- Training set → learn patterns.
- Validation set → tune model.
- Test set → final evaluation.

4.Feature Engineering

- Traditional ML uses this heavily.
- Deep learning often learns features automatically.

5.Model Selection

Examples:

- CNN → images
- RNN/LSTM/Transformer → sequences/text
- Autoencoder → compression/anomaly detection

Deep Learning Pipeline

6. Training

- Forward propagation
- Loss calculation
- Backpropagation
- Weight updates using optimizers like Adam/SGD

7. Evaluation

Common metrics:

- Accuracy
- Precision/Recall
- F1-score
- ROC-AUC
- BLEU (NLP)

8. Hyperparameter Tuning

- Learning rate
- Batch size
- Number of layers
- Epochs
- Dropout rate

9. Deployment

- REST API
- Mobile app
- Edge device
- Cloud inference

Deep Learning Pipeline

10. Monitoring

- Detect drift
- Retrain periodically
- Track latency & accuracy
- Popular tools:
 - Data: Pandas, NumPy
 - DL Frameworks: TensorFlow, PyTorch
 - Deployment: Docker, AWS
 - Experiment Tracking: MLflow
- A compact modern MLOps-style pipeline:
 - Raw Data → Data Pipeline → Training → Validation → Model Registry → CI/CD → Deployment → Monitoring

Comparison: ML vs Deep Learning Pipelines

Aspect	ML Pipeline	Deep Learning Pipeline
Feature Engineering	Manual, domain-driven	Often automated via representation learning
Model Complexity	Simpler (linear models, trees)	Complex (CNNs, RNNs, Transformers)
Data Needs	Works with smaller datasets	Requires massive datasets
Compute Needs	Moderate (CPU)	High (GPU/TPU)
Deployment	Lightweight	Often requires specialized serving infrastructure

What is a tensor?

- Sales team tracks numbers in spreadsheets.
- A single number (revenue: \$5M) is a **scalar** — 0 dimensions.
- A row of monthly figures is a **vector** — 1 dimension.
- A full quarterly table is a **matrix** — 2 dimensions.
- The moment you stack 12 months of tables across 5 regions across 3 product lines, you have a **tensor** — 3+ dimensions.
- TensorFlow got its name because neural networks are machines that transform tensors.

What is a tensor?

Think of it as structured data with dimensions

Scalar	Vector	Matrix	Tensor
Rank 0 · shape () Revenue: \$5M NPS: 72 Churn: 3.2% 	Rank 1 · shape (12,) 12 months of revenue figures 	Rank 2 · shape (12, 5) 12 months × 5 regions 	Rank 3+ · shape (12,5,3) Months × Regions × Product lines 

The name "TensorFlow" = the flow of tensors through a neural network's computation graph

Why Tensor Flow

3,500+

companies in production

180M+

PyPI downloads / month

10 yrs

battle-tested at Google

Production-grade

TF Serving, TF Lite, TF.js — one model, every platform

Enterprise trust

Google, Airbus, Uber, Coca-Cola — long-term backing

Scales to any HW

MirroredStrategy() splits across 8 GPUs in one line

Massive talent pool

Most-searched ML framework on LinkedIn

Full ecosystem

TensorBoard, TF Data, TF Hub, Keras — one toolkit

MLOps built in

TFX: ingest → validate → train → serve, versioned

Why Tensor Flow and Keras

Industry Standard



Used by Google, Twitter, Airbnb & 90%+ of Fortune 500 ML teams.

Two-Layer Simplicity



Keras is the high-level API. TensorFlow is the powerful engine underneath.

Beginner-Friendly



A working neural network in under 10 lines of code.

Production-Ready



The same framework takes you from prototype to deployment at scale.

Tensor Flows

- TensorFlow is an open-source Deep Learning and Machine Learning framework developed by Google.
- It helps developers build and train:
 - Neural Networks
 - Deep Learning Models
 - Computer Vision Systems
 - Natural Language Processing Applications
 - Recommendation Engines
 - Generative AI Applications

Tensor Flows

- Think of TensorFlow as:
- "A factory that automatically learns patterns from data and makes intelligent predictions."
- It can run on:
 - CPU
 - GPU
 - TPU (Google Tensor Processing Unit)
 - Mobile devices
 - Cloud platforms

CPU vs GPU vs TPU



GENERAL PURPOSE

CPU

Central Processing Unit

Cores	4 – 128
Clock speed	3 – 5 GHz
Parallelism	Low
Memory bandwidth	~100 GB/s
ML training speed	Baseline (1x)
Best for	Sequential logic, control flow

PARALLEL COMPUTE

GPU

Graphics Processing Unit

Cores	3,000 – 18,000
Clock speed	1 – 2 GHz
Parallelism	High
Memory bandwidth	~3,000 GB/s
ML training speed	~50x CPU
Best for	Matrix ops, vision models

ML-OPTIMISED

TPU

Tensor Processing Unit

Cores	Systolic array (custom)
Clock speed	~900 MHz
Parallelism	Extreme (tensor- native)
Memory bandwidth	~900 GB/s + HBM
ML training speed	~100x CPU
Best for	Large-scale DL training

Relative training throughput

larger model training on ImageNet-scale data

CPU vs GPU vs TPU

CPU



GPU (A100)



TPU v4 pod



CPU = a professor

Brilliant. Handles any problem. Does one complex thing at a time very well. Great for decisions, branching logic, anything sequential.

GPU = a lecture hall of students

Thousands of simpler cores all working in parallel. Each core is less powerful, but together they can process enormous batches of matrix multiplications simultaneously.

TPU = a factory assembly line

Designed from scratch for one job — matrix multiply-accumulate. Every transistor serves that purpose. Nothing wasted on general compute. Outrageously fast at exactly what DL needs.

When to use each in TensorFlow

CPU

Data preprocessing

Small models

Inference on edge

Debugging

GPU

CNN training

NLP fine-tuning

Most production models

Real-time inference

TPU

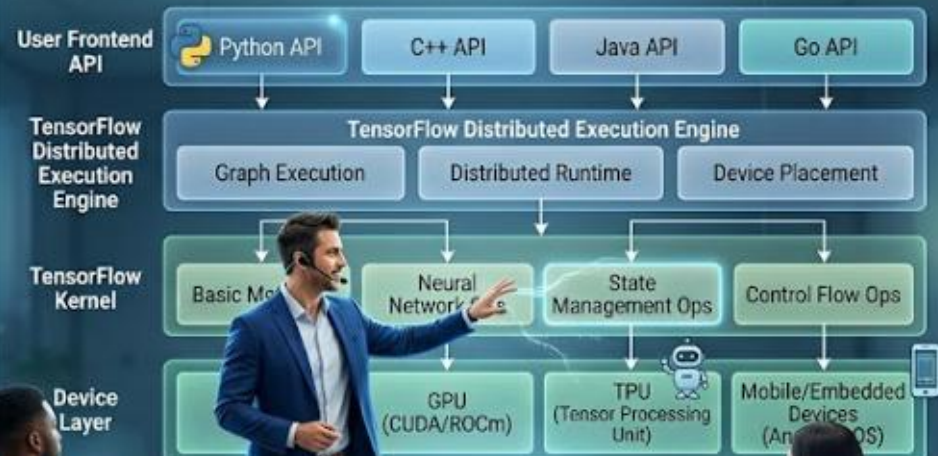
LLM pre-training

Transformer at scale

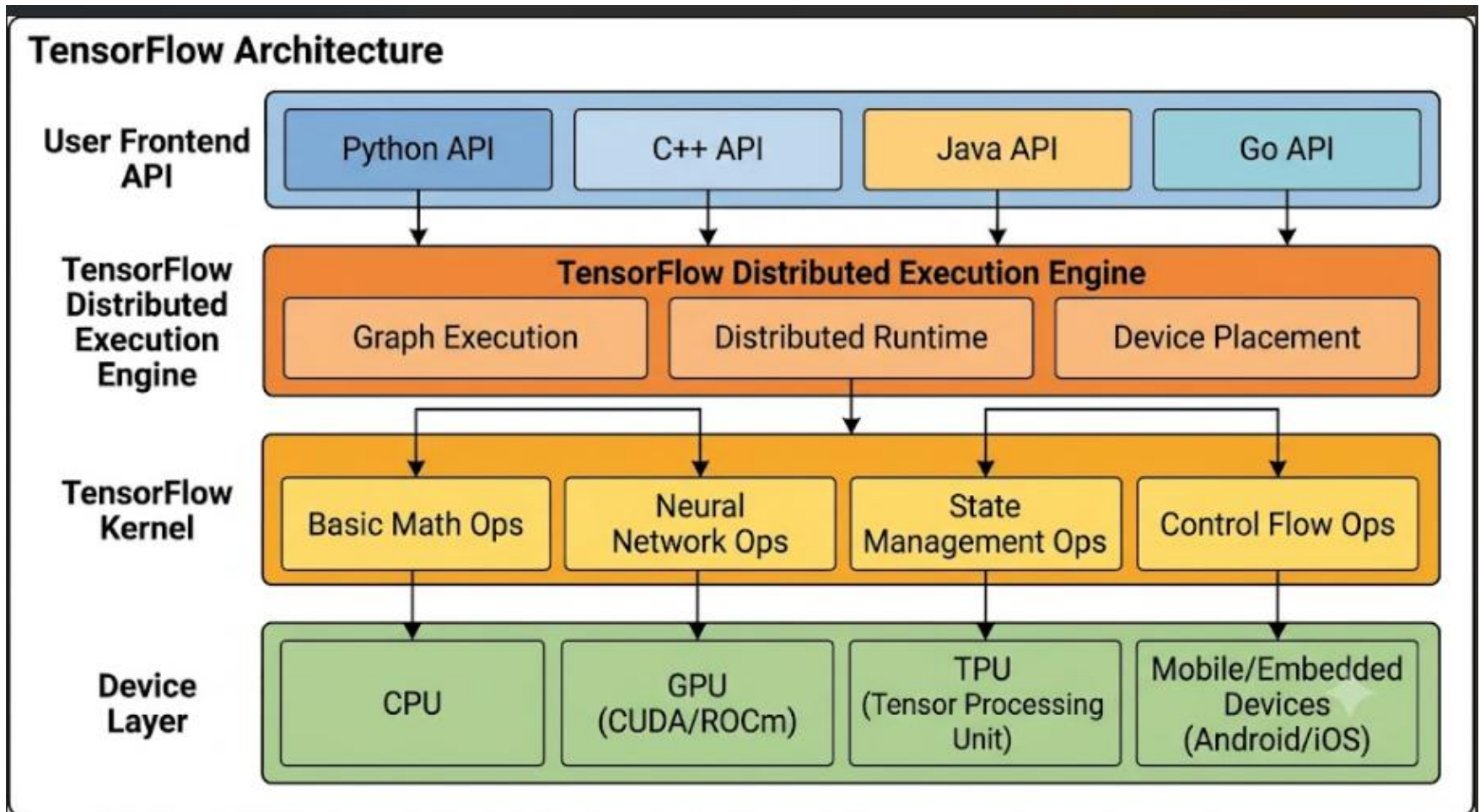
Google-scale workloads

Research frontier

TensorFlow Architecture



TensorFlow Architecture



Tensor Flow Architecture

- **1. The User Frontend API (The Command Center)**
- This is where our teams operate.
- When our data scientists write a line of code, they are using one of these: **Python, C++, Java, or Go.**
- The important thing to understand is that TensorFlow is designed to translate your preference.
- We choose **Python API** for rapid development and testing because it's flexible.
- But if we need maximum speed or to embed it into a C++ application, we can use the **C++ API.**
- This layer is about choice and usability, translating your commands into the internal TensorFlow representation.

Tensor Flow Architecture

- **2. TensorFlow Distributed Execution Engine (The Control Tower)**
- Once we've defined your model at the top, the structure isn't run directly in Python. It's compiled into a **Graph**. This execution engine is the brains of the operation. It has three critical jobs:
- **Graph Execution:** It takes the entire computational plan and manages its flow, ensuring operations execute in the correct order.
- **Distributed Runtime:** This is where scalability lives. If we are running a tiny test, the engine handles it locally. But if we need to scale up—say, to process a petabyte of industrial sensor data—this engine knows how to break the work apart and distribute it across a massive cluster of servers.
- **Device Placement:** The engine looks at the operations and says, "This specific math op is better suited for a GPU," or "This neural net layer should run on a TPU." It makes those decisions automatically, moving data to where the computation is fastest.

Tensor Flow Architecture

- **3. TensorFlow Kernel (The Specialized Math Labs)**
- These are the highly optimized libraries of code written by Google's engineers.
- This isn't generic code; this is laser-focused, optimized math.
- When we run a command to *train a model*, we aren't running generic addition.
- We are running **Neural Network Ops**.
- When our system predicts which engine part is going to fail, it uses **Math Ops** and **Control Flow Ops**.
- This layer ensures the fundamental building blocks of machine learning run with extreme efficiency, squeezing every ounce of performance out of the math.

Tensor Flow Architecture

- **4. The Device Layer (The Muscle)**
- **CPU (Standard Server):** We use this for basic data preparation and small tests. It's reliable but not fast for complex math.
- **GPU (CUDA/ROCm):** This is our workhorse. We deploy GPUs when we need to accelerate model training, turning days of work into hours.

Tensor Flow Architecture

- **4. The Device Layer (The Muscle)**
- **TPU (Tensor Processing Unit):** *This is the differentiator.*
- When we need to deploy massively scaled models *in production*—like global language translation or deep financial analysis—we utilize Google’s custom TPUs.
- This hardware is physically designed *only* to run the exact math of TensorFlow (from Image 1), making it the ultimate tool for deep learning inference.

Tensor Flow Architecture

- **4. The Device Layer (The Muscle)**
- **Mobile/Embedded (Android/iOS):** And the most important segment for today's market: the edge.
- We can condense those powerful models, push them past the APIs, and execute them directly on smartphones and factory sensors.
- This means real-time intelligence *on device* without constant cloud connectivity.

Real world Use cases

Banking — Fraud detection

Tensor: [batch, 50, 28]

LSTM scores every card swipe in <100ms. Catches anomalies before the terminal responds.

Retail — Demand forecasting

Tensor: [store, 365, 12]

30-day SKU-level forecasts. A 10% accuracy gain = billions in saved inventory.

Healthcare — Medical imaging

Tensor: [1, 512, 512, 3]

ResNet-50 matches specialist accuracy for diabetic retinopathy screening.

Manufacturing — Predictive maintenance

Tensor: [machine, 1000, 64]

1D ConvNet on sensor streams detects turbine faults days before failure.

Media — Recommendations

Tensor: [user, 200]

Two-tower model: user embedding × item embedding. Ranks millions of items in real time.

TensorFlow vs Alternatives

TensorFlow choose when...

- ✓ Deploying to production at scale (mobile, web, server)
- ✓ Multi-device / multi-cloud training needed
- ✓ Regulated industry requiring audit & governance
- ✓ Full MLOps pipeline (TFX) is a priority
- ✓ Long-term enterprise support matters

PyTorch choose when...

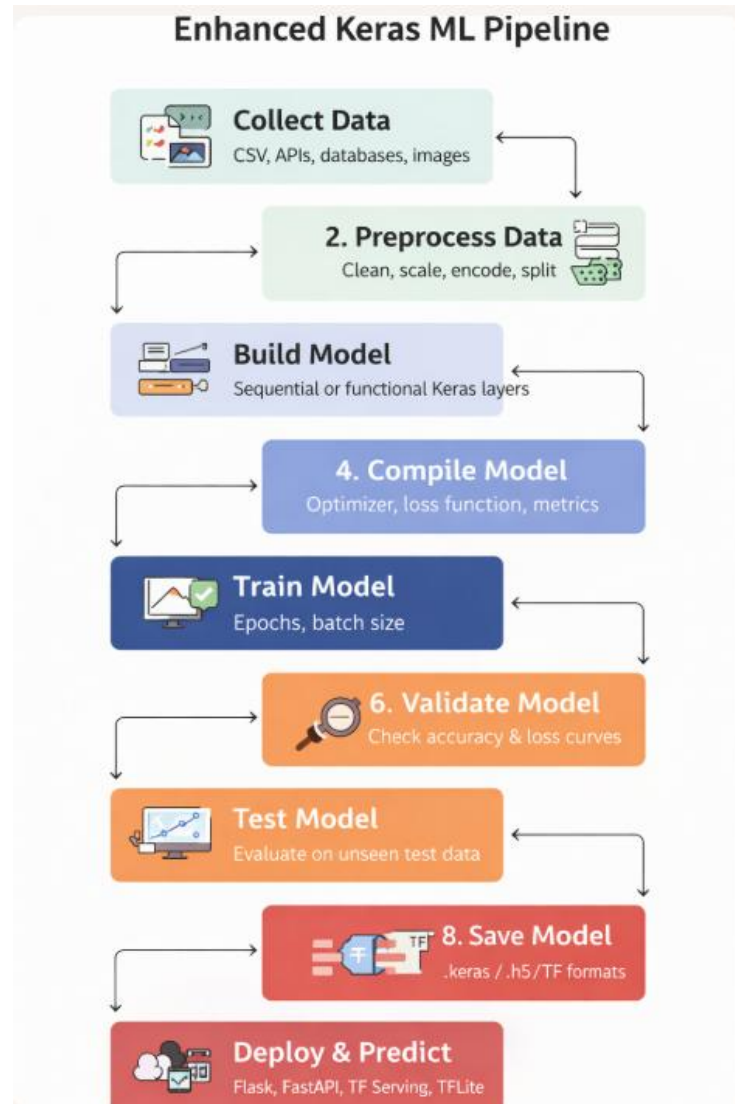
- ✓ Research and rapid experimentation first
- ✓ Team prefers Pythonic, eager-first debugging
- ✓ Academia / publishing papers (dominant there)
- ✓ Hugging Face model ecosystem is the priority
- ✓ Smaller team, fewer deployment constraints

HOW TENSORFLOW MATURED — A BRIEF TIMELINE

- 2011 ● **DistBelief** — Google's internal predecessor. Proved that large-scale neural nets were industrially viable.
- 2015 ● **TensorFlow 1.0 open-sourced**. Google releases it to the world. Becomes the dominant framework within 12 months.
- 2017 ● **Keras integration + TF Lite**. Mobile deployment unlocked. Keras makes it accessible to non-ML engineers.
- 2019 ● **TensorFlow 2.0**. Eager execution by default — finally as easy to debug as PyTorch. `tf.function` for performance.
- 2021+ ● **TPU optimisation + JAX convergence**. Google's custom silicon (TPUs) gives TF models a 10–100× speed advantage for large-scale training. TF and JAX increasingly share infrastructure.

- **Keras** is an open-source, high-level deep learning API written in Python.
- Designed with the philosophy of being "for human beings, not machines," it prioritizes user experience, speed of prototyping, and ease of debugging.
- Originally created by François Chollet in 2015, Keras has evolved significantly—most notably with **Keras 3**, which operates as a unified, multi-backend framework.
- TensorFlow is the car engine. Keras is the steering wheel, dashboard, and pedals.
- We don't need to know how the engine works to drive — but the engine is what gives us the power.

Tensor/Keras Training Flow



Scikit-Learn vs TensorFlow

- **Real-World Example**
- Imagine you want to build a **Loan Approval System**.
- **Scikit-Learn**
- Uses traditional Machine Learning algorithms:
 - Linear Regression
 - Logistic Regression
 - Decision Tree
 - Random Forest
 - SVM
 - K-Means

Scikit-Learn vs TensorFlow

- **Real-World Example**
- **TensorFlow**
- Uses Deep Learning algorithms:
 - Neural Networks
 - CNN
 - RNN
 - LSTM
 - Transformers
 - BERT
 - GPT
 - GAN

Key Differences

Feature	Scikit-Learn	TensorFlow
Type	Machine Learning	Deep Learning
Learning Curve	Easy	Moderate
Training Speed	Fast	Slower
Dataset Size	Small-Medium	Medium-Huge
Images	Limited	Excellent
Text Processing	Basic	Excellent
Speech Recognition	Not Ideal	Excellent
GPU Support	Minimal	Strong
Neural Networks	Basic MLP	Advanced
Production Scale	Good	Excellent

Dataset Recommendation

Records	Preferred Tool
100	Scikit-Learn
1,000	Scikit-Learn
10,000	Scikit-Learn
100,000	Either
1 Million+	TensorFlow

Performance Comparison

Scenario	Winner
House Price Prediction	Scikit-Learn
Customer Churn	Scikit-Learn
Fraud Detection (small data)	Scikit-Learn
Face Recognition	TensorFlow
Speech Recognition	TensorFlow
ChatGPT-like Systems	TensorFlow
Medical Image Analysis	TensorFlow
Recommendation Engine	TensorFlow

The 3 Steps to Every Keras Model

01

Define

Architecture

Choose your layers, neurons, and activation functions. Sequential API for simple models, Functional API for complex ones.

```
model = keras.Sequential([...])
```

02

Compile

Configuration

Set the optimizer (how to learn), loss function (what to minimize), and metrics (what to measure).

```
model.compile(optimizer, loss,  
metrics)
```

03

Fit

Training

Pass your data. Keras handles forward pass, loss calculation, backpropagation, and weight updates automatically.

```
model.fit(X_train, y_train,  
epochs=N)
```

Key Hyperparameters for Beginners

Activation Functions

Activation	When to use
ReLU	Hidden layers (default choice)
Sigmoid	Binary output (0 or 1)
Softmax	Multi-class output
Linear	Regression (no activation)

Loss Functions

Loss Function	When to use
<code>binary_crossentropy</code>	2-class classification
<code>sparse_categorical_crossentropy</code>	Multi-class (integer labels)
<code>categorical_crossentropy</code>	Multi-class (one-hot labels)
<code>mean_squared_error</code>	Regression problems

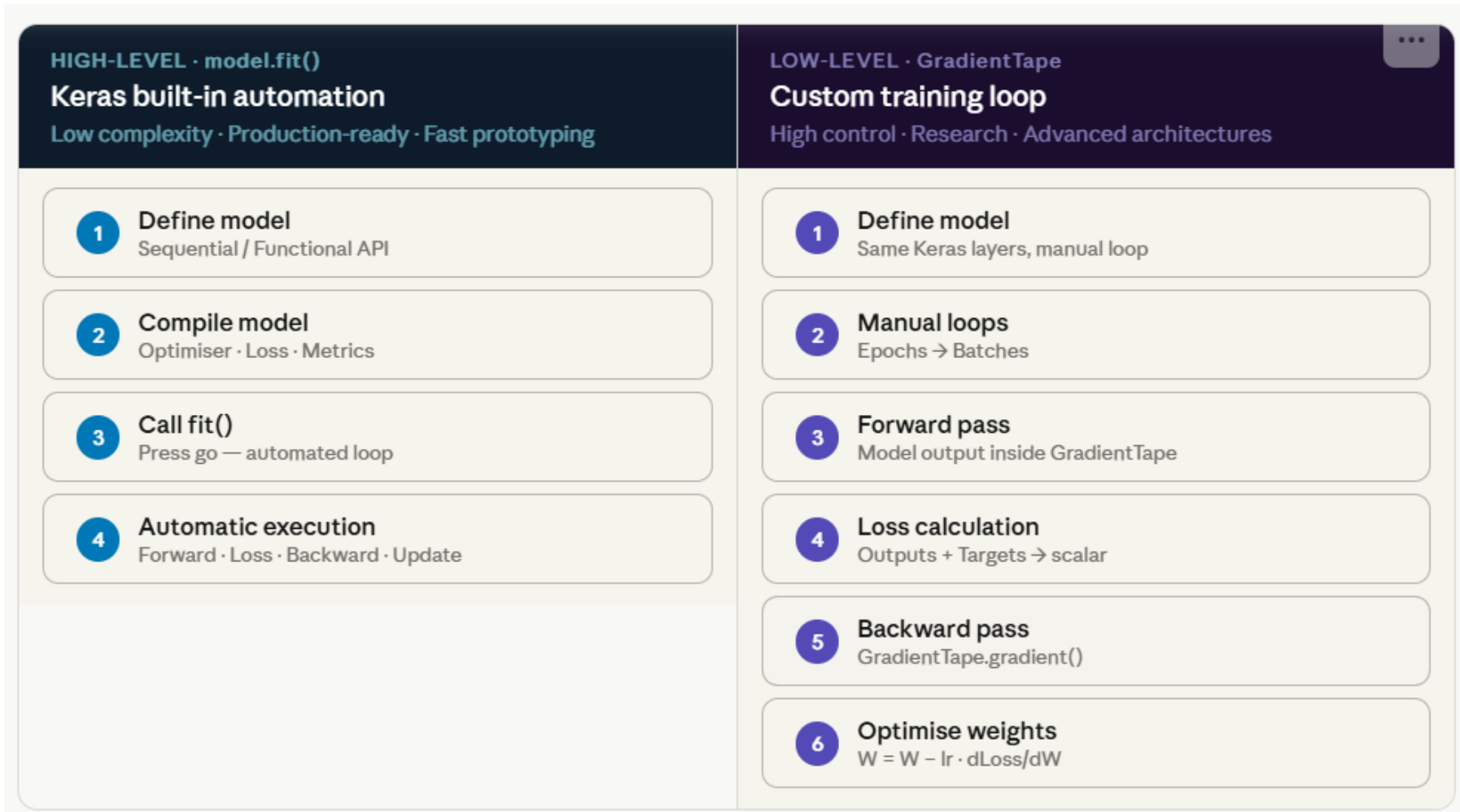
Tensor/Keras Training Flow

Feature	TensorFlow	Keras
Level	Low-level + High-level	High-level
Learning Curve	Steeper	Easier
Coding Effort	More	Less
Flexibility	Very High	High
Development Speed	Slower	Faster
Best For	Research, custom architectures	Business applications
Beginner Friendly	Moderate	Excellent
Production Ready	Yes	Yes

Tensor/Keras Training Flow

- **TensorFlow** is the full framework — it handles tensors, automatic differentiation, GPU acceleration, deployment, and all the low-level maths.
- You could build a neural network using only raw TensorFlow, but it'd take 50+ lines for something simple.
- **Keras** is a high-level API that lives *inside* TensorFlow (as `tf.keras`). It wraps all that complexity into clean, readable code.
- When you write `model.fit()` or `Dense(4, activation='relu')`, you're using Keras — and Keras calls TensorFlow under the hood to do the actual work.

Tensor/Keras Training Flow





Key Features & Architecture

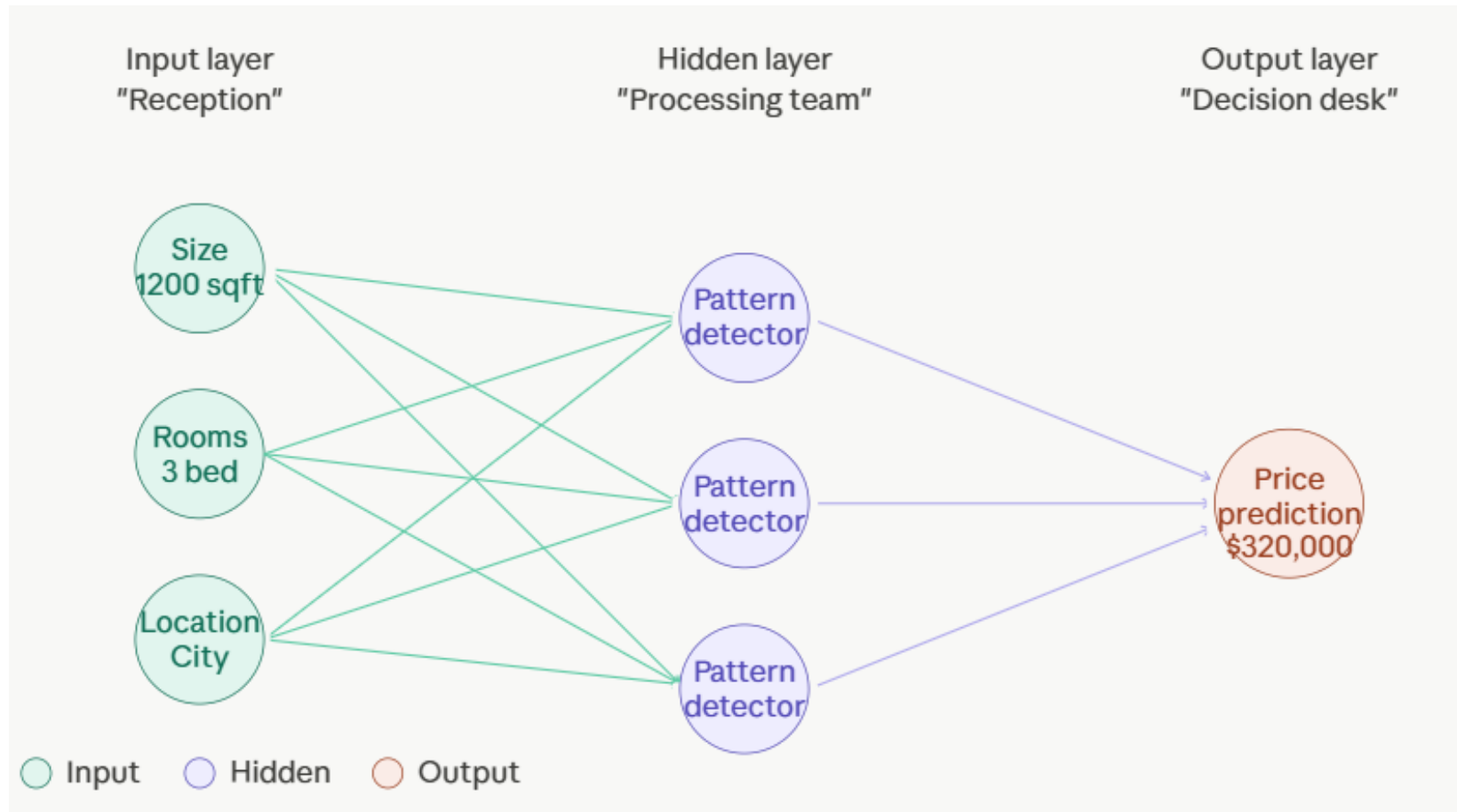


- **Multi Backend Freedom:**
 - Keras does not handle low-level tensor operations itself.
 - Instead, it acts as an interface on top of three major computational engines: JAX, TensorFlow, and PyTorch.
 - We can write our code once and run it on whichever backend offers the best performance for your specific hardware.
- **Performance Optimization:**
 - By leveraging backends like JAX or TensorFlow XLA, Keras can achieve major speedups during training and deployment.
- **Ecosystem Integration:**
 - Through KerasHub (formerly KerasNLP/KerasCV), users have instant access to pre-trained, state-of-the-art foundation models like Gemma, Llama, and Stable Diffusion.

Simple Neural Network

- A neural network is like a **company with departments**.
- Raw information comes in the front door, passes through teams of workers who each process it a little, and a final decision comes out the other end.
- Each worker looks at what they receive, does a small calculation, and passes their output to the next department.

Simple Neural Network



Why Activation Function?

- Imagine a loan approval system.
- Inputs:
 - Salary = 80000
 - Credit Score = 750
 - Experience = 10 years
- A neuron calculates:
 - $Z = W_1X_1 + W_2X_2 + W_3X_3 + \text{Bias}$
- Suppose:
 - $Z = 15$
- Now what?
 - Should 15 mean Approved?
 - Should 15 mean Rejected?
- Activation functions convert this raw value into something meaningful.

Why Activation Function?

Step 1: Input Values

Suppose we have a Loan Approval System.

Feature	Value
Salary	80,000
Credit Score	750
Experience	10 years

These become the neuron inputs:

$$X_1 = 80000$$

$$X_2 = 750$$

$$X_3 = 10$$

Why Activation Function?

Step 2: Assign Weights

The neural network learns weights during training.

Suppose:

$$W_1 = 0.0001$$

$$W_2 = 0.01$$

$$W_3 = 0.2$$

Bias:

$$b = -2$$

Why Activation Function?

Step 3: Calculate Weighted Sum (Z)

Neuron formula:

$$Z = W_1X_1 + W_2X_2 + W_3X_3 + b$$

Substitute values:

$$Z = (0.0001 \times 80000) + (0.01 \times 750) + (0.2 \times 10) + (-2)$$

Salary Contribution

$$0.0001 \times 80000 = 8$$

Credit Score Contribution

$$0.01 \times 750 = 7.5$$

Experience Contribution

$$0.2 \times 10 = 2$$

Add Bias

$$Z = 8 + 7.5 + 2 - 2$$

$$Z = 15.5$$

Approximately:

$$Z = 15$$

Why Activation Function?

Step 3: Calculate Weighted Sum (Z)

Neuron formula:

$$Z = W_1X_1 + W_2X_2 + W_3X_3 + b$$

Substitute values:

$$Z = (0.0001 \times 80000) + (0.01 \times 750) + (0.2 \times 10) + (-2)$$

Salary Contribution

$$0.0001 \times 80000 = 8$$

Credit Score Contribution

$$0.01 \times 750 = 7.5$$

Experience Contribution

$$0.2 \times 10 = 2$$

Add Bias

$$Z = 8 + 7.5 + 2 - 2$$

$$Z = 15.5$$

Approximately:

$$Z = 15$$

Why Activation Function?

Step 4: What Does $Z=15$ Mean?

This is the important question.

Does 15 mean:

✓ Loan Approved?

or

✗ Loan Rejected?

The neuron does not know.

15 is just a raw score.

Why Activation Function?

Step 5: Apply Activation Function

Activation functions convert the raw score into a meaningful output.

Case 1: Step Function

Rule:

$$\text{Output} = \begin{cases} 1 & Z > 0 \\ 0 & Z \leq 0 \end{cases}$$

Since:

$$Z = 15$$

$$15 > 0$$

Output:

$$1$$

Meaning:

✅ Loan Approved

Why Activation Function?

Case 2: Sigmoid Function

Formula:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Substitute:

$$Z = 15$$

$$\sigma(15) = \frac{1}{1 + e^{-15}}$$

$$e^{-15} = 0.000000305$$

$$\begin{aligned}\sigma(15) &= \frac{1}{1 + 0.000000305} \\ &= 0.9999997\end{aligned}$$

Output:

99.99997%

Meaning:

● Very high probability of approval

Why Activation Function?

Case 3: ReLU

Formula:

$$f(x) = \max(0, x)$$

Since:

$$Z = 15$$

$$f(15) = 15$$

Output:

15

ReLU keeps positive values unchanged.

Used mostly in hidden layers.

Real-Time Banking Interpretation

Customer	Z Value	Sigmoid Output	Decision
Poor applicant	-5	0.006	Reject
Average applicant	0	0.50	Borderline
Good applicant	5	0.993	Approve
Excellent applicant	15	0.9999997	Strong Approve

Simple Summary

- **Simple Trainer Summary**

- Inputs = Salary, Credit Score, Experience.
- Neuron multiplies each input by a weight.
- Adds them together with bias.
- Produces a raw score (**$Z = 15$**).
- Activation function converts 15 into a meaningful value.
- Sigmoid converts it into **99.99997% approval probability**.
- Final decision = **Approve Loan**.

Relu vs Sigmoid Activation Function

Pick a student: ☐

S5 — Fail

S1

$z = -4.45$

R:0.00

S:0.01

Fail

S2

$z = -3.59$

R:0.00

S:0.03

Fail

S3

$z = -2.74$

R:0.00

S:0.06

Fail

S4

$z = -1.88$

R:0.00

S:0.13

Fail

S5

$z = -1.03$

R:0.00

S:0.26

Fail

S6

$z = 0.35$

R:0.35

S:0.59

Pass

S7

$z = 1.43$

R:1.43

S:0.81

Pass

S8

$z = 2.28$

R:2.28

S:0.91

Pass

S9

$z = 3.37$

R:3.37

S:0.97

Pass

S10

$z = 4.45$

R:4.45

S:0.99

Pass

ReLU — hidden layers

$$\max(0, z)$$

$z > 0 \rightarrow$ pass signal through unchanged

$z \leq 0 \rightarrow$ output zero, stay silent

Used in: Dense layers 1 & 2

Selected student $z = -1.03$

ReLU output = 0.000

Status: Silent — output = 0

Sigmoid — output layer

$$1 / (1 + e^{-z})$$

Any $z \rightarrow$ squash to 0.0–1.0

Output = probability of passing

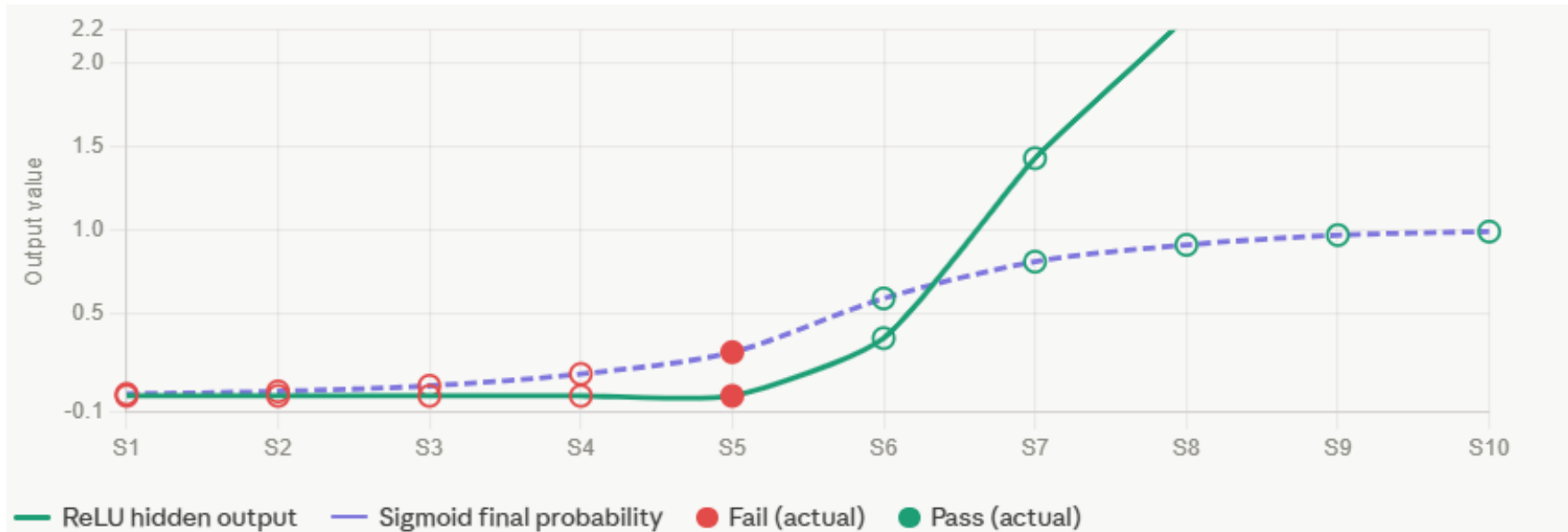
Used in: Final Dense layer

Selected student $z = -1.03$

Sigmoid output = 0.263

Verdict: FAIL (26% — below 50%)

Relu vs Sigmoid Activation Function



Rule of thumb: `relu` for every hidden layer | `sigmoid` only on the final output layer when predicting Pass / Fail

Slide through all 10 students and watch both outputs update live.

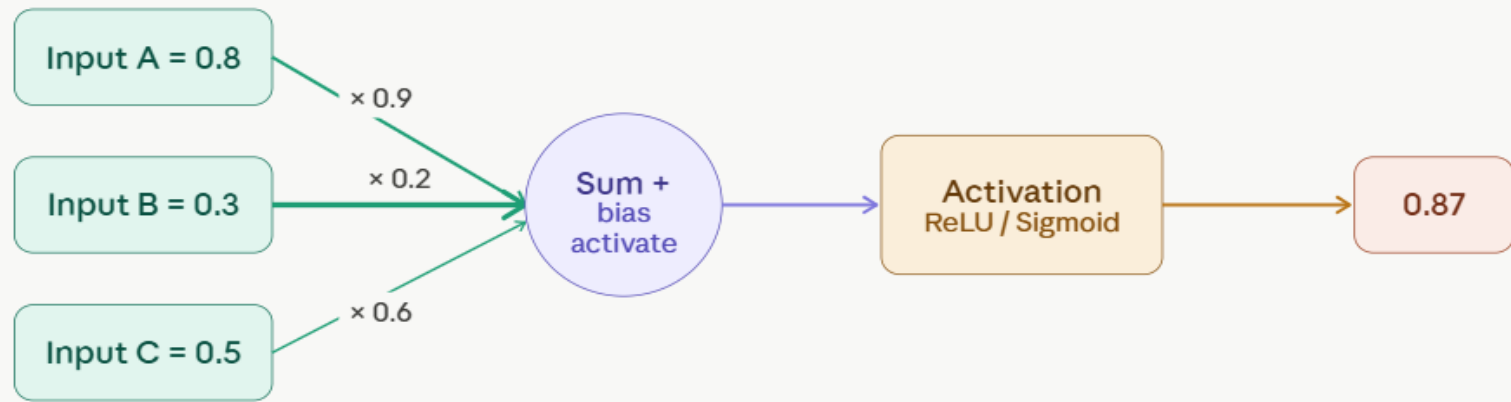
Quick Memory Trick

ReLU = Learning Activation
(Hidden Layers)

Sigmoid = Probability Activation
(Output Layer)

What one neuron actually does

- Think of a single neuron as a **junior analyst**.
- It receives multiple reports, decides how important each one is (that's the *weight*), adds them up, and if the total is significant enough, it fires off a signal.
- Otherwise, it stays silent.



1. Receive inputs

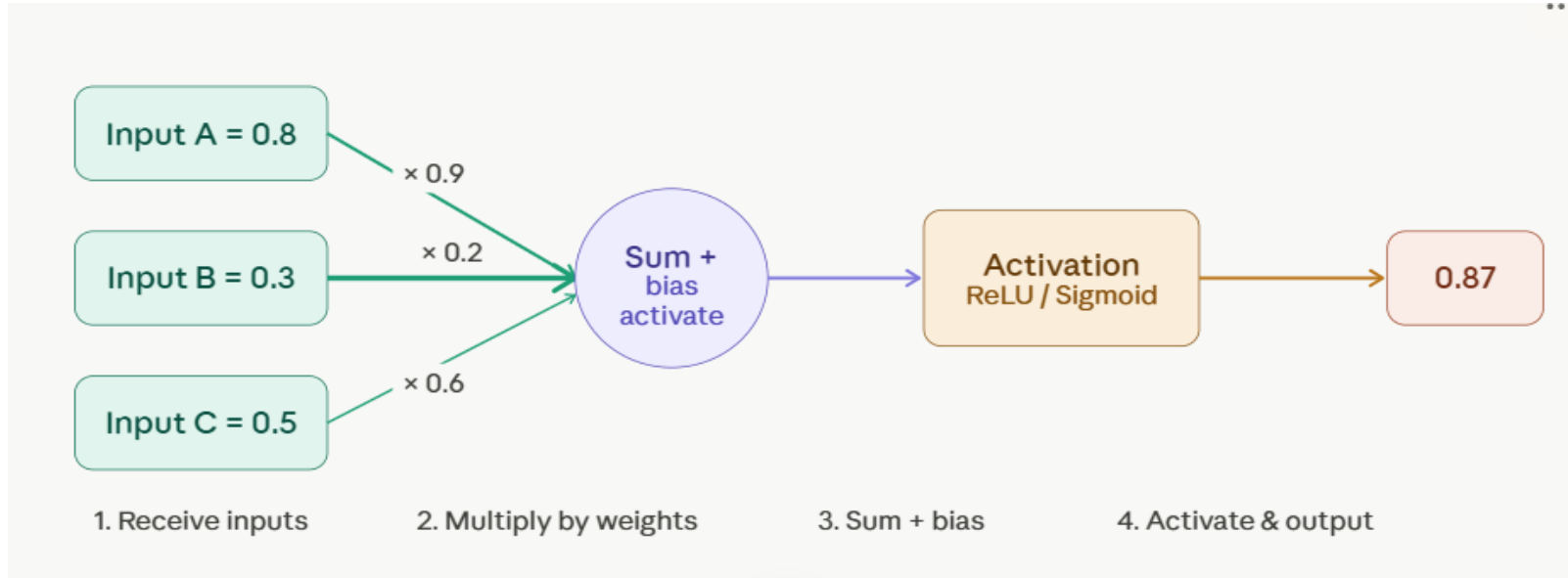
2. Multiply by weights

3. Sum + bias

4. Activate & output

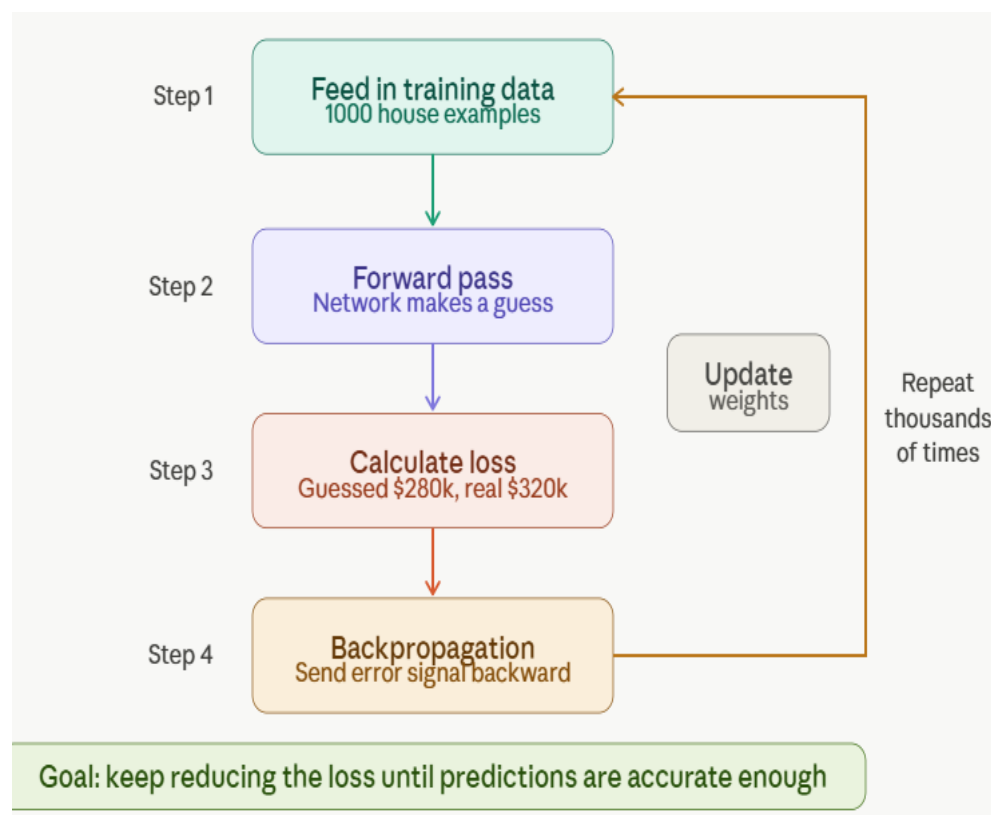
What one neuron actually does

- The thicker the arrow, the more important that input is. Weights are like the seniority of each report — a senior analyst's input (weight 0.9) gets more attention than an intern's (weight 0.2).

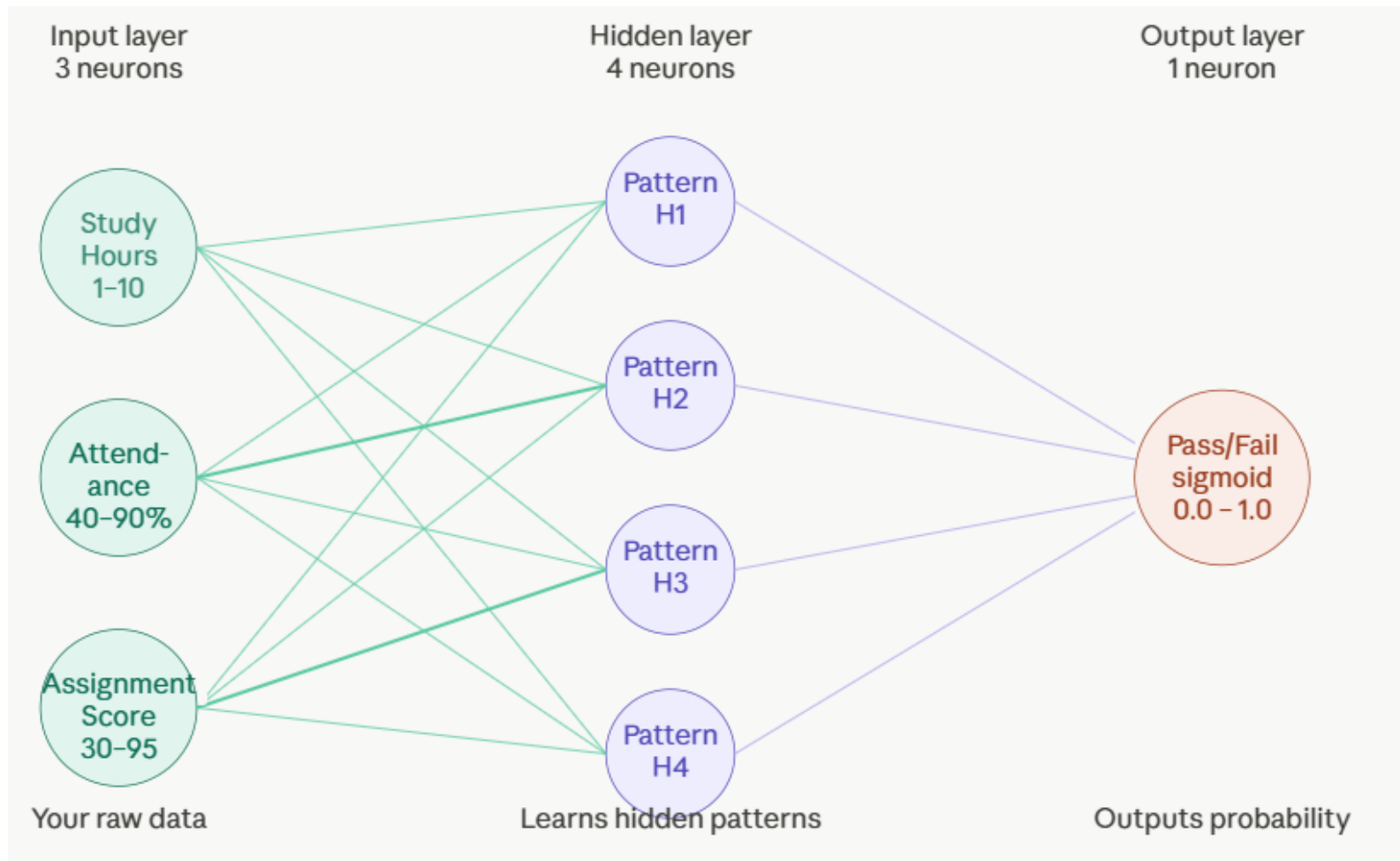


How the network learns (training loop)

- Here's the part most people find magical.
- The network doesn't get programmed with rules — it *learns by making mistakes*.
- Think of it like a new hire's first week.



Student NN



Student NN



Student	Study hours		Attendance		Assignment		Result
S1		1		40%		30	Fail
S2		2		45%		35	Fail
S3		3		50%		40	Fail
S4		4		55%		45	Fail
S5		5		60%		50	Fail
S6		6		70%		60	Pass
S7		7		75%		70	Pass
S8		8		80%		75	Pass
S9		9		85%		85	Pass
S10		10		90%		95	Pass

What each line does

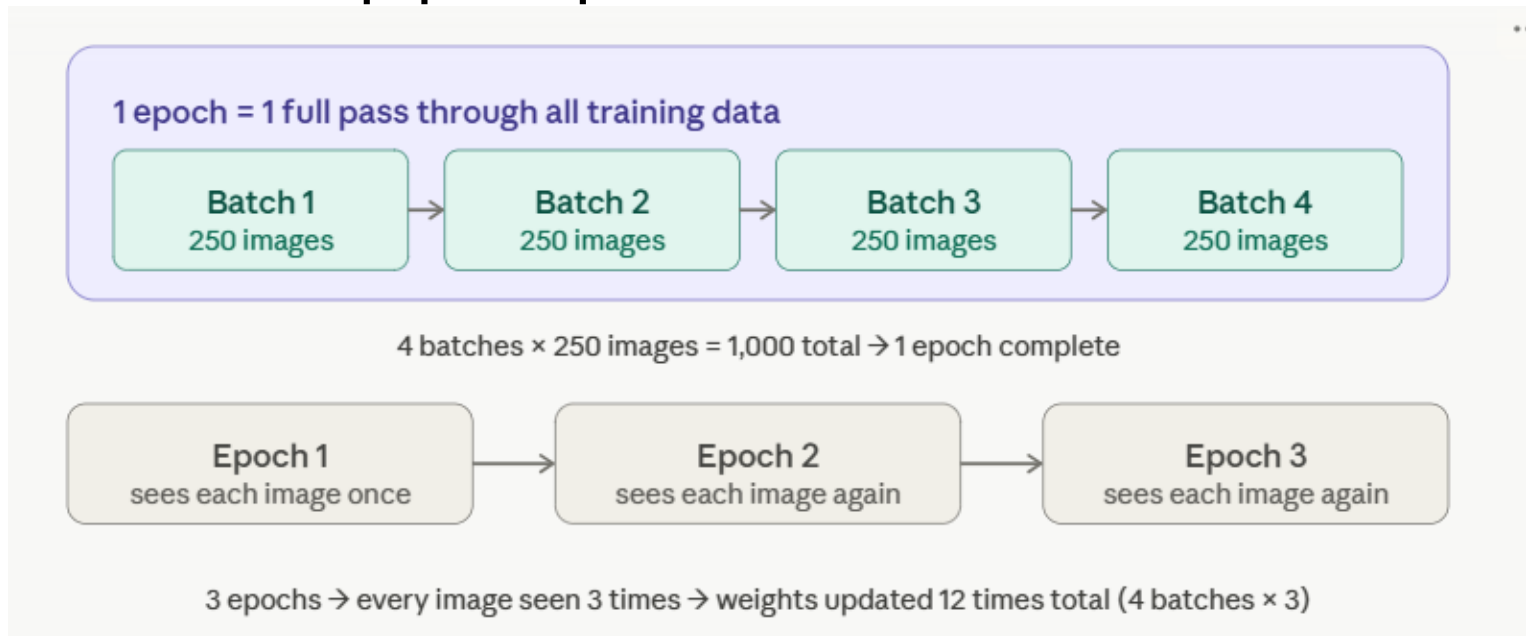
Code section	What it means in plain English
<code>Dense(4, activation='relu')</code>	Hire 4 analysts in the hidden department, each using ReLU — "only speak up if your total is positive"
<code>Dense(1, activation='sigmoid')</code>	One final decision-maker who outputs a number between 0 and 1 — the probability of passing
<code>binary_crossentropy</code>	The penalty formula — how badly did we misjudge? High when we said 0.9 but answer was 0
<code>adam</code> optimizer	A smart coach who adjusts how fast each weight updates based on recent mistakes
<code>epochs=300</code>	300 full review cycles over all 10 student records
<code>model.predict(...)</code>	Ask the trained network: "What would you say about this new student?"

Real-Time Use Cases

Scenario	Inputs	Output
Student result prediction	Study hours, attendance, marks	Pass / Fail
Loan approval	Salary, credit score, experience	Approved / Rejected
Diabetes prediction	Glucose, BMI, age	Diabetic / Not diabetic
Customer churn	Usage, complaints, payment delay	Churn / Not churn
Employee attrition	Salary, experience, rating	Leave / Stay

Epoch

- An **epoch** is one complete pass through your entire training dataset.
- If you have 1,000 images and train for 10 epochs, your model sees each image exactly 10 times total — one full sweep per epoch.



- A few related terms that are easy to confuse:
- **Epoch** — one complete pass through the full training dataset. After every epoch, Keras logs your metrics (loss, val_accuracy, etc.) and fires callbacks like EarlyStopping.
- **Batch** — a small chunk of the dataset processed together. Batch size is typically 32 or 64. The model weights are updated once per batch, not once per epoch.
- **Iteration (step)** — one weight update. If you have 1,000 images and batch size 250, you get 4 iterations per epoch.

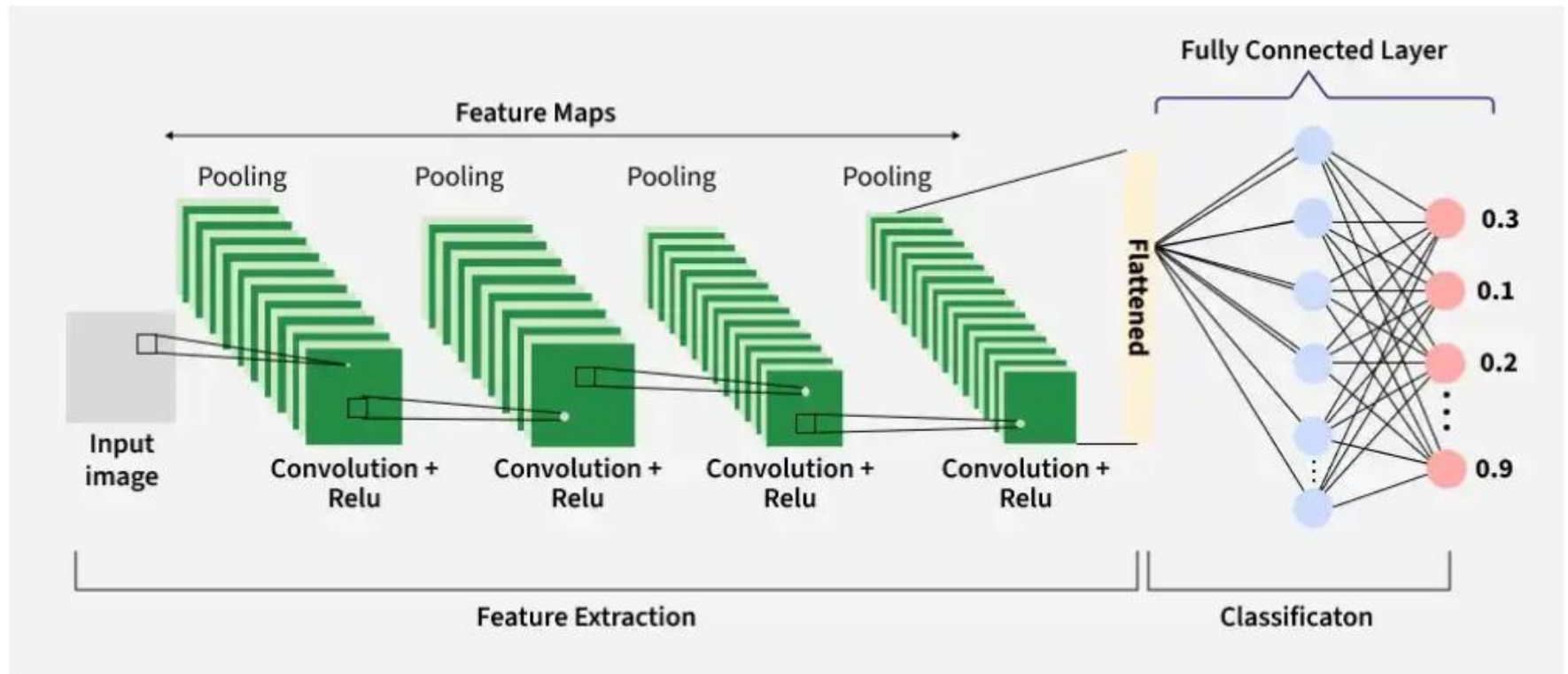
Epoch

- `model.fit(`
- `X_train,` # 1,000 images
- `y_train,`
- `batch_size=250,` # 4 batches per epoch
- `epochs=20,` # 20 full passes → 80 weight
 updates total
- `)`

CNN for Deep Learning

- Convolutional Neural Networks (CNNs), are neural network architectures inspired by the human visual system, designed to process image data by capturing spatial relationships between pixels.
 - Learn hierarchical features from simple edges to complex objects
 - Capture spatial patterns using convolution operations
 - Detect objects regardless of their position in the image
 - Reduce computation by focusing on local regions instead of the entire image

CNN for Deep Learning



CNN for Deep Learning

Input image



Detect patterns

Edges, curves, textures — without hand-coding

Work at every position

Same filter scans the whole image

Build hierarchically

Simple → complex features layer by layer

What Do CNNs Actually Learn?

Early

Layer 1: Edges & Colors

Gabor-like filters. Simple edge detectors at different angles and color gradients. Identical across all vision models.

e.g. / \ — | diagonal edges, color blobs

Mid

Layer 2–3: Textures & Parts

Combinations of edges form textures: fur, scales, fabric, grid patterns. Parts of objects begin to emerge.

e.g. Eyes, fur patches, circular patterns

Deep

Deep Layers: Objects & Concepts

High-level semantic concepts. Neurons activate for 'dog snout', 'wheel', 'human face'. Specific to training data.

e.g. Dog nose detector, tyre detector

Key Components of CNN

- **1. Input Layer**
- The input layer receives the raw image data and passes it to the network for processing.
- In CNNs, input is typically a 3D volume (width $\times$ height $\times$ depth).
- Stores pixel values of the image (e.g., $32 \times 32 \times 3$ for RGB images).
- Preserves the spatial structure of the image for further feature extraction.

Key Components of CNN

- **2. Convolutional Layer**

- The Convolutional Layer is responsible for extracting important features from the input data.
- It applies a set of learnable filters (kernels) that slide over the image.
- Computes the dot product between the filter weights and corresponding image patches, producing feature maps.
- Uses small filters (e.g., 2×2 , 3×3 , 5×5) to scan the input image.
- Generates feature maps that capture patterns such as edges, textures and shapes.
- ***Example:*** Using 12 filters results in an output volume of $32 \times 32 \times 12$.

Key Components of CNN

- **3. Activation Layer**
- The Activation Layer introduces non-linearity into the network by applying an element-wise activation function to the output of the convolution layer.
- This enables the model to learn complex patterns beyond linear relationships.
- Common activation functions include ReLU, Tanh and Leaky ReLU.
- Applied element-wise to the feature maps.
- The output dimensions remain unchanged (e.g., $32 \times 32 \times 12$).

Key Components of CNN

• 4. Pooling Layer

- The Pooling Layer is used to reduce the spatial dimensions of the feature maps, making computation faster, reducing memory usage and helping to prevent overfitting.
- It is typically inserted between convolutional layers in a CNN.
- Common types include Max Pooling and Average Pooling.
- Reduces width and height while keeping depth unchanged.
- ***Example:*** Using 2×2 max pooling with stride 2 reduces the volume from $32 \times 32 \times 12$ to $16 \times 16 \times 12$.

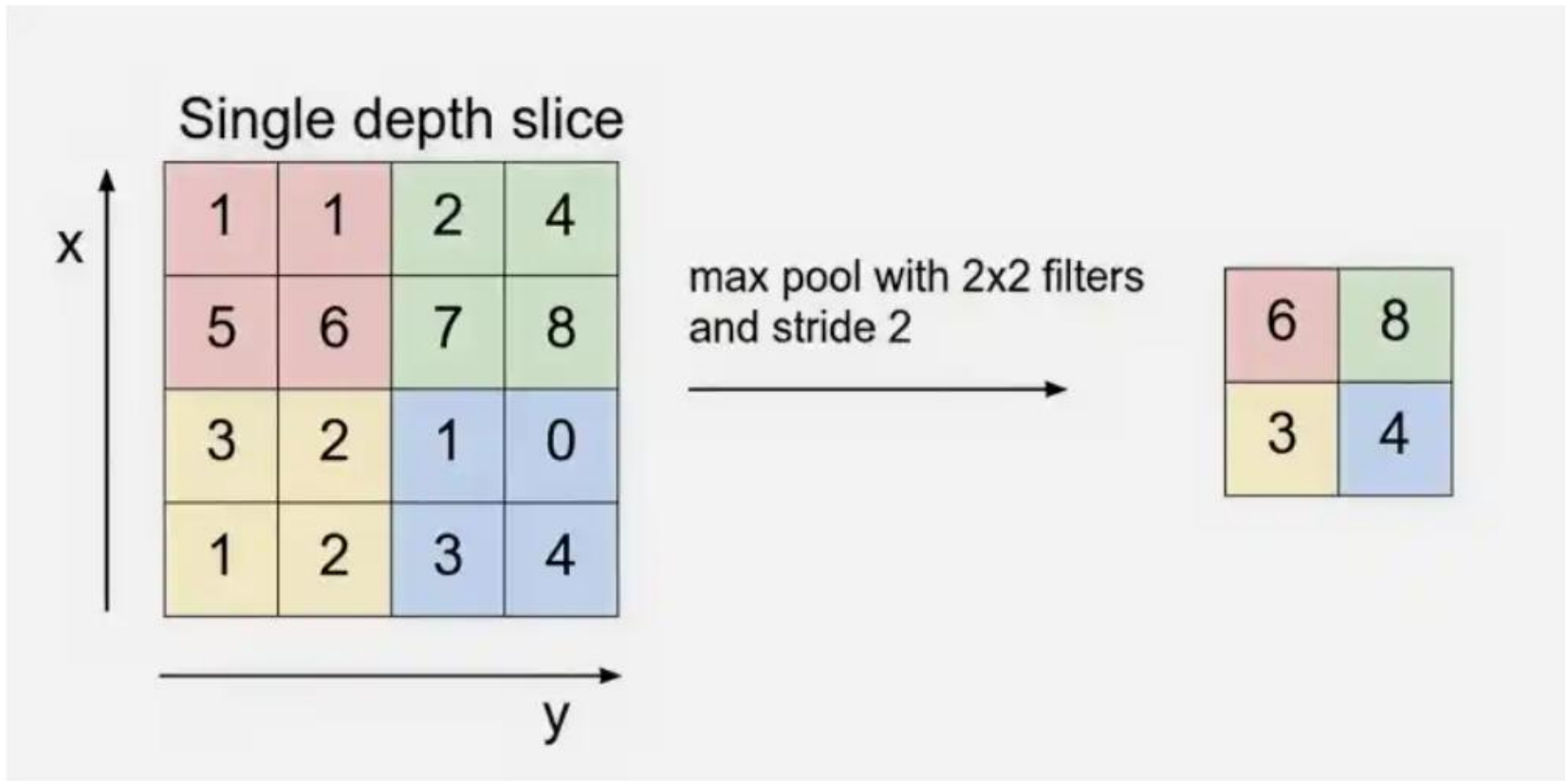
Key Components of CNN

• 4. Pooling Layer

- The Pooling Layer is used to reduce the spatial dimensions of the feature maps, making computation faster, reducing memory usage and helping to prevent overfitting.
- It is typically inserted between convolutional layers in a CNN.
- Common types include Max Pooling and Average Pooling.
- Reduces width and height while keeping depth unchanged.
- ***Example:*** Using 2×2 max pooling with stride 2 reduces the volume from $32 \times 32 \times 12$ to $16 \times 16 \times 12$.

Key Components of CNN

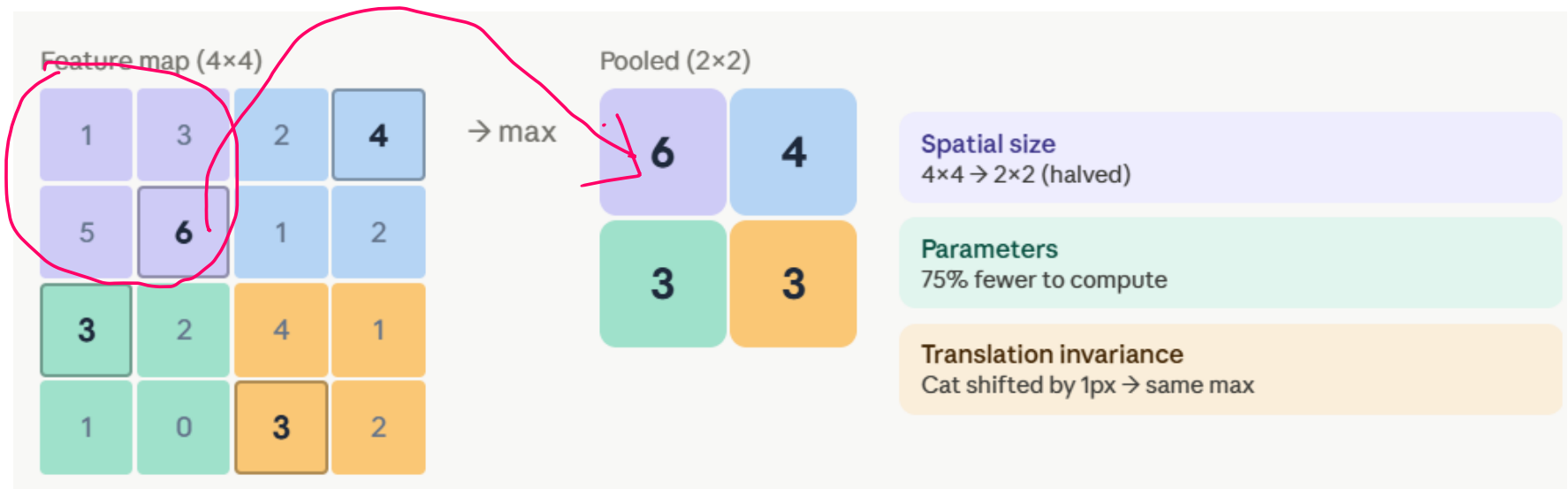
• 4. Pooling Layer



Max Pooling

Key Components of CNN

• 4. Pooling Layer



Key Components of CNN

- **5. Flattening**
- Flattening converts the multi-dimensional feature maps into a one-dimensional vector after convolution and pooling.
- This vector is then passed to the fully connected layer for classification or regression.
- ***Example:*** Flattening $16 \times 16 \times 12$ results in a vector of size 3072 ($16 \times 16 \times 12$).

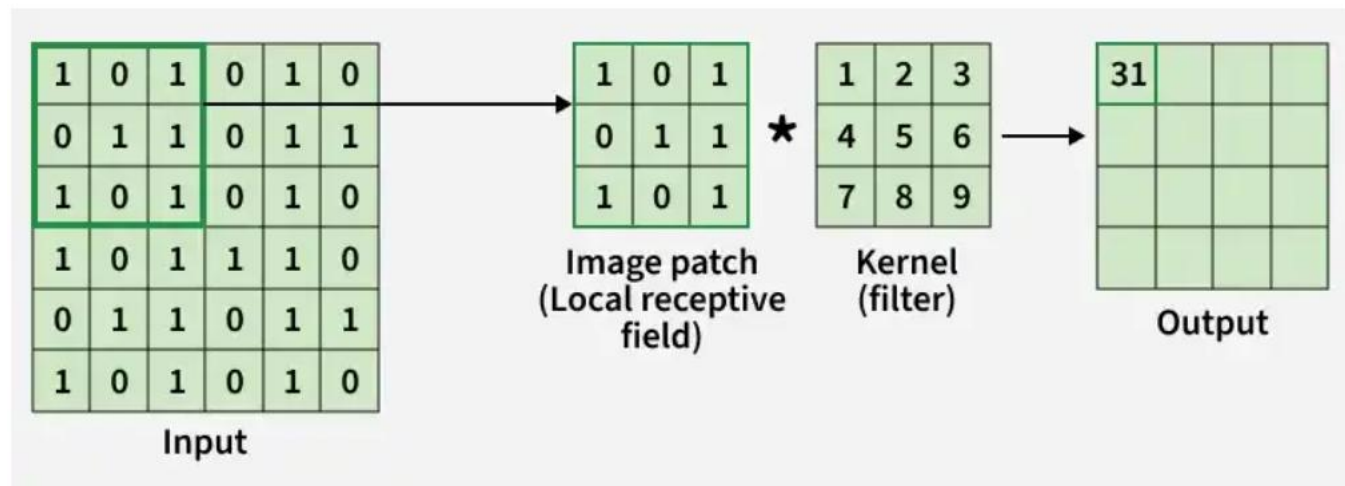
Key Components of CNN

- **6. Fully Connected Layer**
- The fully connected (dense) layer performs high-level reasoning using extracted features and produces the final classification scores.
- ***Example:*** The 3072-length vector is connected to neurons for classification

Key Components of CNN

• 7. Output Layer

- The output layer converts final scores into probabilities using activation functions like Sigmoid (binary classification) or Softmax (multi-class classification).
- **Example:** For 10 classes, Softmax produces 10 probability values each representing the likelihood of a class.



- **Advantages**

- Automatically learn important features from images, videos or audio without manual extraction.
- Highly effective at detecting spatial patterns, edges, textures and shapes.
- Robust to variations like translation, rotation and scaling in input data.
- Can handle large datasets and achieve high predictive accuracy.
- Supports end-to-end training, simplifying the model pipeline.

• Limitations

- Training is computationally intensive and requires significant memory.
- Prone to overfitting if data is limited or regularization is insufficient.
- Requires large amounts of labeled data for optimal performance.
- Limited interpretability difficult to understand learned features.
- Sensitive to adversarial examples and noise in input data.

Pre-trained Model Comparison

Model	Year	Parameters	Speed	Accuracy
LeNet	1998	60K	Very Fast	Low
AlexNet	2012	60M	Fast	Medium
VGG16	2014	138M	Slow	High
VGG19	2014	144M	Slow	High
GoogLeNet	2014	6.8M	Fast	High
ResNet50	2015	25M	Fast	Very High
InceptionV3	2016	24M	Fast	Very High
DenseNet121	2017	8M	Medium	Very High
MobileNetV2	2018	3.4M	Very Fast	High
EfficientNetB0	2019	5.3M	Fast	Very High

Real-Time Usage

- **Medical Image Classification**

- Input:
Chest X-Ray

Pre-trained:
ResNet50

Output:
Normal / Pneumonia

Real-Time Usage

- **Manufacturing Quality Inspection**

- Input:
T-Shirt Image

Pre-trained:
EfficientNet

Output:
Good / Defective

Real-Time Usage

- **Face Recognition**

- Input:
Employee Photo

Pre-trained:
VGGFace / ResNet

Output:
Employee ID

Real-Time Usage

- **Self-Driving Cars**

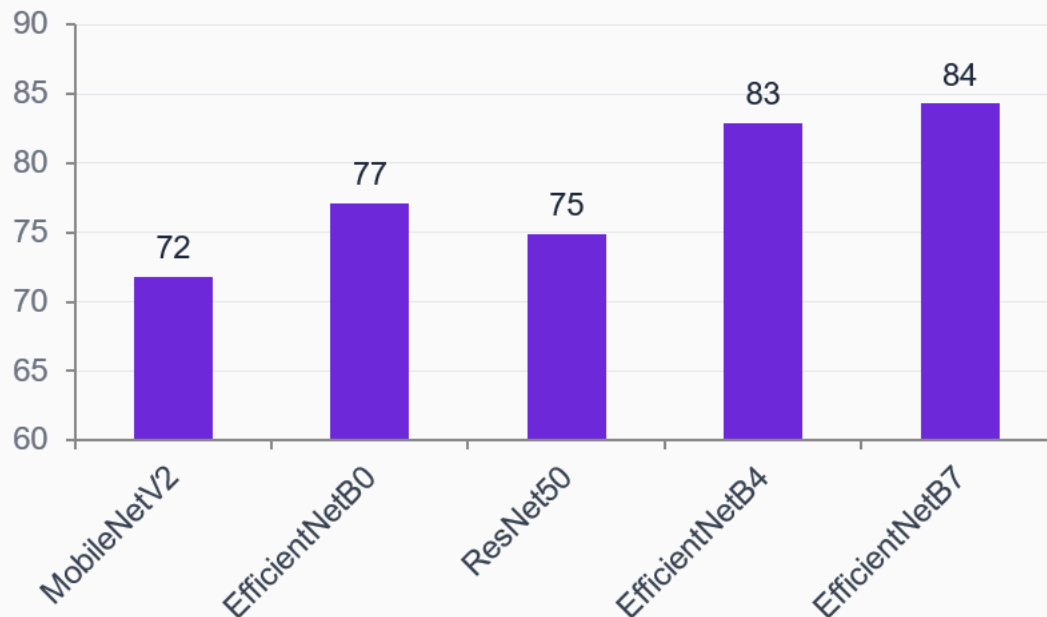
- Input:
Road Camera

Pre-trained:
YOLO + ResNet Backbone

Output:
Cars, Pedestrians, Signals

Pre-trained Model Comparison

Top-1 Accuracy on ImageNet (%)



Quick Selector:

MobileNetV2

Edge / Mobile

Tiny model. Real-time on phone or Raspberry Pi.

EfficientNetB0

Start Here

Best accuracy/compute for most projects.

EfficientNetB4

High Accuracy

Strong baseline when compute allows.

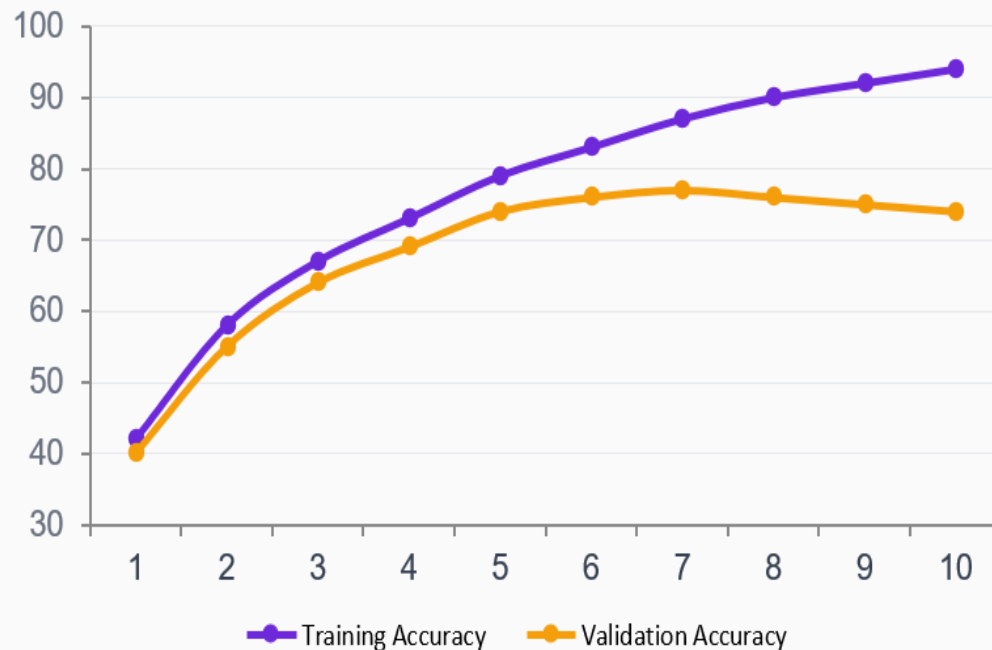
ResNet50

Proven Stable

Widely studied. Easy to debug and replicate.

Evaluating CNN Performance

Accuracy Over Epochs (Overfitting Example)



How to Read Training Curves:

 Both curves rising together

→ Add more epochs

 Train up, val diverging down

→ Add Dropout, augmentation, less capacity

 Both curves low & flat

→ Larger model, more epochs, lower LR

 Val loss oscillates wildly

→ Reduce learning rate 10×

Step 1 — Convolution: scanning with a filter

A small filter (3×3 kernel) slides across the input image. At each position it multiplies its weights by the underlying pixel values and sums the result — producing one value in the output feature map.

Input (5×5)		Filter (3×3)		Feature map (3×3)																																											
<table border="1"> <tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr> </table>	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	1	1	×	<table border="1"> <tr><td>+1</td><td>0</td><td>-1</td></tr> <tr><td>+1</td><td>0</td><td>-1</td></tr> <tr><td>+1</td><td>0</td><td>-1</td></tr> </table>	+1	0	-1	+1	0	-1	+1	0	-1	=	<table border="1"> <tr><td>2</td><td>0</td><td>-2</td></tr> <tr><td>2</td><td>0</td><td>-2</td></tr> <tr><td>2</td><td>0</td><td>-2</td></tr> </table>	2	0	-2	2	0	-2	2	0	-2
1	0	1	0	1																																											
0	1	0	1	0																																											
1	0	1	0	1																																											
0	1	0	1	0																																											
1	0	0	1	1																																											
+1	0	-1																																													
+1	0	-1																																													
+1	0	-1																																													
2	0	-2																																													
2	0	-2																																													
2	0	-2																																													

Key insight: This filter detects vertical edges — where pixel values jump from high to low left-to-right. One filter = one feature map. 32 filters = 32 feature maps stacked into a "volume".

Step 2 — ReLU: keep only positive signals

After each convolution, a ReLU activation is applied: any negative value becomes 0, positive values stay unchanged. This introduces non-linearity — without it, no matter how many layers you stack, the network is just one linear function.

Before ReLU

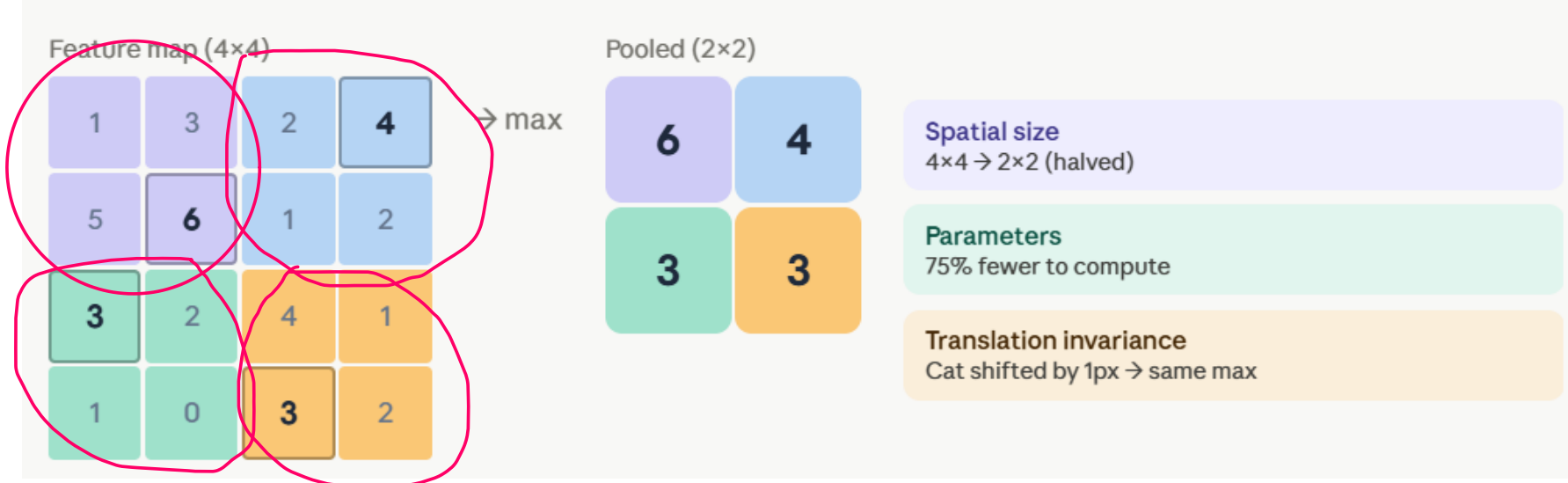
-2	-1	0	0	0	0
1	2	3	1	2	3
-0.5	2.5	-3	0	2.5	0

After ReLU

Why it matters: Negative activations represent "this feature is NOT here". ReLU discards those, letting only positive evidence flow forward. This sparsity is what allows deep networks to train efficiently.

Step 3 — Pooling: shrink & preserve

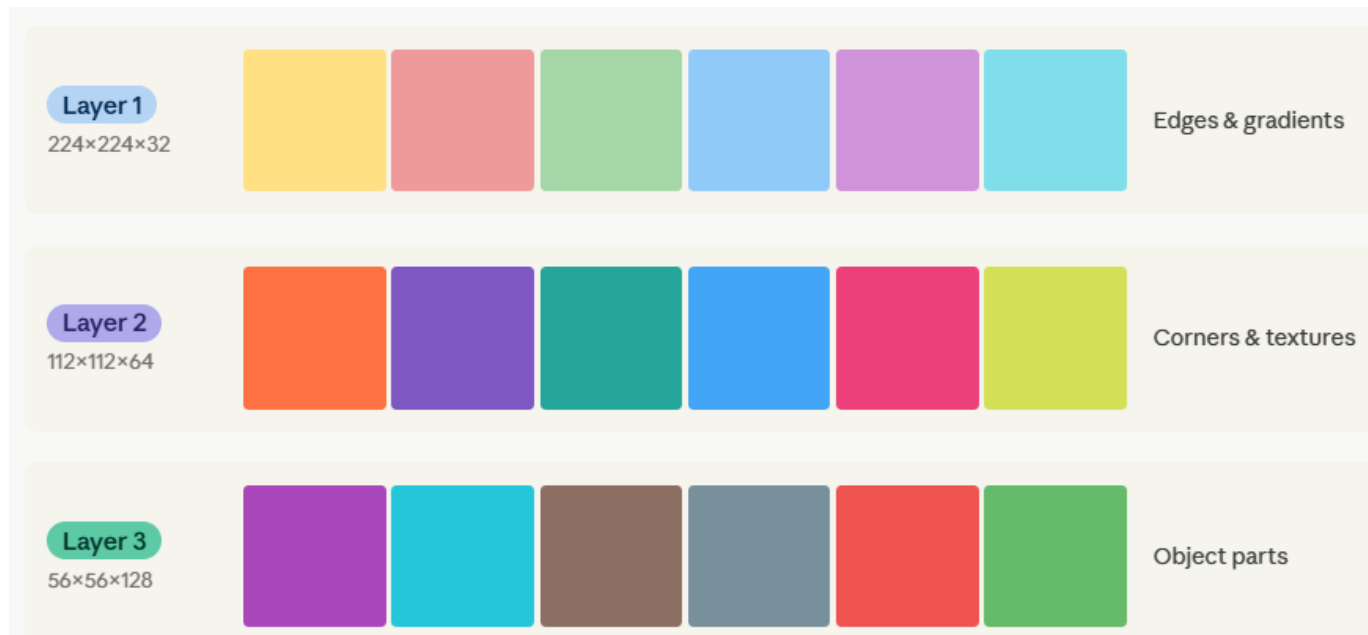
- Max Pooling slides a 2×2 window over the feature map and keeps only the maximum value in each window.
- This halves the spatial dimensions, reducing computation — while keeping the strongest signal regardless of exact position.



Step 4 — Stacking layers: the feature hierarchy



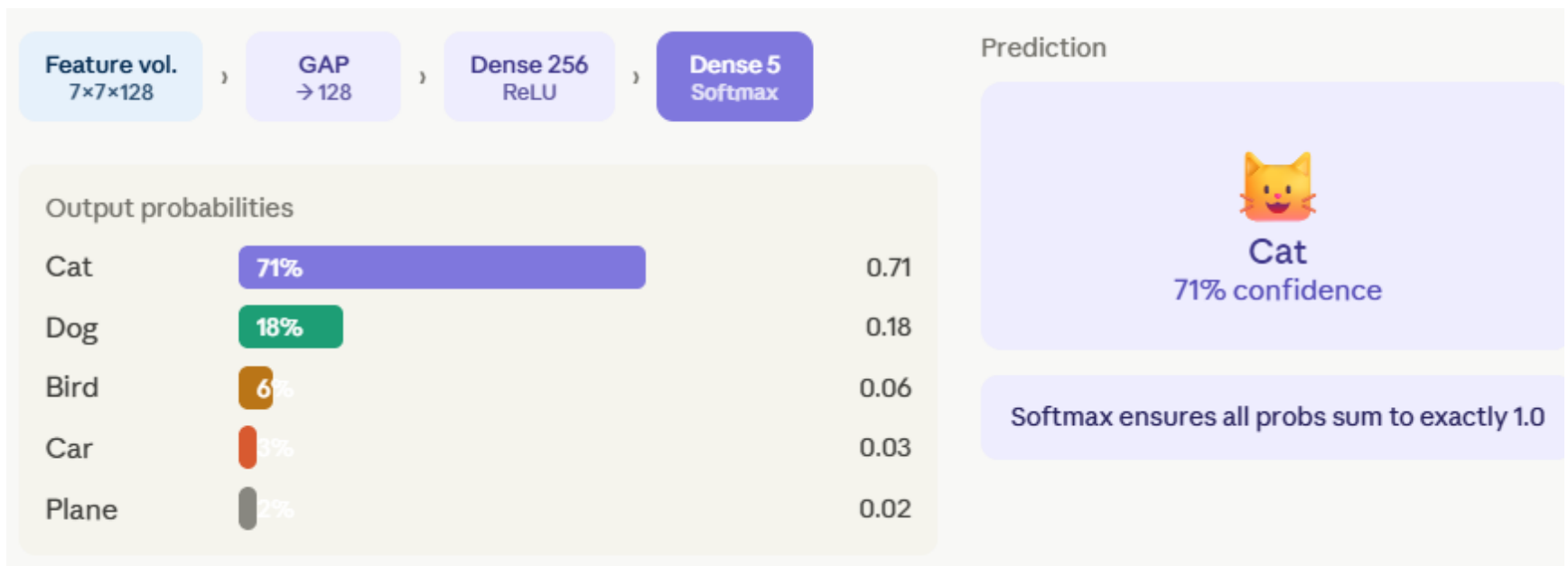
- The power of CNNs comes from stacking Conv+ReLU+Pool blocks.
- Each block sees a wider context and more abstract patterns than the one before. Depth = abstraction.



Rule of thumb: Double the filters each layer (32 → 64 → 128). As spatial size halves (pooling), depth doubles — so total information stays roughly constant across the network.

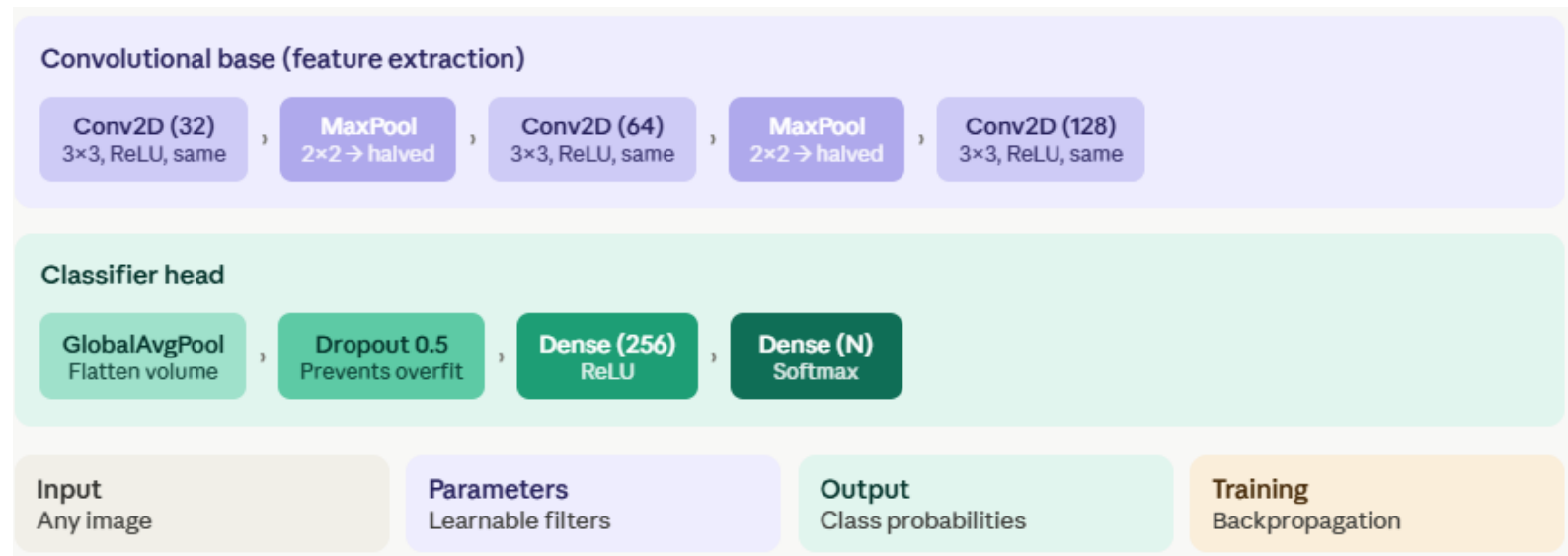
Step 5 — Classifier head: making the prediction

- After the convolutional base extracts features, a GlobalAveragePooling layer collapses the 3D feature volume into a flat vector. Then Dense layers with Softmax produce a probability for each class.



Full architecture at a glance

- A complete CNN pipeline: input image flows through alternating Conv+Pool blocks (feature extraction), then a classifier head makes the final prediction.
- This is the blueprint for nearly every image classification model.



Call backs

- Callbacks are functions or blocks of code that are executed at specific stages of the training process.
- They allow you to interact with the model at various points such as:
 - At the start and end of an epoch
 - Before and after a batch is processed
 - At the start and end of training
- These interactions can be used to implement custom behaviour such as early stopping, learning rate scheduling, saving model checkpoints, logging metrics, and more.

Call backs

```
for epoch, iterator in data_handler.enumerate_epochs():
```

```
    callbacks.on_epoch_begin(epoch)
```

```
    for step in data_handler.steps():
```

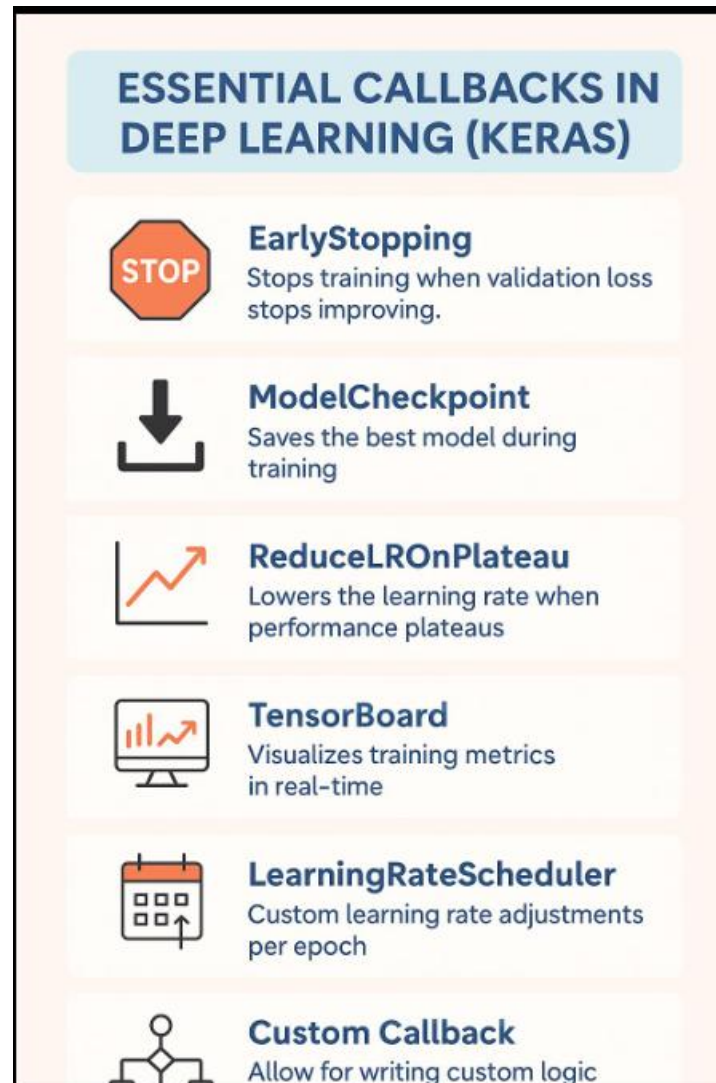
```
        callbacks.on_train_batch_begin(step)
```

```
        train_function(iterator)
```

```
        callbacks.on_train_batch_end(end_step, logs)
```

```
    callbacks.on_epoch_end(epoch, epoch_logs)
```

Essential Call backs in Deep Learning



Call backs

- The three you'll use in almost every CNN project:
- Model Checkpoint — our safety net. Writes the model to disk only when `val_accuracy` actually improves. Without it, if training crashes at epoch 48 you lose everything.
- Early Stopping — our overfitting guard. Once `val_accuracy` stops improving for patience epochs, it halts training and *undoes* the overfit epochs by restoring the best weights. Always set `restore_best_weights=True`.
- Reduce LR On Plateau — Reduce → decrease, LR → Learning Rate, On Plateau → when the monitored metric (such as validation loss) stops improving and becomes flat (plateaus) .
- When `val_loss` flatlines, it multiplies the learning rate by factor (usually 0.5). The smaller steps often let the optimizer find a better minimum that the original learning rate was jumping over.

Call backs

- `model.fit(`
- `X_train, y_train,`
- `epochs=50,`
- `callbacks=[`
- `ModelCheckpoint("best.keras",`
`monitor="val_accuracy", save_best_only=True),`
- `EarlyStopping(monitor="val_accuracy", patience=6,`
`restore_best_weights=True),`
- `ReduceLROnPlateau(monitor="val_loss", factor=0.5,`
`patience=3, min_lr=1e-6),`
- `]`
- `).`

Experiment Tracking

- Experiment tracking is the practice of recording everything about each training run — hyperparameters, metrics, code version, model weights.
- We can compare runs, reproduce results, and know exactly what produced your best model.
- Without it, after 20 experiments we won't remember whether it was $lr=0.001$ or $lr=0.0003$ that gave us 94% accuracy.

Experiment Tracking



Total runs

8

Best val acc

92.8%

Best run

run-008

Lowest val loss

0.261



Filter by optimizer

All optimizers



Sort by

Val accuracy



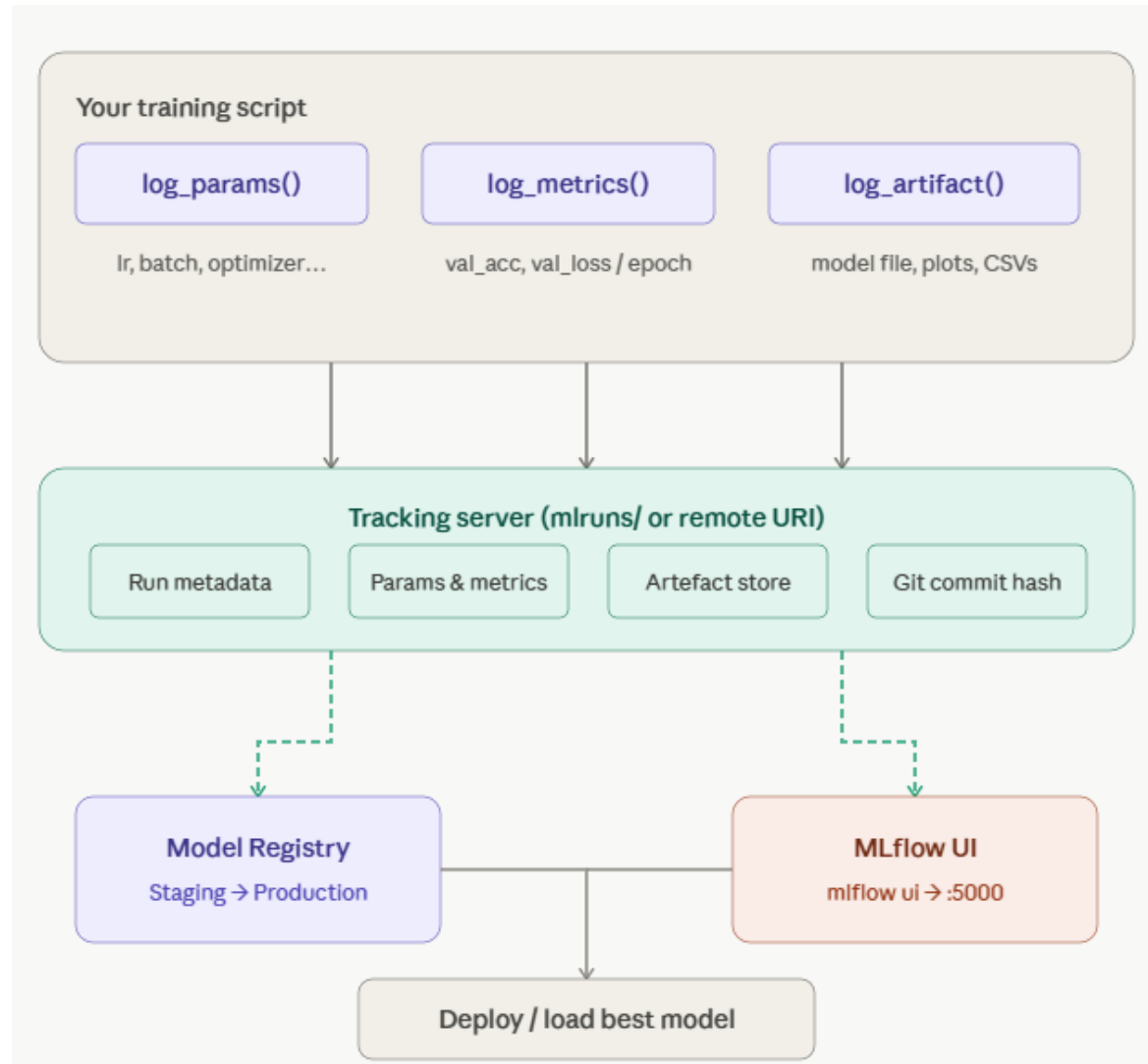
Experiment Tracking

Run	Optimizer	LR	Batch	Dropout	Epochs	Val accuracy	Val loss	Status
run-008	Adam	0.0003	32	0.4	26	93% 	0.261	done
run-002	Adam	0.0003	32	0.5	24	91% 	0.298	done
run-003	Adam	0.0003	64	0.3	22	89% 	0.341	done
run-001	Adam	0.001	32	0.5	18	87% 	0.412	done
run-007	RMSprop	0.001	32	0.4	20	87% 	0.428	done
run-005	SGD	0.001	64	0.4	28	86% 	0.463	done
run-004	SGD	0.01	32	0.5	30	84% 	0.501	done
run-006	Adam	0.0001	32	0.5	15	80% 	0.589	stoppe

Experiment Tracking

- The dashboard above shows what experiment tracking looks like in practice — filter by optimizer, sort by any column, and the best run is highlighted automatically.
- **What we track in every run:**
 - Hyperparameters — learning rate, batch size, optimizer, dropout, architecture choices
 - Metrics per epoch — val_accuracy, val_loss, accuracy, loss
 - Config — random seed, dataset version, data augmentation settings
 - Artifacts — the saved model file, training curves plot

Experiment Tracking – ML FLOW



ML Flow Experiment Tracking

- **ML flow provides a powerful framework for**
 - experiment tracking
 - log parameters
 - log metrics
 - log artifacts
 - log code versions across multiple runs and organize them into experiments for easy comparison.
- It helps you systematically manage model development, avoid chaos in iterative workflows, and ensure reproducibility.

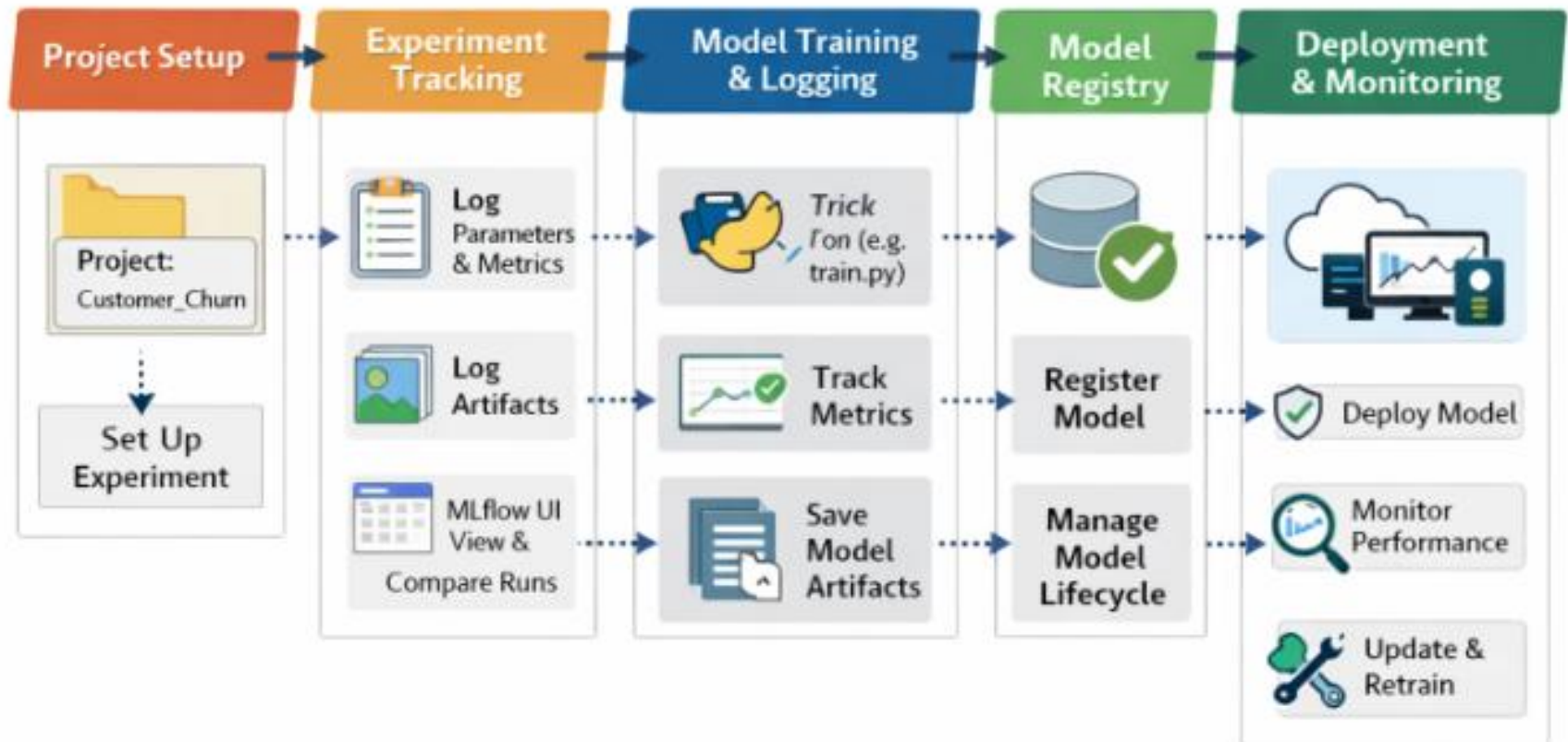
Core Concepts in MLflow Tracking

- **Runs:** Each execution of training code (e.g., `train.py`) is a run. Runs capture metadata (start/end time, parameters, metrics) and artifacts (model weights, plots, logs).
- **Experiments:** High-level containers grouping related runs. Useful for comparing different approaches to the same problem.
- **Models:** Trained artifacts produced during runs. MLflow can register and manage them across the lifecycle.
- **Tracking API:** Functions like `mlflow.log_param()` and `mlflow.log_metric()` record parameters and metrics.
- Auto-logging (`mlflow.autolog()`) simplifies logging for libraries like Scikit-learn, PyTorch, and XGBoost.

Comparison Table

Concept	Definition	Example Use
Run	Single execution of ML code	Training Random Forest with depth=6
Experiment	Collection of related runs	Comparing architectures for image classification
Model	Trained artifact with metadata	Registered XGBoost model for deployment
Artifact	Output files from runs	Confusion matrix, logs, model weights

End to End ML FLOW



Register model

```
mlflow.tensorflow.log_model(  
    pipeline_model,  
    artifact_path="loan_pipeline_model",  
    registered_model_name="LoanApprovalPipelineM  
odel"  
)
```

Start MLflow UI

- In terminal
- `mlflow ui`
- Open:
- `http://localhost:5000`

Deploy model

- `mlflow models serve -m models:/LoanApprovalPipelineModel/1 -p 5001 --no-conda`

Test model



HTTP localhost:5001/invocations

POST localhost:5001/invocations

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded **raw** binary JSON

```
1 {
2   "inputs": [
3     [0.25, -0.42, 1.31]
4   ]
5 }
```

ody Cookies Headers (4) Test Results 200 OK 65

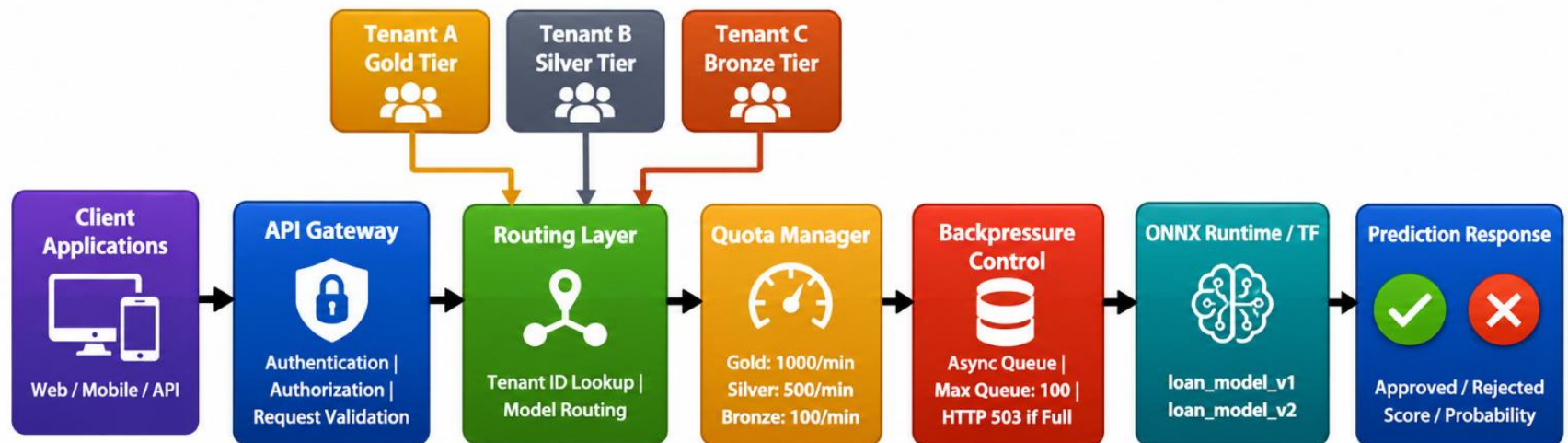
Pretty Raw Preview Visualize JSON

```
1 {
2   "predictions": [
3     [
4       0.7233651876449585
5     ]
6   ]
7 }
```

Test model

```
import requests
payload = {
    "inputs": [
        [0.25, -0.42, 1.31]
    ]
}
response = requests.post(
    "http://127.0.0.1:5001/invocations",
    json=payload
)
print(response.json())
```

Tenants



Model API with Tenants,Quota,Routing and Versioning



ONNX Loan Approval Inference Gateway 0.1.0 OAS 3.1

/openapi.json

default

POST /predict Predict

GET /health Health

Schemas

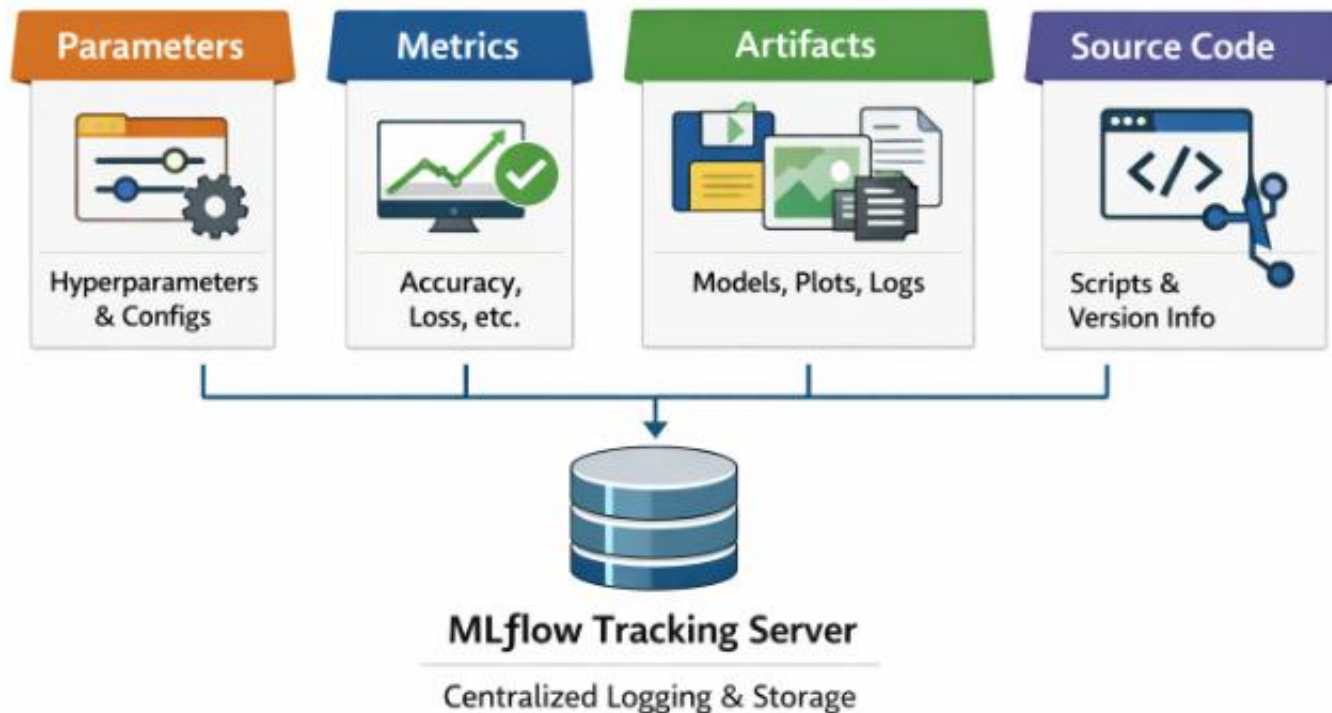
HTTPValidationError > Expand all object

LoanRequest > Expand all object

ValidationError > Expand all object

What gets tracked

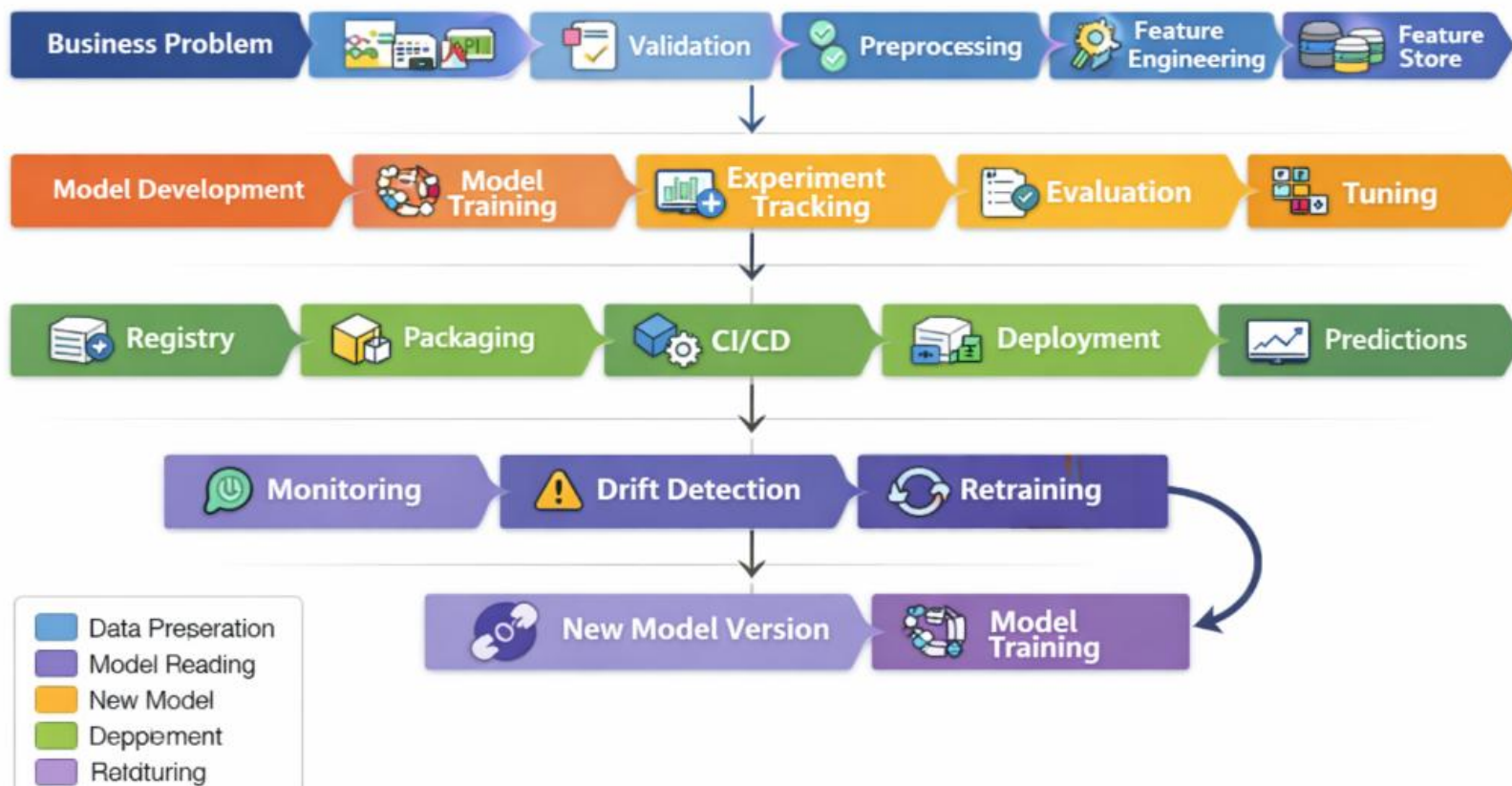
MLflow Tracking Components



Real-Time Machine Learning Flow (Enterprise Project)



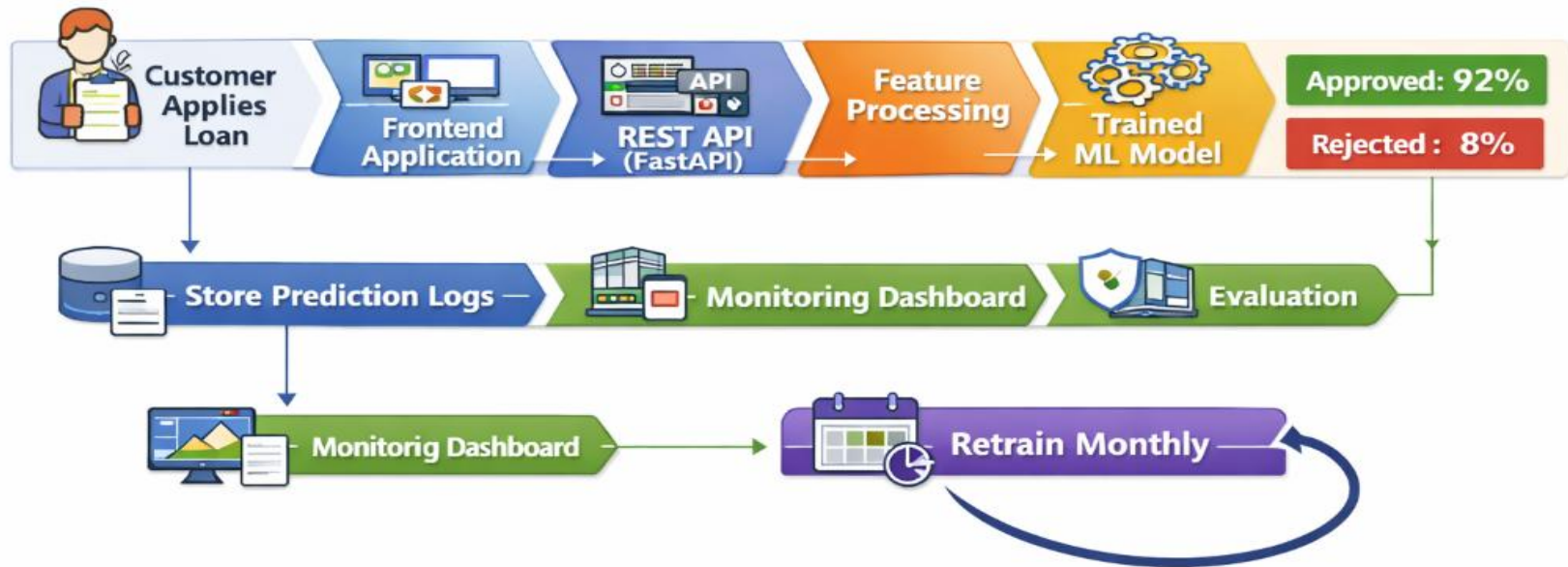
End-to-End ML Lifecycle



Real-Time Machine Learning Flow (Enterprise Project)

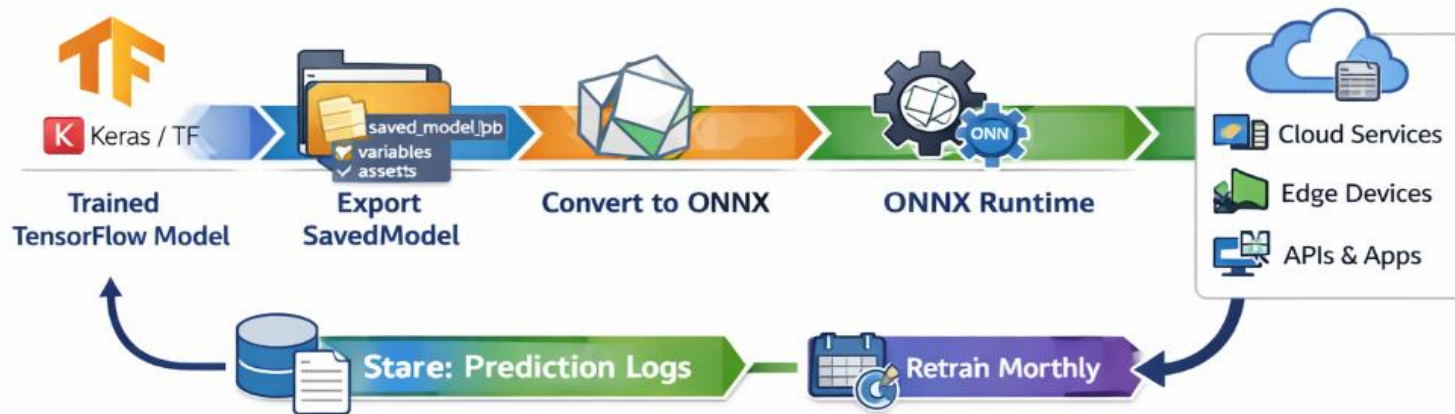


Loan Approval ML Pipeline



Exporting SavedModel; ONNX conversion

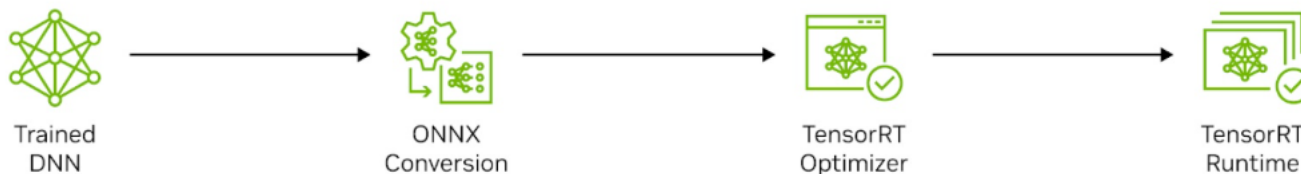
TensorFlow SavedModel to ONNX Conversion



What is TensorRT?

- **TensorRT** is an inference optimization library developed by NVIDIA.
- It takes models trained in frameworks like **TensorFlow**, **PyTorch**, or **ONNX** and converts them into highly optimized runtime engines for deployment on NVIDIA GPUs.
- Used in **data centers, edge devices, and embedded systems** for real-time AI applications such as autonomous driving, NLP, and computer vision.

TensorRT





What is TensorRT?



- **TensorRT** is NVIDIA's SDK for deep learning inference.
- It performs **layer fusion, precision calibration (FP16/INT8), and kernel auto-tuning** to maximize GPU efficiency.
- Ideal for **real-time applications** like autonomous driving, speech recognition, and recommendation systems.



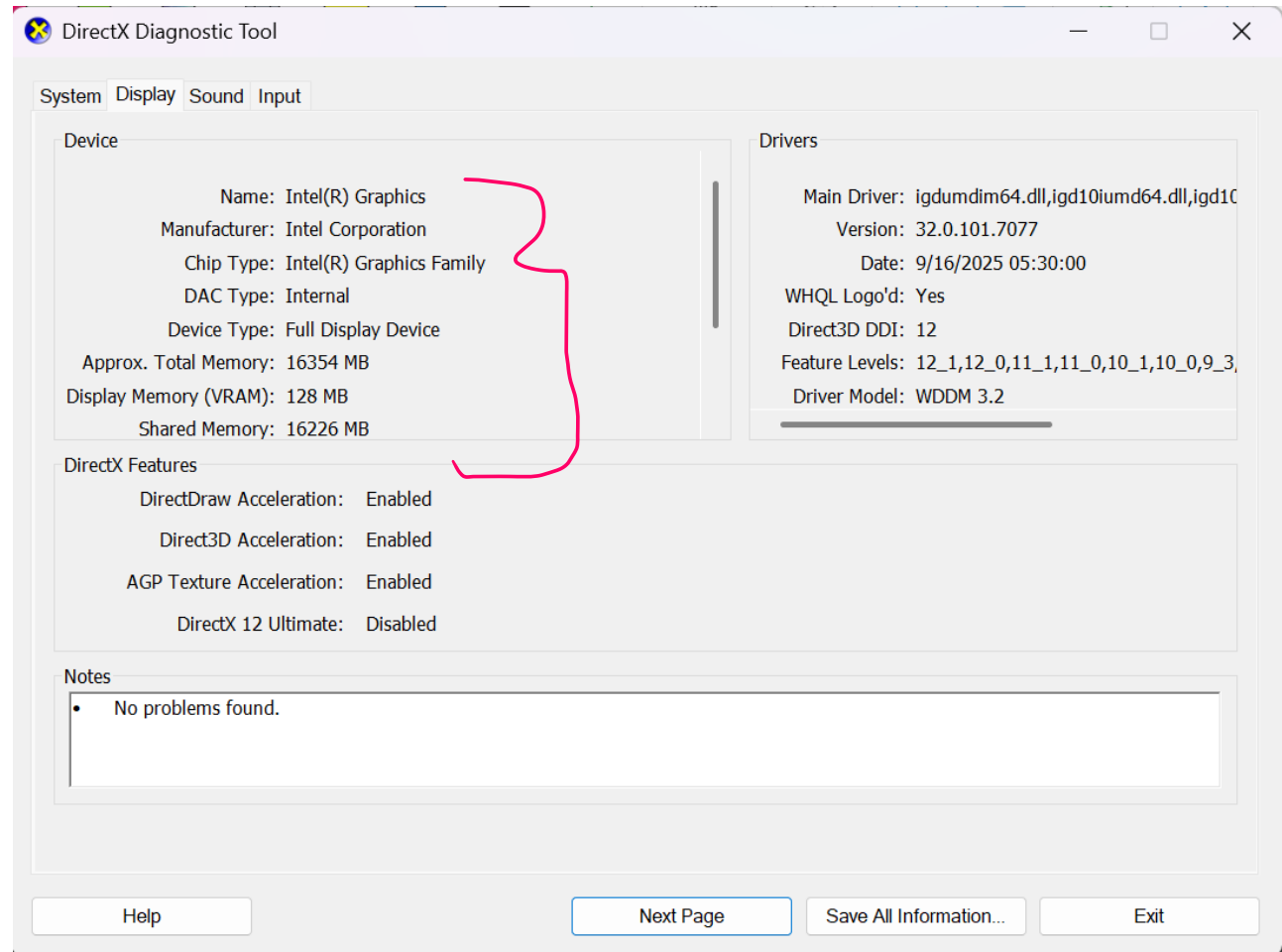
Minimum Hardware Requirements for TensorFlow

Minimum System Configuration

Component	Minimum
CPU	Intel i5 / Ryzen 5
RAM	8 GB
GPU	NVIDIA GTX 1050 4GB
CUDA	Supported version
OS	Windows 10/11 64-bit or Linux
Python	3.10+

Minimum Hardware Requirements for TensorFlow

- How to test
- Run from cmd
 - dxdiag





Minimum Hardware Requirements for TensorRT

- For corporate training and hands-on TensorRT/Triton demos, an **RTX 3050 or RTX 4060 laptop** is a practical starting point.

Why We Need TensorRT

- **Performance Boost:** Delivers up to **36x faster inference** compared to CPU-only execution.
- **Efficiency:** Reduces GPU memory usage and computational overhead.
- **Scalability:** Supports deployment across single GPUs, clusters, and edge devices.
- **Compatibility:** Works with major frameworks and interchange formats like ONNX.
- **Production Readiness:** Provides APIs for C++ and Python, enabling integration into enterprise pipelines.

Key Features of TensorRT

Feature	Description
Precision Calibration	Supports FP32, FP16, INT8, BF16, FP8 for faster computation with minimal accuracy loss.
Layer Fusion	Combines multiple neural network layers into single operations to reduce memory access.
Kernel Auto-Tuning	Selects the most efficient GPU kernels for each layer.
Dynamic Tensor Memory	Efficient GPU memory allocation and reuse during inference.
Quantization	Supports post-training quantization and quantization-aware training.
Multi-Device Inference	Enables distributed inference across multiple GPUs.
Transformer Optimization	Specialized support for large language models and attention mechanisms.
API Extensibility	C++ and Python APIs with plugin support for custom layers.



What is Triton Inference Server?



- **Triton Inference Server** (formerly TensorRT Inference Server) is a **cloud-native serving system**.
- Supports multiple frameworks: **TensorRT, PyTorch, TensorFlow, ONNX Runtime, OpenVINO**.
- Provides **HTTP/gRPC endpoints** for client requests.
- Features:
 - **Dynamic batching**: Groups requests to maximize GPU utilization.
 - **Concurrent model execution**: Runs multiple models or versions simultaneously.
 - **Multi-GPU scaling**: Efficient deployment across clusters.

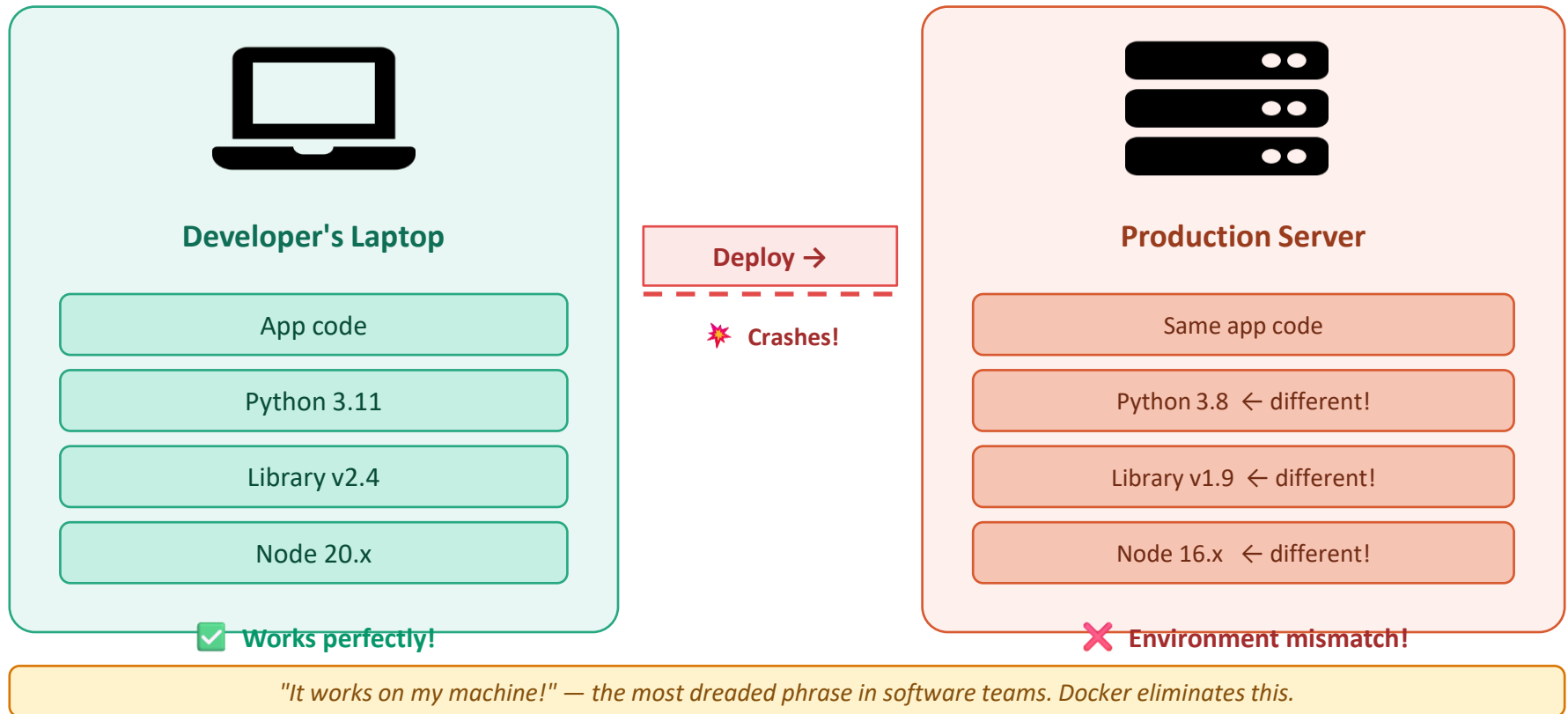
Steps to install server

DEEPLARNING

```
|
|├─ src
|   └─ deepmlapp
|       └─ utils
|
|├─ loan_approval_pipeline.onnx
|├─ loan_approval_model.onnx
|├─ loan_approval_pipeline.keras
|
|└─ model_repository
    └─ loan_model
        ├── config.pbtxt
        └─ 1
            └─ model.onnx
```

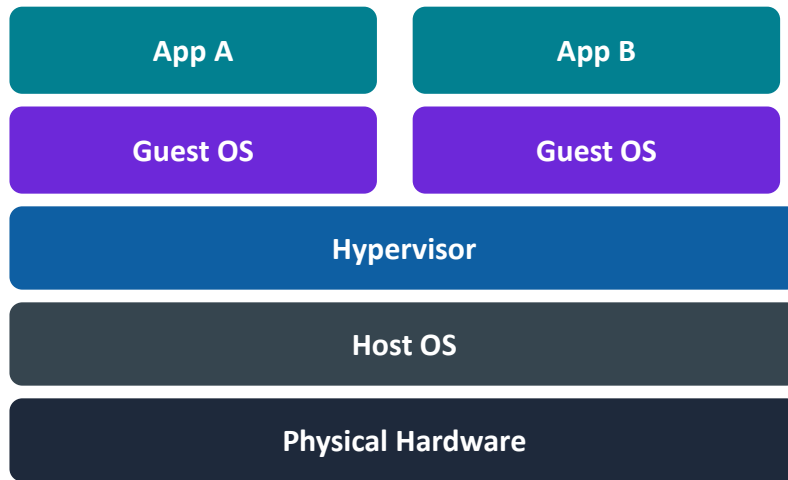
Loanapplication.onnx
renamed to model.onnx

01 The Problem Docker Solves



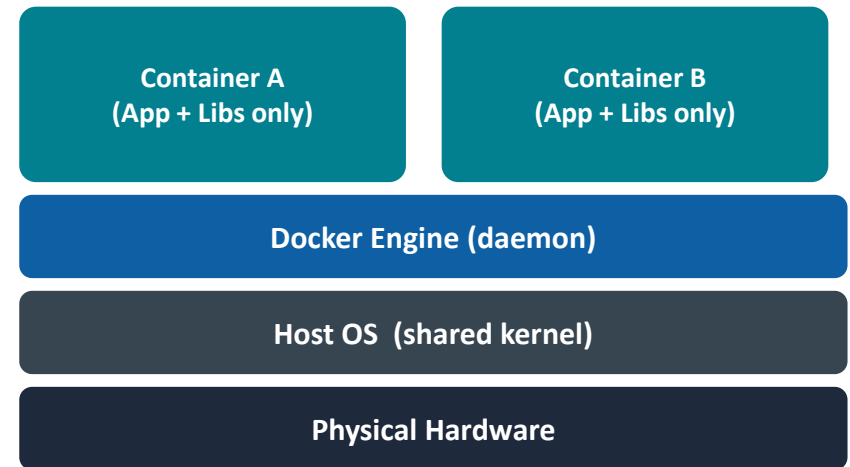
02 Containers vs Virtual Machines

Virtual Machine



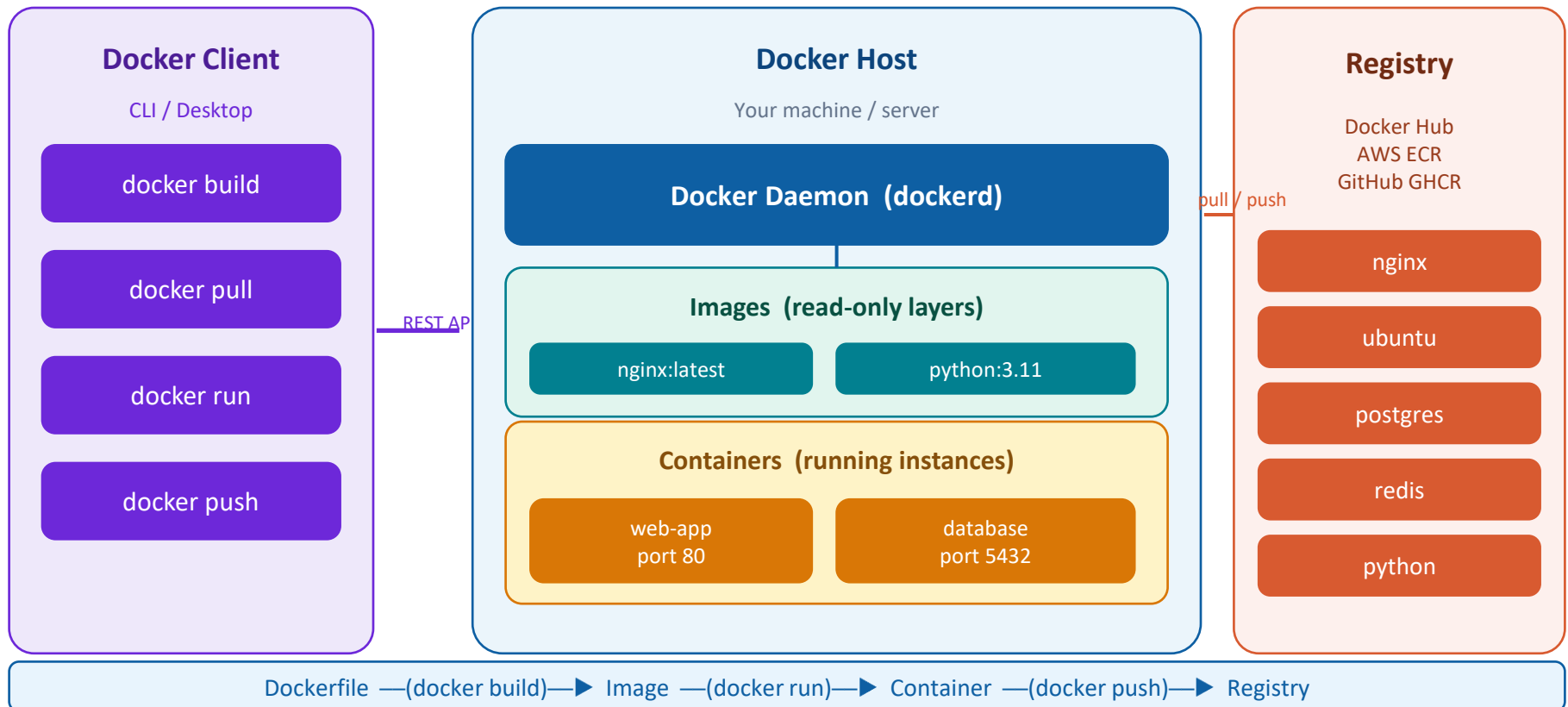
⚠ GBs per VM · Minutes to start

Docker Container

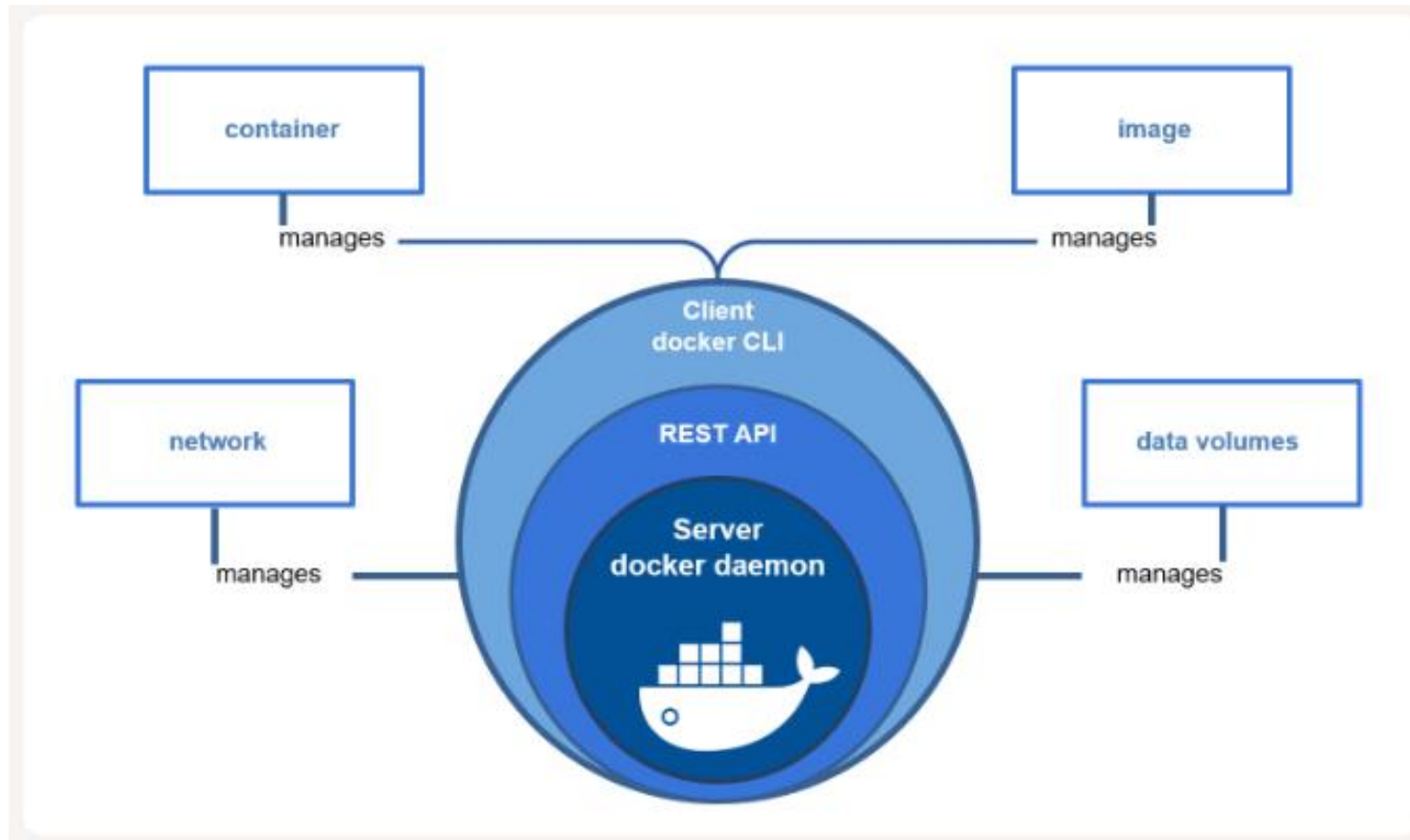


✓ MBs per container · Milliseconds to start

03 Docker Architecture



03 Docker Architecture



04 Core Components



Dockerfile

Plain text file with step-by-step instructions to build an image. Each instruction (FROM, RUN, COPY) creates a new layer.



Image

Read-only layered snapshot built from a Dockerfile. Layers are cached and shared — 2 images sharing a base store it once.



Container

A running instance of an image. Gets a writable layer on top. Isolated filesystem, network, and process space.



Registry

Storage & distribution for images. Docker Hub is the public registry. Companies use ECR, ACR, or GHCR privately.



Docker Daemon

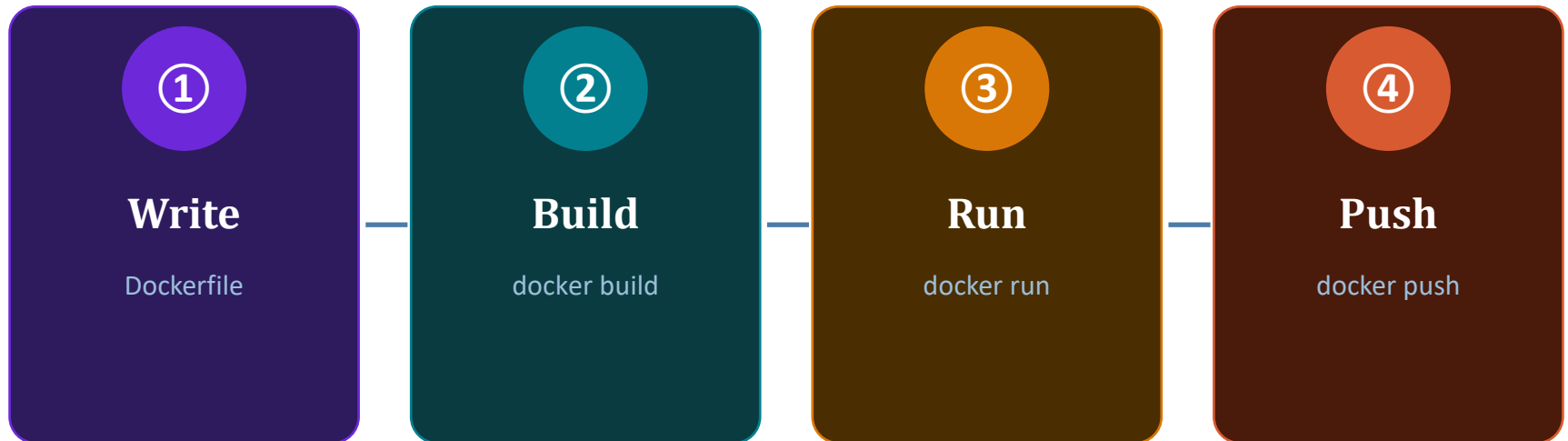
Background service (dockerd) that manages all Docker objects — images, containers, networks, volumes.



Docker Client

The CLI (docker) you type commands into. Sends REST API calls to the daemon. Can connect locally or remotely.

Docker Commands



← *docker pull* — others pull your image from the registry and run it on their machine ←

```
$ docker build -t myapp:1.0 .
```

```
$ docker run -d -p 80:8080 myapp:1.0
```

```
$ docker push myuser/myapp:1.0
```

06 Networking & Volumes

Networking

Bridge network (default) 172.17.0.0/16

web container
172.17.0.2:8080

SQL

db container
172.17.0.3:5432

Port mapping: -p 80:8080

Host:80 → Container:8080

Network types: bridge · host · overlay · none

(bridge = default for single host)

Volumes

- Data survives container removal
- Shared between containers
- Stored at /var/lib/docker/volumes
- Use -v flag when running:

```
-v /host/data:/app/data
```

Without a volume, all data inside a container is lost when the container is deleted.

07 Essential Commands — Cheat Sheet



Images

```
docker build -t myapp:1.0 .
```

```
docker images
```

```
docker pull nginx:latest
```

```
docker rmi myapp:1.0
```

```
docker push user/myapp:1.0
```

Containers

```
docker run -d -p 80:8080 myapp
```

```
docker ps
```

```
docker ps -a
```

```
docker stop <id>
```

```
docker rm <id>
```

```
docker logs <id>
```

```
docker exec -it <id> bash
```

System

```
docker system prune
```

```
docker inspect <id>
```

```
docker stats
```

```
docker network ls
```

```
docker volume ls
```

Summary & What's Next

Key Takeaways

Dockerfile = Recipe (instructions)

Image = Baked cake (read-only snapshot)

Container = A slice being served (running)

Registry = The bakery catalogue (storage)

Daemon = Kitchen manager (background service)

Client = You placing the order (CLI)

What to Learn Next

1

Docker Compose

Multi-container apps

2

Dockerfile best practices

Layer caching & security

3

Docker networking

Bridge, overlay, host

4

Kubernetes

Container orchestration

"Build once, run anywhere." — The Docker promise delivered.

Steps to install server

- `docker run --rm -p 8000:8000 -p 8001:8001 -p 8002:8002 -v "%cd%\model_repository:/models" nvcr.io/nvidia/tritonserver:24.10-py3 tritonserver --model-repository=/models`

Steps to install server



```
Administrator: Command Prompt - docker run --rm -p 8000:8000 -p 8001:8001 -p 8002:8002 -v "E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\deeplearning\model_reposito... --
```

```
+-----+
| loan_model | 1      | READY |
+-----+
```

```
I0610 22:01:09.548171 1 metrics.cc:770] "Collecting CPU metrics"
I0610 22:01:09.548706 1 tritonserver.cc:2598]
```

```
+-----+
```

Option	Value
server_id	triton
server_version	2.51.0
server_extensions	classification sequence model_repository model_repository(unload_dependents) schedule_policy model_configuration system_shared_memory c
uda_shared_memory	binary_tensor_data parameters statistics trace logging
model_repository_path[0]	/models
model_control_mode	MODE_NONE
strict_model_config	0
model_config_name	
rate_limit	OFF
pinned_memory_pool_byte_size	268435456
min_supported_compute_capability	6.0
strict_readiness	1
exit_timeout	30
cache_enabled	0

```
+-----+
```

```
I0610 22:01:09.564265 1 grpc_server.cc:2558] "Started GRPCInferenceService at 0.0.0.0:8001"
I0610 22:01:09.564809 1 http_server.cc:4713] "Started HTTPService at 0.0.0.0:8000"
I0610 22:01:09.608255 1 http_server.cc:362] "Started Metrics Service at 0.0.0.0:8002"
```

Test Model

- C:\Windows\System32>curl
http://localhost:8000/v2/models/loan_model
- {"name":"loan_model","versions":["1"],"platform":"onnx
runtime_onnx","inputs":[{"name":"loan_raw_input","dat
atype":"FP32","shape":[-
1,3]}],"outputs":[{"name":"loan_output","datatype":"FP3
2","shape":[-1,1]}]}
- C:\Windows\System32>

Test Model



```
(deepmlenv) E:\Datascience2026\cognizantaimlapr2026-master\cognizantaimlapr2026-master\deeplearning>python src
\deepmlapp\utils\test_tritonserver.py
200
{
  "model_name": "loan_model",
  "model_version": "1",
  "outputs": [
    {
      "name": "loan_output",
      "datatype": "FP32",
      "shape": [
        1,
        1
      ],
      "data": [
        0.6066855788230896
      ]
    }
  ]
}
Score: 0.6066855788230896
Prediction: Loan Approved
```



TensorRT + Triton: Why They Matter



Feature	TensorRT	Triton
Purpose	Model optimization for inference	Model serving & orchestration
Latency	Ultra-low (optimized kernels)	Managed via batching/concurrency
Frameworks	Works with TensorFlow, PyTorch, ONNX	Supports multiple frameworks simultaneously
Deployment	Embedded/edge devices, GPUs	Cloud, on-prem, Kubernetes
Integration	Converts models to optimized engines	Serves models via HTTP/gRPC



TensorRT + Triton: Why They Matter



Item	TensorRT	Triton
Purpose	Optimizes model	Serves model
Main use	Speed improvement	Production deployment
Input	ONNX / TF / PyTorch	Model repository
Output	TensorRT engine	API endpoint
Level	Model optimization	Model serving platform

Edge Devices

- **Common Examples**
 - Smartphones and tablets
 - IoT sensors and smart meters
 - Security cameras with AI capabilities
 - Industrial controllers and robots
 - Autonomous vehicles
 - Smart speakers
 - Medical monitoring devices
 - Retail point-of-sale systems

- **TensorRT alone:** Best for edge devices or single-GPU inference pipelines.
- **Triton + TensorRT:** Best for **production-scale deployments**, handling **multiple clients, models, and GPUs**.
- **Performance gains:** Triton's batching + TensorRT's kernel optimizations = **high throughput with minimal latency**.
- **Alternatives:** Competes with **TensorFlow Serving** and **TorchServe**, but Triton is often superior in GPU utilization and multi-framework support.

TensorRT/Triton Serving (awareness)

- TensorRT and Triton Inference Server are NVIDIA's key technologies for high-performance model deployment.
- TensorRT optimizes deep learning models for GPU execution.
- Triton provides a scalable serving layer with HTTP/gRPC endpoints, dynamic batching, and multi-framework support.
- Together, they enable low-latency, high-throughput inference in production environments.

Challenges & Considerations

- **Complex setup:** Requires Docker/NVIDIA GPU drivers.
- **Monitoring overhead:** Needs integration with Prometheus/Grafana for observability.
- **Model conversion:** TensorRT optimization step can be tricky (especially for custom layers).
- **Client scaling:** Network latency can still bottleneck performance if not tuned.

⚡ Batch vs Online Inference

Point	Online Inference	Batch Inference
Meaning	Predict immediately for one request	Predict for many records together
Input	Single user/request	CSV, database table, files
Output time	Milliseconds/seconds	Minutes/hours
Example	Loan approval API call	Predict loan approval for 1 lakh customers overnight
Tools	FastAPI, ONNX Runtime, Triton	Airflow, Spark, Pandas, Batch jobs
Best for	Real-time apps	Reports, scoring, analytics
Cost	Higher runtime cost	Usually cheaper
Backpressure	Very important	Less urgent, handled by job queue
Quota	Per request/tenant	Per job/tenant

- **Autoscaling** ensures inference services adapt to workload changes.
- Triton integrates with **Kubernetes Horizontal Pod Autoscaler (HPA)** or **KServe**, scaling pods based on CPU/GPU utilization or request latency.
- Key strategies:
 - **Latency-based scaling**: Add replicas when response times exceed thresholds.
 - **Queue-depth scaling**: Scale when request backlog grows.
 - **GPU utilization scaling**: Monitor GPU metrics (via Prometheus/Grafana).

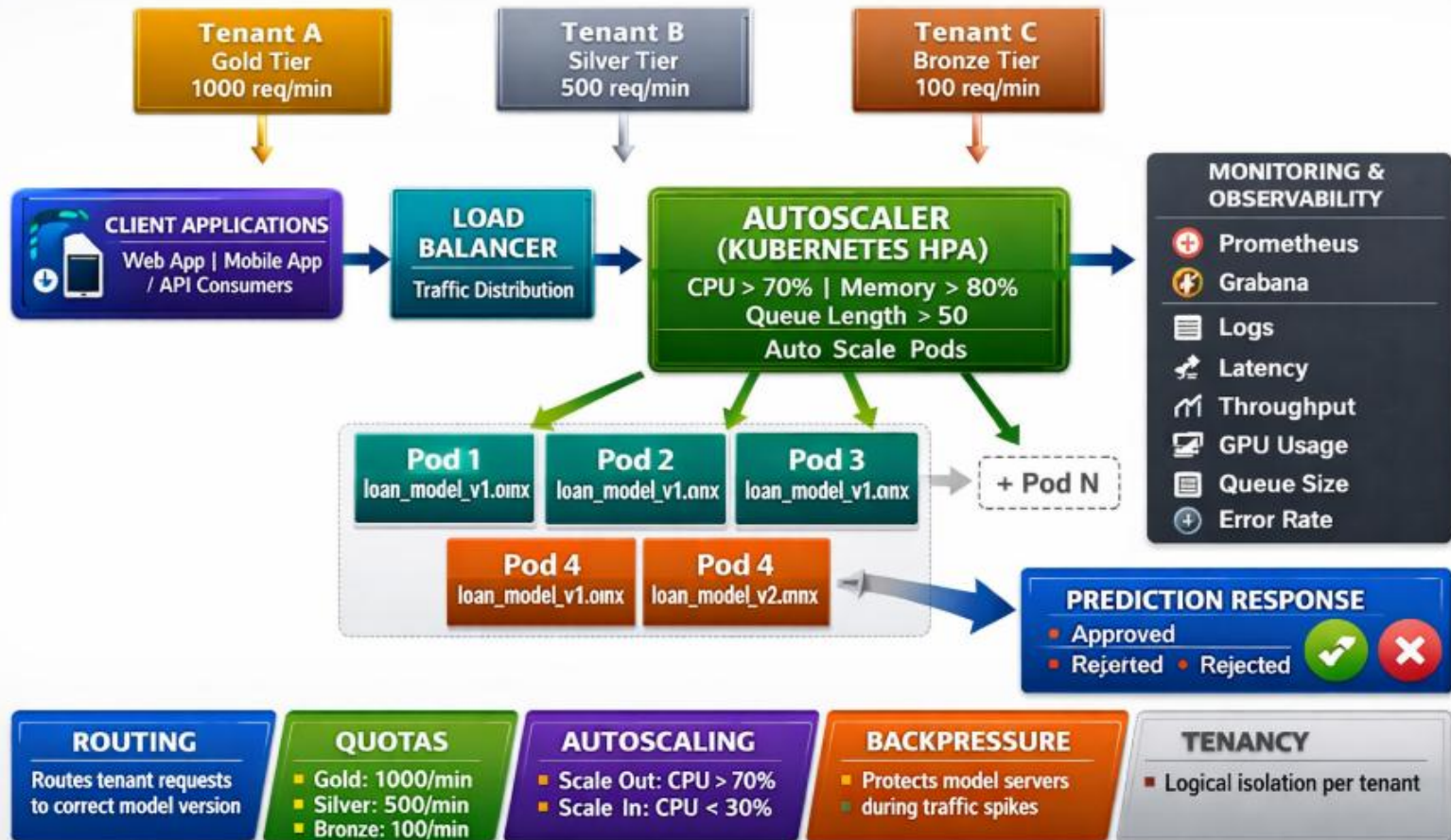
Autoscaling

- `docker build -t loan-onnx-api:v1 .`
- `docker run -p 8000:8000 loan-onnx-api:v1`
- `#autoscale`
- `kubectl autoscale deployment loan-onnx-api --min=1 --max=4 --cpu-percent=20`



Autoscaling

AUTOSCALING FOR ML INFERENCE PLATFORM



- **A/B testing** in inference means routing a fraction of traffic to a new model version while most traffic stays on the baseline.
- Benefits:
 - Compare accuracy, latency, and user impact.
 - Gradual rollout reduces risk.
- Triton can be paired with **Istio/KServe** for traffic splitting (e.g., 90% → v1, 10% → v2).



Shadow Deployments



- **Shadow deployment** sends a copy of live traffic to a new model **without affecting user responses**.
- Useful for:
 - Validating performance under real load.
 - Checking correctness before production rollout.
- Example: Baseline model serves users, shadow model logs predictions for offline comparison.

Putting It Together

- **Batch inference** → offline scoring jobs.
- **Online inference** → Triton serving with autoscaling.
- **A/B testing** → controlled rollout with traffic splitting.
- **Shadow deployment** → silent validation under real traffic.

Questions



Module Summary

- Machine Learning Engineering and ML System Design
- Deep Learning and Scalable Model Serving
- NLP, LLMs and Generative AI Engineering
- Cloud-Native MLOps and Platform Automation

